A Survey of Reinforcement Learning For Economics

Pranjal Rawat, Georgetown University*

March 2026

Abstract

This survey (re)introduces reinforcement learning methods to researchers in the social sciences. The curse of dimensionality limits how far exact dynamic programming can be effectively applied, forcing us to rely on suitably "small" problems or our ability to convert "big" problems into smaller ones. While this reduction has been sufficient for many classical applications, a growing class of economic models resists such reduction. Reinforcement learning algorithms offer a natural, sample-based extension of dynamic programming, extending tractability to problems with high-dimensional states, continuous actions, and strategic interactions. I review the theory connecting classical planning to modern learning algorithms and demonstrate their mechanics through simulated examples in pricing, inventory control, strategic games, and preference elicitation. I also examine the practical vulnerabilities of these algorithms, noting their brittleness, sample inefficiency, sensitivity to hyperparameters, and the absence of global convergence guarantees outside of tabular settings. The successes of reinforcement learning remain strictly bounded by these constraints, as well as a reliance on accurate simulators. That said, when guided by economic structure, reinforcement learning provides a flexible and innovative framework. It stands as an imperfect, but promising, addition to the researcher's toolkit. A companion survey (Rust and Rawat, 2026b) covers the inverse problem of inferring preferences from observed behavior. All simulation code is publicly available.[1]

Keywords: Reinforcement Learning, Economics, Structural Estimation, Inverse Reinforcement Learning, Multi-Agent, Bandits, RLHF

*I am grateful to John Rust for encouraging me to undertake this survey. I thank Simon LeBastard for sharing his RL notes with me.

[1] https://github.com/rawatpranjal/survey-of-reinforcement-learning-in-economics

Contents

1 Introduction

This survey (re)introduces reinforcement learning to researchers studying sequential decision problems. I review the theoretical connections between dynamic programming and reinforcement learning, demonstrating how value iteration, Q-learning, and policy gradient methods are common solution methods to the same class of optimization problems. I then examine applications across several domains, including control problems, pricing and inventory management; structural economic models with high-dimensional state spaces; strategic games in which multi-agent algorithms compute equilibria under imperfect information; bandit problems in which economic structure yields tighter regret bounds; and preference learning. The exposition combines formal theory, practical applications and computational illustration.

Both dynamic programming and reinforcement learning solve the Bellman equation; they differ in the information requirements and the way in which the solution is refined. First, dynamic programming requires knowledge of the transition in the environment and the reward function which allows the reduction of the average Bellman error, reinforcement learning estimates value functions only from sampled transitions (observed sets of state, action, reward, next-state) which only allows reduction of the sampled Bellman error *at that state-action pair*. This allows us to improve policies in domains where it is easier to build a simulator than specify the model of the environment and rewards e.g. board games, physics simulators for robots. Second, dynamic programming makes a "breadth-first" (across all states and actions) update of the solution at each sweep, while reinforcement learning makes a "incremental" (only for the current state and action) update; this greatly reduces the computational burden and enhances scalability.

Reduction of average Bellman errors gives dynamic programming a geometric rate of convergence to the optimal solution, while the incremental updates and reduction of sampled Bellman errors, when combined with "sufficient exploration" of the state-action space, gives reinforcement learning only sublinear convergence guarantees. This is however, quite sufficient in practice, the scalability attained by only sampling transitions and making incremental updates more than makes up for the slower rates of convergence (and brittleness). These approximation methods sacrifice theoretical guarantees. RL algorithms lack the convergence assurances of exact dynamic programming. They exhibit sensitivity to hyperparameters and initialization. They can converge to suboptimal policies without diagnostic indication. This survey presents reinforcement learning as a computationally flexible framework while acknowledging its methodological limitations.

Theory in reinforcement learning trails empirical success, often by years; convergence guarantees, sample complexity bounds, and approximation error characterizations typically arrive after practitioners have demonstrated that an algorithm works. The theoretical insights that eventually follow tend to be deep and structural, and the empirical frontier is itself a productive research frontier. Experiments are brittle, conducted on benchmark environments that are stylized approximations of deployment settings. These benchmarks nonetheless serve a critical coordination function, aligning research effort, enabling reproducible comparison, and exposing failure modes that motivate new theory. Details matter disproportionately in reinforcement learning; small implementation choices can determine whether an algorithm converges or diverges, and the practice of releasing code and documenting hyperparameters, seeds, and preprocessing has proven essential to progress.

This survey focuses on less-surveyed intersections between reinforcement learning and applied decision-making, including the shared theoretical foundations, structural estimation, strategic interaction, bandit problems with domain structure, preference learning, and causal inference. Algorithmic collusion, in which independent pricing algorithms learn to sustain supra-competitive prices (Calvano et al., 2020), is treated in a companion thesis chapter (Rawat, 2026) and omitted here. Reinforcement learning and deep learning methods for solving macroeconomic

models with heterogeneous agents constitute a growing literature with dedicated methodological treatments (Atashbar and Shi, 2022), (Maliar et al., 2021), and (Fernández-Villaverde et al., 2024). Portfolio optimization, optimal execution, and asset pricing via reinforcement learning form a large body of work surveyed comprehensively elsewhere (Hambly et al., 2023). The inverse problem of inferring preferences from observed behavior using inverse reinforcement learning and structural estimation is treated in a companion survey (Rust and Rawat, 2026).

The survey addresses the forward problem, that is, computing optimal policies given a known or simulated environment. Section 3 traces the parallel historical development of dynamic programming and reinforcement learning. Section 4 surveys reinforcement learning algorithms, and Section 5 develops the unified theory connecting planning and learning, with Section 6 examining the empirics of deep RL. Sections 7 and 8 apply reinforcement learning to optimal-control problems and to structural estimation of econometric models. Section 9 treats macroeconomic models, and Section 10 addresses strategic games. Section 11 surveys bandit problems with domain structure. Section 12 covers offline reinforcement learning and Section 13 covers reinforcement learning from human feedback. Section 14 treats causal inference for RL, with Section 15 treating RL methods for causal inference. Section 16 surveys quantile, robust, and constrained reinforcement learning, and Section 17 treats world models and model-based RL. Section 18 concludes.

2 Two Cultures of Sequential Decision-Making

Two intellectual traditions both study sequential decision-making under uncertainty, but they descend from different intellectual traditions. The first is fundamentally an *inference culture*. Its central task is to understand the world. A "model" in this tradition is a specification of preferences, beliefs, constraints, and an equilibrium concept. The RL tradition is instead a *control culture*. Its central task is to act in the world. An RL researcher's "model" is a transition kernel $P(s'|s,a)$ and a reward function $r(s,a)$. These are different mathematical objects serving different scientific purposes.

The inference tradition concentrates its effort on specifying and estimating the objective function and the law of motion that governs the environment. The entire structural econometrics enterprise (demand estimation, dynamic discrete choice) is devoted to recovering these primitives from data, and the optimal policy is a byproduct that falls out once the primitives have been identified. The control tradition inverts this emphasis. Engineers typically take the objective and the dynamics as given (the cost function is specified, the plant physics are known or measurable) and focus on whether the optimal policy can be computed, approximated, and deployed under real-time and robustness constraints. The entire controls enterprise (PID, LQR, MPC, RL) is about computing and implementing the policy under different assumptions about what the agent knows and what it can compute.

The two cultures maintain different relationships with data. Econometricians work primarily with observational data, where endogeneity is the central obstacle; agents sort, markets clear, and unobservables correlate with the variables of interest. Identification, the question of whether the data can distinguish the true model from observationally equivalent alternatives, is the defining challenge, and it disciplines every modeling choice (functional forms, equilibrium definitions, instrumental variables, regression discontinuities, natural experiments). RL researchers have traditionally enjoyed what might be called *simulator omnipotence*. They own the data-generating process, can inject arbitrary variation through exploration policies, and can generate millions of on-policy rollouts at negligible marginal cost. Their binding constraint is computational (can the algorithm converge before the compute budget runs out?) rather than statistical (is the estimator consistent given the endogeneity structure of the data?). This asymmetry shapes everything downstream, from what counts as a valid result to what the word "model" means. To an econometrician, a model is a set of falsifiable restrictions on the joint

distribution of observables and primitives. To an RL researcher, a model is a simulator you can call.

The two fields also share a vocabulary ("agent," "learning," "model," "policy") whose meanings diverge in ways that create persistent confusion. The subsections below provide a systematic translation.

2.1 Core Distinctions

In RL, *prediction* refers to estimating $V^\pi(s)$ or $Q^\pi(s,a)$ for a fixed policy π (policy evaluation), not forecasting observable variables. *Control* refers to finding the policy π^* that maximizes expected discounted return (policy optimization), not the inclusion of regressors. Because prediction concerns evaluating a specific policy, a closely related question is whether the data was generated by the policy being studied. *On-policy* methods evaluate and improve the same policy that generates the data. *Off-policy* methods learn about a target policy π from data generated by a different behavioral policy μ. This distinction is central to causal inference (Section 14), where off-policy evaluation is precisely counterfactual policy evaluation.[2]

Online RL learns while interacting with the environment, collecting new data as a consequence of its own actions. *Offline* RL (also called batch RL) learns exclusively from a fixed dataset of previously collected transitions, with no ability to gather additional samples. The offline setting is closer to standard empirical work, where the dataset is given and the analyst cannot run new experiments; the online setting corresponds more closely to adaptive experimental design or sequential decision problems. Note that "online" in RL carries no timing constraint; it means only that the agent generates fresh experience. *Real-time* RL, by contrast, imposes hard deadlines on the perception-action loop, as in robotics or mechatronics. Every real-time RL system is online, but most online RL (games, recommender systems) are not real-time.

In RL, the word *model* refers strictly to the environment's dynamics, the transition kernel $P(s'|s,a)$ and reward function $r(s,a)$. A *model-based* RL algorithm explicitly constructs or is given a mathematical representation of P and r, then computes a policy by planning through that representation (for example, using a simulator or the known rules of a game, as in AlphaZero). A *model-free* algorithm computes the value function or policy directly from experienced transitions without ever building an explicit representation of the transition probabilities. "Model-free" need not mean the algorithm lacks access to any model of the environment. A model-free algorithm could interact with a simulator that internally implements a complete computerized model of the environment; the distinction is that the algorithm never extracts or plans through the transition probabilities, treating the simulator as a black box that merely returns sample transitions. However, it is possible that a "model-free" algorithm could also be implemented "in-field" where there is genuinely no access to a "model" of the environment but only direct access to the environment itself (for reasons discussed later, this is rarely done).

Neither "model-free" nor "model-based" maps onto the reduced-form versus structural distinction in econometrics. Both labels refer to whether the algorithm uses an explicit representation of P and r, not to whether the analyst makes structural assumptions about preferences or equilibrium. A "model-based" RL algorithm needs only a representation of P and r, regardless of whether those objects arise from structural assumptions about human behavior. Conversely, "model-free" does not mean "assumption-free"; both variants operate within an MDP, which itself imposes the Markov assumption on the state. In the inference tradition, "model" refers to a set of agents, preferences, exclusion restrictions, and equilibrium concepts. It is therefore possible to specify a rich structural model but solve for its equilibrium using a model-free RL algorithm as a computational tool, as in Section 8.

[2] "Counterfactual" here is used in the interventional sense, asking "what would happen if we deployed policy π instead of μ?" This is distinct from the structural counterfactual in Oberst and Sontag (2019), which conditions on the specific realized trajectory and asks what would have happened to *this* individual under a different action, requiring a fully specified structural causal model rather than just the observational distribution under μ.

Every RL system passes through two phases. In the *training phase*, the agent interacts with an environment (simulated or real) and updates its parameters. In the *execution phase* (also called the deployment phase), the policy is frozen and used to make decisions without further updates. This distinction is critical for interpreting "online." Online training in RL almost always takes place inside a simulator (and not in the "real world"). AlphaGo Zero trained online through millions of self-play games; when it faced Lee Sedol, its weights were frozen and it was purely executing its trained policy. Some deployed systems in Section 7 followed this pattern cleanly; DiDi's dispatch system trained a value function from historical trip data, then deployed with fixed weights. Others blur the boundary. The hotel revenue management system in Section 7.2 updated Q-values from realized returns after each completed episode during live operation, making it an *in-field* learner rather than a frozen executor.[3] Bandit algorithms (Section 11) also learn in-field by design, updating demand estimates from real customer responses during deployment.

The word *inference* is overloaded. In machine learning, "inference" refers to executing a frozen model, a forward pass producing outputs from inputs. In statistics, "inference" means statistical inference, the construction of standard errors, confidence intervals, and hypothesis tests. This survey uses "inference" exclusively in the statistical sense and "execution" or "deployment" when referring to applying a trained model (whether in a computer or in field). Established terms such as "Bayesian inference" and "variational inference," which refer to inference about parameters or distributions, are used where appropriate. The key takeaway is that most RL convergence results and sample complexity guarantees refer to the training phase, and interpreting them as claims about deployed performance requires additional argument. Table 1 summarizes these distinctions.

Table 1: The reinforcement learning lifecycle grid. Most RL research operates in the top-left cell. Readers from the inference tradition typically picture the bottom-left when they hear "online."

	Training (parameters updated)	Execution (parameters frozen)
Simulator	· AlphaGo Zero self-play · RL solver in structural estimation · Bandit calibration via simulation	· Policy benchmarks on synthetic environments
Historical data	· RLHF reward model from preference logs · Causal OPE from observational records · Logged-bandit policy learning	· Off-policy evaluation of a target policy
Live market ("in-field")	· Bandit pricing experiments · Hotel RM Q-learning from live episodes	· DiDi dispatch with frozen weights · AlphaGo vs. Lee Sedol

A typical applied RL pipeline moves through the grid sequentially, from pre-training on historical logs (middle-left) to refinement in a simulator (top-left) to deployment with frozen weights (bottom-right). Bandits illustrate this fluidity; even a bandit algorithm that will ultimately learn in-field is typically calibrated in simulation and tuned on historical logs before any live deployment, because in-field exploration incurs real financial cost. The systems that do operate in-field arrive with exploration parameters, initial policies, and demand priors shaped by extensive offline preparation.

[3] "In-field" is not standard RL vocabulary. We introduce it here to distinguish live-market online learning, where exploration has real financial cost, from the far more common case of online learning inside a simulator.

The Tmall e-commerce pricing project of Liu et al. (2019) (Section 7.3) illustrates this migration concretely. The team pre-trained a DQN from logged specialist pricing decisions (historical data, training), then ran offline evaluation of the candidate policy on held-out transaction logs before any live deployment (historical data, execution). The evaluated policy was deployed for 15 to 30 day field experiments on live Tmall traffic, with the agent receiving reward and observation signals from the market environment (live market, execution and training). Liu et al. (2019) note that no accurate simulator exists for e-commerce pricing, so the project skipped the simulator row entirely, jumping from historical pre-training directly to live deployment. Not every application traverses all six cells of Table 1, but the grid clarifies which cells a given project could be working on.

2.2 Overlapping Terminology

In the inference tradition, an *agent* is always a human decision-maker (a consumer, worker, or firm) or a social planner whose choices are the object of study. In RL, "the agent" is the learning algorithm itself, or more precisely an algorithm deployed on behalf of a human decision-maker, and the human, if present at all, is part of the environment providing reward signals.

In RL the *environment* is a formal object encompassing everything outside the agent (the DGP, other agents, market clearing conditions), whereas in the inference tradition "environment" refers more loosely to market structure or institutional rules.[4]. RL speaks of *rewards* where the other tradition speaks of *utility*. The mapping is not exact: a reward $r(s, a)$ is a known, externally specified function that the algorithm maximizes, whereas utility $u(x, d)$ in the inference tradition is a primitive of preferences that the analyst must recover or estimate from observed choices.

In RL, the return $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$ is the discounted sum of future rewards from time t onward, the random variable whose expectation defines the value function. The RL usage is closer to what is called the "present discounted value" of a stream of payoffs. A single complete sequence of interactions from an initial state to termination, $(s_0, a_0, r_1, s_1, \ldots, s_T)$, is called an *episode* (synonymously, *trajectory* or *rollout*).[5] Where a statistician speaks of the *outcome*, meaning the dependent variable Y in a regression, RL has no single analog: the reward r_t is the per-period outcome, the return G_t is the cumulative outcome, and the value function $V^{\pi}(s)$ is the expected cumulative outcome conditional on state.

Learning has several distinct meanings. In decision theory (Bayesian learning, adaptive expectations), learning refers to agents forming and refining beliefs about unknown parameters of their environment. In supervised machine learning, learning means statistical estimation, fitting the weights of a parameterized model to minimize a loss function over data. In reinforcement learning, "learning" is primarily computation. When an RL agent "learns" a Q-function, it is executing a recursive stochastic approximation algorithm to find the fixed point of the Bellman operator. We say the algorithm is "learning" because it improves its policy iteratively through simulated or real experience, but mathematically it is solving a fixed-point problem. In many applications throughout this survey, particularly Section 8, the RL algorithm is simply a numerical method for solving the Bellman equation; no actual human-like learning from experience is taking place. Finally, while RL draws inspiration from animal psychology (Section 3.1), it is a drastic simplification of biological learning. Tabula rasa RL algorithms require millions of iterations of trial and error to discover policies that animals acquire rapidly. Real-world animal learning relies on innate priors, basic physical knowledge, and parental nurturing and should be

[4] A source of confusion is *generative model*. In machine learning broadly, this means a model of the joint distribution $P(X, Y)$ or a synthetic data generator (GANs, diffusion models). In RL theory, a generative model is a simulator oracle that, given any (s, a), returns a sample $s' \sim P(\cdot|s, a)$ and reward $r(s, a)$; the usage implies random-access simulation, stronger than sequential online interaction but weaker than knowing P analytically.

[5] The closest statistical analogs are a single panel unit's time series, a realization of a stochastic process, or one "history" in a dynamic model.

distinct from RL-style "learning".

Adaptive learning in macroeconomics has used ideas formally analogous to reinforcement learning for decades, though under different names and with different motivations. In the adaptive learning literature initiated by Marcet and Sargent (1989), boundedly rational agents update their beliefs about equilibrium parameters using recursive least-squares and other Robbins-Monro-type stochastic approximation algorithms. The convergence criterion in that literature, E-stability, evaluates whether the ordinary differential equation (ODE) associated with the mapping from the perceived to the actual law of motion is locally asymptotically stable at the rational expectations equilibrium. This fixed-point stability requirement is analogous to the contraction conditions governing the convergence of temporal-difference and Q-learning algorithms in RL. Borkar and Meyn (2000) make this mathematical connection explicit: they prove the convergence of Q-learning and actor-critic methods using the ODE method, explicitly citing Sargent (1993) as a parallel application of the exact same framework to boundedly rational agents.

Active learning appears in both fields but means different things. In the inference tradition, active learning denotes Bayesian experimentation where an agent endogenously chooses what information to acquire, trading off the cost of exploration against the option value of better future decisions. In machine learning, active learning is a supervised learning protocol in which the algorithm selects which unlabeled examples to query an oracle for labels, minimizing annotation cost rather than maximizing cumulative reward. The term *exploration* is ubiquitous in RL but rarely used in the inference tradition. In RL, exploration is a modular subcomponent of a larger algorithm, a procedure for choosing informative actions that may be quite crude (such as ε-greedy random action selection) or more principled (UCB, Thompson sampling). The closest analog in the other tradition is *optimal experimentation* in the Bayesian bandit tradition (Rothschild 1974), where the value of information is derived endogenously from the agent's dynamic program, not bolted on as a separate heuristic. The inference tradition also uses *Bayesian learning* to describe the same phenomenon, but always within a fully specified model of beliefs and preferences; in RL, exploration can be a purely algorithmic device with no decision-theoretic foundation.

Bootstrapping in statistics refers to Efron's resampling method (Efron, 1979); in RL, it means updating a value estimate using another value estimate rather than a complete realized return, as when a TD algorithm uses the target $r_{t+1} + \gamma V(s_{t+1})$ that depends on the current, uncertain estimate $V(s_{t+1})$ (Sutton, 1988). *Function approximation* in RL refers to representing value functions or policies using parameterized function classes (linear combinations of basis functions, kernel methods, or neural networks), which statisticians will recognize as sieve estimation or nonparametric series estimation, the approximation of an unknown function by projection onto a finite-dimensional basis. *Calibration* carries unrelated meanings. In the inference tradition, calibration means choosing model parameters to match a set of empirical moments or stylized facts (Kydland and Prescott 1982). In machine learning, calibration refers to probability calibration, ensuring that predicted probabilities match observed frequencies, a statistical property of a classifier's outputs.

The term *bandit* itself carries different mathematical content across disciplines. The classical multi-armed bandit in statistics (Thompson, 1933; Rothschild, 1974; Gittins, 1979) is a Bayesian sequential allocation problem. The state is the agent's posterior belief over the unknown arm distributions, and the solution is the Gittins index, an optimal allocation rule derived from the theory of optimal stopping. In the RL and computer science literature, bandits are instead framed as frequentist regret-minimization problems. Algorithms such as UCB provide worst-case bounds on cumulative regret $\sum_{t=1}^{T}(\mu^* - \mu_{A_t})$ without requiring Bayesian priors. The two traditions ask fundamentally different questions, Bayesian optimality of the full sequential problem versus minimax regret rates over adversarial or stochastic environments, and their answers are not directly comparable. Section 11 adopts the regret framework because it connects

more naturally to the sample complexity concerns that arise in field experiments.

The term *contextual bandit* is especially liable to misreading. To a reader from the inference tradition, the "context" is simply the state variable x_t, and a contextual bandit looks like an MDP with an unknown reward parameter. What the RL literature signals by "context" is a specific structural restriction on transitions, the agent's action has no causal effect on the next context, so that $P(x_{t+1} \mid x_t, a_t) = P(x_{t+1})$. This exogeneity assumption separates the exploration problem (learning which arm is best given the current context) from the planning problem (choosing actions that influence future states). When contexts evolve exogenously, there is no long-horizon credit assignment, and the problem reduces to repeated one-period optimization under uncertainty. The term "contextual" therefore flags a modeling assumption about dynamics, not merely the presence of observable covariates.

The RL literature frames some recommender systems as contextual bandits, where user covariates are the context, the recommendation is the arm, and a click or rating is the reward. One might instead view movie recommendation as a two-sided learning problem in which the platform learns user preferences while users simultaneously explore the catalog and update their own tastes. The bandit formulation absorbs the user's utility maximization into the environment's reward signal and treats user arrivals as exogenous. This is a modeling choice, not a fact about the world. Also, the object called a "bandit" in this formulation, a one-step decision under exogenous context, is a slightly different mathematical object than the Bayesian sequential allocation problem of Gittins (1979), even though both carry the same name and are related. [6] RL abstracts away much of this complexity (human learning, strategic interaction) to fit the problem into the standard MDP framework (π, V, P, r).

The word *policy* is overloaded. In the inference tradition, a "policy" is a rule set by a government, central bank, or regulator, a tax schedule, a subsidy, an interest rate rule, a licensing requirement. These rules are part of the *environment* in which private agents (consumers, firms) optimize. "Policy evaluation" in this tradition means changing some aspect of the environment and asking how agents would respond: if the earned income tax credit were expanded, how would labor supply shift? The standard workflow proceeds in two steps: first estimate or calibrate the structural model (preferences, technology, market interactions), then simulate counterfactuals under the alternative rule. In RL, "policy" means the agent's own decision rule $\pi(a|s)$, and "policy evaluation" means computing $V^\pi(s)$, the expected return from following that rule in a fixed environment. RL typically assumes access to a high-fidelity simulator, a physics engine, a game, a digital twin, whose rules do not change; the interesting question is how the agent should behave within those rules, not what happens if the rules change. When the environment does shift, RL treats it as a nuisance (sim-to-real transfer, domain adaptation) rather than the object of study; in offline RL (Section 12), distributional shift between the training data and the deployed environment becomes a central concern, bringing RL closer to standard counterfactual reasoning.[7]

Identification means different things in the two fields. In statistics, a parameter θ is identified if it is uniquely pinned down by the combination of the data-generating process and the model's maintained assumptions; formally, no two distinct parameter values $\theta \neq \tilde{\theta}$ can generate the same distribution of observable data (Lewbel, 2019). Identification is a logical property of the model, not a statistical one; if identification fails, no amount of additional data or more sophisticated estimation will recover the parameter, because multiple parameter values are observationally equivalent. In RL, there is little general identification discourse. The closest analogs are narrow and domain-specific. In inverse reinforcement learning, reward identifiability asks whether the

[6] Lattimore (2016) proves that the Gittins index with a flat Gaussian prior achieves finite-time regret of the same order as UCB, and that the index decomposes as posterior mean plus an exploration bonus that shrinks toward zero near the horizon, structurally resembling but differing from the UCB confidence bound.

[7] A notable exception is Tomasev et al. (2020), where AlphaZero was used to evaluate alternative chess rule sets proposed by Kramnik, asking how optimal play changes under modified game rules. This is precisely an environment counterfactual.

reward function can be uniquely recovered from observed optimal behavior; Kim et al. (2021) formalize this for MaxEnt MDP models, showing that for deterministic dynamics, strong identifiability requires the domain graph to be coverable and aperiodic.[8] In model-based RL, system identification refers to learning the transition dynamics $\hat{T}$ well enough for the resulting policy to perform well, a usage inherited from control theory rather than statistics (Ross and Bagnell, 2012). RL researchers focus on out-of-sample performance of the learned controller, so whether parameters are identified in the statistical sense matters less; what matters is that the policy generalizes.[9]

Regret diverges across fields. In decision theory, regret is either an emotion that modifies preferences under uncertainty (Loomes and Sugden 1982) or a static minimax decision criterion for choosing among actions when probabilities are unknown (Savage 1951, Manski 2004). In RL and computer science, regret is a dynamic performance metric, the cumulative gap between the agent's returns and those of the best fixed policy in hindsight, and sublinear growth of this quantity is the standard benchmark for online learning algorithms. In the other tradition, *efficiency* concerns welfare: Pareto efficiency asks whether any reallocation could improve one agent's utility without harming another, and allocative efficiency asks whether goods flow to their highest-valued uses. In RL, efficiency concerns resources: sample efficiency measures how many environment interactions an algorithm needs to find a good policy, and computational efficiency measures the operations required per timestep.

In the inference tradition, the *discount factor* is a structural parameter encoding time preference, an agent's intrinsic willingness to trade present for future consumption. Koopmans (1960) derived it axiomatically from preference postulates, principally stationarity and impatience, showing that these behavioral axioms uniquely imply an exponential discounted utility representation with a single discount factor strictly between zero and one (Bleichrodt et al., 2008). This parameter is treated as a deep primitive of preferences, to be estimated from data on intertemporal choices, and deviations from constant discounting (such as the quasi-hyperbolic present bias of Laibson 1997) are studied as substantive behavioral phenomena. In RL, the discount factor has historically served a more pragmatic role, ensuring that infinite-horizon returns remain finite and that the Bellman operator is a contraction, thereby guaranteeing convergence of dynamic programming algorithms. It is typically treated as a hyperparameter to be tuned for computational performance rather than a structural claim about the agent's preferences. Pitis (2019) bridges this gap, axiomatically deriving discounting in RL from rationality postulates and showing that the fixed discount factor is best understood as an optimizing representation rather than a literal description of time preference.

2.3 Structural Equivalences

Beyond terminological differences, several formal objects in RL and the inference tradition are mathematically identical. The softmax (or Boltzmann) policy used throughout RL is the multinomial logit model of McFadden (1974a). The RL softmax policy selects actions according to

$$\pi(a \mid s) = \frac{\exp(Q(s,a)/\tau)}{\sum_{a' \in \mathcal{A}} \exp(Q(s,a')/\tau)}, \tag{1}$$

where $\tau > 0$ is a temperature parameter. In the discrete choice framework, $Q(s,a)$ plays the role of the deterministic component of utility $v(a \mid x)$, and τ is the scale parameter of the Type I extreme value (Gumbel) taste shocks ε_a. As $\tau \to 0$, the policy converges to the greedy

[8] Kim et al. (2021) explicitly note that the literature on identifying dynamic discrete choice models (Rust, 1994) addresses "an equivalent problem" to reward identifiability in IRL, providing a direct bridge between the two fields.

[9] The one RL subfield where identification is central, inverse reinforcement learning, is the subject of the companion survey (Rust and Rawat, 2026).

(deterministic) policy, just as the logit choice probability concentrates on the utility-maximizing alternative as the variance of taste shocks vanishes.

The entropy regularization commonly added to RL objectives is the inclusive value (or log-sum-exp) from the discrete choice literature. The soft value function

$$V^{\text{soft}}(s) = \tau \log \sum_{a \in \mathcal{A}} \exp(Q(s,a)/\tau) \tag{2}$$

is identical to the McFadden surplus function $W(x) = \tau \log \sum_a \exp(v(a|x)/\tau) + C$, where C is Euler's constant. In the structural estimation literature, this object appears as the Emax function in dynamic discrete choice models following Rust (1987).

The action-value function $Q^\pi(s,a)$ is the choice-specific value function $v_\theta(x,d)$ of the DDC literature (Table 2). The advantage function $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ is therefore the choice-specific value net of the ex-ante value, a quantity that appears in the Hotz and Miller (1993) CCP estimator for dynamic discrete choice models.[10]

The two fields arrived at this shared mathematics from opposite directions. In RL, entropy regularization began as a pragmatic trick to prevent premature convergence and encourage exploration (Williams and Peng, 1991), and only decades later did Ziebart (2010) and Levine (2018) provide decision-theoretic foundations showing that the resulting policies are robust to model misspecification. In the inference tradition, the softmax emerged not from any concern about exploration but from the random utility framework of McFadden (1974a), where agents are fully informed and choose deterministically; the apparent randomness arises entirely from the econometrician's inability to observe all relevant taste variation. The RL agent randomizes because it is ignorant of the environment; the economic agent appears to randomize because the observer is ignorant of the agent's preferences.

2.4 Notation

Table 2 maps the notation used throughout this survey to the most common econometric equivalents.

Table 2: Notation mapping between reinforcement learning and the inference tradition.

RL Term	Symbol	Inference Tradition	Symbol
State	$s \in \mathcal{S}$	State variable, covariate	x_t, Ω_t
Action	$a \in \mathcal{A}$	Choice, control variable	d_t, u_t[11]
Reward	$r(s,a)$	Per-period utility, payoff	$u(x,d)$
Discount factor	γ[12]	Discount factor	β
Policy	$\pi(a\|s)$	Decision rule, CCP	$P(d\|x)$
Value function	$V^\pi(s)$	Ex-ante value function	$\bar{V}_\theta(x)$
Q-function	$Q^\pi(s,a)$	Choice-specific value function	$v_\theta(x,d)$
Return	G_t	Present discounted value	$\sum_{k=0}^{\infty} \beta^k u_{t+k}$
Transition	$P(s'\|s,a)$	State transition law	$f(x_{t+1}\|x_t,d_t)$
Learning rate	α	Step size	α_n
TD error	δ_t[13]	Bellman residual at sample	–

[10] These equivalences reflect a common convex-analytic structure. The log-sum-exp value function and the negative Shannon entropy are Fenchel conjugates of each other, so maximizing expected payoffs with an entropy bonus and recovering utilities from observed choice probabilities are two views of the same optimization. Chiong et al. (2016) build on this conjugate duality to develop identification and estimation methods for dynamic discrete choice models, and Fosgerau and Sørensen (2022) show that the underlying structure extends well beyond the logit case.

3 A Brief History of Reinforcement Learning

Reinforcement learning draws on animal psychology, game-playing programs, and optimal control theory in roughly equal measure. Thorndike's law of effect and behaviourist trial-and-error learning provided the notion of "reinforcement" as formalized by Rescorla-Wagner. Chess and checkers programs of Shannon and Samuel from the 1950s onward gave researchers concrete problems on which to test ideas about machine learning. The Bellman-Howard-Blackwell dynamic programming framework provided the recursive structure and language.

3.1 Animal Psychology

Controlled experiments on rats, cats, and dogs inspired the "gridworld" environments still used today, and shaped how the field conceptualizes "training" agents through "reward signals". Thorndike (1898) placed cats in puzzle boxes with latched doors and food visible outside. Across 15 different box configurations, the cats initially engaged in undirected behavior such as clawing at the walls, pushing against the bars, reaching through openings. The first cat to escape the simplest box required 160 seconds of random activity before accidentally pressing the latch. By the 24th trial, the same cat pressed the latch directly within 6 seconds. The learning curves showed gradual, continuous improvement rather than sudden insight. From these experiments the Law of Effect was formulated: responses (actions) followed by satisfaction (positive rewards) are "stamped in" and more likely to recur, while those followed by discomfort (negative rewards) are "stamped out."

Pavlov (1927) noted that dogs salivated not only at food itself but at the sight of an empty food bowl, the sound of footsteps, and other stimuli that preceded feeding (i.e. the state). To measure these "psychic secretions" precisely, he surgically implanted fistulas (tubes allowing external collection) to collect saliva. In the canonical experiment, a metronome sounded before food delivery. After 20–40 pairings, the metronome alone elicited salivation. The response followed not from the stimulus itself but from what it "predicted". In reinforcement learning terms, the conditioned stimulus is a state s, and the learned expectation of food is the value function $V(s)$.

Kamin (1969) demonstrated that learning requires more than mere co-occurrence in time. In Phase I of his blocking experiment, rats learned that a noise predicted a shock and developed a conditioned fear response to the noise alone. In Phase II, a compound stimulus of noise plus light was paired with the same shock. When the light was subsequently presented alone, no fear response occurred. The light was "blocked" (ignored) because the noise already predicted the shock perfectly. There was no prediction error (or "surprise") to drive learning about the light. Once the noise was established as a predictor, the light added no new information and thus no new learning occurred.

Rescorla and Wagner (1972) formalized the blocking phenomenon as a prediction-error learning rule. Translating to RL notation:[14]

$$V(s) \leftarrow V(s) + \alpha\,\delta, \quad \text{where } \delta = r - V(s) \tag{3}$$

This is temporal difference learning with $\gamma = 0$, without discounting of future rewards. Each stimulus s begins with $V(s) = 0$. On each trial, the organism observes the stimuli present, receives outcome $r \in \{0, 1\}$, and updates values according to δ. The model is purely predictive; there are no actions, only learned expectations about reward.[15] The model's power lay in

[14] The original notation used V_i for associative strength of stimulus i, α_i for stimulus salience (noticeability), β_j for learning rate, λ_j for maximum conditioning (1 if reward present, 0 otherwise), and $V_{\text{tot}} = \sum_k V_k$ for total prediction. The correspondence is: $V_i \to V(s)$, $\alpha_i\beta_j \to \alpha$, $\lambda_j \to r$, $V_{\text{tot}} \to V(s)$. See Sutton and Barto (1990) for details.

[15] The Rescorla-Wagner update is mathematically identical to the Widrow-Hoff least mean squares rule.

prediction, not just explanation. It correctly predicted overexpectation, whereby two separately conditioned stimuli combined and reinforced together each lose value because their summed prediction exceeds r.

3.2 Board Games

Chess and checkers are sequential decision problems. The board position is a state $s \in \mathcal{S}$, a legal move is an action $a \in \mathcal{A}(s)$, and the resulting position is a successor state $s' = T(s, a)$ determined by the rules of the game. The game outcome provides a terminal reward $r \in \{+1, 0, -1\}$ for win, draw, or loss. An evaluation function $f(P)$ that scores a position corresponds to a value function $V(s)$ estimating expected outcome. These games are deterministic (no chance moves in chess), fully observable (both players see the entire board), and zero-sum (one player's gain is the other's loss). The adversarial structure introduces a second player whose actions a' must be anticipated.

Shannon (1950) posed the fundamental question: can we program a general-purpose computer to play chess, and if so, what principles should guide the design? The challenge is computational. A chess game averages 40 moves per player with roughly 30 legal moves available at each position. Shannon estimated that exhaustive search through all possible games would require examining approximately $30^{80} \approx 10^{120}$ positions, a number exceeding the atoms in the observable universe. This is the curse of dimensionality applied to games, where state space size grows exponentially with the number of sequential decisions. Shannon calculated that brute-force enumeration would require 10^{90} years at any foreseeable computing speed. The curse intensifies with game complexity. Chess has approximately 10^{47} legal positions, shogi 10^{71}, and Go 10^{171}.[16]

Shannon distinguished two approaches. Type A strategies search all continuations to a fixed depth H, building a complete game tree and evaluating every leaf ("rote-learning"). Type B strategies search selectively, examining only variations deemed important by some criterion ("generalization"). Either approach requires an evaluation function $f(P)$ to score positions where search terminates. Shannon proposed linear evaluation:

$$f(P) = \sum_i w_i \phi_i(P) \tag{4}$$

with features ϕ_i for material, mobility, pawn structure, and king safety. Weights w_i were hand-tuned. The minimax principle governs adversarial search. In a two-player zero-sum game, the value of a position satisfies

$$V(s) = \max_{a \in \mathcal{A}(s)} \min_{a' \in \mathcal{A}'(s')} V(T(s, a, a')) \tag{5}$$

where the maximizing player moves first and the minimizing opponent responds optimally. This is model-based planning, since the transition function T is known exactly from the game rules. The computational problem is how to use limited search resources effectively given the exponential tree.

Both Type A and Type B strategies truncate the game tree at depth H and substitute the evaluation function for exact continuation values. This is approximate dynamic programming. The true value $V^*(s)$ satisfies a recursive equation, but computing it exactly is infeasible, so the recursion is truncated and terminal values approximated. Deeper lookahead (H-step search) builds larger trees; rollout policies extend search by simulating play to the end using a fast base policy.[17] The evaluation function serves as a heuristic substitute for exact computation. Shan-

[16] State space estimates from Igami (2020).

[17] Monte Carlo tree search, developed later, samples rollouts rather than enumerating all branches, enabling deeper effective lookahead in games with large branching factors.

non did not implement a chess program; the 1950 paper is theoretical, outlining the architecture that shaped fifty years of game engines.

Samuel (1959) built a checkers program for the IBM 704 that could improve through experience. The program played against itself, generating virtually unlimited training data at no cost. It parameterized the value function as a linear combination of hand-crafted features (piece advantage, mobility, king safety):

$$V(s;\mathbf{w}) = \sum_i w_i \phi_i(s) \tag{6}$$

After each move from s_t to s_{t+1}, weights were updated by temporal difference:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[V(s_{t+1};\mathbf{w}) - V(s_t;\mathbf{w})\right] \nabla_{\mathbf{w}} V(s_t;\mathbf{w}) \tag{7}$$

In a 1965 match, World Champion W.F. Hellman won all four games played by mail, but was played to a draw in one game. After learning from 173,989 book moves, the program agreed with the book-recommended move (or rated only 1 move higher) 64% of the time without lookahead. With lookahead and minimaxing, it followed book moves "a much higher fraction of the time."

The linear parameterization compresses the value function from 10^{20} table entries to dozens of weights, the essential response to the curse of dimensionality. Samuel's architecture, minimax search to depth H with the learned evaluation function scoring leaves, is an early instance of rollout, where tree search simulates forward, truncating at H and substituting $V(s)$ for exact continuation values.[18] The conceptual apparatus of modern game-playing AI (self-play, evaluation learning, tree search, and function approximation) was present in the 1950s.

3.3 Optimal Control

Bellman (1957) considered multi-stage decision processes in which a system occupies state $s \in \mathcal{S}$, the decision-maker chooses action $a \in \mathcal{A}$, the system transitions to $s' \sim P(\cdot|s,a)$, and a reward $r(s,a,s')$ accrues. The objective is to maximize cumulative reward over a finite or infinite horizon. The classical approach treats an N-stage process as a single N-dimensional optimization. Bellman calculated the consequence. A 10-stage process with 10 grid points per variable requires 10^{10} function evaluations; at one evaluation per second, 10^{10} evaluations require 2.77 million hours. He called this exponential growth the curse of dimensionality. His solution was the principle of optimality, namely that an optimal policy has the property that, whatever the initial state and initial decision, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision. This principle yields the Bellman equation:

$$V^*(s) = \max_{a \in \mathcal{A}} \left[r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^*(s') \right] \tag{8}$$

The equation reduces an N-dimensional problem to a sequence of N one-dimensional problems. Value iteration computes V^* by iterating $V_{k+1}(s) = \max_a [r(s,a) + \gamma \sum_{s'} P(s'|s,a) V_k(s')]$. The monograph applied the method to resource allocation, inventory control, bottleneck scheduling, gold mining under uncertainty, and multi-stage games.[19]

Howard (1960) observed that value iteration converges slowly for problems of indefinite duration. His alternative, policy iteration, solves for the value function of a fixed policy and then improves the policy directly. Given policy π, policy evaluation computes the gain g (average reward per period) and relative values v_i by solving the linear system $g + v_i = q_i + \sum_j p_{ij} v_j$

[18] Bertsekas (2021) interprets this as approximate dynamic programming, where offline training (learning V) combined with online planning (tree search) implements a Newton-like step for the Bellman equation.

[19] Chapter IX shows how dynamic programming derives classical variational conditions from the functional equation. The continuous-time analogue is the Hamilton-Jacobi-Bellman equation.

for $i = 1, \ldots, N$, where q_i is the expected immediate reward in state i and p_{ij} is the transition probability under π. Policy improvement then selects, for each state i, the action k maximizing $q_i^k + \sum_j p_{ij}^k v_j$. Howard proved that each iteration strictly increases the gain unless the policy is already optimal, and the algorithm terminates in finitely many steps. For a problem with 50 states and 50 actions per state, exhaustive enumeration must consider $50^{50} \approx 10^{85}$ policies; policy iteration finds the optimum in a handful of iterations. Howard demonstrated the method on a toymaker's production problem, taxicab dispatch in three city zones, and automobile replacement timing.

Blackwell (1965) established the measure-theoretic foundations for discounted dynamic programming with general state and action spaces. He proved that the Bellman operator T defined by $Tu(s) = \sup_a [r(s,a) + \gamma \int u(s')P(ds'|s,a)]$ is a contraction with modulus γ: $\|Tu - Tv\|_\infty \leq \gamma \|u - v\|_\infty$. Banach's fixed-point theorem then guarantees a unique bounded solution V^* to the Bellman equation, with $\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty$ under value iteration. The central result concerns stationary policies, which use the same decision rule $f : \mathcal{S} \to \mathcal{A}$ at every period regardless of history. Blackwell proved that if the action space is finite, there exists an optimal stationary policy. For countable action spaces, ϵ-optimal stationary policies exist for every $\epsilon > 0$. These results justify the focus on memoryless policies, since optimal behavior depends only on the current state, not on the history of past states and actions. The Bellman equation, Howard's policy iteration, and Blackwell's existence theorems constitute the planning framework. Given complete knowledge of P and r, these methods compute optimal policies exactly. The challenge of learning without such knowledge is the central problem of reinforcement learning.[20]

4 Reinforcement Learning Algorithms

4.1 The Classical Synthesis

4.1.1 Monte Carlo Estimation

When $P(s'|s,a)$ and $r(s,a)$ are unknown, the obvious approach is to use Monte Carlo (MC) to approximate them. These methods estimate value functions from sampled *episodes* $(s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_T)$. The realized *return* from state s_t is

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1} \tag{9}$$

First-visit MC prediction averages G_t over episodes for each state s, counting only its first occurrence per episode. Each first-visit return is an independent draw from the return distribution, so the sample mean converges almost surely to $V^\pi(s)$ by the strong law of large numbers (Sutton and Barto, 2018). An incremental update is,

$$V(s) \leftarrow V(s) + \alpha[G_t - V(s)]$$

For MC control, we can estimate action-values $Q(s,a)$ by averaging first-visit returns from each state-action pair, then improve the policy greedily: $\pi(s) = \mathit{argmax}_a\, Q(s,a)$. Under exploring starts (every (s,a) pair begins an episode infinitely often), this alternation converges to Q^*.[21]

[20] For comprehensive treatments of dynamic programming in economics, including numerical methods, computational complexity, and the curse of dimensionality, see Rust (2008) and Rust (1996).

[21] Tsitsiklis (2002) proved convergence even when the policy improves after every episode rather than waiting for complete evaluation. In practice, exploring starts is infeasible, so on-policy variants use ε-greedy exploration instead. These converge to Q^* provided the exploration schedule satisfies the greedy-in-the-limit with infinite exploration (GLIE) condition: every state-action pair is visited infinitely often, and $\varepsilon_t \to 0$ so the policy converges to greedy in the limit.

4.1.2 Sutton (1988)

Monte Carlo has two limitations. The agent must wait for episode termination to compute G_t, ruling out continuing tasks. And G_t is unbiased ($\mathbb{E}[G_t \mid S_t = s] = V^\pi(s)$) but high-variance, because it sums random rewards over the entire *trajectory*.

Sutton (1988) proposed temporal difference (TD) learning to fix both problems. TD(0) replaces the full return G_t with a one-step target:

$$V(s_t) \leftarrow V(s_t) + \alpha \left[r_{t+1} + \gamma V(s_{t+1}) - V(s_t) \right] \tag{10}$$

The target $r_{t+1} + \gamma V(s_{t+1})$ depends on one random reward and one random transition, so its variance is low. The cost is bias. The bootstrap target $V(s_{t+1})$ is the agent's current estimate, not the true value. This is "*bootstrapping*".[22] As V improves, the bias shrinks; the low variance persists regardless. Sutton demonstrated this tradeoff on a five-state random walk where TD(0) converged faster than Monte Carlo with less data.

The general TD(λ) update interpolates between these extremes through an eligibility trace.[23]

$$V(s_t) \leftarrow V(s_t) + \alpha \, \delta_t \, e_t(s) \tag{11}$$

where $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ is the TD error and $e_t(s) = \gamma\lambda \, e_{t-1}(s) + \mathbb{1}\{s = s_t\}$ is the eligibility trace for state s. Setting $\lambda = 0$ yields TD(0); setting $\lambda = 1$ recovers Monte Carlo returns. Intermediate λ trades off variance against bias. [24]

4.1.3 Watkins (1989)

TD(0) learns value functions $V(s)$, but converting these to actions (the control problem) still requires knowing transition probabilities. Given $V(s')$ for all successor states, the agent needs to know which action leads to which successor.

Watkins and Dayan (1992a), formalizing Watkins's 1989 PhD thesis, provided the solution. Instead of learning $V(s')$ learn $Q(s, a)$ (the action value function or the "quality" of actions function) directly, the expected return from taking action a in state s and then behaving optimally. The optimal policy is then $\pi^*(s) = \mathit{argmax}_a \, Q^*(s, a)$, requiring no model to act. The Bellman optimality equation provides the fixed-point condition:

$$Q^*(s, a) = \mathbb{E}\left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right] \tag{12}$$

Q-learning achieves this via the update

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right] \tag{13}$$

Q-learning converges to Q^* under standard regularity conditions (Section 5.2). The maximization over a' makes Q-learning *off-policy*:[25] the update target uses the greedy action at the next state regardless of the action actually taken. Therefore the agent can follow an ε-greedy exploration strategy or even a fully random policy, while learning about the optimal policy

[22] See Section 2 for the distinction between RL bootstrapping and Efron's resampling procedure.

[23] The eligibility trace records which states were recently visited, allowing credit assignment to propagate backward in time. States visited more recently receive stronger updates when the TD error is observed.

[24] Dayan (1992) proved convergence of TD(λ) for general λ in the tabular case; Jaakkola et al. (1994) gave a unified stochastic approximation proof covering both TD and Q-learning; Tsitsiklis and Van Roy (1997) extended the analysis to linear function approximation.

[25] See Section 2 for the on-policy/off-policy distinction. Q-learning learns about the greedy policy $\pi^*(s) = \mathit{argmax}_a \, Q(s, a)$ while collecting data with an exploratory policy. This exploratory policy needs only to be sufficiently "exploratory" and admits a wide range of policies; including a fully random policy.

directly.

4.1.4 Williams (1992)

Value-based methods learn an action-value function and derive a policy from it. Williams (1992) derived an alternative, policy gradients, that optimize the policy directly by gradient ascent on expected return

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)\, G_t\right] \tag{14}$$

where $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k+1}$ is the discounted return from time t. The log-derivative trick allows the gradient to be estimated from sampled trajectories without differentiating through the environment dynamics.

Consider a robot arm that must apply a continuous torque $a \in \mathbb{R}$ to reach a target angle. Q-learning requires computing $\max_a Q(s,a)$ at every update, which becomes a nested optimization problem [26] when the action space is continuous. A policy gradient method sidesteps the issue. Parameterize the policy as a Gaussian $\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s),\, \sigma_\theta^2(s))$,[27] sample an action, observe the return, and update θ by REINFORCE. Virtually all continuous-control results in reinforcement learning descend from the policy gradient framework for this reason.

4.1.5 Tesauro (1994)

Tesauro (1994)'s TD-Gammon demonstrated that temporal difference learning with neural network function approximation could achieve expert-level play in a domain with approximately 10^{20} legal positions[28].

Backgammon was far beyond tabular methods, yet TD-Gammon trained a feedforward neural network to estimate the probability of winning from any board position. A hidden layer[29] of 80 sigmoid units fed a single sigmoid output.

$$\hat{V}(s) = \sigma\big(\mathbf{w}^\top \sigma(W\mathbf{x}(s) + \mathbf{b}) + c\big) \tag{15}$$

where σ is the logistic sigmoid, W is the input-to-hidden weight matrix, and $\mathbf{w}$ is the hidden-to-output weight vector. The network was trained by self-play using TD(λ) with $\lambda = 0.7$. After each move from position s_t to s_{t+1}, the weights $\boldsymbol{\theta}$[30] were updated by

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\big[\hat{V}(s_{t+1}) - \hat{V}(s_t)\big]\, \mathbf{e}_t \tag{16}$$

where α[31] is the step size, and the eligibility trace $\mathbf{e}_t = \sum_{k=1}^{t} \lambda^{t-k} \nabla_{\boldsymbol{\theta}} \hat{V}(s_k)$ accumulates expo-

[26] In continuous action spaces, $\max_a Q(s,a)$ has no closed-form solution in general and must be solved numerically at every Bellman update. Discretizing a d-dimensional action space on a grid of m points per dimension costs $O(m^d)$ evaluations per update step.

[27] The Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ has density $(2\pi\sigma^2)^{-1/2} \exp(-(a-\mu)^2/2\sigma^2)$. Here $\mu_\theta(s)$ and $\sigma_\theta(s)$ are neural network outputs parameterizing the policy mean and standard deviation.

[28] The state $\mathbf{x}(s)$ was a 198-dimensional binary encoding of the raw board (four units per board point per player indicating checker counts, plus bar and borne-off counts). The output was a four-component vector estimating probabilities of each game outcome (White/Black $\times$ normal win/gammon), and the move maximizing expected outcome among all legal moves was selected at each step.

[29] A feedforward neural network stacks an input layer, one or more hidden layers, and an output layer; each unit in a hidden layer computes a weighted sum of its inputs and applies a nonlinear activation, here the logistic sigmoid $\sigma(x) = 1/(1+e^{-x})$, which maps any real number to $(0,1)$ and is identical to the binary logit link function.

[30] $\boldsymbol{\theta}$ denotes the full vector of network weights and biases, generalizing the scalar θ used for policy parameters in earlier sections.

[31] The *learning rate* α controls the step size of each parameter update. TD learning uses a semi-gradient step rather than true gradient descent, but α plays the same role: too large and updates overshoot; too small and convergence is slow.

nentially decayed gradients of past predictions.[32] At game's end, $\hat{V}(s_{t+1})$ is replaced by the outcome $z \in \{0, 1\}$.

A single neural network $\hat{V}$ serves as the evaluation function for both players. At each turn, the current player selects the legal move maximizing $\hat{V}(s')$ from its own perspective. As the network improves, it generates stronger play on both sides, producing harder training games that drive further improvement. The dice rolls ensure diverse board positions without requiring an explicit exploration mechanism.[33]

4.1.6 SARSA (1994)

Q-learning learns the optimal action-value function regardless of the policy generating experience. This off-policy property is useful but introduces complications when combined with function approximation. Rummery and Niranjan (1994) introduced SARSA as an *on-policy*[34] alternative that learns the value of the policy actually being followed. The name derives from the quintuple $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$ used in each update:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right] \tag{17}$$

The key difference from Q-learning is that SARSA bootstraps from the action a_{t+1} actually taken at the next state, rather than the greedy action $argmax_{a'} Q(s_{t+1}, a')$. This makes the algorithm on-policy, since the target depends on the behavior policy generating the data.

If the agent follows an ε-greedy policy, SARSA converges to $Q^{\varepsilon\text{-greedy}}$, not Q^*[35] policies and standard step-size conditions (Section 5.2). This distinction matters when exploration is costly. Consider the cliff-walking problem, where an agent must traverse a gridworld with a cliff along one edge. The optimal path runs along the cliff edge (shortest route), but the ε-greedy policy occasionally falls off. Q-learning learns the optimal path because it evaluates the greedy policy; the agent falls off during learning but the Q-values reflect the optimal route. SARSA learns a safer path further from the cliff because it evaluates the actual exploratory policy; it accounts for the fact that exploration sometimes leads to catastrophic states.

4.1.7 Baird (1995)

Baird (1995) constructed a six-state star MDP demonstrating divergence of Q-learning with linear function approximation. The MDP has five outer states that all transition to a single inner state under the target policy. The off-policy behavior samples states uniformly. With linear function approximation, the weights grow without bound under repeated Q-learning updates. The source of instability is the interaction of three components, namely bootstrapping (updating from estimated values rather than observed returns), off-policy learning (training on data from a different policy than the target), and function approximation (representing the value function with a parameterized model). Sutton and Barto (2018) later named this the deadly triad. Any two components can be combined safely; all three together permit divergence. The mechanism underlying this instability is analyzed in Section 5.3.

This result explains the asymmetry between Tesauro's success and Baird's failure. TD-Gammon used bootstrapping and function approximation but was on-policy, with training data

[32] In the neural network case, the eligibility trace $\mathbf{e}_t \in \mathbb{R}^{|\boldsymbol{\theta}|}$ is a vector accumulating exponentially-weighted gradients, extending the scalar state-based trace to parameter space.

[33] Version 2.1, trained on 1,500,000 games with 2-ply search, achieved near-parity with former world champion Bill Robertie and discovered novel positional strategies subsequently adopted by the human backgammon community.

[34] See Section 2 for the on-policy/off-policy distinction.

[35] SARSA converges to Q^* under GLIE (greedy in the limit with infinite exploration). A GLIE schedule explores all state-action pairs infinitely often but converges to the greedy policy asymptotically. The standard ε-greedy policy with $\varepsilon_t \to 0$ is one such schedule.

coming from the same self-play policy whose value was being estimated. Baird's counterexample used all three components and diverged. Baird also proposed a constructive solution, namely residual gradient algorithms that perform gradient descent on the mean-squared Bellman residual, guaranteeing convergence at the cost of a different fixed point.

4.1.8 Actor-Critic Methods (2000)

The idea of maintaining both a policy (*actor*) and a value function (*critic*) dates to Barto et al. (1983), who used a two-component system to solve the pole-balancing task. Actor-critic methods address the high variance of REINFORCE by replacing Monte Carlo returns with bootstrapped TD targets as the learning signal. The critic learns a value function $V(s)$ by TD updates.

$$V(s_t) \leftarrow V(s_t) + \alpha_c \, \delta_t, \quad \delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t) \tag{18}$$

The actor updates the policy using the TD error as a sample of the advantage.

$$\theta \leftarrow \theta + \alpha_a \nabla_\theta \log \pi_\theta(a_t|s_t) \, \delta_t \tag{19}$$

The TD error δ_t estimates $A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$, the advantage of action a_t over the average action.[36] Positive advantages indicate the action was better than expected; negative advantages indicate it was worse.

Konda and Tsitsiklis (2000) provided the first convergence proof for actor-critic algorithms with function approximation, showing convergence to a stationary point under two-timescale learning rates ($\alpha_c \gg \alpha_a$) and a compatibility condition on the critic architecture (Section 5.5).

4.1.9 Natural Policy Gradient (2001)

Standard gradient descent treats all parameter directions equally, but policy parameters define probability distributions whose natural geometry is not Euclidean. Kakade (2001) introduced the natural policy gradient, which measures progress in distribution space rather than parameter space. The update uses the Fisher information matrix $F(\theta)$:

$$F(\theta) = \mathbb{E}_{s \sim d^\pi, a \sim \pi_\theta} \left[\nabla_\theta \log \pi_\theta(a|s) \nabla_\theta \log \pi_\theta(a|s)^\top \right] \tag{20}$$

The natural gradient is

$$\tilde{\nabla}_\theta J(\theta) = F(\theta)^{-1} \nabla_\theta J(\theta) \tag{21}$$

This direction is invariant to reparameterization of the policy. In the tabular softmax[37] case, a single natural gradient step with unit step size recovers one step of exact policy iteration (Section 5.4).

The computational bottleneck is inverting $F(\theta) \in \mathbb{R}^{d \times d}$. Practical implementations use conjugate gradient methods to solve $F(\theta)x = \nabla_\theta J(\theta)$ without forming F explicitly. This approach was later scaled to deep neural networks by TRPO and PPO.

4.1.10 Fitted Value Iteration and Fitted Q-Iteration (2005)

Tabular Q-learning maintains a separate entry for every state-action pair. When $|\mathcal{S}|$ is large or the state space is continuous, as in most applied problems, this is infeasible. *Fitted Q-Iteration* (FQI) (Ernst et al., 2005) replaces the tabular update with a supervised regression step: given a batch of transitions, fit a function approximator to the Bellman targets.

[36] That δ_t is an unbiased estimate of the advantage follows from the policy gradient theorem (Sutton et al., 2000).

[37] The softmax function maps a vector $\mathbf{z}$ to a probability distribution: $\text{softmax}(z_i) = \exp(z_i) / \sum_j \exp(z_j)$. A softmax policy parameterizes action probabilities as $\pi_\theta(a|s) = \text{softmax}(\theta_{s,a})$.

Let $\mathcal{F}$ be a function class mapping $\mathcal{S} \times \mathcal{A} \to \mathbb{R}$. Initialize $Q_0 \equiv 0$. At each iteration $k = 0, 1, \ldots, K-1$: (i) draw N transitions (s_i, a_i, r_i, s_i') from a generative model; (ii) construct regression targets $y_i^{(k)} = r_i + \gamma \max_{a'} Q_k(s_i', a')$; (iii) set $Q_{k+1} \leftarrow \arg\min_{f \in \mathcal{F}} \frac{1}{N} \sum_{i=1}^{N} (f(s_i, a_i) - y_i^{(k)})^2$. The output is the greedy policy $\pi_K(s) = \arg\max_a Q_K(s, a)$.

The regression step replaces the exact Bellman application with a projection onto $\mathcal{F}$: $Q_{k+1} = \Pi_{\mathcal{F}} \mathcal{T} Q_k$, where $\mathcal{T}$ is the Bellman optimality operator and $\Pi_{\mathcal{F}}$ is the L^2-projection under the sample distribution. Fitted Value Iteration (FVI) (Munos and Szepesvári, 2008) applies the same idea to the value function directly: $V_{k+1} = \Pi_{\mathcal{F}} \mathcal{T}^* V_k$, where $(\mathcal{T}^* V)(s) = \max_a \{r(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s')\}$.

With feature matrix $\Phi \in \mathbb{R}^{|\mathcal{S}| \times d}$ (rows $\phi(s)^\top$) and per-action weight vectors $\theta_a \in \mathbb{R}^d$, each FQI regression step for action a reduces to the normal equations:

$$\theta_a^{(k+1)} = \left(\Phi^\top \Phi\right)^{-1} \Phi^\top y_a^{(k)}, \tag{22}$$

where $y_a^{(k)}(s) = r(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} \phi(s')^\top \theta_{a'}^{(k)}$. Computation is $O(d^2|\mathcal{S}| + d^3)$ per action per iteration. The FVI update takes the same form with a single weight vector θ_V:

$$\theta_V^{(k+1)} = \left(\Phi^\top \Phi\right)^{-1} \Phi^\top V_{\text{target}}^{(k)}, \tag{23}$$

where $V_{\text{target}}^{(k)}(s) = \max_a \{r(s, a) + \gamma \sum_{s'} P(s'|s, a) \phi(s')^\top \theta_V^{(k)}\}$. The finite-sample error theory for these methods is developed in Section 5.2.5.

4.2 The Deep Learning Era

4.2.1 Deep Q-Networks (2015)

Mnih et al. (2015) trained a single convolutional neural network[38] to play 49 Atari 2600 games directly from pixel inputs ($210 \times 160 \times 3$) and a scalar score, using no game-specific features.

The architecture processed four consecutive frames through three convolutional layers and a fully connected layer.[39] The network $Q(s, a; \theta)$ was trained to minimize the squared temporal difference error

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \tag{24}$$

Two innovations stabilized learning. *Experience replay* (Lin, 1992) stored transitions (s, a, r, s') in a buffer $\mathcal{D}$ and sampled uniformly for training, breaking the temporal correlation between consecutive updates. A *target network* used a frozen copy of parameters θ^-, updated periodically,[40] so the regression target does not shift with each gradient step.

DQN exceeded human-level performance on 29 of 49 games using a single architecture and hyperparameters.[41] Games requiring long-horizon planning or sparse rewards, such as Montezuma's Revenge, remained difficult.

[38] A convolutional neural network applies learned spatial filters to detect local patterns in grid-structured data, commonly used for image inputs.

[39] A fully connected layer computes $y = Wx + b$ where every input unit is connected to every output unit with learned weights W and biases b; it is the affine transformation familiar from linear regression, followed by a nonlinear activation.

[40] The buffer held 10^6 transitions; the target network θ^- was synchronized to θ every $C = 10{,}000$ steps.

[41] *Hyperparameters* are design choices fixed before training begins, such as network depth, learning rate, and replay buffer size; they are not estimated by gradient descent.

4.2.2 TRPO and PPO (2015, 2017)

Policy gradient methods suffer from a practical instability: a single large gradient step can move the policy into a region where performance collapses and recovery is slow.

Schulman et al. (2015) addressed this with Trust Region Policy Optimization (TRPO). TRPO solves the constrained optimization problem

$$\max_{\theta} \; L_{\theta_{\text{old}}}(\theta) \quad \text{subject to} \quad \bar{D}_{\text{KL}}(\theta_{\text{old}}, \theta) \leq \delta \tag{25}$$

where L is a surrogate objective based on the advantage function $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$.[42]

Schulman et al. (2017) proposed Proximal Policy Optimization (PPO) as a simpler alternative. PPO replaces the KL constraint with a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min\Big(r_t(\theta)\hat{A}_t,\; \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t \Big) \right] \tag{26}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio. The clipping removes the incentive for $r_t(\theta)$ to move outside the interval $[1-\varepsilon, 1+\varepsilon]$, penalizing large policy updates without explicitly computing a divergence measure.

PPO outperformed A2C, TRPO, and the cross-entropy method on continuous control benchmarks and achieved the highest average reward on 30 of 49 Atari games among the methods tested. PPO became the default policy optimization algorithm for large-scale RL applications, demonstrating that constraining the magnitude of policy updates is essential for stable optimization.

4.2.3 Soft Actor-Critic (2018)

Haarnoja et al. (2018) introduced Soft Actor-Critic (SAC), which adds entropy regularization to the actor-critic framework. The agent maximizes expected return plus an entropy bonus:

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t \left(r_t + \tau \mathcal{H}(\pi_\theta(\cdot|s_t)) \right) \right] \tag{27}$$

where $\mathcal{H}(\pi) = -\sum_a \pi(a) \log \pi(a)$ is the entropy and $\tau > 0$ is a temperature parameter. The entropy bonus encourages exploration by penalizing deterministic policies. The optimal policy under this objective is softmax in the Q-values: $\pi^*(a|s) \propto \exp(Q^*(s,a)/\tau)$, connecting to discrete choice models in econometrics.

SAC maintains two Q-networks (to reduce overestimation bias) and a policy network. The soft Bellman operator for the critic is:

$$(T^\pi Q)(s,a) = r(s,a) + \gamma \mathbb{E}_{s'} \left[V(s') \right], \quad V(s) = \mathbb{E}_{a\sim\pi}[Q(s,a) - \tau \log \pi(a|s)] \tag{28}$$

SAC is off-policy (using experience replay), handles continuous actions naturally, and achieves state-of-the-art sample efficiency on continuous control benchmarks. The entropy regularization provides automatic exploration without ε-greedy schedules.

[42] The Kullback-Leibler divergence $D_{\text{KL}}(p\|q) = \sum_x p(x) \log(p(x)/q(x))$ measures statistical distance between distributions. The bar denotes expectation over states: $\bar{D}_{\text{KL}} = \mathbb{E}_s[D_{\text{KL}}(\pi_{\theta_{\text{old}}}(\cdot|s)\|\pi_\theta(\cdot|s))]$.

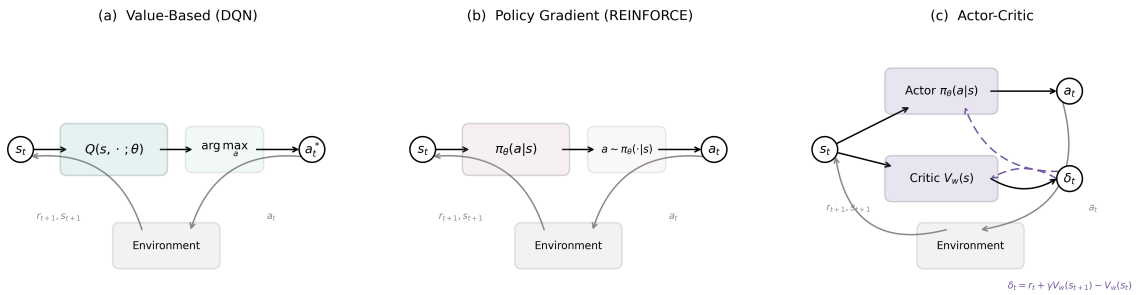

Figure 1: Architecture comparison of three core algorithm families: value-based, policy gradient, and actor-critic. (a) DQN maps states to Q-values for all actions, selecting the argmax. (b) REINFORCE maps states to a probability distribution over actions, then samples. (c) Actor-Critic maintains separate policy and value networks; the critic's TD error δ_t feeds back into both during training, yielding lower-variance policy updates than REINFORCE's Monte-Carlo return. Each panel includes the environment feedback loop in which the agent's action produces a reward and next state.

4.2.4 Control as Probabilistic Inference

The *control-as-inference* framework (Todorov, 2006; Kappen, 2011; Ziebart, 2010; Levine, 2018) recasts the preceding algorithms as instances of probabilistic inference (computation, not statistical inference) in a single graphical model.[43] The payoff is that every advance in approximate inference (variational methods, message passing, amortized inference) becomes a candidate RL algorithm, and the forward problem (finding the optimal policy given rewards) and the inverse problem (recovering rewards from observed behavior) become two queries in the same model. The construction introduces a binary "optimality" variable $\mathcal{O}_t \in \{0, 1\}$ at each time step, appended to the standard state-action-dynamics chain (Figure 2). The distribution over this variable is defined as

$$P(\mathcal{O}_t = 1 \mid s_t, a_t) = \exp(r(s_t, a_t)/\tau) \tag{29}$$

where $\tau > 0$ is a temperature parameter and rewards satisfy $r \leq 0$.[44] This is a modeling assumption, not a derivation from first principles; the exponential form is chosen so the resulting posterior matches entropy-regularized RL.

The RL problem becomes a probabilistic inference query. Given that all future time steps are optimal ($\mathcal{O}_{t:T} = 1$), what is $P(a_t \mid s_t, \mathcal{O}_{t:T} = 1)$? Define backward messages $\beta_t(s, a) = P(\mathcal{O}_{t:T} = 1 \mid s_t = s, a_t = a)$, the probability that everything from t onward is optimal given the agent is in state s taking action a. These are computed by backward recursion:

$$\beta_T(s, a) = \exp(r(s, a)/\tau) \tag{30}$$

$$\beta_t(s, a) = \exp(r(s, a)/\tau) \sum_{s'} P(s'|s, a)\, \beta_{t+1}(s') \quad (t < T) \tag{31}$$

where $\beta_{t+1}(s') \propto \sum_{a'} \beta_{t+1}(s', a')$ marginalizes over future actions under a uniform prior. By Bayes' rule, the posterior over actions is $P(a_t \mid s_t, \mathcal{O}_{t:T} = 1) = \beta_t(s_t, a_t) \,/\, \sum_{a'} \beta_t(s_t, a')$. Defining $Q(s, a) = \tau \log \beta_t(s, a)$, so that backward messages in log-space correspond to soft

[43] The framework is a mathematical language, much like random utility in econometrics, that reveals structural connections rather than deriving algorithms from first principles.

[44] The non-positive reward assumption ensures $\exp(r/\tau) \in (0, 1]$. Any bounded reward can be shifted to satisfy this without changing the optimal policy. Alternatively, the optimality variable can be replaced by an undirected potential $\Phi(s_t, a_t) = \exp(r(s_t, a_t)/\tau)$, yielding a conditional random field formulation that removes this restriction (Ziebart, 2010).

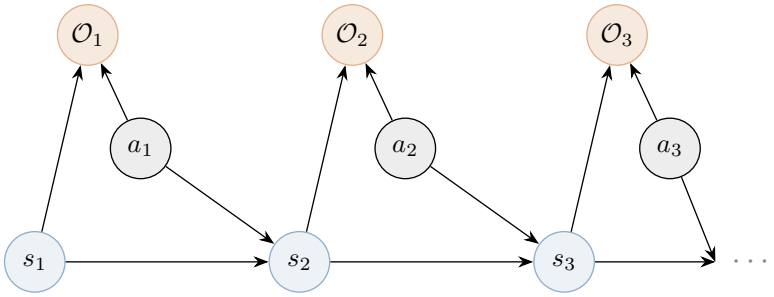

Figure 2: The control-as-inference graphical model. States s_t and actions a_t jointly determine the next state s_{t+1} (dynamics) and the optimality variable $\mathcal{O}_t$ (reward signal). Optimality nodes $\mathcal{O}_t$ are observed as $\mathcal{O}_t = 1$; the posterior over actions $P(a_t \mid s_t, \mathcal{O}_{t:T} = 1)$ yields the optimal policy.

value functions, and $V(s) = \tau \log \sum_a \exp(Q(s,a)/\tau)$[45], the posterior becomes $\pi(a \mid s) = \exp\big((Q(s,a) - V(s))/\tau\big)$, the softmax over Q-values.

Exact inference in this graphical model under stochastic dynamics produces risk-seeking policies. The backup becomes $Q(s,a) = r(s,a) + \tau \log \mathbb{E}_{s'}[\exp(V(s')/\tau)]$, which overweights unlikely favorable transitions because the agent implicitly assumes it can influence the dynamics.[46] The framework corrects this by applying structured variational inference, restricting the variational family to distributions that match the true dynamics $P(s'|s,a)$. This yields the soft Bellman equations

$$Q(s,a) = r(s,a) + \gamma\, \mathbb{E}_{s' \sim P}[V(s')] \tag{32}$$

$$V(s) = \tau \log \sum_{a \in \mathcal{A}} \exp\left(\frac{Q(s,a)}{\tau}\right) \tag{33}$$

with the optimal policy given by $\pi(a \mid s) = \exp\big((Q(s,a) - V(s))/\tau\big)$. The operator $\mathcal{T}^{\text{soft}}$ defined by Equations (32)–(33) is a γ-contraction in $\|\cdot\|_\infty$, with the same convergence rate as the standard Bellman operator.[47] As $\tau \to 0$, the log-sum-exp converges to the hard maximum and the soft Bellman equations reduce to the standard Bellman optimality equations. The value function in Equation (33) is identical to McFadden's social surplus function from discrete choice theory (McFadden, 1978), completing the connection noted in the preceding subsection.

Classical RL already employs exploration strategies (ε-greedy, UCB), but these are designed separately from the optimization objective. In the probabilistic framework, the objective itself maximizes expected reward and policy entropy simultaneously.[48] The Q-function, the value function, and the exploration behavior are all derived from a single objective. Different approximation strategies applied to this single model recover familiar algorithms. When the backward messages in Equations (30)–(31) are computed exactly in the tabular setting, the result is soft value iteration. Estimating the ELBO gradient via the likelihood ratio trick yields max-ent policy gradients, identical to REINFORCE with $-\tau \log \pi(a_t|s_t)$ added to the reward at each step.

[45] Since $\beta_t(s) \propto \sum_a \beta_t(s,a) = \sum_a \exp(Q(s,a)/\tau)$, we have $V(s) = \tau \log \sum_a \exp(Q(s,a)/\tau)$.

[46] Under exact inference, the posterior dynamics $P(s'|s,a,\mathcal{O}_{1:T} = 1)$ differ from the true dynamics $P(s'|s,a)$, biasing the inferred transitions toward favorable outcomes. The log-expectation-of-exponentials exceeds the expectation by Jensen's inequality. This optimism is an artifact of exact inference in the model, not a feature.

[47] The proof follows from the log-sum-exp being 1-Lipschitz in the sup norm; see Haarnoja et al. (2017), Theorem 1.

[48] The max-ent objective is $\max_\pi \sum_t \mathbb{E}[r(s_t,a_t) + \tau \mathcal{H}(\pi(\cdot|s_t))]$. This equals the evidence lower bound (ELBO) of the graphical model, a quantity from variational inference that lower-bounds the log-likelihood of the observed optimality evidence $\log P(\mathcal{O}_{1:T} = 1)$. Maximizing the ELBO with respect to the policy is equivalent to finding the best approximation to the true posterior, measured by KL divergence; see Blei et al. (2017) for a review of variational inference.

Fitting parameterized Q_ϕ and V_ψ networks to approximate the backward messages gives Soft Actor-Critic (Haarnoja et al., 2018). Fitting Q_ϕ alone and extracting the policy implicitly via the softmax gives soft Q-learning (Haarnoja et al., 2017). Trust-region methods such as TRPO and PPO do not optimize the max-ent objective, but each policy update step solves a max-ent subproblem with the old policy as prior, so the per-step structure mirrors the framework even though the global target remains standard reward maximization (Schulman et al., 2015, 2017). Hard-max algorithms (Q-learning, DQN, standard policy gradient) are not direct instances of the framework; they are recovered in the zero-temperature limit $\tau \to 0$, where the softmax collapses to the argmax. Reading the graphical model in the reverse direction, treating the reward as the unknown and observed behavior as evidence of optimality, yields maximum entropy inverse reinforcement learning (Ziebart et al., 2008), connecting to the structural estimation methods discussed in Section 8.

The framework has practical limitations. The converged fixed point is Q^*_{soft}, not Q^*; at any $\tau > 0$, the policy is suboptimal for the original reward-maximization objective. The entropy bonus produces undirected exploration, spreading probability mass rather than targeting uncertain states. In safety-critical environments, the objective assigns nonzero probability to catastrophic actions. The practical benefits of the probabilistic perspective, including robustness to model perturbations and smooth optimization landscapes, are most apparent in continuous-control and robotics settings. In perfectly simulated environments with exact rewards (Atari, Go), the hard-max algorithms that dominate those domains do not require this probabilistic formulation, as the following subsection illustrates.

4.2.5 AlphaGo Zero (2017)

The game of Go has approximately 10^{170} legal positions and a branching factor of roughly 250, far beyond the reach of brute-force search. Hand-crafted evaluation functions, which had succeeded in chess, failed here because positional concepts like influence, territory, and group viability are holistic and contextual. Monte Carlo tree search (MCTS) had achieved amateur-level play by using random simulations to estimate position values, but progress had stalled below professional strength. Silver et al. (2016) broke through by combining supervised learning from 30 million human expert positions, reinforcement learning via self-play, and MCTS with learned value and policy networks; the resulting system defeated Lee Sedol four games to one in March 2016. A year later, Silver et al. (2017) showed that none of the human data was necessary.

AlphaGo Zero uses a single convolutional neural network $f_\theta(s) = (\mathbf{p}, v)$ that takes a board position s and outputs both a policy vector $\mathbf{p}$ over legal moves and a scalar value v estimating the probability of winning. The input representation consists of 17 binary planes on the 19×19 board encoding the raw game state without hand-crafted features.[49] The architecture uses residual blocks,[50] which allow training of very deep networks.

During play, each move is selected by running MCTS, which conducts 1,600 simulated games from the current position to estimate move quality. Each simulation proceeds in four phases, illustrated in Figure 3. In the selection phase, the algorithm starts from the current position and traverses the partially built search tree by choosing at each node the action that maximizes $Q(s,a) + c_{\text{puct}} \cdot P(s,a) \cdot \sqrt{\sum_b N(s,b)} \,/\, (1 + N(s,a))$, where $Q(s,a)$ is the current average value of action a, $P(s,a)$ is the prior probability from the neural network, and $N(s,a)$ is the visit count.[51] In the expansion and evaluation phase, when the traversal reaches a position not yet

[49] Eight planes for Black's stone positions over the last eight moves, eight for White's, and one indicating which color plays next. The history planes allow the network to detect ko situations and infer the trajectory of play.

[50] A residual block computes $\mathbf{x} + g(\mathbf{x})$ rather than just $g(\mathbf{x})$, where g is a learned transformation. The skip connection allows gradients to flow through very deep networks without vanishing. AlphaGo Zero's 40-block architecture has 79 parameterized layers.

[51] The constant c_{puct} controls exploration. Actions visited often have well-estimated Q values but a shrinking

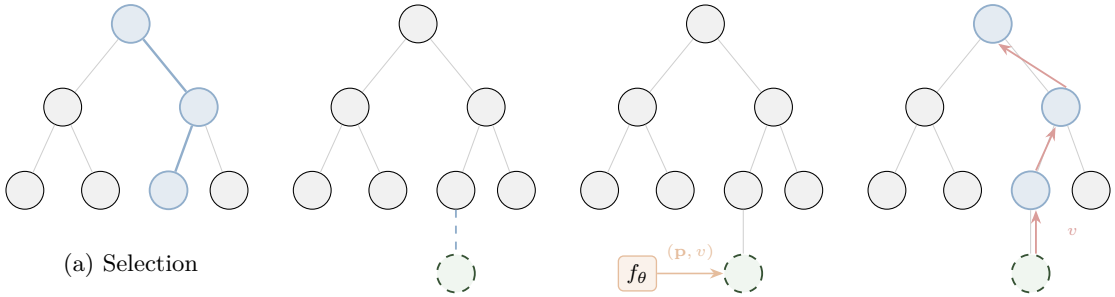

Figure 3: The four phases of a single MCTS simulation in AlphaGo Zero. (a) Selection traverses the tree from the root, choosing at each node the action maximizing a UCB-like score balancing exploitation (Q) and exploration (P/N). (b) Expansion adds a new leaf node when the traversal reaches an unexplored position. (c) The neural network f_θ evaluates the new position, producing move priors $\mathbf{p}$ and a value estimate v. (d) Backup propagates v along the traversed path, updating mean values $Q(s, a)$ and visit counts $N(s, a)$ at each edge.

in the tree, the neural network evaluates it in a single forward pass, producing a policy vector $\mathbf{p}$ and a value estimate v; the policy initializes prior probabilities $P(s', a) = p_a$ for each child edge. In the backup phase, the value v propagates back up the traversed path, incrementing each edge's visit count $N(s, a)$ and updating its mean value $Q(s, a)$. After all 1,600 simulations, the algorithm selects the move with the highest visit count at the root.

The training loop generates self-play games. At each board position s_t during a game, the program runs MCTS to produce improved move probabilities $\boldsymbol{\pi}_t$, where $\pi_t(a) \propto N(s_t, a)^{1/\tau}$ and τ is a temperature parameter controlling exploration.[52] A move a_t is sampled from $\boldsymbol{\pi}_t$ and played. At game end, the outcome $z \in \{-1, +1\}$ is recorded. Each position becomes a training triple $(s_t, \boldsymbol{\pi}_t, z)$, and the network parameters are updated to minimize

$$\ell(\theta) = (z - v)^2 - \boldsymbol{\pi}^\top \log \mathbf{p} + c\|\theta\|^2 \tag{34}$$

where the first term is a value prediction loss, the second is a policy cross-entropy loss,[53] and the third is L_2 regularization.[54] The key mechanism is a virtuous cycle. MCTS serves as a policy improvement operator, since the search probabilities $\boldsymbol{\pi}$ are stronger than the raw network outputs $\mathbf{p}$. Training the network to match $\boldsymbol{\pi}$ distills the search improvements back into the network, and the improved network in turn produces better MCTS. After 72 hours of self-play on 4 TPUs, AlphaGo Zero surpassed all previous versions, including the one that defeated Lee Sedol, and discovered novel strategies not previously seen in human play.[55]

Go was well-suited to this architecture. Its fixed 19×19 board maps naturally to convolutional networks, its perfect information and deterministic transitions make MCTS's tree structure exact, and the binary game outcome provides an unambiguous training signal. Igami

exploration bonus; rarely visited actions have uncertain values but a large bonus. This is a continuous analogue of the upper confidence bound (UCB) strategy from bandit theory.

[52] Early in the game ($t \leq 30$), $\tau = 1$ so moves are sampled proportionally to visit counts, encouraging diverse openings. Later, $\tau \to 0$ and the most-visited move is selected deterministically. Dirichlet noise is also added to root priors, $P(s, a) = (1 - \varepsilon)p_a + \varepsilon\eta_a$ with $\eta \sim \text{Dir}(0.03)$, ensuring all legal moves can be explored despite strong network priors.

[53] The cross-entropy loss $-\boldsymbol{\pi}^\top \log \mathbf{p}$ measures how well the predicted distribution $\mathbf{p}$ matches the target $\boldsymbol{\pi}$; it equals zero when the distributions are identical.

[54] L_2 regularization penalizes the squared magnitude of parameters, $c\|\theta\|^2$, analogous to ridge regression in econometrics.

[55] The system that defeated Lee Sedol in March 2016 used fixed network weights throughout the match; no parameter updates occurred between or during games. This illustrates the training-execution distinction (Section 2): the months of self-play constituted the training phase, while the five-game match was purely execution.

(2020) interprets the architecture in econometric terms, where the policy network is a conditional choice probability (CCP) estimator, the value network is a conditional value function (CVF) estimator, and the system performs CCP estimation and forward simulation jointly, connecting to the approach of Hotz and Miller (1993) in dynamic discrete choice.

4.2.6 Decision Transformers (2021)

Chen et al. (2021) proposed replacing Bellman backups with autoregressive sequence modeling. The Decision Transformer conditions a causal GPT-style Transformer on trajectories $\tau = (\hat{R}_1, s_1, a_1, \hat{R}_2, s_2, a_2, \ldots)$, where $\hat{R}_t$ is the *return-to-go* (desired future cumulative reward). At test time, conditioning on a high target return extracts a high-performing policy without any temporal-difference learning. Janner et al. (2021) extended this to the Trajectory Transformer, modeling entire trajectories as flat token sequences with continuous dimensions discretized into bins, enabling planning via beam search over trajectories.

The approach has fundamental limitations that Bellman-based methods do not share. Brandfonbrener et al. (2022) proved that return-conditioned supervised learning (RCSL) recovers optimal policies only under assumptions strictly stronger than those needed for dynamic programming: near-deterministic dynamics, expert data coverage, and a unique mapping from returns to optimal actions. In stochastic environments, high returns may reflect environmental luck rather than good decisions, causing RCSL to imitate lucky-but-suboptimal trajectories. Paster et al. (2022) demonstrated this concretely: in a simple gambling MDP, the Decision Transformer conditioned on high returns selects risky gambles over the optimal safe action, even with infinite data.[56]

A second limitation is that RCSL cannot *stitch* suboptimal trajectory segments. If the dataset contains two trajectories that each visit a useful intermediate state but from different starting points, Bellman-based methods can compose the better segments by propagating values backward through the shared state. Sequence models, which predict forward autoregressively, cannot perform this backward composition.

5 The Theory of Reinforcement Learning

5.1 The Geometry of Dynamic Programming

Value iteration (VI) and policy iteration (PI) are the workhorses of dynamic programming. VI applies the Bellman operator repeatedly until convergence; PI alternates between *policy evaluation* (solving a linear system) and *policy improvement* (taking the greedy action). PI converges faster. Why? The answer reveals a connection between dynamic programming and numerical optimization. Policy iteration is Newton's method applied to the Bellman equation.

5.1.1 Value Iteration as Picard Iteration

Consider the Bellman optimality operator T acting on value functions.

$$(TV)(s) = \max_{a \in \mathcal{A}} \left\{ r(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s,a) V(s') \right\}. \tag{35}$$

This operator is nonlinear due to the max. Value iteration applies T repeatedly: $V_{k+1} = TV_k$. Since T is a γ-contraction in the supremum norm (Denardo, 1967), Banach's fixed-point theorem

[56] Emmons et al. (2022) showed that a simple two-layer MLP matches the Transformer architecture on D4RL benchmarks, suggesting the autoregressive structure provides conditional density estimation rather than temporal reasoning. The essential element is conditioning on the right outcome variable, not the architecture.

guarantees $\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty$.[57] This is Picard iteration, with linear convergence at rate γ.[58] The iteration count to reduce error by a factor of δ is $k = \log(\delta)/\log(1/\gamma)$ (Bertsekas, 1996).

5.1.2 Policy Iteration as Newton's Method

Policy iteration takes a different approach. At the current value estimate $\tilde{V}$, define the greedy policy $\tilde{\pi}(s) = argmax_a\{r(s,a) + \gamma \sum_{s'} P(s'|s,a)\tilde{V}(s')\}$. The *policy evaluation* step solves the linear fixed-point equation $V = T^{\tilde{\pi}}V$ exactly, where $T^{\tilde{\pi}}$ is the policy-specific Bellman operator

$$(T^{\tilde{\pi}}V)(s) = r(s, \tilde{\pi}(s)) + \gamma \sum_{s'} P(s'|s, \tilde{\pi}(s))V(s'). \tag{36}$$

The geometric structure, formalized by Puterman and Brumelle (1979), is as follows: the linear operator $T^{\tilde{\pi}}$ is a supporting hyperplane to the nonlinear operator T at the current iterate.[59] Specifically, the operators satisfy tangency: $T^{\tilde{\pi}}\tilde{V} = T\tilde{V}$, so the linearization agrees with the nonlinear operator at the current iterate. They also satisfy support: $T^{\tilde{\pi}}V \leq TV$ for all V, meaning the linear operator lies weakly below the nonlinear one everywhere, just as a tangent line lies below a convex function. Policy evaluation solves for the fixed point of this linearization exactly. This is precisely the structure of Newton's method; linearize the nonlinear equation at the current point, solve the linearized system, and iterate.[60][61]

Theorem 1 (Policy Improvement, Howard (1960))**.** *Let π_k be the current policy with value V^{π_k}, and let π_{k+1} be the greedy policy with respect to V^{π_k}:*

$$\pi_{k+1}(s) = \underset{a \in \mathcal{A}}{argmax} \left\{ r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^{\pi_k}(s') \right\}. \tag{37}$$

Then $V^{\pi_{k+1}}(s) \geq V^{\pi_k}(s)$ for all $s \in \mathcal{S}$, with strict inequality at some state unless π_k is already optimal.

The consequence is finite termination. Since there are at most $|\mathcal{A}|^{|\mathcal{S}|}$ deterministic policies and each PI step strictly improves the value function (Theorem 1), PI reaches the exact optimum in finitely many iterations. While VI requires $k = \log(100)/\log(1/\gamma)$ iterations to reduce error by a factor of 100 (Bertsekas, 1996), PI typically converges in 5–10 iterations regardless of γ.[62][63]

[57] This is the Contraction Mapping Theorem. The identical mathematical structure governs convergence of value function iteration in consumption-savings models, competitive equilibrium computation, and Bellman equation solution.

[58] Picard iteration is $x_{k+1} = f(x_k)$ for finding roots of $x = f(x)$. When f is a contraction, convergence is geometric. The rate γ means each iteration reduces the error by a fixed proportion; more patient agents (higher γ) face slower convergence because the operator contracts less per step.

[59] A supporting hyperplane to a convex function at x_0 is a linear function ℓ with $\ell(x_0) = f(x_0)$ and $\ell(x) \leq f(x)$ everywhere. In plainer terms: $T^{\tilde{\pi}}$ is the tangent-line approximation to T from elementary calculus, extended to function spaces. The policy operator $T^{\tilde{\pi}}$ plays this role for the Bellman operator.

[60] The Newton interpretation of policy iteration has precursors in Kleinman (1968) for Riccati equations in linear-quadratic control and Pollatschek and Avi-Itzhak (1969) for stochastic games.

[61] Algebraically: consider finding the root of $G(V) = V - TV = 0$. The Bellman operator T is piecewise affine, not smooth: T is affine on each region where the greedy policy is constant, with kinks at boundaries where the optimal action switches. This makes G a semismooth function in the sense of Qi and Sun (1993). At any iterate V_k where the greedy policy $\tilde{\pi}$ is unique (a generic condition), T is locally affine: $TV = r^{\tilde{\pi}} + \gamma P^{\tilde{\pi}}V$, so $G'(V_k) = I - \gamma P^{\tilde{\pi}}$. The Newton step $V_{k+1} = V_k - [G'(V_k)]^{-1}G(V_k) = (I - \gamma P^{\tilde{\pi}})^{-1}r^{\tilde{\pi}}$ is exactly the policy evaluation solution. At the non-smooth boundary points where two actions tie, any element of the B-subdifferential yields the same iterate because the two candidate linearizations produce the same fixed point.

[62] Ye (2011) proves PI is strongly polynomial with iteration count $O\big(\frac{|\mathcal{S}||\mathcal{A}|}{1-\gamma} \log \frac{|\mathcal{S}|}{1-\gamma}\big)$, resolving a long-standing conjecture. This bound is for fixed γ; Fearnley (2010) constructs examples requiring exponentially many iterations when γ is allowed to vary with $|\mathcal{S}|$.

[63] For continuous-state problems discretized on a grid (the norm in economics), the finite-termination argument

At $\gamma = 0.90$, VI needs 44 iterations while PI needs only 5–8; at $\gamma = 0.95$, VI requires 90 versus 5–8; at $\gamma = 0.99$, VI needs 459 iterations while PI still converges in 5–10.

Bertsekas (2022b) extends this interpretation to a broad class of dynamic programming problems. The Newton structure applies whenever the Bellman operator can be written as a pointwise maximum over linear operators: $T = \max_\pi T^\pi$. This includes not only infinite horizon problems with discounting but also optimal stopping problems (job search, option exercise), average cost optimization (inventory, queueing), and minimax formulations for adversarial settings.[64][65] The practical implication is that algorithms with policy-improvement structure (evaluate a policy exactly or approximately, then improve) inherit Newton-like convergence behavior, while pure value-iteration methods (apply T directly) are limited to linear convergence.[66]

5.1.3 Simulation Study: The Brock–Mirman Economy

The Brock and Mirman (1972) optimal growth model provides a concrete demonstration. A planner chooses capital k' to maximize $\sum_{t=0}^{\infty} \beta^t \log(c_t)$ subject to the resource constraint $c_t + k_{t+1} = z_t k_t^\alpha$, where productivity $z_t \in \{0.9, 1.1\}$ follows a Markov chain with persistence 0.8. I set $\alpha = 0.36$, $\beta = 0.96$, and discretize capital on a 500-point grid covering two productivity states (1,000 states). This model admits the closed-form policy $k'(k, z) = \alpha\beta z k^\alpha$, providing an exact benchmark.

The discretized model makes the PI–Newton equivalence concrete. Define the Bellman residual $G(V) = V - TV$ on $\mathbb{R}^n$ where $n = 1{,}000$ (500 capital grid points $\times$ 2 productivity states). At iterate V_k, let π_k denote the greedy policy. Since π_k is unique at V_k (generically), T is locally affine: $TV = r^{\pi_k} + \gamma P^{\pi_k} V$, so the residual becomes $G(V) = (I - \gamma P^{\pi_k})V - r^{\pi_k}$ with Jacobian $G'(V_k) = I - \gamma P^{\pi_k}$. The Newton update is

$$V_{k+1} = V_k - [G'(V_k)]^{-1} G(V_k) = (I - \gamma P^{\pi_k})^{-1} r^{\pi_k}, \tag{38}$$

which is exactly the policy evaluation step: solving $V = r^{\pi_k} + \gamma P^{\pi_k} V$ for V. Each PI iteration is one Newton step on the Bellman residual, explaining the 11-iteration convergence on a 1,000-state problem.[67]

The VI iteration count follows from the contraction bound: each iteration reduces the Bellman residual by the factor $\beta = 0.96$, requiring $k = \lceil \log(\epsilon / \|TV_0 - V_0\|_\infty) / \log \beta \rceil = 567$

still applies to the discretized problem, but the number of grid points n enters the bound. Santos and Rust (2004) establish a three-tier convergence result for PI applied to discretized dynamic programs: order ≈ 1.5 globally for general interpolation schemes, quadratic convergence locally when the value function approximation is concave and piecewise linear, and superlinear convergence for general smooth interpolation. The formal error constants $C(h)$ in their quadratic bound degrade as the grid mesh $h \to 0$, but the iteration count is empirically independent of grid size.

[64] Rust (1996) surveys successive approximation and policy iteration methods for economic models, comparing their performance on the bus engine replacement problem. Zhang (2023) extends randomized policy iteration to multi-agent problems where the control is m-dimensional, reducing per-iteration complexity from exponential to linear in m.

[65] These problems appear under different names such as "stochastic shortest path" for optimal stopping, "average-cost MDP" for long-run average optimization, and "model predictive control" (or receding-horizon control) for finite-horizon replanning.

[66] Blackwell (1965) proves that for discounted MDPs with finite state and action spaces, a stationary deterministic policy $\pi : \mathcal{S} \to \mathcal{A}$ exists that is optimal for all initial states simultaneously. This "uniform optimality" has three implications: (1) the search space reduces from history-dependent or stochastic policies to static maps, justifying neural networks that condition only on current state; (2) the optimal policy is independent of the initial distribution d_0, so changing where episodes start does not require retraining; (3) PI and VI are guaranteed to converge to the same globally optimal policy regardless of initialization.

[67] Santos and Rust (2004) is the definitive reference on PI convergence for discretized economic models of precisely this type. Their analysis of the Brock–Mirman growth model establishes that PI iteration counts are empirically independent of grid resolution (7–11 iterations across all grid sizes in Table 3), consistent with the Newton interpretation: Newton's method converges in a number of steps determined by the nonlinearity of the operator, not the dimension of the discretization.

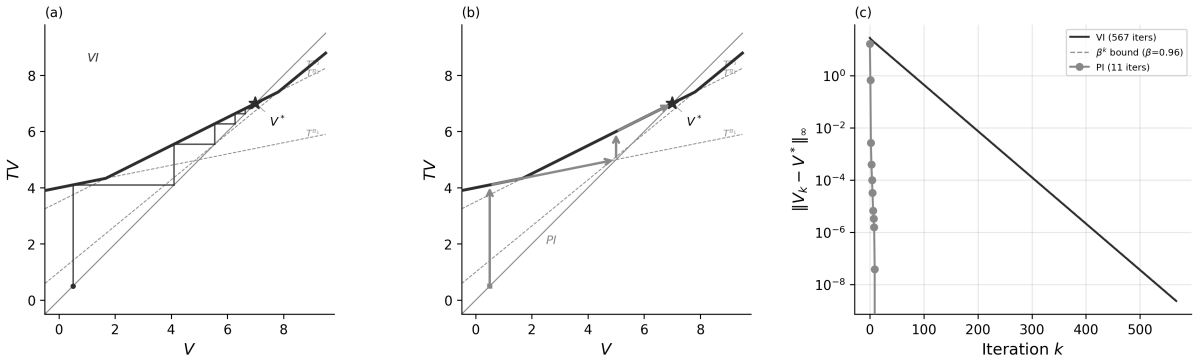

Figure 4: The Brock–Mirman economy ($\alpha = 0.36$, $\beta = 0.96$, 1,000 states). (a) Value iteration on a scalar Bellman equation. The staircase iterates $V_{k+1} = TV_k$, converging at the linear rate γ. (b) Policy iteration as Newton's method. Each step solves for the fixed point of the active policy operator T^{π_k}, jumping to the tangent line's intersection with the diagonal. (c) Sup-norm error $\|V_k - V^*\|_\infty$ for the discretized model; VI requires 567 iterations, PI converges in 11.

Table 3: **Brock–Mirman economy: VI vs PI vs LP.**

Regime	Method	Iterations	Time (s)	$\|V - V^*\|_\infty$
1. Contraction	VI	567	32.66	9.7e-11
	PI	11	0.62	0.0e+00
2. LP dual	VI	—	0.006	—
	LP	—	0.023	2.3e-09
3. Rate (n_k=10)	VI	567	0.003	—
	PI	7	0.000	—
3. Rate (n_k=200)	VI	567	3.097	—
	PI	10	0.067	—

iterations for tolerance $\epsilon = 10^{-10}$.[68]

Figure 4(a)–(b) provide a geometric interpretation for a scalar Bellman equation with three policies. Each policy operator $T^{\pi_i}V = r^{\pi_i} + \gamma_i V$ is affine; the Bellman operator $T = \max_i T^{\pi_i}$ is their upper envelope, a convex piecewise-linear function. Panel (a) shows VI: the staircase iterates $V_{k+1} = TV_k$ by alternating between the T curve and the 45° line, converging at the linear rate γ. Panel (b) shows PI: at each iterate, the algorithm identifies the active policy operator and solves for its fixed point on the 45° line, jumping directly to the intersection. This is a Newton step, where each T^{π_k} is a supporting hyperplane to T at V_k, and the fixed point of the linearization is the Newton iterate. The scalar picture extends to $\mathbb{R}^n$: the affine operator $T^{\pi_k}V = r^{\pi_k} + \gamma P^{\pi_k}V$ supports T at V_k, and its fixed point $(I - \gamma P^{\pi_k})^{-1}r^{\pi_k}$ is the Newton iterate from equation (38). Finite termination follows because T has finitely many affine pieces; the iteration count depends on the number of policy switches, not the state-space dimension.

Table 3 and Figure 4 confirm the theory. VI requires 567 iterations at rate $\beta^n = 0.96^n$; PI converges in 11, a 50× reduction predicted by the Newton interpretation. The Manne (1960) LP recovers the same value function to solver precision ($\|V_{\text{LP}} - V_{\text{VI}}\|_\infty < 10^{-8}$).[69]

[68] At $\beta = 0.99$, the weaker contraction yields approximately four times as many iterations, since $\log(0.96)/\log(0.99) \approx 4$.

[69] PI wall-clock time scales favorably: at $n_k = 200$, PI is roughly 50× faster than VI (Table 3), because the $O(n^3)$ per-iteration cost of policy evaluation is offset by 7–10 total iterations versus 567.

5.2 Value Learning Methods

5.2.1 Stochastic Approximation Foundations

When P is unknown, a single sampled transition (s, a, r, s') can replace the expectation $\mathbb{E}_{s' \sim P(\cdot|s,a)}[V(s')]$. The mathematical foundation is stochastic approximation, developed by Robbins (1952).[70] Consider the problem of finding x^* such that $g(x^*) = 0$, where g cannot be evaluated directly but one can observe noisy samples $g(x) + \epsilon$. The Robbins-Monro iteration is:

$$x_{t+1} = x_t - \alpha_t[g(x_t) + \epsilon_t], \tag{39}$$

where ϵ_t is zero-mean noise. Under two conditions on the step sizes, this converges to x^* with probability one. The conditions are $\sum_{t=0}^{\infty} \alpha_t = \infty$ (sufficient exploration, ensuring that learning never ceases) and $\sum_{t=0}^{\infty} \alpha_t^2 < \infty$ (diminishing noise, ensuring the variance of cumulative updates is finite). The canonical choice $\alpha_t = 1/(t+1)$ satisfies both conditions.

5.2.2 Q-Learning and SARSA

Q-learning (Watkins and Dayan, 1992a) is Robbins-Monro applied to the Bellman equation for action-value functions.[71] Define the Q-factor Bellman operator:

$$(FQ)(s, a) = r(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q(s', a'). \tag{40}$$

The Q-learning update, upon observing transition (s_t, a_t, r_t, s_{t+1}), is

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]. \tag{41}$$

The logic of Q-learning is best understood as a Monte Carlo approximation of the Bellman contraction. The true Bellman operator involves an integral over the transition distribution, $(FQ)(s, a) = r + \gamma \int \max_{a'} Q(s', a')\, dP(s'|s, a)$. Since P is unknown, this integral cannot be computed analytically. However, a sample transition (s, a, r, s') acts as a single-point Monte Carlo estimate of this integral. The Q-learning update is simply an exponential moving average (with weight α_t) between the current estimate and this noisy Monte Carlo target. Because F is a γ-contraction in the supremum norm, the expected update drives the estimate toward the fixed point Q^*, provided the noise in the Monte Carlo sample averages out over time (which the Robbins-Monro conditions ensure) (Tsitsiklis, 1994).[72]

Convergence requires two conditions. Exploration (visiting all state-action pairs infinitely often) ensures identification. The Robbins-Monro step-size conditions ($\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$) balance tracking versus noise suppression. Watkins and Dayan (1992a) and Jaakkola et al. (1994) formalize these; Tsitsiklis (1994) provides the general framework.

[70] The modern theory of stochastic approximation, including convergence rates and the ODE (ordinary differential equation) method for analyzing iterates, is developed in Kushner and Clark (1978) and Borkar and Meyn (2000). The ODE method shows that the expected trajectory of the stochastic iterates tracks the solution of a deterministic differential equation $\dot{x} = -g(x)$, providing stability conditions via Lyapunov theory.

[71] The "Q" in Q-learning stands for "quality," following Watkins and Dayan (1992a), who used $Q(s, a)$ to denote the quality (expected return) of taking action a in state s. The term "Q-factor" is used interchangeably with "action-value function" throughout the RL literature.

[72] Q-learning can be viewed as root-finding for the expected Bellman residual: the goal is to find parameters such that $\mathbb{E}_{(s,a,r,s')}[\delta_t] = 0$, where $\delta_t = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$ is the temporal difference error. The update is a stochastic gradient step on the mean-squared Bellman error, but with a critical distinction: the gradient is "semi-gradient" because the target $\max_{a'} Q(s', a')$ is treated as a fixed constant rather than a function of the parameters being updated. This simplifies computation but disconnects the update from true gradient descent, requiring the specific stability conditions of Tsitsiklis (1994).

The choice of step-size schedule has quantitative consequences: Even-Dar and Mansour (2003) show that polynomial schedules $\alpha_t = 1/t^\omega$ with $\omega \in (1/2, 1)$ achieve convergence rate $O(1/t^{1-\omega})$, creating an explicit tradeoff between speed and stability. Recent work by Li et al. (2024a) establishes that Q-learning with variance-reduced updates achieves minimax-optimal sample complexity $\tilde{O}(|\mathcal{S}||\mathcal{A}|/(1-\gamma)^3\epsilon^2)$, matching information-theoretic lower bounds.[73] Model-free learning is possible. The optimal value function can be found without ever estimating the transition probabilities.[74]

SARSA provides an on-policy variant.[75] Instead of taking the maximum over next actions, SARSA uses the action actually taken:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t \left[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]. \tag{42}$$

This solves for the value function of the behavior policy π rather than the optimal policy. Singh et al. (2000) prove convergence under the same step-size conditions, provided the behavior policy converges to a stationary distribution. Q-learning is noisy value iteration on Q-factors; SARSA is noisy *policy evaluation*.[76] The Robbins-Monro conditions ensure that the noise averages out faster than the signal decays, allowing asymptotic convergence despite using only single-sample estimates.

5.2.3 Multi-Step Returns and TD(λ)

The updates in (41) bootstrap from a single successor state. More generally, one can bootstrap from n steps ahead. The n-step return is

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n V(S_{t+n}). \tag{43}$$

Setting $n = 1$ recovers the TD(0) target; letting $n \to \infty$ (or reaching a terminal state) gives the Monte Carlo return.

The λ-*return* (Sutton, 1988) averages all n-step returns with geometrically decaying weights:

$$G_t^\lambda = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}, \qquad \lambda \in [0, 1]. \tag{44}$$

This is the *forward view*: at time t, look forward at all possible truncation horizons and take a weighted average. The parameter λ controls a bias-variance tradeoff: lower λ gives higher bias (more bootstrapping) but lower variance; higher λ gives lower bias but higher variance, since more of the update relies on stochastic returns rather than value estimates.

[73] Vanilla Q-learning (without variance reduction) has tight complexity $\tilde{\Theta}(|\mathcal{S}||\mathcal{A}|/(1-\gamma)^4\epsilon^2)$, worse by a factor of $1/(1-\gamma)$ due to maximization bias inflating variance. The optimal cubic rate requires variance-reduced updates (Wainwright, 2019; Sidford et al., 2018). By contrast, naive model-based RL (estimate $\hat{P}$ from samples, then solve by planning) achieves the optimal $(1-\gamma)^{-3}$ rate with no special tricks (Agarwal et al., 2020b), illustrating the statistical cost of discarding transition structure.

[74] Model-free methods are essential when (a) the environment is a physical system with dynamics too complex to write down, or (b) the agent learns directly from interaction. Note that "model-free" does not mean "atheoretical." The agent does not store $P(s'|s, a)$ explicitly, but the Q-function serves as an implicit model encoding long-run consequences.

[75] SARSA is named for the quintuple $(S_t, A_t, R_t, S_{t+1}, A_{t+1})$ used in each update (Rummery and Niranjan, 1994). Convergence requires the GLIE (Greedy in the Limit with Infinite Exploration) condition: the behavior policy must explore all actions infinitely often while converging to a greedy policy. ε-greedy with $\varepsilon_t \to 0$ satisfies this.

[76] Bhandari et al. (2021) provide finite-time analysis of TD learning, showing that the convergence rate depends on the mixing time of the Markov chain under the behavior policy. Faster mixing (less serial correlation in the state sequence) yields faster convergence.

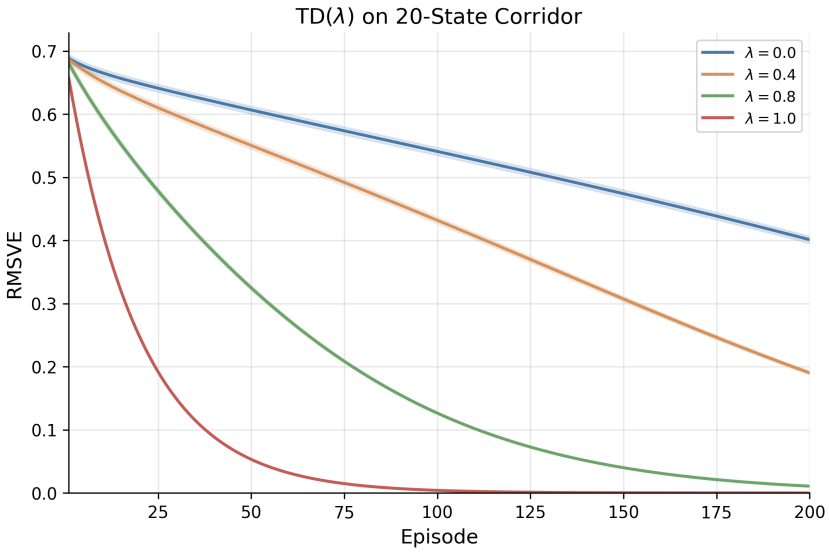

Figure 5: RMSVE vs. episodes for TD(λ) on the 20-state corridor. Shaded regions show ± 1 SE over 20 seeds.

The forward view requires waiting until the end of the episode to compute G_t^λ. The *backward view* computes the same total update incrementally. At each step, compute one TD error $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ and distribute it to all states via an *eligibility trace*:

$$e_t(s) = \gamma\lambda\, e_{t-1}(s) + \mathbb{1}\{s = S_t\}, \qquad V(s) \leftarrow V(s) + \alpha\, \delta_t\, e_t(s). \tag{45}$$

The trace $e_t(s)$ acts as a fading memory of recently visited states: it spikes when s is visited and decays by $\gamma\lambda$ per step. Each TD error δ_t updates every state in proportion to its current trace, enabling $O(|\mathcal{S}|)$ per-step credit assignment without storing trajectories.[77] The historical development of eligibility traces is discussed in Section 3.

Under linear function approximation, TD(λ) converges to a unique fixed point with approximation error bounded by $\frac{1-\lambda\gamma}{\sqrt{1-\gamma^2}}$ times the best-in-class error (Equation 51 and the bound in Section 5.3; Tsitsiklis and Van Roy, 1997). Higher λ tightens this bound, approaching the projection of V^π as $\lambda \to 1$.

5.2.4 Simulation Study: Credit Assignment in a Corridor

A 20-state deterministic corridor ($s \in \{0, \ldots, 19\}$, action: move right, reward $+1$ on the transition into the terminal state $s = 19$, $\gamma = 0.99$) isolates the credit-assignment mechanism. The true value function is $V^*(s) = \gamma^{18-s}$ for $s \leq 18$ (and $V^*(19) = 0$). TD(λ) performs policy evaluation for four values of λ across 20 seeds and 200 episodes.

Table 4 and Figure 5 show that higher λ propagates the sparse terminal reward signal backward through the corridor faster: TD($\lambda = 1$) reaches RMSVE < 0.05 in fewer episodes than TD(0), which must wait for many episodes before the reward signal diffuses back to early states through one-step bootstrapping alone.

[77] The forward and backward views produce identical total weight changes over a complete episode (Sutton, 1988). The backward view is preferred in practice because it operates online (updating after each transition) rather than requiring the full episode to be stored. Practical variants include *replacing traces* ($e_t(s) = 1$ when $s = S_t$, capping the trace at 1 instead of accumulating), *Dutch traces* (van Seijen et al., 2016), and off-policy extensions such as Retrace(λ) (Munos et al., 2016) and V-trace (Espeholt et al., 2018).

Table 4: TD(λ) on 20-state corridor. Mean $\pm$ SE over 20 seeds, 200 episodes, $\gamma = 0.99$, $\alpha = 0.05$.

λ	Final RMSVE	Episodes to RMSVE < 0.05
0.0	0.4012 ± 0.0056	> 200
0.4	0.1902 ± 0.0028	> 200
0.8	0.0108 ± 0.0002	141 ± 1
1.0	0.0000 ± 0.0000	52 ± 0

5.2.5 Finite-Sample Theory of Fitted Methods

Fitted Q-Iteration and Fitted Value Iteration (Definition 4.1.10, Section 4.1.10) replace exact Bellman applications with projected regression steps. The projection introduces approximation error that compounds across iterations.

Define the inherent Bellman approximation error

$$\varepsilon_{\text{approx}} = \inf_{f \in \mathcal{F}} \|\mathcal{T} f - f\|_{p,\mu}, \tag{46}$$

the smallest residual achievable when the Bellman operator maps any element of $\mathcal{F}$ back to itself. Munos and Szepesvári (2008) show that after K iterations with N i.i.d. samples per iteration,

$$\|V_K - V^*\|_{p,\rho} \;\le\; C_{\rho,\mu} \left[\gamma^K \|V_0 - V^*\|_{p,\rho} + \frac{\varepsilon_{\text{approx}}}{(1-\gamma)^2} + O\!\left(\frac{1}{\sqrt{N}}\right) \right], \tag{47}$$

where $C_{\rho,\mu}$ is a *concentrability coefficient* bounding the ratio of future-state distributions under the evaluation distribution ρ relative to the data distribution μ.[78] Three terms drive the error: the geometric decay γ^K (initialization bias), the approximation error $(1-\gamma)^{-2}\varepsilon_{\text{approx}}$ (bias from function class), and the estimation error $O(1/\sqrt{N})$ (variance from finite samples). When $\mathcal{F}$ contains V^* exactly, $\varepsilon_{\text{approx}} = 0$ and the bound recovers exact convergence as $K \to \infty$. The $(1-\gamma)^{-2}$ amplification, one factor of $(1-\gamma)^{-1}$ more than in tabular Q-learning, reflects error accumulation across approximate DP steps: each regression step introduces bias, and this bias compounds over K iterations.

Both algorithms solve projected Bellman equations. When $V^* \in \text{span}(\Phi)$ exactly, FVI converges to V^* in a single projected iteration, since the normal equations (23) recover θ_V^* satisfying $\Phi\theta_V^* = V^*$. For FQI, the per-action Q-functions satisfy $Q^*(s, a^*(s)) = V^*(s)$ at the optimal action $a^*(s)$, so when $Q^*(\cdot, a)$ is also representable in $\text{span}(\Phi)$ for each a, FQI recovers consistent value estimates $\phi(s)^\top \theta^*_{a^*(s)} = \phi(s)^\top \theta_V^*$ at convergence. Whether FQI succeeds therefore depends on the geometry of the problem: when $Q^*(\cdot, a) \notin \text{span}(\Phi)$, as on the Brock–Mirman economy, where the per-action Q-function requires fractional-power terms $k^{-n\alpha}$ outside the log-polynomial span, FQI stalls at error 1.65 while FVI converges to 0.001; when $Q^*(x, u)$ is exactly quadratic in (x, u), as on the linear-quadratic control problem (Section 5.2.6), both FVI and FQI converge to near-zero error. Section 5.2.7 shows that replacing the linear basis with a nonlinear parametric model that matches the log-Cobb–Douglas structure of Q^* restores FQI convergence on the Brock–Mirman economy.

5.2.6 Simulation Study: Fitted Methods on Linear-Quadratic Control

Linear-quadratic control (LQC) is a setting where both V^* and Q^* are quadratic polynomials, so the fitted method comparison is analytically transparent. The model has scalar state $x \in [-4, 4]$

[78] The concentrability coefficient $C_{\rho,\mu}$ measures how well the data distribution μ covers future states reachable under optimal policies from ρ. When μ is the state-action distribution of the optimal policy itself, $C_{\rho,\mu} = 1$. Distribution mismatch, common when batch data comes from a sub-optimal behavior policy, inflates $C_{\rho,\mu}$ and worsens the bound. Antos et al. (2008) extend these results to continuous action spaces and single-trajectory data.

and action $u \in [-2, 2]$, deterministic dynamics $x' = ax + bu$ with $a = 0.5$, $b = 1.0$, reward $r(x, u) = -(x^2 + u^2)$, and discount $\gamma = 0.95$. The parameters are chosen so that $x' = 0.5x + u \in [-4, 4]$ whenever $(x, u) \in [-4, 4] \times [-2, 2]$, making the grid strictly invariant. The optimal value function satisfies $V^*(x) = -Px^2$, where P solves $\gamma b^2 P^2 + P(1 - \gamma(a^2 + b^2)) - 1 = 0$, yielding $P \approx 1.129$. The optimal Q-function is $Q^*(x, u) = -(1 + \gamma P a^2)x^2 - 2\gamma P ab\, xu - (1 + \gamma P b^2)u^2 \approx -1.268x^2 - 1.073xu - 2.073u^2$, which lies exactly in $\text{span}\{x^2, xu, u^2\} \subset \text{span}\{x, x^2, u, u^2, xu\}$. FVI uses state features $\phi_V(x) = [x, x^2]^\top \in \mathbb{R}^2$ (no intercept, since $V^*(0) = 0$); FQI uses state-action features $\phi_Q(x, u) = [x, x^2, u, u^2, xu]^\top \in \mathbb{R}^5$. Both use a 301-point state grid and 201-point action grid. DQN uses a two-layer network of 64 units per layer with ReLU activations, an experience replay buffer of 50,000 transitions, a hard target-network update every 500 gradient steps, and rewards scaled by a factor of $1/20$ to stabilize training.

Both methods recover the analytical solution to machine precision (Table 5, Figure 6): FVI and FQI converge in under 10 iterations with errors below 10^{-3}, matching the known polynomial structure of Q^*. DQN also converges (error 5.6×10^{-1} after 100,000 gradient steps) with no prior knowledge of the feature basis. The contrast with Brock–Mirman is exact: FQI succeeds here because $Q^*(x, u) \in \text{span}(\Phi_Q)$, while it fails on Brock–Mirman because $Q^*(\cdot, a) \notin \text{span}(\Phi)$.

Method	Iterations	Error vs V^*	P (recovered)	Key coefficient
Exact VI (discrete)	25	1.12e-03	—	—
FVI	9	3.23e-04	1.1294	$\hat{\theta}_V^{x^2} = -1.1294$
FQI	10	9.37e-05	1.2682	$\hat{\theta}_Q^{xu} = -1.0729$
DQN (2×64 ReLU)	100000	5.64e-01	—	—
Analytical ($V^* = -Px^2$)	—	0	1.1294	$c_{xu} = -1.0729$

Table 5: Fitted weights and convergence metrics for FVI, FQI, and DQN on linear-quadratic control. The FVI x^2 coefficient recovers the Riccati solution $P \approx 1.129$. The FQI quadratic coefficients match the analytical Q^* to four decimal places. FVI and FQI achieve max error below 10^{-3} against the analytical V^*; DQN achieves error 5.6×10^{-1} after 100,000 gradient steps with no feature basis specified.

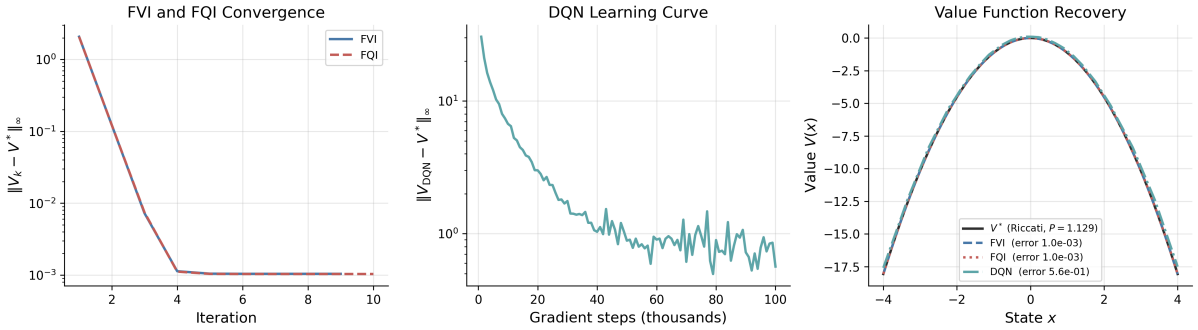

Figure 6: LQC convergence of FVI and FQI (left), DQN learning curve (middle), and value function recovery for all three methods (right). FVI and FQI reduce $\|V_k - V^*\|_\infty$ to near-zero in under 10 iterations, exploiting the known polynomial structure of Q^*. DQN declines from error 6.7 to 0.56 over 100,000 gradient steps with no feature basis specified.

5.2.7 Simulation Study: Basis Representability on the Brock–Mirman Economy

The Brock–Mirman stochastic growth model (Brock and Mirman, 1972) provides the negative case. The economy has $|\mathcal{S}| = 100$ states ($N_K = 50$ capital grid points, $N_Z = 2$ productivity levels) and $|\mathcal{A}| = 50$ actions. We use the same log-polynomial basis $\phi(k, z) = [1, \log k,\ k/\bar{k},\ (k/\bar{k})^2,\ (k/\bar{k})^3] \otimes [\mathbb{1}_{z=z_\ell},\ \mathbb{1}_{z=z_h}]$ from the theory discussion above, which contains

V^* up to residual $\|\Pi_\Phi V^* - V^*\|_\infty = 0.0002$. To test whether the failure is inherent to FQI or to the basis, we add two methods that use the structurally correct per-action feature $\log(zk^\alpha - k')$, the log-consumption implied by the Cobb–Douglas technology. Oracle-FQI treats $\alpha = 0.36$ as known and runs standard OLS per action with three parameters $[\mathbb{1}_{z=z_\ell}, \mathbb{1}_{z=z_h}, \log(zk^\alpha - k')]$. NLLS-FQI estimates α jointly via concentrated least squares: for each candidate α, it solves conditional OLS for intercepts and slope, then optimizes α to minimize total residual sum of squares, initialized at the deliberately wrong value $\alpha_0 = 0.5$.[79]

Table 6 and Figure 7 confirm the diagnosis: basis representability, not algorithmic failure. FVI converges near the projection floor of the log-polynomial basis; linear FQI stalls at error 1.65, confirming $Q^*(\cdot, a) \notin \text{span}(\Phi)$. Oracle-FQI and NLLS-FQI, using the structurally correct log-consumption feature, match exact VI with error below 10^{-4}. NLLS-FQI recovers $\hat{\alpha} = 0.3600$ in a single iteration, demonstrating that the same FQI algorithm succeeds when the function class contains Q^*.

Method	$\|V - V^*\|_\infty$	$\|V - V^*\|_{\text{rms}}$	Policy agreement (%)	Iterations
Exact VI	0.0000	0.0000	98.0	—
FVI (linear)	0.0010	0.0008	99.0	341
FQI (linear)	1.6521	1.4805	4.0	339
Oracle-FQI	0.0000	0.0000	98.0	341
NLLS-FQI ($\hat{\alpha} = 0.3600$)	0.0000	0.0000	98.0	341

Table 6: Convergence metrics for five methods on the Brock–Mirman economy ($N_K = 50$, $N_Z = 2$, $\gamma = 0.96$). Policy agreement is measured against the closed-form optimal policy $k' = \alpha\beta z k^\alpha$.

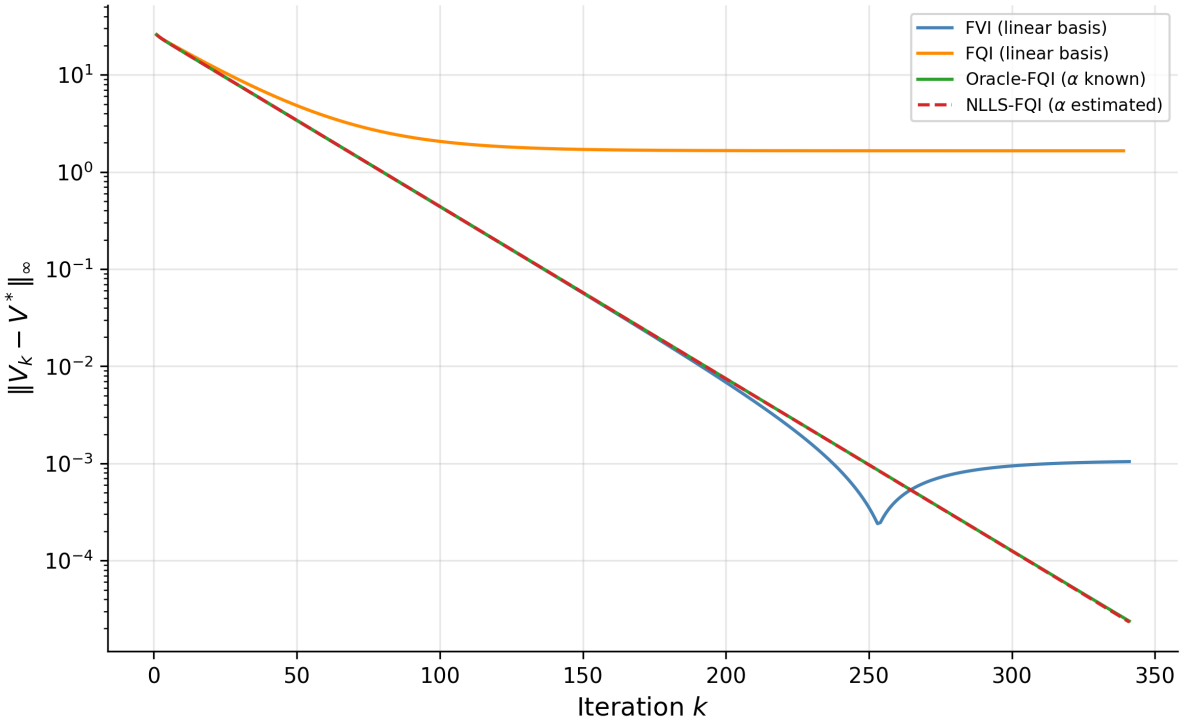

Figure 7: Left: convergence of $\|V_k - V^*\|_\infty$ for FVI, linear FQI, Oracle-FQI, and NLLS-FQI on the Brock–Mirman economy. Right: NLLS-FQI estimated α trajectory, converging from $\alpha_0 = 0.5$ to the true $\alpha = 0.36$ in one iteration.

[79] Observations where $zk^\alpha - k' \leq 0$ (infeasible consumption) contribute a penalty equal to the mean squared target, preventing the optimizer from improving RSS by shrinking the feasible set.

5.2.8 Rollout, Lookahead, and AlphaZero

Two constructions bridge the Newton interpretation of Section 5.1 to practical algorithms. Given a base policy μ with value function V^μ, the *rollout* policy selects

$$\mu_R(s) = \underset{a \in \mathcal{A}}{argmax} \left\{ r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^\mu(s') \right\}. \tag{48}$$

This is one step of policy iteration starting from μ: the Policy Improvement Theorem (Theorem 1) guarantees $V^{\mu_R}(s) \geq V^\mu(s)$ for all s, with strict inequality unless μ is already optimal (Bertsekas, 2021).[80] Given an arbitrary approximate value function $\tilde{V}$ (not necessarily the value of any policy), the *one-step lookahead* policy selects

$$\tilde{\pi}(s) = \underset{a \in \mathcal{A}}{argmax} \left\{ r(s,a) + \gamma \sum_{s'} P(s'|s,a)\, \tilde{V}(s') \right\}. \tag{49}$$

When $\tilde{V} = V^\mu$, lookahead and rollout coincide. The distinction matters because rollout inherits the monotone improvement guarantee (it starts from the value of a policy), while lookahead from an arbitrary $\tilde{V}$ has no such monotonicity. The Newton interpretation from Section 5.1 explains why lookahead nevertheless helps: both constructions solve the linearized Bellman equation at the current iterate.[81]

An ℓ-step lookahead extends this by applying $(\ell - 1)$ steps of value iteration before the final greedy selection. The first $(\ell - 1)$ steps are ordinary Bellman contractions, each shrinking the approximation error by a factor of γ. Only the final step, the greedy policy improvement, constitutes the Newton step.[82] The resulting error bound is

$$\|V^{\tilde{\pi}} - V^*\|_\infty \leq \gamma^\ell \, \|\tilde{V} - V^*\|_\infty, \tag{50}$$

where the γ^ℓ factor reflects ℓ total contractions (Bertsekas, 2022a, Prop. 2.3.1). Deep lookahead compensates for poor approximation through repeated contraction, not through repeated Newton steps.

Recall the AlphaGo Zero system from Section 4.2.5, where a neural network $f_\theta(s) = (\mathbf{p}, v)$ outputs a prior policy $\mathbf{p}$ and a value estimate v for any board position s. During play, the network does not act alone: Monte Carlo Tree Search runs simulated games from the current position, using v to evaluate leaf nodes and $\mathbf{p}$ to guide which branches to explore.[83] The network provides $\tilde{V}$; MCTS applies multi-step lookahead through selective tree expansion. Table 7 makes the correspondence explicit.

The gap between network-only play and network-plus-search is the contraction factor γ^H, where H is the effective search depth. The network provides a rough starting point $\tilde{V}$; MCTS applies the Bellman operator through deep lookahead, shrinking the approximation error by γ per level of search.[84]

[80] Bertsekas uses cost-minimization notation throughout his work, writing min where standard RL uses max and defining value as accumulated cost rather than reward. I translate to the reward-maximization convention used elsewhere in this chapter; the mathematics are equivalent with reversed inequalities.

[81] Rollout requires a simulator (generative model) that can be queried from arbitrary states, a stronger assumption than the trajectory-based access of Q-learning and SARSA.

[82] Bertsekas (2021) states this explicitly: "whatever follows the first step of the lookahead is preparation for the Newton step." The preceding value iteration steps have linear convergence at rate γ; only the terminal improvement step has superlinear character.

[83] AlphaGo Zero uses 1,600 MCTS simulations per move for Go; the generalized AlphaZero algorithm uses 800 simulations per move across chess, shogi, and Go (Silver et al., 2018a, Table S3).

[84] MCTS adds UCB exploration and selective tree expansion beyond the literal lookahead framework. The Newton interpretation explains why lookahead helps at all, namely, the final greedy selection over the search tree is a policy improvement step. It does not explain the specific mechanisms (upper confidence bounds, progressive

Step	Policy Iteration	AlphaZero
Initialize	Arbitrary π_0	Random network f_θ
Evaluate	Solve $V = T^{\pi_k} V$ exactly	Network value head $v \approx V^{\pi_k}$
Improve	$\pi_{k+1} = argmax_a\{r + \gamma P V^{\pi_k}\}$	MCTS simulations from v
Iterate	Repeat until π is stationary	Retrain f_θ on self-play outcomes

Table 7: Policy iteration and AlphaZero follow the same evaluate-improve loop. The network provides approximate policy evaluation; MCTS provides approximate policy improvement via selective tree search.

5.3 The Central Challenge: The Deadly Triad

State spaces are too large for lookup tables (Go has 10^{170} states; most economic models have continuous state variables). Practitioners must combine three ingredients: *function approximation* (to handle large state spaces), bootstrapping (to learn from single transitions rather than complete episodes), and off-policy learning (to learn about the optimal policy while exploring, or to reuse old data). Each is desirable in isolation. Their interaction, known as the *deadly triad* (Sutton and Barto, 2018, Ch. 11), is the central open problem in reinforcement learning theory.

Off-policy learning is preferred for three reasons. First, sample efficiency: transitions collected under any behavioral policy can be reused to evaluate or improve a different target policy, amortizing the cost of data collection. Discarding data because it was generated by a superseded policy is wasteful. Second, exploration and exploitation separate cleanly: the agent can follow an exploratory policy (e.g., ε-greedy) to ensure adequate state-space coverage while simultaneously learning the optimal deterministic policy. On-policy methods such as SARSA entangle the two, learning the value of the exploratory policy rather than the optimal one. Third, off-policy evaluation answers "what would have happened under policy π?" from data generated by policy μ, the counterfactual question at the heart of policy comparison.

5.3.1 The Projected Bellman Operator

With function approximation $V(s) \approx \phi(s)^\top \theta$, the parameter vector θ is shared across states. Updating θ to improve the value estimate at one state simultaneously changes the estimate at every other state. The algorithm can no longer apply the Bellman operator T^π to each state independently; instead, it applies T^π to compute a target, then *projects* the result back onto the function space (the span of the features ϕ). The composed operator is ΠT^π, the *projected Bellman operator*, where Π denotes this projection (Tsitsiklis and Van Roy, 1997). Convergence of the approximate iteration $\theta_{k+1} = \Pi T^\pi \theta_k$ requires ΠT^π to be a contraction.

In the on-policy setting, Π minimizes squared error weighted by d^π, the stationary distribution of the policy being evaluated, so $\Pi V = \arg\min_{\hat{V} \in \text{span}(\Phi)} \|V - \hat{V}\|_{d^\pi}$, where $\|V\|_{d^\pi}^2 = \sum_s d^\pi(s) V(s)^2$. The Bellman operator T^π is a γ-contraction in the same d^π-norm. Because both operators use the same norm, the projection Π is *orthogonal*, meaning the residual $V - \Pi V$ is perpendicular to the approximation subspace. The Pythagorean theorem then gives $\|\Pi V\|_{d^\pi}^2 + \|V - \Pi V\|_{d^\pi}^2 = \|V\|_{d^\pi}^2$, so $\|\Pi V\|_{d^\pi} \leq \|V\|_{d^\pi}$.[85] The projection cannot expand distances. The composition therefore contracts:

$$\|\Pi T^\pi V_1 - \Pi T^\pi V_2\|_{d^\pi} \leq \underbrace{\|\Pi\|}_{\leq 1} \cdot \underbrace{\|T^\pi V_1 - T^\pi V_2\|_{d^\pi}}_{\leq \gamma \|V_1 - V_2\|_{d^\pi}} < \|V_1 - V_2\|_{d^\pi}. \tag{51}$$

widening) that make MCTS computationally efficient.

[85] The same argument holds in $\mathbb{R}^n$. Projecting a vector onto a subspace never makes it longer. This is the geometric content of the Cauchy-Schwarz inequality. In the function-approximation setting, the "vector" is a value function, the "subspace" is the span of features, and "length" is the d^π-weighted L^2 norm.

A unique fixed point $\Phi\theta^*$ exists, and TD(λ) converges to it with probability one. The resulting approximation error satisfies $\|\Phi\theta^* - V^\pi\|_{d^\pi} \leq \frac{1-\lambda\gamma}{\sqrt{1-\gamma^2}}\|\Pi V^\pi - V^\pi\|_{d^\pi}$, bounding the TD solution's error by a multiple of the best possible approximation error (Tsitsiklis and Van Roy, 1997, Theorem 1).

5.3.2 Why Off-Policy Learning Diverges

In the off-policy setting, samples come from a behavior distribution $\mu \neq d^\pi$. The projection now minimizes error under μ, but the Bellman operator T^π still contracts in the d^π-norm. The two operators measure distance in different norms. The projection is no longer orthogonal in the d^π-norm; it is *oblique*. Unlike orthogonal projections, oblique projections can expand distances, with $\|\Pi_\mu\|_{d^\pi}$ exceeding 1 in the worst case. If $\|\Pi_\mu\|_{d^\pi} > 1/\gamma$, the expansion from projection overwhelms the γ-contraction from the Bellman operator, and the fixed-point iteration diverges.

This divergence is not overfitting. Overfitting occurs when the approximator memorizes training data at the expense of generalization; collecting more data helps. Divergence means the parameter vector θ grows without bound, producing arbitrarily large value estimates that bear no relation to the true values. More data does not help; the algorithm itself is unstable. The distinction matters because the remedies are entirely different. Regularization and early stopping address overfitting, while the deadly triad requires structural changes to the algorithm.

Baird (1995) constructed a six-state star MDP that makes this failure concrete. All rewards are zero, so the true value is $V^*(s) = 0$ for every state. A lookup table learns this immediately. The MDP has a star topology: states 1 through 5 each transition to state 6, and state 6 transitions to itself. Linear function approximation uses a shared weight w_1 across all states plus a state-specific weight, with $V(s) = 2w_1 + w_s$ for $s \in \{1, \ldots, 5\}$ and $V(6) = 2w_1 - w_6$. Training samples all transitions equally often (uniform distribution, not d^π). The dynamics are as follows.[86] When $V(6)$ is large and positive, the TD target $\gamma V(6)$ exceeds $V(s)$ for states 1 through 5, producing positive TD errors that push w_1 upward. At state 6, the TD target is $\gamma V(6) < V(6)$, producing a negative TD error that pushes w_1 downward. But states 1 through 5 are each visited as often as state 6, so w_1 receives five upward pushes for every one downward push. The shared weight diverges to $+\infty$. The on-policy distribution would concentrate mass on state 6 (the absorbing state), counterbalancing the upward pressure; uniform sampling destroys this balance.[87]

Each element of the triad is individually necessary for divergence. Without function approximation (tabular), the projection is the identity and Q-learning's contraction applies directly. Without bootstrapping (Monte Carlo returns), targets are independent of current value estimates and the problem reduces to supervised regression. Without off-policy learning, samples come from d^π, the projection is orthogonal, and the Tsitsiklis-Van Roy convergence guarantee holds.

5.3.3 Resolutions

Three classes of algorithms restore convergence, each neutralizing a different component of the triad.

Target networks weaken bootstrapping. Instead of updating toward $r + \gamma Q(s'; \theta)$, where the target moves with each parameter update, DQN (Mnih et al., 2015) updates toward $r +$

[86] Baird (1995) also presents an MDP variant with two actions per state, demonstrating that Q-learning diverges under the same mechanism. The mechanism is identical: shared parameters create cross-state coupling that uniform sampling cannot counterbalance.

[87] The gradient of $\|Q - TQ\|^2$ requires two independent next-state samples from the same (s, a), since $\nabla\mathbb{E}[(Q - \mathbb{E}[r + \gamma V(s')])^2]$ involves $\mathbb{E}[\cdot] \cdot \nabla\mathbb{E}[\cdot]$ and $\mathbb{E}[XY] \neq \mathbb{E}[X]\mathbb{E}[Y]$. This "double-sampling" requirement is impractical (Baird, 1995), so practitioners use semi-gradient TD, treating the bootstrap target as a constant. This semi-gradient structure makes off-policy TD vulnerable to projection mismatch.

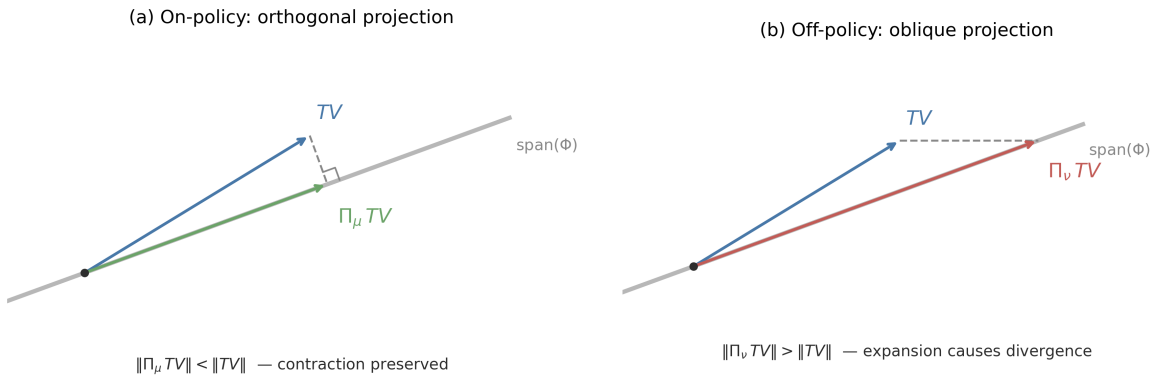

Figure 8: Geometry of the projected Bellman operator in $\mathbb{R}^2$. The gray line is the function approximation subspace $\text{span}(\Phi)$; the blue arrow is TV, the Bellman update. (a) On-policy: the orthogonal projection Π_μ drops TV perpendicularly onto the subspace, preserving the contraction. (b) Off-policy: the oblique projection Π_ν (under the behavior distribution) reaches a point further from the origin than TV itself, causing expansion.

$\gamma Q(s'; \theta^-)$, where θ^- is a slowly-updated copy of the parameters.[88] The regression target becomes quasi-static, converting the coupled fixed-point problem into a sequence of supervised learning problems. Zhang et al. (2021) prove that this two-timescale scheme converges to a regularized TD fixed point with linear function approximation. Fellows et al. (2023) show that target networks recondition the Jacobian of the TD update: the spectral radius of the composed update operator depends on the target network update frequency k, and for sufficiently large k the spectral radius drops below 1 even in off-policy settings with nonlinear function approximation.

Gradient TD methods fix the projection mismatch. Sutton et al. (2009) reformulate the projected Bellman error as a saddle-point problem $\min_\theta \max_y L(\theta, y)$, yielding algorithms (GTD, GTD2, TDC) that perform true stochastic gradient descent on the mean-squared projected Bellman error.[89] These methods converge off-policy with linear function approximation because they eliminate the semi-gradient approximation that causes the norm mismatch.

Regularization shrinks the projection operator. Lim and Lee (2024) add an ℓ_2 penalty $-\eta\theta$ to the Q-learning update. This changes the projection from $\Pi = X(X^\top DX)^{-1}X^\top D$ to $\Pi_\eta = X(X^\top DX + \eta I)^{-1}X^\top D$. As the regularization strength η increases, the projection "shrinks" toward the origin. For sufficiently large η, $\gamma\|\Pi_\eta\| < 1$, restoring the contraction property. The algorithm converges to a biased but stable fixed point, with the bias controlled by η.

5.4 Policy Learning Methods

Value-based methods find fixed points of the Bellman operator. Policy-based methods parameterize the policy directly as $\pi_\theta(a|s)$ and maximize expected return $J(\theta) = \mathbb{E}_{\pi_\theta}[\sum_{t=0}^{\infty} \gamma^t R_t]$ by gradient ascent. This formulation sidesteps the Bellman equation entirely and frames reinforcement learning as constrained optimization.

[88] Experience replay (Lin, 1992) complements target networks by breaking temporal correlation in the training data. The two mechanisms address different sources of instability: target networks stabilize the bootstrap target, while replay stabilizes the sampling distribution.

[89] The saddle-point formulation introduces auxiliary variables y of the same dimension as θ, doubling the parameter count and requiring a second learning rate. Bhandari et al. (2021) provide finite-time convergence rates for these two-timescale algorithms.

5.4.1 The Policy Gradient Theorem

The policy gradient theorem, proved independently by Williams (1992) for the episodic case and Sutton et al. (2000) for the general discounted setting, provides a tractable expression for the gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a|s)\, Q^{\pi_\theta}(s,a)\right], \tag{52}$$

where $d^{\pi_\theta}(s) = (1-\gamma)\sum_{t=0}^{\infty}\gamma^t \mathbb{P}(s_t = s|\pi_\theta)$ is the discounted state visitation distribution. The fundamental econometric challenge in optimizing $J(\theta)$ is that this distribution depends on θ through the environment's dynamics. A naive derivative would require $\nabla_\theta d^{\pi_\theta}(s)$, which implies differentiating the unknown transition matrix $P(s'|s,a)$.

The policy gradient theorem sidesteps this entirely via a likelihood ratio (or score function) trick.[90] The theorem transforms a sensitivity analysis problem (how does the system evolve?) into a simpler expectation problem (what is the correlation between the score $\nabla \log \pi$ and the value Q?). The gradient can be written as an expectation under the current policy, weighted by action-values, without requiring $\nabla_\theta d^{\pi_\theta}$. The transition dynamics $P(s'|s,a)$ do not appear; the gradient is estimable via sample averages from trajectories alone.

5.4.2 REINFORCE and Variance Reduction

REINFORCE (Williams, 1992) is the simplest policy gradient algorithm. Sample a trajectory $(s_0, a_0, r_0, s_1, \ldots)$, compute the return $G_t = \sum_{k=0}^{\infty}\gamma^k r_{t+k}$ from each time step, and update:

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)\, G_t. \tag{53}$$

This is an unbiased estimator of $\nabla_\theta J(\theta)$, but its variance is high because a single trajectory provides a noisy estimate of Q^{π_θ}. Despite high variance, REINFORCE converges to a globally optimal policy in the tabular setting.[91]

5.4.3 Natural Policy Gradient and Gradient Domination

Standard gradient descent treats all parameter directions equally. But small changes in θ can cause large changes in the policy distribution π_θ. The natural policy gradient (Kakade, 2001), building on the natural gradient framework of Amari (1998), accounts for this curvature by preconditioning with the Fisher information matrix.[92] The relationship between NPG and standard PG parallels that between Fisher scoring and gradient ascent in MLE. Both precondition with the inverse Fisher information matrix $F(\theta)^{-1}$, achieving parameterization invariance and quadratic convergence near the optimum.

$$\tilde{\nabla}_\theta J(\theta) = F(\theta)^{-1}\nabla_\theta J(\theta), \quad F(\theta) = \mathbb{E}_{s,a}\left[\nabla_\theta \log \pi_\theta(a|s) \nabla_\theta \log \pi_\theta(a|s)^\top\right]. \tag{54}$$

Why does NPG recover policy iteration? Standard gradient ascent is sensitive to parameterization: it takes the steepest step in Euclidean parameter space, where units depend on how the policy is parameterized. NPG takes the steepest step in distribution space (measured by KL-divergence), which is invariant to reparameterization. In the tabular case, Kakade (2001, Theorem 2) proves that this geometric correction aligns the gradient exactly with the greedy

[90] The score function $\nabla_\theta \log \pi_\theta(a|s)$ is the same mathematical object that appears in the Cramér-Rao bound and the score test in maximum likelihood estimation. The policy gradient is a covariance, $\nabla J = \mathrm{Cov}_{d^{\pi_\theta}\times\pi_\theta}(\nabla \log \pi_\theta, Q^{\pi_\theta})$, measuring how sensitive the log-likelihood of the policy is to parameter changes, weighted by action quality.

[91] The baseline $b(s)$ subtracted from G_t reduces variance while preserving unbiasedness, since $\mathbb{E}[\nabla_\theta \log \pi_\theta(a|s) b(s)] = 0$ for any baseline independent of a.

[92] The Fisher information $F(\theta) = \mathbb{E}[\nabla \log p \nabla \log p^\top]$ measures curvature of the log-likelihood and appears in the Cramér-Rao bound. Here it measures how policy distributions change with parameters.

policy $\tilde{\pi}$ from policy iteration. With step size 1 (and exact estimation), NPG performs one full Newton step; with smaller step sizes, it performs damped Newton updates. This explains its rapid convergence: NPG approximates the quadratic convergence of finding a fixed point rather than the linear convergence of hill-climbing.

The RL objective $J(\theta)$ is non-convex in θ. For researchers trained to distrust gradient methods on non-convex objectives, the natural concern is convergence to spurious local optima. For tabular softmax policies (one free parameter per state-action pair), this concern is unfounded. The landscape is "benign" in a precise sense. Agarwal et al. (2021a) prove that $J(\theta)$ satisfies a *gradient domination* condition (also called Polyak-Łojasiewicz, or PL). The PL condition has the same functional form as the strong convexity condition for guaranteeing linear convergence of gradient descent, but it applies to non-convex functions: whenever $\|\nabla J(\theta)\|$ is small, the policy must be near-optimal. Formally, the sub-optimality $J(\pi^*) - J(\pi_\theta)$ is bounded by a constant times $\|\nabla J(\theta)\|^2$. The implication is immediate: any point where the gradient vanishes is globally optimal. The non-convex landscape has no false peaks, no spurious local maxima. Gradient ascent cannot get trapped.

Mei et al. (2020) sharpen this result for softmax parameterization, proving explicit convergence rates. These guarantees are specific to the tabular parameterization. With function approximation, the PL condition does not hold. Agarwal et al. (2021a, Theorem 6.2) show that NPG with log-linear or smooth policy classes (including neural networks) converges to a neighborhood of the optimum whose radius depends on the approximation error of the policy class, not to the global optimum itself.[93]

In the tabular setting, NPG achieves more: *dimension-free* convergence. Standard gradient ascent on $J(\theta)$ has a convergence rate that depends on the smoothness constant, which scales with $|\mathcal{S}|$. NPG circumvents this by preconditioning with the Fisher information matrix $F(\theta)^{-1}$. The mechanism is that the state-visitation distribution $d^{\pi_\theta}(s)$ appears in both $\nabla J(\theta)$ and $F(\theta)$; when computing $F^{-1}\nabla J$, these terms cancel analytically. The resulting update rule is equivalent to soft policy iteration and converges at rate $O(1/(1-\gamma)^2\epsilon)$, independent of $|\mathcal{S}|$ and $|\mathcal{A}|$ (Xiao, 2022).

5.4.4 Trust Region Methods

NPG requires computing and inverting the Fisher information matrix $F(\theta)$, which scales as $O(d^2)$ in parameters and is impractical for neural networks. TRPO (Schulman et al., 2015) approximates the natural gradient using conjugate gradient methods[94] without forming F explicitly, and enforces trust regions via line search.[95] Shani et al. (2020) prove convergence for adaptive trust region methods that adjust the constraint radius dynamically. PPO (Schulman et al., 2017) simplifies further by replacing the hard KL constraint with a clipped surrogate ob-

[93] However, "no spurious local optima" does not imply "easy optimization." The landscape is dominated by vast plateaus (saddle points) where gradients vanish. Without sufficient exploration, the probability of visiting relevant states decays exponentially with the horizon, rendering the gradient exponentially small. Global convergence requires the starting distribution to have adequate coverage relative to the optimal policy's visitation distribution, formalized as the "distribution mismatch coefficient" by Agarwal et al. (2021a). The Natural Policy Gradient addresses this by preconditioning with the Fisher Information Matrix, making the update direction covariant: invariant to invertible linear transformations of the parameter space. This standardizes units across parameters, preventing stalling on plateaus caused by poor parameter scaling. Li et al. (2022) make this quantitatively precise: vanilla softmax policy gradient requires iterations doubly exponential in the effective horizon $1/(1-\gamma)$ because score functions are exponentially small in directions corresponding to suboptimal actions.

[94] Conjugate gradient is an iterative method for solving linear systems $Ax = b$ without forming A explicitly, requiring only matrix-vector products Av. With k iterations it costs $O(kd)$ versus $O(d^3)$ for direct inversion, making it feasible for neural networks with millions of parameters.

[95] Trust region methods build on the performance difference lemma: for any two policies π and π', $J(\pi') - J(\pi) = \frac{1}{1-\gamma}\mathbb{E}_{s\sim d^{\pi'}}[\sum_a \pi'(a|s)A^\pi(s,a)]$, where $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ is the advantage function. This identity bounds how much policy improvement is possible and motivates constraining updates to regions where advantage estimates remain accurate (Kakade, 2002).

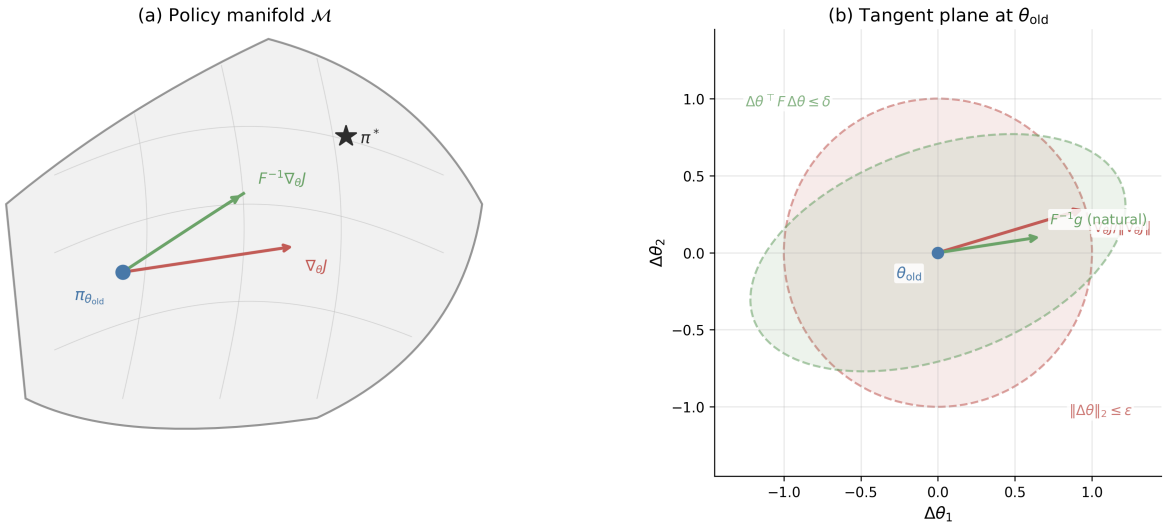

Figure 9: Information geometry of the natural policy gradient. *Left*: the policy manifold $\mathcal{M}$ with Euclidean gradient $\nabla_\theta J$ (red) and natural gradient $F^{-1}\nabla_\theta J$ (green) from the current iterate $\pi_{\theta_{\text{old}}}$ toward the optimal policy π^*. *Right*: tangent plane at θ_{old} showing the Euclidean unit ball $\|\Delta\theta\|_2 \le \varepsilon$ (red) and the KL unit ball $\Delta\theta^\top F \Delta\theta \le \delta$ (green), with the respective steepest-ascent directions.

jective, trading theoretical guarantees for computational simplicity. The geometric foundation of trust region methods lies in information geometry. The space of policies $\{\pi_\theta : \theta \in \mathbb{R}^d\}$ forms a statistical manifold, and the natural distance between two nearby policies is the KL divergence, not the Euclidean distance between their parameters (Amari, 1998). To second order, $\mathrm{KL}(\pi_\theta \| \pi_{\theta+\Delta\theta}) \approx \frac{1}{2}\Delta\theta^\top F(\theta)\Delta\theta$, where $F(\theta)$ is the Fisher information matrix. Two parameter vectors θ and θ' that are far apart in Euclidean distance may correspond to nearly identical distributions, while nearby parameters may produce radically different policies. The natural gradient corrects for this by measuring steepest ascent in KL-divergence rather than Euclidean norm. Figure 9 illustrates the distinction: on the policy manifold, the Euclidean gradient $\nabla_\theta J$ points in a direction that ignores curvature, while the natural gradient $F^{-1}\nabla_\theta J$ follows the manifold's intrinsic geometry toward the optimum.

TRPO formalizes this insight as a constrained optimization problem. At each iteration, TRPO maximizes the importance-weighted surrogate

$$\max_\theta \; L(\theta) = \mathbb{E}_{s\sim d^{\pi_{\theta_{\text{old}}}}}\left[\sum_a \frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} A^{\pi_{\theta_{\text{old}}}}(s,a)\right] \quad \text{s.t.} \quad \mathrm{KL}(\pi_{\theta_{\text{old}}} \| \pi_\theta) \le \delta, \tag{55}$$

where $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ is the advantage function. Linearizing $L(\theta)$ around θ_{old} and applying the quadratic KL approximation yields a Lagrangian whose closed-form solution is

$$\theta_{\text{new}} = \theta_{\text{old}} + \sqrt{\frac{2\delta}{g^\top F^{-1} g}}\, F^{-1} g, \qquad g = \nabla_\theta L(\theta)\big|_{\theta_{\text{old}}}. \tag{56}$$

This is precisely the natural gradient direction, scaled so that the step saturates the KL budget δ. The step size is determined entirely by the trust region geometry, not by a learning rate hyperparameter. In practice, TRPO solves the linear system $Fv = g$ via conjugate gradient and performs a backtracking line search to enforce the KL constraint exactly.[96]

[96] Conjugate gradient solves $Fv = g$ iteratively using only matrix-vector products Fv, which can be computed via automatic differentiation without forming F explicitly. With k iterations it costs $O(kd)$ versus $O(d^3)$ for direct inversion, making it feasible for neural networks with millions of parameters.

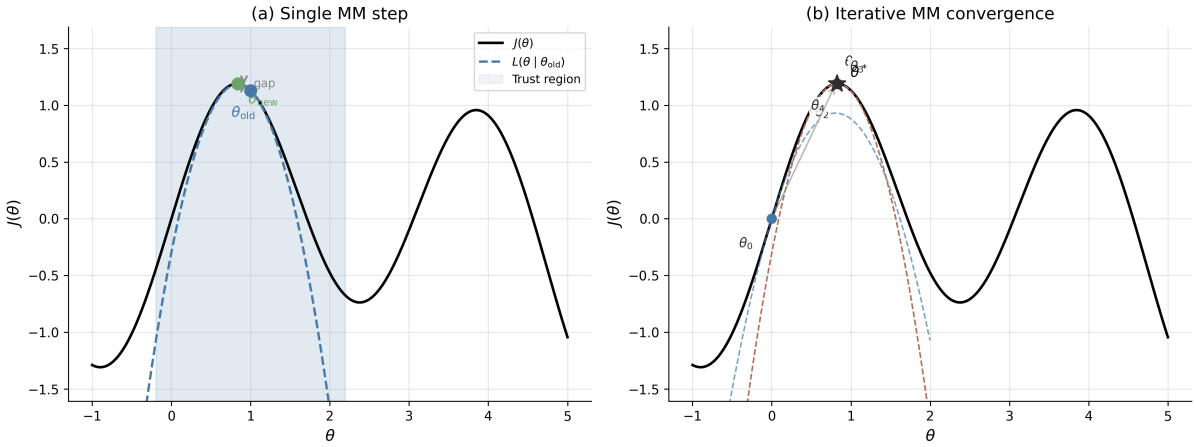

Figure 10: Majorization-minimization interpretation of trust region updates. *Left*: the surrogate $L(\theta|\theta_{\text{old}})$ (dashed) lower-bounds $J(\theta)$ (solid) and is tight at θ_{old}; the trust region (shaded) constrains the step. The gap between $L(\theta_{\text{new}})$ and $J(\theta_{\text{new}})$ is the guaranteed improvement. *Right*: iterative MM convergence from θ_0 through four surrogates (dashed, colored by iteration) to θ^*.

The theoretical guarantee underlying TRPO is a majorization-minimization (MM) argument. The surrogate $L(\theta)$ is a local lower bound on $J(\theta)$ that is tight at θ_{old}: $L(\theta_{\text{old}}) = J(\theta_{\text{old}})$ and $L(\theta) \leq J(\theta)$ within the trust region.[97] Maximizing L within the trust region therefore guarantees monotonic improvement: $J(\theta_{\text{new}}) \geq L(\theta_{\text{new}}) \geq L(\theta_{\text{old}}) = J(\theta_{\text{old}})$. This is the same pattern as the EM algorithm in statistics, where the E-step constructs a surrogate (the ELBO) and the M-step maximizes it.[98] Shani et al. (2020) prove convergence for adaptive trust region methods that adjust δ dynamically. Figure 10 illustrates this mechanism: each surrogate is a lower bound that touches J at the current iterate, and sequential maximization produces monotonically improving iterates converging to θ^*.

PPO (Schulman et al., 2017) replaces the hard KL constraint with a clipped surrogate objective. Let $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ denote the importance ratio. PPO maximizes

$$L^{\text{clip}}(\theta) = \mathbb{E}_t\left[\min(r_t(\theta)A_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)A_t)\right], \tag{57}$$

where ε (typically 0.1–0.2) bounds the ratio. When $A_t > 0$, clipping prevents r_t from exceeding $1+\varepsilon$; when $A_t < 0$, it prevents r_t from falling below $1-\varepsilon$. The resulting feasible region is not an ellipsoid in parameter space but rather a non-convex set determined by the ratio constraint at each sampled state-action pair. PPO requires no Fisher information computation and uses only first-order gradients, making it the dominant method in large-scale applications including RLHF (Section 13). Figure 11 illustrates all three mechanisms in the LQC monetary policy setting, where the non-ellipsoidal PPO feasible region is visible in contrast to TRPO's KL ellipse.

The trust region framework connects naturally to econometric optimization. The Levenberg-Marquardt algorithm for nonlinear least squares uses a similar trust region mechanism, interpolating between gradient descent and Gauss-Newton steps.[99] More broadly, the Fisher informa-

[97] The bound follows from the performance difference lemma: $J(\pi') - J(\pi) = \frac{1}{1-\gamma}\mathbb{E}_{s\sim d^{\pi'}}[\sum_a \pi'(a|s)A^\pi(s,a)]$. Replacing $d^{\pi'}$ with d^π introduces error controlled by the KL divergence between the two policies (Kakade, 2002).

[98] In EM, $Q(\theta|\theta^{(t)})$ lower-bounds the log-likelihood and is tight at $\theta^{(t)}$. Each M-step guarantees $\ell(\theta^{(t+1)}) \geq Q(\theta^{(t+1)}|\theta^{(t)}) \geq Q(\theta^{(t)}|\theta^{(t)}) = \ell(\theta^{(t)})$. The TRPO bound has the same structure with L playing the role of Q and J playing the role of ℓ.

[99] Levenberg-Marquardt solves $\min_\theta \|r(\theta)\|^2$ by adding a damping term λI to the Gauss-Newton Hessian approximation $J^\top J$, which is equivalent to constraining the step to a trust region whose radius decreases with λ. TRPO replaces $J^\top J$ with the Fisher information matrix $F(\theta)$.

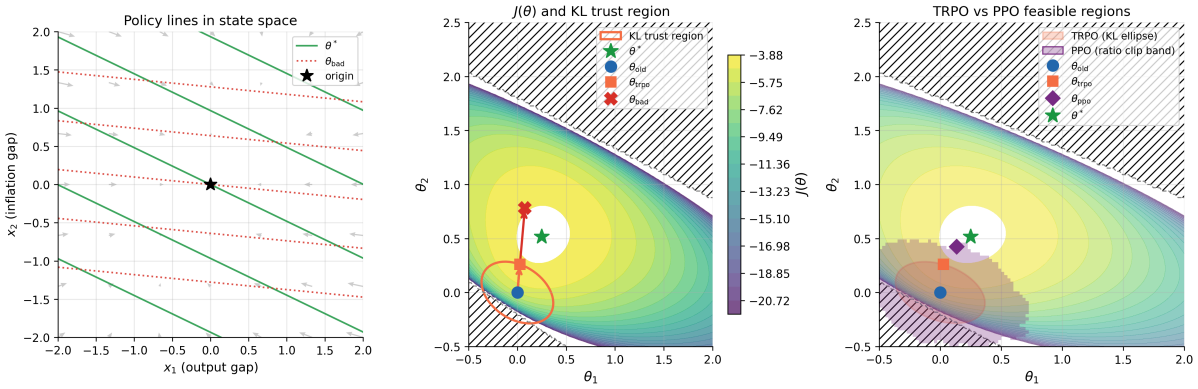

Figure 11: Trust region methods in the LQC monetary policy setting. A central bank learns a Taylor rule $u_t = -(\theta_1 x_{1t} + \theta_2 x_{2t})$ mapping output gap x_1 and inflation gap x_2 to an interest rate instrument. *Left*: policy contour lines in state space for the current iterate θ_{old}, optimal weights θ^*, and the unconstrained gradient step θ_{bad}, with phase arrows showing closed-loop dynamics under θ_{old}. *Center*: expected return $J(\theta_1, \theta_2)$ in parameter space; hatching marks the unstable region. The KL trust region ellipse bounds the TRPO step; the unconstrained gradient step overshoots into the unstable region. *Right*: TRPO feasible region (KL ellipse) and PPO feasible region (50% ratio-clip band over 200 sampled state-action pairs) overlaid on $J(\theta)$, with the respective constrained steps marked.

tion matrix that defines TRPO's trust region is the same object that appears in the Cramér-Rao bound: it measures the statistical precision of the policy parameterization. The natural gradient adapts step sizes to this precision, taking large steps in well-identified directions and small steps where the data provide little information about the policy.

5.5 Hybrid Methods

REINFORCE estimates policy gradients from sample returns (unbiased, high variance); TD methods use bootstrapped targets $r + \gamma V(s')$ (lower variance, biased when V is approximate). Actor-critic methods combine both. The critic estimates the value function, the actor updates the policy using the critic's estimates.

5.5.1 Actor-Critic Architecture and Two-Timescale Convergence

The theoretical foundation is two-timescale stochastic approximation (Konda and Tsitsiklis, 2000), building on the two-timescale ODE convergence theory of Borkar (1997).[100] Run two concurrent learning processes:

$$\text{Critic (fast):} \quad \theta_{t+1} = \theta_t + \alpha_t^{(c)} \delta_t \nabla_\theta \hat{V}(s_t; \theta_t), \tag{58}$$

$$\text{Actor (slow):} \quad \omega_{t+1} = \omega_t + \alpha_t^{(a)} \delta_t \nabla_\omega \log \pi_\omega(a_t | s_t), \tag{59}$$

where $\delta_t = r_t + \gamma \hat{V}(s_{t+1}; \theta_t) - \hat{V}(s_t; \theta_t)$ is the TD error. The critic updates the value function parameters θ; the actor updates the policy parameters ω.

[100] The original analysis of Konda and Tsitsiklis (2000) uses the average-cost formulation with TD error $\delta_t = c(X_t, U_t) - \Lambda + V(X_{t+1}) - V(X_t)$, where Λ is the average cost. The discounted variant presented here follows by replacing the average-cost baseline with $\gamma V(s')$. A related but distinct ODE stability framework for single-timescale stochastic approximation, including Q-learning and TD learning, appears in Borkar and Meyn (2000).

Convergence requires the critic to learn faster than the actor.

$$\lim_{t\to\infty} \frac{\alpha_t^{(a)}}{\alpha_t^{(c)}} = 0, \quad \text{with both satisfying Robbins-Monro conditions.} \tag{60}$$

Under this separation, the actor sees a quasi-stationary critic: from the actor's perspective, the critic provides approximately correct value estimates at each step.[101] The actor's updates are then approximately unbiased policy gradient steps. Konda and Tsitsiklis (2000) prove convergence to a stationary point of $J(\omega)$ (i.e., $\nabla J(\omega) \to 0$).

Convergence requires a structural condition on the critic. The critic's feature vectors must span the actor's score functions $\nabla_\omega \log \pi_\omega(a|s)$, so that the critic's approximation error lies orthogonal to the policy gradient direction. Under this compatibility condition, the critic's projection error does not bias the actor's gradient estimates.[102]

A2C (Advantage Actor-Critic) is the synchronous variant: collect a batch of transitions, compute TD errors, and update both networks. A3C (Mnih et al., 2016) parallelizes this across multiple workers updating a shared parameter server asynchronously.[103]

5.5.2 Entropy Regularization and Soft Actor-Critic

SAC (Soft Actor-Critic) (Haarnoja et al., 2018) extends the actor-critic framework with entropy regularization, building on the soft Q-learning algorithm of Haarnoja et al. (2017). The agent maximizes the entropy-augmented objective:

$$J_\tau(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t \left(R_t + \tau \mathcal{H}(\pi_\theta(\cdot|s_t))\right)\right], \tag{61}$$

where $\mathcal{H}(\pi) = -\sum_a \pi(a) \log \pi(a)$ is the entropy and $\tau > 0$ is the temperature parameter. Geist et al. (2019) later provided the unifying theoretical framework, showing that entropy regularization converts the Bellman optimality operator's non-smooth hard max into a smooth log-sum-exp, and that the resulting soft Bellman operator remains a γ-contraction, preserving the convergence guarantees of standard dynamic programming.[104]

Entropy regularization also addresses the deadly triad directly. By maintaining policy stochasticity, the behavior policy used for data collection remains close to the target policy being optimized. This reduces the distribution mismatch between the stationary distribution under the behavior policy and the update targets, mitigating the off-policy instability leg of the

[101] The two-timescale structure is analogous to nested optimization in structural estimation, where an inner loop solves for equilibrium given parameters and an outer loop searches over parameters. The critic's inner loop (policy evaluation) must converge before the actor's outer loop (policy improvement) takes a step. Wu et al. (2020) provide finite-time convergence rates ($\tilde{O}(\epsilon^{-2.5})$ sample complexity) for two-timescale actor-critic with linear approximation, and Tian et al. (2023) establish analogous rates for single-timescale actor-critic with multi-layer neural networks.

[102] The compatible function approximation theorem first appears in Sutton et al. (2000) and is the key structural requirement in Konda and Tsitsiklis (2000). It constrains the critic architecture to be "compatible" with the actor parameterization, the same condition that makes the natural policy gradient equal to the critic's weight vector in Kakade (2002).

[103] Parallel workers decorrelate gradient estimates by exploring different parts of the state space simultaneously, removing the need for an experience replay buffer. Rigorous convergence theory for A3C's lock-free asynchronous parameter updates remains an open problem; existing analyses of asynchronous stochastic approximation (Qu and Wierman, 2020) address classical asynchrony (different state-action pairs updated at different times on a single trajectory), not the parallel-worker gradient setting.

[104] The optimal policy under entropy regularization is $\pi^*(a|s) \propto \exp(Q^*(s,a)/\tau)$, which is precisely the McFadden (1974a) logit choice probability with systematic utility $Q^*(s,a)$ and scale parameter τ. The entropy-regularized value function is the log-sum-exp of Q-values, corresponding to the inclusive value (log-sum) operator in nested logit models. This equivalence between the soft-control framework and dynamic discrete choice models with EV1 taste shocks is developed formally in Rust and Rawat (2026), Appendix A.

triad. Cen et al. (2022) formalize a second benefit: entropy regularization accelerates convergence of the natural policy gradient from $O(1/\epsilon)$ to $O(\log(1/\epsilon))$, providing a precise sense in which smoothing the policy landscape aids optimization. The actor-critic structure separates identification from optimization: the critic solves a regression problem (estimate V^π from data), while the actor solves an optimization problem (improve π using the estimated values).

5.5.3 Error Amplification Under Approximate Value Functions

Two questions remain important: how does approximation error propagate to policy quality, and how does computational complexity scale with problem size? Singh and Yee (1994) bound the policy degradation from value function errors (an independent derivation appears in Bertsekas and Tsitsiklis 1996, Proposition 6.1). If $\hat{V}$ approximates V^* with error $\|\hat{V} - V^*\|_\infty \leq \epsilon$, and $\hat{\pi}$ is the greedy policy with respect to $\hat{V}$, then:

$$\|V^* - V^{\hat{\pi}}\|_\infty \leq \frac{2\gamma}{1-\gamma}\epsilon. \tag{62}$$

At $\gamma = 0.99$, the amplification factor is $2 \cdot 0.99/0.01 = 198$. A 1% error in value function approximation yields at most 198% error in policy value.[105] This bound is pessimistic but finite: approximate value functions do not cause unbounded policy degradation.

5.5.4 Sample Complexity of Planning

Classical dynamic programming complexity scales with the state space size $|\mathcal{S}|$. For problems like Go, where $|\mathcal{S}| \approx 10^{170}$, exact computation is impossible. Kearns et al. (2002) prove that with access to a generative model[106] (a simulator that samples transitions from any state-action pair), near-optimal planning is possible with *no dependence on* $|\mathcal{S}|$. The cost is exponential dependence on the effective horizon $H = \log(R_{\max}/(\epsilon(1-\gamma)))/\log(1/\gamma)$: the sparse sampling algorithm requires $O((|\mathcal{A}|/\epsilon)^H)$ simulator calls.[107] For γ near 1, $H \approx (1-\gamma)^{-1}\log(R_{\max}/\epsilon)$, so the method is practical only for short effective horizons or moderate discount factors. Classical DP scales linearly in $|\mathcal{S}|$ but polynomially in H; sparse sampling eliminates state-space dependence at the cost of exponential horizon dependence. This explains why MCTS succeeds in large state spaces with bounded lookahead.

The minimax-optimal sample complexity for planning with a generative model, when queries to arbitrary state-action pairs are permitted, is $\Theta(|\mathcal{S}||\mathcal{A}|/((1-\gamma)^3\epsilon^2))$ (Azar et al., 2013). This bound scales linearly in $|\mathcal{S}|$ but polynomially in $1/(1-\gamma)$, the opposite regime from sparse sampling. Agarwal et al. (2020a) show that the plug-in model-based approach (learn $\hat{P}$ from samples, then plan with $\hat{P}$) achieves this minimax rate, establishing that model-based RL is statistically optimal. Li et al. (2024b) further tighten this result by breaking the $|\mathcal{S}||\mathcal{A}|/(1-\gamma)^2$

[105] The Singh-Yee bound is worst-case and not tight in general; massoud Farahmand et al. (2010) derive tighter bounds under smoothness assumptions on the MDP. The amplification factor $2\gamma/(1-\gamma)$ is a sensitivity analysis: it quantifies how errors in the "inputs" (value estimates) propagate to "outputs" (policy quality), analogous to errors-in-variables bias in regression. The discount factor γ controls sensitivity; more patient agents face larger amplification.

[106] A "generative model" in RL is a simulator that, given any state-action pair (s, a), returns a sampled next state $s' \sim P(\cdot|s, a)$ and reward r. This is unrelated to "generative models" in machine learning (GANs, diffusion models) or "generative processes" in Bayesian econometrics. The distinction matters: planning with a generative model is strictly easier than learning from a single trajectory, because the agent can query arbitrary states rather than following a sequential path.

[107] The sparse sampling algorithm builds a random tree of depth H from the current state, sampling C successor states per action at each node, then estimates values by averaging leaf rewards back up the tree. Its running time is $O((C|\mathcal{A}|)^H)$, which is exponential in H but entirely independent of $|\mathcal{S}|$. Kearns et al. (2002) also prove a lower bound of $\Omega(2^H)$ generative model calls for any planning algorithm, so exponential horizon dependence is unavoidable in the worst case.

sample-size barrier, showing that the minimax rate is achievable with total sample size as low as $|\mathcal{S}||\mathcal{A}|/(1-\gamma)$.[108]

5.6 Fundamental Tradeoffs

The choice between methods involves distinct tradeoffs rooted in DP structure. Value-based methods target Q^* via the Bellman contraction (Szepesvári, 2010). In the tabular case convergence is guaranteed, but function approximation introduces the deadly triad. Policy-based methods optimize π_θ directly. Modern theory (Agarwal et al., 2021a) establishes global convergence for softmax policies, with high variance as the practical weakness rather than local traps. Actor-critic methods combine both (Konda and Tsitsiklis, 2000), using the critic for low-variance value estimates while the actor inherits policy gradient's global convergence. Each family traces to DP foundations.

Four additional trade-offs pervade reinforcement learning. First, *exploration versus exploitation*: should the agent act on its current best estimate or gather information to improve future decisions? Lai and Robbins (1985) establish the fundamental lower bound: any consistent policy must incur regret at least logarithmic in the number of periods. Naive exploration (ε-greedy) requires samples exponential in the horizon; strategic exploration (UCB, optimism in the face of uncertainty) reduces this to polynomial (Auer et al., 2002a), formalizing the value of targeted experimentation.[109]

Second, *model-based versus model-free*: model-based methods learn a transition model $\hat{P}(s'|s,a)$ and plan with it (Sutton, 1990); model-free methods learn value functions or policies directly from transitions. The Dyna architecture (Sutton, 1990) bridges these by generating simulated experience from the learned model to supplement real transitions. Model-based methods are sample-efficient (each transition updates the entire model, which improves value estimates for all states) but suffer asymptotic bias if the model class is misspecified; model-free methods are asymptotically unbiased but sample-inefficient, using each transition for a single gradient step.[110]

Third, *on-policy versus off-policy* (Sutton and Barto, 2018, Ch. 5–7): on-policy methods (SARSA, REINFORCE) learn from data generated by the current policy, ensuring stability but discarding past experience; off-policy methods (Q-learning, DQN) reuse stored experience via replay buffers, gaining sample efficiency but risking the instabilities of the deadly triad.

Fourth, *bias versus variance in advantage estimation*: REINFORCE uses the full Monte Carlo return (unbiased but high variance); actor-critic methods (Konda and Tsitsiklis, 2000) use bootstrapped TD targets (low variance but biased by the critic's approximation error). Generalized Advantage Estimation in PPO (Schulman et al., 2015) interpolates between these extremes via a parameter $\lambda \in [0,1]$, where $\lambda = 1$ recovers Monte Carlo returns and $\lambda = 0$ recovers one-step TD. The value function baseline is the variance-minimizing choice, motivating the actor-critic architecture as a bias-variance compromise.

[108] Recent extensions push these results beyond standard MDPs. Clavier et al. (2024) study the robust MDP setting where the agent must plan under model uncertainty with sa-rectangular or s-rectangular uncertainty sets, establishing minimax rates for robust policy optimization. Wang et al. (2025c) extend the analysis to risk-sensitive objectives under the iterated CVaR criterion.

[109] The exploration-exploitation tradeoff is the subject of the bandits chapter, where the multi-armed bandit framework provides the sharpest analysis. In full MDPs, exploration is harder because the agent must learn not just reward distributions but also transition dynamics, compounding the information requirement.

[110] Model misspecification in RL is the analog of omitted variable bias in econometrics: if the learned model omits relevant state variables or misspecifies the functional form of transitions, the resulting policy is biased regardless of sample size. Moerland et al. (2023) provide a comprehensive survey of model-based RL, analyzing the model-bias versus sample-efficiency tradeoff across method families.

5.7 Conclusion

RL algorithms are not mysterious. They are asymptotic approximations to classical dynamic programming operators, justified by the mathematics of contractions, stochastic approximation, and gradient domination. Value iteration becomes Q-learning (Watkins and Dayan, 1992a; Tsitsiklis, 1994) when expectations are replaced by single samples. Policy iteration becomes the natural policy gradient (Kakade, 2001; Agarwal et al., 2021a) when the greedy improvement step is approximated by gradient ascent, and NPG recovers PI exactly in the tabular case. The stochastic approximation framework, from the foundational work of Robbins (1952) through the ODE method of Borkar and Meyn (2000), guarantees that under appropriate step-size conditions, noisy iterates converge to the same fixed points as their deterministic counterparts. Reinforcement learning is not a departure from dynamic programming but an extension of it. Tabular RL and RL with linear function approximation rest on solid theoretical foundations. Deep RL lacks comparable guarantees: convergence remains an open problem, and empirical successes remain case-specific.

6 The Empirics of Deep Reinforcement Learning

I review the empirical pathologies of deep reinforcement learning, their causes, and the diagnostic tools that expose them.

6.1 The Moving Target Problem

In supervised learning, the loss function is a *fixed function* of the training data and model parameters. Deep reinforcement learning does not enjoy this property. Each gradient step moves both the current value estimates and the targets simultaneously, creating a "nonstationary" optimization landscape. The target network heuristic introduced by Mnih et al. (2015) slows target drift by periodically freezing a copy of the network, but does not eliminate it. Therefore Bellman residual is a poor proxy for the accuracy of the value function.

Fujimoto et al. (2022) formalize this observation. Let Q^π denote the true value function for policy π, let $\Delta(s,a) = Q(s,a) - Q^\pi(s,a)$ denote the value error, and let $\varepsilon(s,a) = Q(s,a) - (r + \gamma \mathbb{E}_{s',a'}[Q(s',a')])$ denote the Bellman error. Substituting the definition of Q^π yields the key identity: $\varepsilon(s,a) = \Delta(s,a) - \gamma \mathbb{E}_{s',a'}[\Delta(s',a')]$. The Bellman error is a *difference* of value errors at consecutive states, not the value error itself. If the errors $\Delta(s,a)$ and $\Delta(s',a')$ are correlated across time—the network is wrong in the same direction at successive state-action pairs—they cancel in the difference and the Bellman error is small regardless of how large the individual errors are.[111]

The second failure mode is specific to finite datasets: Fujimoto et al. (2022, Corollary 1) show that over an incomplete dataset, zero Bellman error is consistent with arbitrarily large value error, because the network can fit unobserved successor pairs to whatever values make the observed residuals vanish.[112]

The dual failure mode appears in policy gradient methods: Ilyas et al. (2020) find that even when PPO's surrogate objective improves monotonically, episode return can plateau or decline, because the surrogate gradient is poorly aligned with the gradient of the true return.[113]

[111] In the extreme case, shifting all Q-values by the constant $c/(1-\gamma)$ leaves the Bellman error unchanged at zero while increasing value error by $c/(1-\gamma)$.

[112] The Bellman equation uniquely identifies Q^π when enforced over the entire MDP, but over an incomplete dataset it admits infinitely many solutions. Whenever a transition (s',a') that is reachable from the dataset is not itself in the dataset, the network is free to set $Q(s',a')$ at unobserved pairs to whatever value makes the residual vanish on the observed ones, unconstrained by any loss. A network can thus reach near-zero training loss while the value function remains arbitrarily inaccurate over the full state space.

[113] Ilyas et al. (2020) examine the surrogate objective used in Proximal Policy Optimization (Schulman et al., 2017). The PPO clipping mechanism is designed to keep policy updates within a trust region by bounding the

6.2 The Reproducibility Crisis and Sensitivity to Random Seeds

Henderson et al. (2018) trained five leading policy gradient algorithms (PPO, TRPO, DDPG, TD3, SAC) on six MuJoCo benchmark environments, holding all hyperparameters fixed and varying only the random seed. The resulting learning curves from different seeds were non-overlapping: a seed that performed well under one algorithm performed comparably to a different algorithm's best seeds, making cross-algorithm comparison unreliable.[114] Agarwal et al. (2021b) quantify the damage: comparing point estimates from 5 runs per task on Atari 100k yields Type I error exceeding 50%, meaning a random noise injection appears beneficial in half of all comparisons.

Agarwal et al. (2021b) propose the *interquartile mean* (IQM) as a replacement for mean and median when comparing algorithms. The IQM discards the top and bottom 25% of runs before averaging, reducing sensitivity to outlier seeds. Using these tools and stratified bootstrap confidence intervals, Agarwal et al. (2021b) find that several widely-cited algorithmic improvements on Atari 100k vanish or reverse when statistical uncertainty is accounted for.[115][116]

6.3 Value Overestimation and Spikes

Q-learning uses the Bellman optimality update

$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a'), \tag{63}$$

where the maximum is taken over the estimated Q-values of all actions at the successor state. Thrun and Schwartz (1993) identify a positive bias intrinsic to this update: if the Q-value estimates contain noise with mean zero, the maximum over noisy estimates is biased upward by Jensen's inequality. An agent that uses a single network for both action selection ($\arg\max$) and value estimation ($\max Q$) systematically overestimates the values of every state it visits, biasing the Bellman target upward at every update step. The bias compounds through bootstrapping: overestimated targets produce overestimated updates, which produce further overestimated targets.

van Hasselt (2010) propose double Q-learning as a remedy: maintain two independent Q-networks Q_A and Q_B. Use Q_A to select the greedy action at s', but use Q_B to evaluate that action. Because the two networks are trained on different data, their errors are approximately independent, and the positive bias largely cancels. van Hasselt et al. (2016b) implement this as Double DQN, using the online network for action selection and the periodically-frozen target network for evaluation. On 49 Atari games, Double DQN reduces overestimation by a factor of 3–5 and improves median performance by 20% relative to DQN.

Fujimoto et al. (2018) observe that Double DQN's correction is incomplete in continuous-action settings, where the target network and online network remain correlated through shared updates. They propose Clipped Double Q-learning: compute two Q-value estimates Q_1, Q_2 with separate networks trained on the same data, and use $y = r + \gamma \min(Q_1(s', a'), Q_2(s', a'))$

probability ratio $\pi_\theta(a|s)/\pi_{\theta_{\text{old}}}(a|s)$. The gradient of the surrogate is poorly aligned with the gradient of the true return, particularly in later training phases where the policy has diverged from the behavior policy used to collect the replay data. The loss metric that practitioners monitor throughout training is measuring something other than what they care about.

[114] Differences as large as 2,000 points in final episode return arose from seed variation alone.

[115] They also introduce performance profiles, which plot the fraction of tasks and seeds where an algorithm achieves performance above a threshold τ, as τ varies from 0 to the maximum. Performance profiles reveal the full shape of the score distribution rather than collapsing it to a single statistic.

[116] The fragility extends to hyperparameters. Eimer et al. (2023) conduct a systematic study of hyperparameter sensitivity across 6 algorithms and 17 environments, finding that default hyperparameters from published papers perform competitively in the specific environments used in those papers but generalize poorly across environments. Patterson et al. (2024) synthesize these findings into an empirical design handbook, recommending at minimum 10 seeds per configuration, IQM-based comparisons, and preregistration of hyperparameter search protocols.

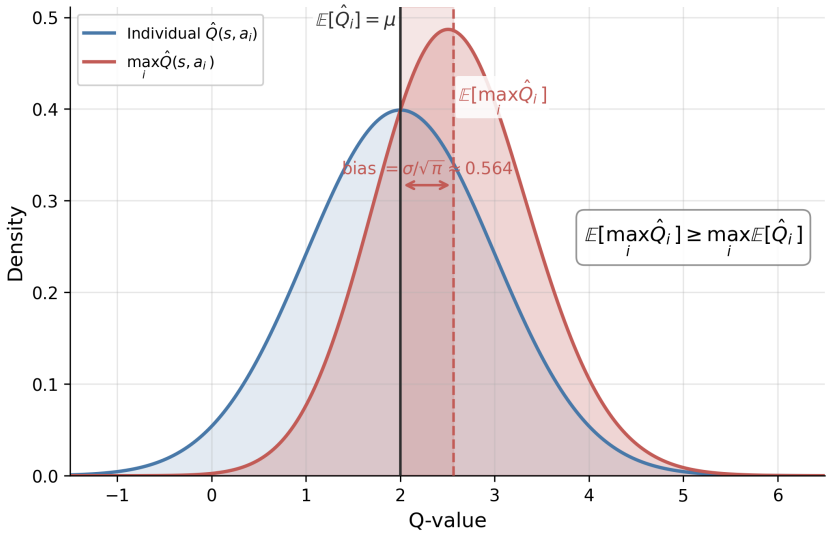

Figure 12: Overestimation bias from Jensen's inequality with $n = 2$ actions. Blue: density of individual $\hat{Q}(s, a_i) \sim \mathcal{N}(\mu, \sigma^2)$. Red: density of $\max_i \hat{Q}(s, a_i)$, shifted right by $\sigma/\sqrt{\pi} \approx 0.56$. The shaded gap is the bias. With $n = 100$ actions the bias exceeds 2.5σ.

as the Bellman target. The minimum operator introduces pessimistic underestimation, which is conservative but avoids the explosive positive bias.[117]

DQN prevents outright divergence, but van Hasselt et al. (2018) find that *soft divergence*, defined as a temporary spike in value estimates by more than 10% followed by recovery, occurs in the majority of DQN training runs across 57 Atari games.[118] The deadly triad does not announce itself as a training failure: the value estimates may spike and recover, leaving no visible trace in the loss curve while corrupting the policy.

Kumar et al. (2021) describe a continuous manifestation of the deadly triad: *implicit under-parameterization*.[119]

6.4 Plasticity Loss and Primacy Bias

A network that trains well at step $t = 100$ may be incapable of learning at step $t = 100{,}000$, even if the data quality at the later step is higher. Lyle et al. (2022) call this *capacity loss*: the network's ability to update its own weights degrades progressively during training, measured by the fraction of effective parameters and the network's ability to fit new random labels. Lyle et al. (2023) extend this to *plasticity loss*, identifying dead ReLU neurons, weight norm growth, and feature rank collapse as three distinct mechanisms, not all of which co-occur.

Nikishin et al. (2022) identify *primacy bias* as a specific cause. Because the replay buffer is filled incrementally, early transitions are oversampled relative to later ones, and the network over-fits to early environment transitions, corrupting representations throughout the remainder of training.[120]

[117] Ciosek et al. (2019) note that clipped double Q can cause excessive pessimism under high uncertainty, proposing optimistic actor-critic as a counterweight.

[118] Larger networks diverge more frequently than smaller ones, counter to the usual intuition that more expressive models should generalize better.

[119] Neural networks trained with bootstrapped TD objectives progressively lose their effective rank, with fewer and fewer neurons contributing distinct directions in the representation. This rank collapse is silent (training loss continues to decrease) but the network's ability to represent new information degrades over time, a form of capacity loss distinct from but related to plasticity loss.

[120] Nikishin et al. (2022) show that 100 initial "priming" steps of excessive gradient updates degrade a SAC agent's performance for hundreds of thousands of subsequent steps. Sokar et al. (2023) measure the fraction of dormant neurons (units with near-zero activation across the replay buffer) accumulating monotonically during

The proposed remedies fall into three categories: periodic resets, continual backpropagation, and architectural interventions.[121]

6.5 Implementation Dominates Algorithmic Innovation

Engstrom et al. (2020) find that PPO with the clipping mechanism disabled performs indistinguishably from the full algorithm; TRPO (Schulman et al., 2015) with the same code-level additions matches PPO.[122] Andrychowicz et al. (2021) identify observation normalization, orthogonal initialization, and learning rate annealing as the three choices accounting for most variance across 250,000 agents and 250 hyperparameter configurations. Huang et al. (2022) catalog 37 implementation details required to reproduce PPO on Atari;[123] omitting any produces materially different results.

The most consequential implementation detail is the distinction between termination and truncation (Pardo et al., 2018). In reinforcement learning, an episode can end for two distinct reasons: *termination*, where the environment reaches a natural absorbing state (the pole falls in CartPole, the robot falls in locomotion tasks), and *truncation*, where the episode is cut short by an external time limit. At a termination, the value of the successor state is zero: $V(s_{\text{term}}) = 0$. At a truncation, the episode is merely paused and the successor state has non-zero value: $V(s_{\text{trunc}}) \neq 0$. Treating truncated transitions as terminated substitutes zero for a non-zero bootstrap value at every time limit boundary, corrupting every Bellman update in the vicinity of episode boundaries. Pardo et al. (2018) show that this conflation degrades performance by 20–40% on standard MuJoCo benchmarks.[124]

6.6 Replay Buffer Pathologies and Reward Scaling

Experience replay (Lin, 1992) decouples the data collection and learning processes, allowing a single transition to be used for multiple gradient updates. The replay ratio (the number of gradient updates per environment step) governs the trade-off between sample efficiency and data staleness. Zhang and Sutton (2017) show that increasing the replay ratio beyond a modest threshold degrades performance.[125]

Schaul et al. (2016) propose prioritized experience replay (PER).[126] Fedus et al. (2020) revisit PER on large-scale Atari experiments and find that uniform sampling from a large enough buffer matches or outperforms PER, while being simpler to implement and tune.

Reward scaling introduces a separate class of failure modes. Standard DQN (Mnih et al., 2015) clips rewards to $[-1, +1]$ across all environments to stabilize training. van Hasselt et al.

training. Dohare et al. (2024) find that standard deep networks lose all plasticity within a few million gradient steps in continual learning tasks.

[121] Periodic resets reinitialize the last layers of the network while retaining the replay buffer, allowing the agent to forget overfit representations without discarding experience (Nikishin et al., 2022; D'Oro et al., 2023). Continual backpropagation replaces neurons with near-zero utility at each gradient step rather than waiting for a global reset (Dohare et al., 2024). Architectural interventions—layer normalization (Lyle et al., 2025), orthogonal initialization, spectral normalization—reduce the rate at which plasticity is lost by stabilizing gradient magnitudes and preventing weight norm growth.

[122] The non-clipping components that suffice are: observation normalization, reward normalization, value function clipping, global gradient norm clipping, orthogonal weight initialization, and the Adam optimizer.

[123] These include reward clipping to $[-1, 1]$, frame stacking to 4 frames, a specific episode termination convention, and a numerically stable normalization of the advantage estimate.

[124] The Gymnasium API (Towers et al., 2024) enforces the distinction by returning separate `terminated` and `truncated` flags, but most pre-2022 codebases conflate them in the `done` flag.

[125] At high replay ratios, the distribution shift between the current policy and the behavior policy that generated the stored data grows large enough to violate the off-policy assumptions of Q-learning. The relationship is non-monotone and environment-dependent, making replay buffer size a sensitive hyperparameter with no universal default.

[126] PER samples transitions with probability proportional to the magnitude of their TD error, on the argument that high-error transitions are the most informative.

(2016a) observe that reward clipping changes the objective: clipped rewards make all positive events equivalent regardless of magnitude, so the agent learns to maximize the frequency of positive events rather than their cumulative value. This substitution can produce policies that are locally rational under the clipped reward but qualitatively suboptimal under the true reward. van Hasselt et al. (2016a) propose PopArt as a remedy.[127]

Skalse et al. (2022) formalize reward hacking and show that any non-constant proxy reward can in principle be exploited by a sufficiently capable optimizer.[128]

6.7 Simulation Study: Bellman Error and Value Error in Offline Policy Evaluation

The MDP uses $s = (k, z)$ with k on a 50-point log-spaced capital grid and $z \in \{0.9, 1.1\}$ following a Markov chain with persistence 0.8. Actions are next-period capital choices on the same grid. Reward is $\log(zk^\alpha - k')$ with $\alpha = 0.36$, $\beta = 0.96$. Rewards are shifted by $-\bar{r}$ (mean reward over feasible pairs) to center Q^* near zero, which avoids initialization issues without altering the optimal policy. The offline dataset $\mathcal{D}$ consists of $T = 2{,}000$ transitions simulated from the closed-form optimal policy $k^*(k, z) = \alpha\beta z k^\alpha$. Since the optimal policy concentrates capital near its steady state, $\mathcal{D}$ covers only 11 of the 4,795 feasible (s, a) pairs—0.2% coverage—the distribution mismatch condition in Fujimoto et al. (2022, Corollary 1).

Two algorithms are trained on $\mathcal{D}$.[129] BRM minimizes $\mathbb{E}_{\mathcal{D}}[(Q(s,a) - (r + \gamma \max_{a'} Q(s', a')))^2]$ where both $Q(s, a)$ and $Q(s', a')$ use the current network; gradients flow through both sides simultaneously. The key consequence is that the network can zero the residual at an observed pair (s, a) by co-moving $Q(s, a)$ and $Q(s', a')$ together, rather than moving either toward Q^*. This is the opposite of supervised learning, where labels are fixed external targets that do not move with the model weights. FQE prevents this by using a frozen target network for $Q(s', a')$; the network must reduce $Q(s, a)$ toward a target that does not respond to its own gradient steps. Every 500 steps we record the Bellman error on $\mathcal{D}$ (current network on both sides) and the value error $\frac{1}{|\mathcal{F}|} \sum_{(s,a)\in\mathcal{F}} |Q_\theta(s, a) - Q^*(s, a)|$ over all 4,795 feasible pairs. As a supervised baseline, OLS regression of $\log c$ on $\log k$ and $\log z$ is estimated on expanding windows.[130]

The OLS baseline shows tight coupling between training and test loss (Pearson $r = -1.000$; Table 8). The RL results reproduce Fujimoto et al. (2022) in the economic model: BRM achieves Bellman error 816× lower than FQE, yet both methods have nearly identical value error over the full MDP, yielding a VE/BE ratio three orders of magnitude higher for BRM. Both mechanisms from Section 6.1 operate: error cancellation on the 11 observed pairs and unconstrained $Q(s', a')$ at the 4,784 unobserved pairs.

6.8 Discussion and Recommendations

Track episode return and policy entropy alongside training loss; entropy collapse and stagnating return are early warning signs of plasticity loss (Section 6.4). Use PPO or SAC as default baselines before implementing custom algorithms. Report at least 10 seeds per configuration with IQM-based comparisons (Section 6.2).

[127] PopArt normalizes targets to have unit variance while adjusting the output layer so that the policy remains invariant to the normalization. PopArt allows consistent learning across reward scales spanning several orders of magnitude without reward clipping.

[128] Skalse et al. (2022) define a proxy as *unhackable* if increasing expected proxy return cannot decrease expected true return. Their main result states that for the set of all stochastic policies, two reward functions are unhackable only if one of them is constant. For deterministic policies and finite policy sets, non-trivial unhackable pairs exist, but the conditions are stringent.

[129] 50,000 gradient steps per seed, 3 seeds; two-layer MLP with 64 hidden units, Adam at 5×10^{-4}; target network updated every 500 steps.

[130] Noise $\sigma = 0.30$; windows from $n = 10$ to 2,000; held-out test set of 500 transitions.

Method	Final BE on $\mathcal{D}$	Final VE (all states)	VE/BE ratio
BRM (seed 42)	2.09×10^{-4}	7.061	33,808
BRM (seed 123)	2.91×10^{-4}	7.450	25,630
BRM (seed 777)	3.12×10^{-4}	7.278	23,320
BRM (mean)	2.71×10^{-4}	7.263	27,586
FQE (seed 42)	0.2037	6.927	34
FQE (seed 123)	0.2173	7.465	34
FQE (seed 777)	0.2425	7.281	30
FQE (mean)	0.2212	7.224	33
OLS ($n = 2{,}000$)	OOS MSE $= 0.096$	OOS $R^2 = 0.112$	$r = -1.000$

Table 8: Bellman error on dataset $\mathcal{D}$ and value error on the full MDP for BRM and FQE trained on offline Brock–Mirman data. BE is mean squared Bellman error evaluated with the current network on both sides; VE is mean absolute deviation from Q^* over all 4,795 feasible state-action pairs.

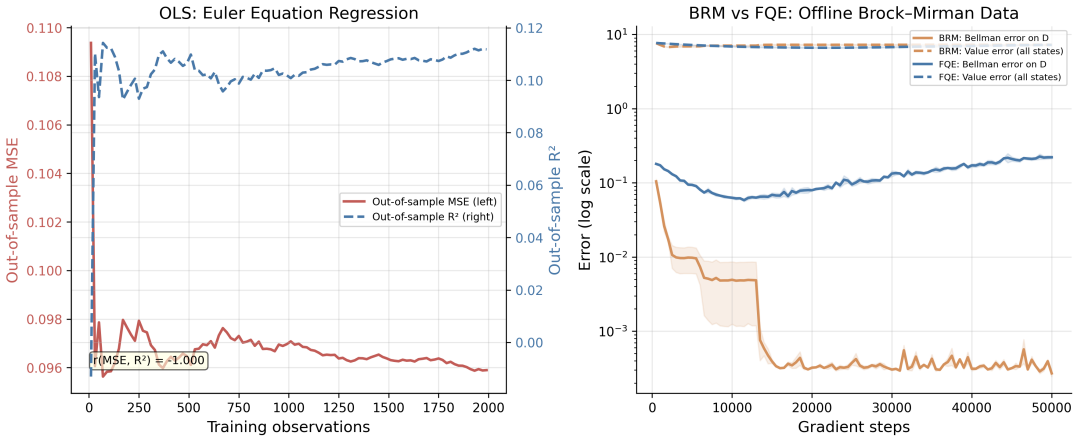

Figure 13: Left: OLS regression of log consumption on log capital and log productivity, estimated on expanding windows from the Brock–Mirman optimal policy. Out-of-sample MSE (left axis, red) and out-of-sample R^2 (right axis, blue) track each other with Pearson $r = -1.000$. Right: BRM (orange) and FQE (blue) trained on offline Brock–Mirman data $\mathcal{D}$ ($T = 2{,}000$ transitions, 0.2% state-space coverage); note the log scale on the y-axis. Solid lines show Bellman error on $\mathcal{D}$ (current network both sides); dashed lines show mean absolute value error against Q^* over all 4,795 feasible state-action pairs; shaded bands are ± 1 SE over 3 seeds.

7 Reinforcement Learning for Optimal Control

A handful of organizations have deployed reinforcement learning beyond simulation, achieving measurable gains on specific large-scale problems. Each deployment required substantial domain engineering and scientific tuning; these remain exceptional cases rather than standard practice. I review the most prominent field deployments, including ride-hailing dispatch at DiDi, data center cooling at Google, hotel revenue management, and financial order execution, before concluding with a simulation study on the bus engine replacement problem. In each case, the RL agent's parameters were updated during a training phase conducted in simulation or on historical data, and the resulting policy was deployed with fixed weights (Section 2), with the exception of the hotel revenue management system, which learned in-field.

7.1 Ride-Hailing Dispatch

Each driver-passenger assignment changes the spatial distribution of available drivers, making ride-hailing dispatch a sequential optimization problem at a scale (tens of millions of daily rides at DiDi) where exact dynamic programming is intractable.

Qin et al. (2021) formalized DiDi's order dispatching as a semi-Markov decision process[131] where each driver is an independent agent. The state of a driver consists of location (discretized into hexagonal zones) and time (bucketed into intervals). The action is the order assigned to the driver, with the option to remain idle. The reward is the trip fare. State transitions are determined by trip destinations, as completing an order transports the driver from origin to destination, changing their spatial state. The stochasticity arises from future demand, which determines the available actions at each state. The per-driver value function $V^\pi(s)$ represents the expected cumulative fare a driver can earn from state s under dispatching policy π:

$$V^\pi(s) = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^{\tau_k} r_k \mid s_0 = s, \pi\right], \tag{64}$$

where τ_k is the time to complete the k-th trip and r_k is the corresponding fare. The platform's objective is to maximize total driver income across the fleet, which decomposes into the sum of individual driver value functions.

Each dispatching window (a few seconds), the platform collects open orders and available drivers, constructs a bipartite graph, and solves a linear assignment problem. The edge weights are computed as the advantage of each driver-order match relative to the driver's current state value.

$$w_{ij} = \hat{r}_i + \gamma^{\hat{\tau}_i} V(s'_j) - V(s_j), \tag{65}$$

where $\hat{r}_i$ is the predicted fare for order i, $\hat{\tau}_i$ is the estimated trip duration, and s'_j is the destination state. The value function is learned via temporal-difference methods from historical trip data. DiDi's Cerebellar Value Network (CVNet; Tang et al., 2019) uses hierarchical coarse-coding[132] with a multi-resolution hexagonal grid, enabling transfer learning[133] across cities and robustness to data sparsity in low-traffic zones.

Production deployment across more than 20 Chinese cities demonstrated modest but consistent improvements of 0.5–2% on key metrics, gains that required the full CVNet infrastructure

[131] A semi-Markov decision process generalizes the standard MDP by allowing variable time between decisions. The discount factor γ^{τ_k} depends on the actual time elapsed τ_k rather than applying a fixed per-step discount.

[132] *Coarse-coding* represents a value function as a weighted sum over overlapping rectangular or hexagonal tiles that partition the state space (Sutton and Barto, 2018); each state activates the tiles that contain it, and a hierarchical variant stacks tiles at multiple resolutions to allow both coarse and fine generalization simultaneously.

[133] *Transfer learning* initializes a model for a new task (a new city) with parameters trained on related tasks (existing cities), on the assumption that learned representations of traffic and demand patterns generalize across contexts and require less data to reach good performance in the new setting.

(hierarchical coarse-coding, multi-resolution grids, transfer learning across cities) to achieve. Table 9 summarizes the reported gains from A/B tests using time-slice rotation, where algorithms alternate control of the platform in 3-hour blocks to avoid interference effects.

Table 9: DiDi dispatch deployment results from Qin et al. (2021) and Tang et al. (2019).

Metric	Improvement vs. Baseline	Scale
Total driver income	+0.5–2.0%	Tens of millions daily rides
Order response rate	+0.5–2.0%	Platform-wide
Fulfillment rate	+0.5–2.0%	Platform-wide

Li et al. (2019) addressed the coordination challenge using mean-field multi-agent RL. With thousands of drivers making simultaneous decisions, the full multi-agent state space is intractable. The mean-field approximation replaces individual driver states with an aggregate distribution, allowing each driver to condition on the density of nearby drivers rather than their exact locations. Experiments on DiDi data showed improved fleet utilization compared to single-agent baselines.

Han et al. (2022b) reported complementary results from Lyft, where the dispatching system optimizes driver assignment and repositioning jointly. Their value decomposition architecture[134] assigns credit to individual driver decisions within the global objective. The production system demonstrated improvements in rider wait times and driver utilization, providing a second data point suggesting that similar approaches may transfer across platforms.

At DiDi's volume, even 1% gains represent hundreds of thousands of additional completed rides per day, because dispatching is fundamentally a fleet positioning problem: today's assignments determine tomorrow's driver distribution.

7.2 Hotel Revenue Management

Budget hotel chains face a capacity allocation problem: how to dynamically distribute rooms across rate segments defined by discount level, with booking channels such as direct platforms and online travel agencies mapped to segments offering discounts ranging from less than 15% to over 40%. Demand is uncertain, cancellations are difficult to forecast, and hotel managers resist black-box optimization systems that override their judgment. Chen et al. (2023a) deployed reinforcement learning at China Lodging Group (CLG), a budget hotel chain operating approximately 2,000 hotels across China, and conducted field experiments measuring the impact of RL-based capacity allocation on actual hotel revenue.

Their system uses a two-step design. The RL agent observes the state (t, s), where t indexes the booking period within a $T = 10$-period episode and s is the average revenue per room sold to date, and selects an average discount level $a \in \{10\%, 20\%, 30\%\}$. A linear program then converts this scalar recommendation into a feasible capacity allocation across rate segments, accounting for each hotel's channel preferences.[135] This decomposition addresses practitioner acceptance, since managers understand a single discount recommendation, and circumvents the need to explicitly model demand arrivals or cancellation rates.

The field experiment randomly assigned five Hanting-brand hotels in Shanghai to the treatment group from ten candidates, with 271 additional Shanghai hotels serving as a donor pool

[134] Value decomposition decomposes the platform's global matching objective into individual driver value functions, each estimable via temporal-difference learning. The global optimum is recovered by optimizing the sum of individual values.

[135] The RL component uses a modified on-policy Monte Carlo method with ε-greedy exploration, updating Q-values from realized returns after each completed episode, making it an in-field learner in the terminology of Section 2.

for synthetic control estimation.[136] Table 10 reports the average treatment effects over the pilot period (March–June 2015).

Table 10: Field experiment results from Chen et al. (2023a), measured via synthetic control.

Metric	Improvement	p-value
Occupancy rate (OR)	+5.15%	0.1361
Average daily rate (ADR)	+5.93%	0.1160
Revenue per available room (RevPAR)	+11.80%	0.0090

The RevPAR gain is heterogeneous across treatment hotels; some improved primarily via higher occupancy rates, others via higher average daily rates, and others via both channels simultaneously.

7.3 E-Commerce Dynamic Pricing

E-commerce platforms face a pricing problem at a scale that defeats human specialists. Alibaba's Tmall.com lists millions of SKUs across thousands of product categories, each requiring daily price adjustments that account for demand elasticity, competitor behavior, inventory levels, and promotional calendars. Liu et al. (2019) deployed deep reinforcement learning agents for automated pricing on Tmall.com beginning July 2018, conducting field experiments on thousands of SKUs over several months.

The pricing MDP for product i has state $s_{i,t} \in \mathbb{R}^m$ comprising four feature groups: price features (current and historical prices, price-to-cost ratio), sales features (units sold, conversion rate), customer traffic features (unique visitors $\text{uv}_{i,t}$, page views), and competitiveness features (price rank among similar products). The pricing period is $d = 1$ day. Actions are either discrete, $a_{i,t} \in \{1, \ldots, K\}$ indexing price bins uniformly spaced between product-specific bounds $[P_{i,\min}, P_{i,\max}]$, or continuous, $a_{i,t} \in \mathbb{R}$. The reward is the difference of revenue conversion rates (DRCR):

$$r_{i,t} = \frac{\text{revenue}_{i,t}}{\text{uv}_{i,t}} - \frac{\text{revenue}_{i,t-\tau}}{\text{uv}_{i,t-\tau}}, \tag{66}$$

where $\text{uv}_{i,t}$ is unique visitors in period t and τ is a reference lag.[137]

Two algorithms are deployed: DQN for the discrete action formulation and DDPG for continuous actions. Both are pre-trained from demonstrations using historical specialist pricing actions (DQfD, DDPGfD) to address cold-start.[138][139]

DDPG with continuous action space outperformed DQN with discrete bins across all daily pricing experiments, and both substantially outperformed manual expert pricing.

[136] Synthetic control constructs a weighted combination of untreated hotels whose pre-treatment trend matches each treated hotel, avoiding the selection bias of simple before-after comparisons.

[137] DRCR normalizes revenue by traffic to remove demand fluctuations unrelated to pricing. The differencing further removes product-specific level effects, yielding a reward signal that is more concave than raw revenue and improves convergence stability.

[138] Pre-populating replay buffers with rollouts from heuristic or scripted policies is a general technique for reducing exploration burden in deep RL. In robotic grasping, Kalashnikov et al. (2018) collected 200,000 scripted grasping attempts at 15–30% success rate, sufficient to bootstrap a vision-based policy to 96% success; see Ibarz et al. (2021) for a survey of these and related warm-start strategies.

[139] Standard A/B testing is infeasible because Chinese e-commerce regulations prohibit displaying different prices to different customers for the same product simultaneously. Liu et al. (2019) instead evaluate using difference-in-differences against "simi-products" (similar products not subject to algorithmic pricing) as controls.

Table 11: Field experiment results from Liu et al. (2019) on Tmall.com. DRCR improvement is relative to the simi-product control group.

Experiment	Method	DRCR Improvement	Duration
Markdown (500 luxury SKUs)	DQN ($K = 9$)	+37.5%	15 days
Daily (1000 FMCGs)	DQN ($K = 100$)	5.10×	30 days
Daily (1000 FMCGs)	DDPG (continuous)	6.07×	30 days

7.4 Financial Order Execution

The theoretical benchmark for optimal execution is the Almgren-Chriss framework (Almgren and Chriss, 2001), which derives optimal deterministic schedules under linear impact assumptions. A trader liquidating Q shares over T periods faces a tradeoff between timing risk and market impact. With risk-aversion parameter λ, price volatility σ, and temporary impact coefficient η, the optimal remaining inventory at time t follows the hyperbolic sine schedule:

$$x^*(t) = Q \cdot \frac{\sinh(\kappa(T-t))}{\sinh(\kappa T)}, \quad \kappa = \sqrt{\frac{\lambda\sigma^2}{\eta}}. \tag{67}$$

This deterministic trajectory is the benchmark any adaptive method must beat. It prescribes front-loading or back-loading depending on the risk-impact balance, but cannot respond to realized order flow or spread dynamics.

Nevmyvaka et al. (2006) applied tabular Q-learning to execution on real limit order book data from NASDAQ stocks.[140] The state at time t is $s_t = (t, q_t, \psi_t, \Delta_t)$, where $t \in \{1, \ldots, T\}$ is time remaining, $q_t \in \{0, 1, \ldots, Q\}$ is inventory remaining, ψ_t is the discretized bid-ask spread, and Δ_t is the discretized signed volume imbalance.[141] Actions $a_t \in \{0, \delta, 2\delta, \ldots, q_t\}$ specify shares to execute in the current interval. The reward is the negative per-period slippage contribution:

$$r_t = -a_t(p_t^{\text{exec}} - p_0), \tag{68}$$

where p_0 is the mid-quote at order arrival and p_t^{exec} is the average execution price. Total implementation shortfall is $IS = -\sum_{t=1}^{T} r_t/Q$.[142] Because the objective is cost minimization, the Q-learning update uses min over next-period actions:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\Big[r_t + \gamma \min_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\Big]. \tag{69}$$

The agent learns to condition on market microstructure signals: trading aggressively when spreads are narrow, waiting when order flow predicts favorable price movement, and accelerating near the deadline.

The RL agent reduces execution costs by 12–19% over Almgren-Chriss. These results informed subsequent work on adaptive execution, though independently verified production deployments remain scarce in the public literature.

[140] The dataset covers 500 trading days of millisecond-level limit order book snapshots for AMZN, QCOM, and NVDA. The state space contains approximately 10,000 states; the horizon is $T = 60$ seconds with discount $\gamma = 1$.

[141] Signed volume imbalance is the difference between buy and sell order volume near the best prices, normalized by total volume; positive imbalance typically predicts short-term price increases.

[142] Implementation shortfall measures the total cost of executing a trade relative to the benchmark mid-quote at arrival. It captures market impact, timing cost, and opportunity cost of unexecuted shares.

Table 12: Execution results from Nevmyvaka et al. (2006) on NASDAQ stocks. TWAP is the time-weighted average price baseline (equal-sized trades at uniform intervals).

Stock	RL vs. TWAP	RL vs. Almgren-Chriss	Data
AMZN	−18.2%	−14.6%	500 days LOB
QCOM	−22.1%	−19.3%	500 days LOB
NVDA	−15.8%	−12.1%	500 days LOB

7.5 Supply Chain Inventory Management

Multi-echelon inventory systems coordinate ordering decisions across supply chain stages, where each stage's order becomes the next stage's incoming shipment. A retailer orders from a warehouse, which orders from a distribution center, which orders from a factory. The sequential nature creates complex dependencies, since an order placed upstream today affects downstream availability many periods later. The state space grows exponentially in the number of echelons, with K stages each having M possible inventory levels yielding M^K states. Classical inventory theory provides closed-form solutions for special cases, most notably the echelon base-stock policy of Clark and Scarf (1960).

The state $s = (I_1, \ldots, I_K)$ records on-hand inventory at each stage, where stage 1 faces customer demand and stage K is the most upstream. The action $q \in \{0, 1, \ldots, Q_{\max}\}$ is the order quantity placed at stage K. Demand $D_t \sim F_D$ arrives at stage 1 each period; unfilled demand is backordered. Shipments flow downstream, with stage k receiving what stage $k+1$ shipped in the previous period. The per-period cost combines holding, backorder, and ordering components.

$$c_t = h \sum_{k=1}^{K} I_k^+ + b \cdot (D_t - I_1)^+ + c_o \cdot q_t, \tag{70}$$

where $I_k^+ = \max(0, I_k)$ is on-hand inventory, $(x)^+ = \max(0, x)$, h is holding cost per unit, b is backorder cost per unit, and c_o is ordering cost. The objective is to minimize expected discounted total cost.

Gijsbrechts et al. (2022) conducted a systematic evaluation of deep RL against classical base-stock policies across lost sales, dual sourcing, and multi-echelon configurations. The classical benchmark is the echelon base-stock policy, in which each stage k maintains an echelon inventory position and orders to bring this position to a target level S_k.[143] For single-echelon systems with backorders, the optimal base-stock level is given by the newsvendor critical fractile $S^* = F_D^{-1}(b/(b+h))$. Table 13 summarizes results from their multi-echelon experiments.

Table 13: Multi-echelon inventory results from Gijsbrechts et al. (2022).

Echelons	DRL Cost Gap vs. Base-Stock	States	Convergence
2	+1.2%	$\sim 10^2$	Achieved
3	+6.8%	$\sim 10^3$	Achieved
4	+12.3%	$\sim 10^5$	Partial
6	—	$\sim 10^8$	Failed

Where classical solutions exist, RL underperforms: the base-stock policy remains highly competitive when properly calibrated, and DRL required millions of training transitions to

[143] Formally, the echelon inventory position at stage k equals on-hand inventory at k plus all inventory at downstream stages $1, \ldots, k-1$ plus in-transit inventory, minus backorders. The echelon base-stock policy of Clark and Scarf (1960) is optimal for serial systems with linear costs and backorders.

approach performance levels achievable through closed-form calculation. RL struggles particularly with multi-echelon coupling, where upstream orders affect downstream costs many periods later.[144] The value proposition for RL lies in problems where analytical solutions are unavailable: non-stationary demand, complex operational constraints, or cost structures that do not admit tractable decomposition.[145] Even in these settings, successful deployment remains rare and requires extensive simulation infrastructure, domain expertise, and careful calibration against classical baselines.

7.6 Real-Time Bidding

Real-time bidding for display advertising presents a budget pacing problem: an advertiser must allocate a fixed budget B_0 across a campaign of T auctions to maximize total conversions. Each impression is a second-price auction; the advertiser submits a bid and pays the second-highest bid if they win. The challenge is that bidding aggressively depletes budget early, while conservative bidding leaves value on the table.

Wu et al. (2018) formalized this as an MDP with state $s_t = (B_t, t, w_t)$, where B_t is remaining budget, t is auctions remaining, and w_t is the recent win rate. The action is a bid multiplier $\lambda_t \in [\lambda_{\min}, \lambda_{\max}]$, so the actual bid is $b_t = \lambda_t \cdot \bar{b}$, where $\bar{b}$ is a base bid calibrated to the estimated value-per-impression. The clearing price c_t is drawn from a distribution F_c estimated from historical data; the advertiser wins if $b_t \geq c_t$. The per-auction reward is:

$$r_t = \mathbb{1}\{b_t \geq c_t\} \cdot \text{cvr}_t, \tag{71}$$

where $\text{cvr}_t \in \{0, 1\}$ is the conversion indicator. Budget evolves as $B_{t+1} = B_t - \mathbb{1}\{b_t \geq c_t\} \cdot c_t$, and the episode terminates when $B_t \leq 0$ or $t = 0$.[146]

The agent is a deep Q-network trained on a simulator calibrated to production RTB data. Table 14 reports performance relative to rule-based pacing baselines. The DQN agent improves total conversions by conserving budget during high-competition periods and bidding aggressively when clearing prices are low, a pattern the rule-based approaches cannot learn.

Table 14: Real-time bidding results from Wu et al. (2018). Performance is relative to linear pacing on a simulator calibrated to production data.

Method	Conversions vs. Linear Pacing	Budget Utilization
Linear pacing	baseline	98%
Hard threshold	−8.3%	91%
Proportional (ECPC)	+4.1%	97%
DQN (Wu et al., 2018)	+11.7%	99%

[144] Credit assignment refers to the difficulty of determining which past actions caused a delayed reward or cost. In multi-echelon systems, an upstream ordering decision today may not affect customer-facing costs for many periods, making it difficult for RL to learn the causal connection.

[145] Residual RL (Silver et al., 2018b) offers a middle ground: rather than replacing classical policies, learn a correction $\pi_{\text{final}}(s) = \pi_{\text{classical}}(s) + \pi_{\text{learned}}(s)$, preserving the classical solution's performance while allowing RL to handle constraints or non-stationarity the analytical method cannot. This connects to perturbation methods in applied mathematics, where one linearizes around a known solution and solves for deviations. See Ibarz et al. (2021) for a survey of residual and demonstration-augmented RL in robotics.

[146] The state space is discretized into budget bins and time bins. Wu et al. (2018) augment the state with bid landscape features derived from historical clearing price distributions, giving the agent information about current market competitiveness.

7.7 Simulation Study: Bus Engine Replacement

I conclude with a simulation demonstrating that RL matches dynamic programming on a classical benchmark. The bus engine replacement problem, introduced by Rust (1987), models a fleet manager's monthly decision whether to replace engines based on accumulated mileage. Replacement incurs a fixed cost but resets mileage to zero; continued operation incurs maintenance costs increasing in mileage.

I extend the single-engine problem to a fleet of N engines with a capacity constraint limiting replacements per period. The state $s = (m_1, \ldots, m_N)$ records discretized mileage for each engine. Actions are subsets of engines to replace, subject to the capacity constraint. The per-period cost is $c(s, a) = \alpha \sum_i m_i + \beta|a|$, where α is the operating cost per unit mileage and β is the replacement cost.[147] Mileage evolves deterministically; replaced engines reset to $m = 0$; others increment by one bin.[148]

Figure 14 compares dynamic programming, DQN, and heuristic baselines across fleet sizes.

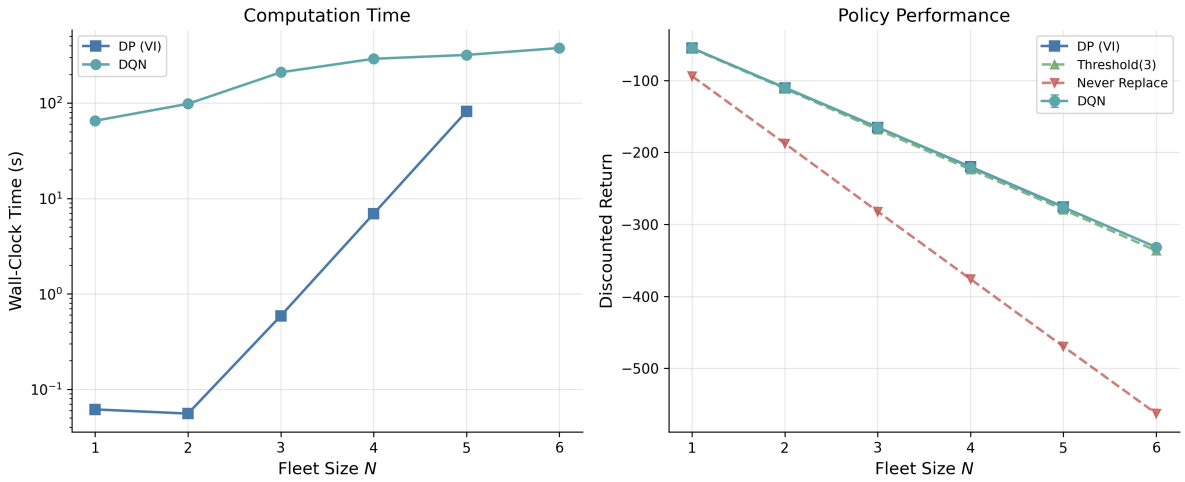

Figure 14: Bus engine replacement benchmark. Left: computation time vs. fleet size (log scale). Right: discounted return vs. fleet size for DP, DQN, and heuristic baselines. At $N = 6$ (46,656 states), DP is infeasible (no data point).

For $N = 1$ through 5 where both methods are computable, DQN matches DP within 1% of the optimal discounted return. At $N = 6$ (46,656 states), DP is infeasible but DQN produces a policy. The threshold heuristic, which replaces engines above a mileage cutoff, provides a reasonable baseline but cannot account for capacity constraints or the joint state of multiple engines. The never-replace heuristic performs poorly due to accumulated mileage costs, confirming that the cost structure creates a non-trivial replacement decision.

[147] The mileage-dependent operating cost $\alpha \sum_i m_i$ follows Rust's original specification $c(x, \theta_1) = \theta_{11} x$, creating a non-trivial threshold replacement policy. The fleet extension uses deterministic mileage increments to isolate the combinatorial scaling challenge that arises from the joint state of multiple engines.

[148] With $M = 6$ mileage bins, the state space is 6^N: 1,296 states at $N = 4$, 7,776 at $N = 5$, 46,656 at $N = 6$. Value iteration is feasible for $N \leq 5$.

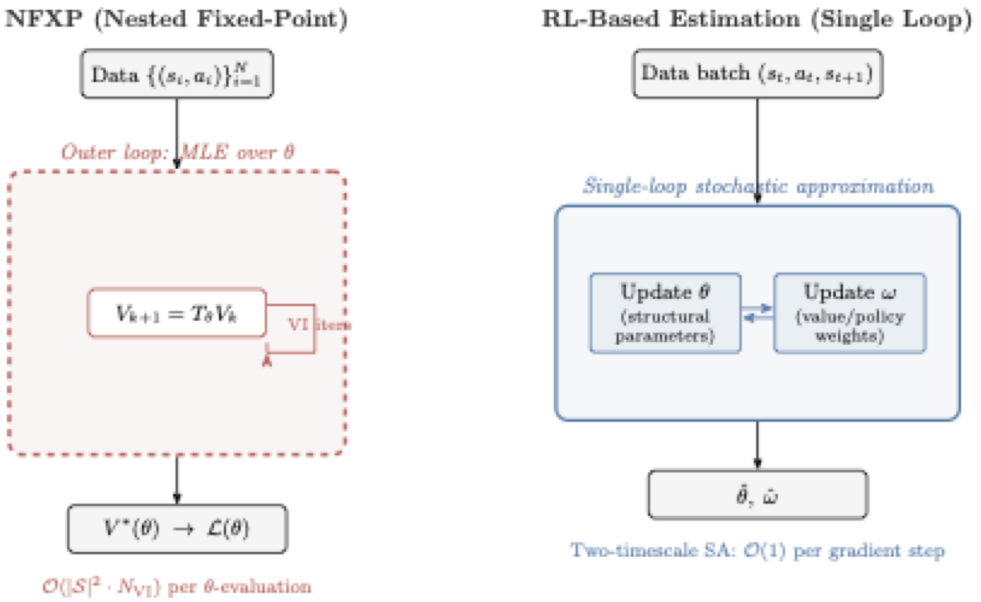

Figure 15: NFXP versus RL-based structural estimation. Top: the nested fixed-point algorithm evaluates the likelihood by solving the Bellman equation to convergence inside each optimizer step. Bottom: single-loop stochastic approximation updates structural parameters θ and value/policy weights ω simultaneously from data batches. The NFXP algorithm is due to Rust (1987).

8 Structural Estimation with Reinforcement Learning

Several recent papers have used RL training loops,[149] namely Q-learning, temporal-difference learning, policy gradient, and actor-critic methods, to solve structural economic models at scales where conventional dynamic programming fails.[150] Throughout this chapter I adopt a unified notation. An MDP is a tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s'|s,a)$ is the transition kernel, $r(s,a)$ is the per-period reward, and $\gamma \in [0,1)$ is the discount factor.[151] A policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ maps states to distributions over actions. The value function under π is $V^\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^\infty \gamma^t r(s_t, a_t) \mid s_0 = s\right]$, and the action-value function is $Q^\pi(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V^\pi(s')]$. The optimal value function satisfies $V^*(s) = \max_{a \in \mathcal{A}} Q^*(s,a)$.

The canonical structural estimation framework for MDPs was formulated by Rust (1994), whose nested fixed-point (NFXP) algorithm embeds the Bellman equation inside a maximum likelihood estimator. The methods reviewed in this chapter replace the inner fixed-point computation with RL-based approximations.

8.1 Single-Agent Structural Estimation

8.1.1 TD Learning for CCP Estimation

Adusumilli and Eckardt (2022) adapt temporal-difference (TD) learning to estimate the recur-

[149] Throughout this chapter, the RL training loop runs entirely inside the econometrician's computational model; the agent never interacts with real economic agents or markets but serves as a numerical method for solving the Bellman equation within a structural estimation procedure (Section 2). There is no execution phase in the usual sense.

[150] I exclude papers that use neural network function approximation without an RL training mechanism. Inverse reinforcement learning is treated in the sister survey (Rust and Rawat, 2026).

[151] Several of the papers reviewed here use β for the discount factor, following economics convention. I translate all results to γ for consistency with the RL literature and the rest of this survey.

sive terms that arise in CCP-based estimation, entirely avoiding specification or estimation of transition densities.

The CCP approach requires computing two functions $h : \mathcal{A} \times \mathcal{S} \to \mathbb{R}^d$ and $g : \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ that solve the recursive equations

$$h(a, s) = z(s, a) + \gamma \, \mathbb{E}[h(a', s') \mid a, s], \tag{72}$$

$$g(a, s) = e(a, s) + \gamma \, \mathbb{E}[g(a', s') \mid a, s], \tag{73}$$

where $e(a, s) = \gamma_{\mathrm{E}} - \ln P(a|s)$ under logit errors (γ_{E} denoting the Euler constant), and the expectation is over the next-period state-action pair (s', a') given the transition kernel $P(s'|s, a)$ and the observed policy $P(a|s)$. Both h and g satisfy a Bellman-like recursion under the observed (data-generating) policy, not the optimal policy, so standard TD learning applies directly. Adusumilli and Eckardt (2022) propose two methods.

The first is the linear semi-gradient method.[152] Approximate $h(a, s) \approx \phi(a, s)^\top w$ where $\phi : \mathcal{A} \times \mathcal{S} \to \mathbb{R}^p$ is a vector of basis functions (e.g., polynomials in state variables) and $w \in \mathbb{R}^p$ are the weights to be estimated. The TD(0) fixed-point equation is

$$\mathbb{E}\left[\phi(a, s) \left(\phi(a, s) - \gamma \phi(a', s')\right)^\top\right] w = \mathbb{E}\left[\phi(a, s) \, z(s, a)\right]. \tag{74}$$

The sample analog replaces population expectations with averages over the observed panel.

$$\hat{w} = \left(\frac{1}{n(T-1)} \sum_{i=1}^{n} \sum_{t=1}^{T-1} \phi_{it} \left(\phi_{it} - \gamma \phi_{i,t+1}\right)^\top\right)^{-1} \left(\frac{1}{n(T-1)} \sum_{i=1}^{n} \sum_{t=1}^{T-1} \phi_{it} \, z_{it}\right), \tag{75}$$

where $\phi_{it} = \phi(a_{it}, s_{it})$ and $z_{it} = z(s_{it}, a_{it})$. This requires inverting a $p \times p$ matrix, where p is the number of basis functions, making computation trivial in most settings. No transition density estimation is needed, since the method uses only observed sequences of current and next-period state-action pairs.

The second method is approximate value iteration (AVI), which iterates the Bellman-like operator using nonparametric regression. At iteration k, one constructs pseudo-outcomes

$$Y_{it}^{(k)} = z_{it} + \gamma \, \hat{h}^{(k-1)}(a_{i,t+1}, s_{i,t+1}) \tag{76}$$

and then fits $\hat{h}^{(k)}$ by regressing $Y_{it}^{(k)}$ on (a_{it}, s_{it}) using any machine learning method, including LASSO, random forests, or neural networks.[153] This is the first DDC estimator compatible with arbitrary ML prediction methods, enabling application to very high-dimensional state spaces.

With $\hat{h}$ and $\hat{g}$ in hand, structural parameters are recovered by pseudo-maximum likelihood estimation (PMLE). For continuous state spaces, Adusumilli and Eckardt (2022) derive a locally robust correction to the PMLE criterion that accounts for the nonparametric first-stage estimation of value terms, restoring $\sqrt{n}$-convergence of $\hat{\theta}$.[154] The PMLE score is $m(a, s; \theta, h, g) = \partial_\theta \ln \pi(a, s; \theta, h, g)$, where π is the logit choice probability with continuation value $V(a, s) = h(a, s)^\top \theta + g(a, s)$. The naive estimator solves $\mathbb{E}_n[m(a, s; \theta, \hat{h}, \hat{g})] = 0$, but with continuous states this moment condition is not orthogonal to the first-stage estimates and

[152] The method is called "semi-gradient" because it computes the gradient of only the prediction $\phi(a, s)^\top w$ with respect to w, not the full TD error including the bootstrap target $\phi(a', s')^\top w$. This avoids differentiating through the target but sacrifices guaranteed convergence in some settings.

[153] *LASSO* (Tibshirani, 1996) adds an ℓ_1 penalty to a regression loss, shrinking many coefficients to zero for sparse solutions; *random forests* average many decision trees fit to random subsamples and feature subsets for nonparametric estimation; *neural networks* compose affine transformations with elementwise nonlinearities across multiple layers.

[154] An estimator has $\sqrt{n}$-convergence if its error shrinks at rate $n^{-1/2}$ where n is the sample size. This is the standard parametric rate; slower rates (e.g., $n^{-1/4}$) indicate efficiency loss from nonparametric first stages.

converges slower than $\sqrt{n}$. The locally robust moment adds a debiasing correction:

$$\zeta = m(a, s; \theta, h, g) - \lambda(a, s; \theta) \left\{ z(s, a)^\top \theta + \gamma\, e(a', s') + \gamma\, V(a', s') - V(a, s) \right\}, \tag{77}$$

where $\lambda(a, s; \theta)$ solves a backward recursion that propagates the influence of estimation error through the dynamic structure.[155] The corrected estimator $\hat{\theta}_{LR}$ solves $\mathbb{E}_n[\zeta_n] = 0$ and is computationally no harder than the naive PMLE, since the correction is constant in θ.

Theorem 2 (Adusumilli and Eckardt (2022), Theorem 1)**.** *Under regularity conditions, the linear semi-gradient estimator $\hat{h}$ satisfies $\|\hat{h} - h\|_2 = O_P(n^{-1/2}(T-1)^{-1/2})$, where $\|\cdot\|_2$ denotes the $L^2(P)$ norm.*[156]

Theorem 3 (Adusumilli and Eckardt (2022), Theorem 5)**.** *Under regularity conditions on the ML method used in AVI, the locally robust PMLE estimator $\hat{\theta}_{LR}$ satisfies $\sqrt{n}(\hat{\theta}_{LR} - \theta^*) \xrightarrow{d} \mathcal{N}(0, \Sigma)$ for an explicit variance Σ, even when the state space is continuous.*

Monte Carlo experiments on a dynamic firm entry model with seven structural parameters and five continuous state variables show that the TD-based estimators achieve a 4- to 11-fold reduction in mean squared error compared to CCP estimators using state-space discretization.

For dynamic discrete games, the method extends naturally. Standard CCP-based estimation of games requires integrating out other players' actions, which becomes intractable with many players or continuous states. TD learning avoids this entirely, since it works directly with the joint empirical distribution of states and their successors. The "integrating out" is done implicitly within sample expectations.

8.1.2 Policy Gradient for DDC Estimation

Hu and Yang (2025) combine policy gradient methods with the Simulated Method of Moments (SMM) to estimate DDCs, with particular focus on models with unobserved state variables.

The outer loop is SMM.[157] Define a vector of data moments $\mathbf{M}_d$ computed from the observed panel. For candidate structural parameters θ and transition parameters ξ, simulate the model to produce simulated moments $\mathbf{M}_s(\theta, \xi)$. The estimator minimizes

$$(\hat{\theta}, \hat{\xi}) = \arg\min_{\theta, \xi} \left(\mathbf{M}_d - \mathbf{M}_s(\theta, \xi)\right)^\top \mathbf{W} \left(\mathbf{M}_d - \mathbf{M}_s(\theta, \xi)\right), \tag{78}$$

where $\mathbf{W}$ is a positive definite weight matrix. Computing $\mathbf{M}_s(\theta, \xi)$ requires solving for the optimal policy under (θ, ξ), which is the inner-loop problem.

The inner loop parametrizes the choice probability directly as a logistic function of state variables. For a binary choice $J_t \in \{0, 1\}$, the general form is $\Pr(J_t = 1 \mid \boldsymbol{X}_t; \boldsymbol{\gamma}(\theta)) = \text{logistic}(\boldsymbol{X}_t \boldsymbol{\gamma}(\theta))$, where $\boldsymbol{\gamma}(\theta)$ are policy parameters that depend on the structural parameters.[158] In their application to a Rust bus engine model with unobserved bus condition S_t^*, this takes the form

$$\Pr(J_t = 1 \mid X_t, S_t^*, t; \boldsymbol{\gamma}) = \frac{\exp(\gamma_0 + \gamma_1 t + \gamma_2 X_t + \gamma_3 S_t^*)}{1 + \exp(\gamma_0 + \gamma_1 t + \gamma_2 X_t + \gamma_3 S_t^*)}, \tag{79}$$

[155] The term in braces is the temporal-difference error of the continuation value V. The adjoint λ weights this TD error by its marginal impact on the PMLE score. See Online Appendix B.3 of Adusumilli and Eckardt (2022) for the derivation.

[156] The $L^2(P)$ norm is $\|f\|_2 = (\int f(x)^2\, dP(x))^{1/2}$, measuring average squared deviation under the probability measure P. This is the natural norm for mean-squared-error analysis.

[157] SMM (Gourieroux et al., 1993) estimates structural parameters by matching moments from simulated data to moments from observed data, avoiding direct evaluation of the likelihood function.

[158] The linear index can be replaced by higher-order terms of $\boldsymbol{X}_t$ or deep neural networks; the method requires only that the gradient $\nabla_\gamma \log \pi_\gamma$ has a closed form.

where t enters the index directly to capture time-varying replacement incentives. The policy parameters are updated by REINFORCE-style gradient ascent, applying the policy gradient theorem (Sutton et al., 1999):

$$\nabla_{\boldsymbol{\gamma}} V(\boldsymbol{\gamma}) = \mathbb{E}\left[\sum_{t=0}^{T} \nabla_{\boldsymbol{\gamma}} \log \pi_{\boldsymbol{\gamma}}(J_t \mid X_t, S_t^*, t)\, Q^{\pi_{\boldsymbol{\gamma}}}(X_t, S_t^*, J_t)\right], \tag{80}$$

where $Q^{\pi_{\boldsymbol{\gamma}}}$ is the action-value function under the current policy, estimated by Monte Carlo returns from forward-simulated trajectories.

The main contribution is handling unobserved state variables. When state variables are only partially observed, the policy in (79) is parametrized as a function of both X_t and S_t^*, and the algorithm forward-simulates trajectories of both observed and unobserved variables.

Building on the nonparametric identification results of Hu and Shum (2012), the outer-loop SMM targets moments from five consecutive periods of observed data, which suffice to separately identify the structural parameters θ and transition parameters ξ without requiring the econometrician to observe S_t^*. No discretization of continuous unobserved states is needed; the same algorithm handles both discrete and continuous unobserved heterogeneity.

For each candidate (θ, ξ) in the outer minimization (78), the inner loop runs policy gradient until convergence, producing optimal policy parameters $\boldsymbol{\gamma}^*(\theta, \xi)$. These are used to simulate data and compute $\mathbf{M}_s(\theta, \xi)$.

On an extended Rust bus engine model with a continuous unobserved bus condition following an AR(1) process, estimates of seven structural parameters are centered around their true values across 400 simulations. On a discrete-unobservable variant, the method matches the precision of Arcidiacono and Miller (2011)'s two-step EM algorithm at comparable computation times, though the advantage diminishes as more inner-loop iterations are used for precision.

8.2 Dynamic Oligopoly and Strategic Interaction

Dynamic oligopoly models combine game theory and dynamic programming: firms choose actions strategically while anticipating competitors' strategies, and the state space grows combinatorially in the number of firms.

8.2.1 Q-Learning in Dynamic Procurement Auctions

Asker et al. (2020) develop a computational framework for analyzing dynamic procurement auctions with serially correlated asymmetric information. Their approach builds on the Experience-Based Equilibrium (EBE) concept of Fershtman and Pakes (2012), which computes equilibria by simulating industry trajectories and updating strategies toward best responses. EBE evaluates values only on recurrently visited states rather than the full state space, making it feasible for large state spaces. While Fershtman and Pakes (2012) use a stochastic approximation algorithm to update continuation values from simulated industry trajectories, Asker et al. (2020) add explicit value-function updates via stochastic approximation.

The model is a repeated first-price sealed-bid auction with two firms. Each firm i maintains a private inventory state $\omega_{i,t}$ (stock of unharvested timber, in their application). The state evolves endogenously, as winning an auction increases inventory while harvesting depletes it. Each firm's private state is not observed by its competitor except at periodic revelation events.

The key computational innovation is the use of Q-learning to compute equilibrium strategies. Each firm i maintains a Q-function $Q_i : \mathcal{S}_i \times \mathcal{A}_i \to \mathbb{R}$, where $\mathcal{S}_i$ encodes firm i's information set (its own inventory, beliefs about the competitor's inventory, public history) and $\mathcal{A}_i$ is its action set (participation decision and bid level). The Q-function satisfies

$$Q_i(s, a) = \mathbb{E}\left[r_i(s, a, a_{-i}) + \gamma \max_{a' \in \mathcal{A}_i} Q_i(s', a') \;\middle|\; s, a\right], \tag{81}$$

where $r_i(s, a, a_{-i})$ is firm i's per-period profit given the state, its own action a, and the competitor's action a_{-i}, and the expectation is taken over the competitor's strategy and the stochastic transitions.

Firms update Q-values using sample averaging.

$$Q_i(s,a) \leftarrow Q_i(s,a) + \frac{1}{h_k(s,a)} \left[r_i + \gamma \max_{a' \in \mathcal{A}_i} Q_i(s', a') - Q_i(s,a) \right], \tag{82}$$

where $h_k(s, a)$ is the number of times state-action pair (s, a) has been visited.[159] The equilibrium computation proceeds iteratively, with firms simultaneously updating their Q-functions based on simulated play against each other's current strategies. Strategies are derived from Q-values using an ε-greedy rule or Boltzmann exploration.[160]

Asker et al. (2020) add a boundary consistency condition to the EBE concept that restricts behavior at the boundary of the recurrent state class, reducing the multiplicity of equilibria. Their numerical analysis reveals that information sharing between firms can, through increased precision of beliefs about competitor states, induce firms to spend more time in states where competition is less intense. The dynamic RL-computed equilibrium yields qualitatively different predictions from both static analysis and myopic ($\gamma = 0$) benchmarks. With dynamics, information sharing decreases average bids and increases average profits, while the myopic benchmark shows negligible effects.

The limitation of this approach is the tabular representation; the Q-function is stored as a lookup table over discretized states and actions, restricting applicability to models with moderate state-space dimension.

8.2.2 TD Learning for Merger Analysis with Innovation

Hollenbeck (2019) uses RL to solve a dynamic oligopoly model with endogenous mergers, entry, exit, and quality investment. The model extends the Ericson-Pakes framework to study how horizontal mergers affect innovation incentives.

The industry state is $\Omega = (\omega_1, \ldots, \omega_n)$ where $\omega_i \in \{1, \ldots, \omega_{\max}\}$ is firm i's product quality. Firms produce differentiated goods and compete in prices (Bertrand competition with logit demand). In each period, firms simultaneously choose investment levels, entry/exit decisions, and potentially initiate merger negotiations.

Each firm i computes its continuation value $V_i(\Omega)$ from the industry state. Because the state space is the product of all firms' quality levels plus industry structure (number of active firms, recent mergers), exact dynamic programming is infeasible for industries with more than two or three firms. Hollenbeck (2019) instead uses temporal-difference learning to estimate values from simulated industry trajectories.

The value function update for firm i is[161]

$$V_i(\Omega) \leftarrow V_i(\Omega) + \alpha \left[\Pi_i(\Omega, \mathbf{a}) + \gamma V_i(\Omega') - V_i(\Omega) \right], \tag{83}$$

where $\Pi_i(\Omega, \mathbf{a})$ denotes firm i's per-period profit given industry state Ω and the joint action profile $\mathbf{a}$ (investment, entry/exit, merger decisions),[162] γ is the discount factor, and Ω' is the

[159] This sample-averaging rule ($\alpha_k = 1/h_k$) is equivalent to maintaining the running mean of observed returns, as distinct from fixed-α Q-learning which gives exponentially decaying weight to older observations. The update is applied for all actions, including counterfactual actions not taken, using the observed state transition.

[160] ε-greedy selects the greedy action $argmax_a\, Q(s, a)$ with probability $1 - \varepsilon$ and a uniformly random action otherwise. Boltzmann exploration selects action a with probability $\propto \exp(Q(s, a)/\tau)$ where $\tau > 0$ is a temperature parameter.

[161] This stochastic approximation update, introduced by Pakes and McGuire (1994) for dynamic oligopoly computation, is closely related to temporal-difference learning in the RL literature. The original algorithm uses visit-count averaging ($\alpha_k = 1/k$ where k counts visits to each state) rather than a fixed learning rate.

[162] I use Π_i for firm i's per-period profit to avoid confusion with policy π.

realized next-period state. The algorithm uses ε-decreasing exploration to prevent convergence to locally suboptimal equilibria.

The equilibrium computation follows the Pakes-McGuire iterative scheme.[163] The algorithm simulates long industry histories, updates each firm's value function via (83), re-derives best-response strategies from updated values, and repeats.

The central finding is that horizontal mergers, while reducing static consumer surplus in the short run, create a strong incentive for entry and investment. Firms enter with negative static profits because the prospect of a lucrative buyout justifies the initial investment. The result is substantially higher long-run innovation and consumer welfare with mergers than without. This finding reverses the standard static antitrust prediction and can only emerge in a dynamic model where firms are forward-looking.

Two related papers merit brief mention. Lomys and Magnolfi (2024) develop structural estimation methods for strategic settings where agents use learning algorithms (specifically regret-minimizing rules) rather than playing a fixed equilibrium. They impose an "asymptotic no-regret" condition as a minimal rationality requirement and derive identification results for payoff parameters. Covarrubias (2022) uses deep RL to study oligopolistic pricing in a New Keynesian framework, representing firms' pricing policies as neural networks $\pi_\phi(a|s)$. The method uncovers multiple equilibria ranging from competitive to collusive pricing.[164]

8.3 Auction Equilibria and Mechanism Design

Closed-form equilibrium bidding strategies exist only for narrow families of valuation distributions and auction formats, and numerical methods scale poorly with the number of bidders and items.

8.3.1 RL for Sequential Price Mechanisms

Brero et al. (2021) use RL to design optimal sequential price mechanisms (SPMs), a class of indirect auction mechanisms where agents are approached in sequence and offered menus of items at posted prices. SPMs generalize both serial dictatorship and posted-price mechanisms and essentially characterize all strongly obviously strategyproof (SOSP) mechanisms (Pycia and Troyan, 2023).[165]

The mechanism design problem is formulated as a partially observable Markov decision process (POMDP).[166] The state at round t includes the set of remaining items $\rho_t^{\text{items}} \subseteq [m]$, remaining agents $\rho_t^{\text{agents}} \subseteq [n]$, the partial allocation $\mathbf{x}_t$, and the agents' (unobserved) valuation functions. The action $a_t = (i_t, \{p_j^t\}_{j \in \rho_{t-1}^{\text{items}}})$ specifies which agent to visit next and what prices to offer. The observation is the agent's purchase decision, from which the mechanism can update beliefs about valuations. The reward is the objective function evaluated at the final allocation:

$$r = g(\mathbf{x}_T, \boldsymbol{\tau}_T; \mathbf{v}), \tag{84}$$

[163] The Pakes-McGuire algorithm (Pakes and McGuire, 1994) computes Markov perfect equilibria by iterating: (1) given current value functions, compute best-response strategies; (2) given strategies, update value functions via simulation. Convergence is not guaranteed but works well in practice.

[164] The most prominent example of algorithmic collusion is Calvano et al. (2020), who showed that independent Q-learning agents in a repeated Bertrand pricing game learn to sustain supra-competitive prices and punish deviators without explicit communication. This result and its implications for competition policy are treated in the companion thesis chapter (Rawat, 2026).

[165] A mechanism is strategyproof if truthful reporting is a dominant strategy. It is obviously strategyproof if this dominance is apparent even to boundedly rational agents; strongly obviously strategyproof (SOSP) adds that dominance holds at every information set (Pycia and Troyan, 2023).

[166] In a POMDP, the agent cannot directly observe the full state. It maintains a belief distribution over possible states, updated via Bayes' rule as new observations arrive. This captures the mechanism designer's uncertainty about bidder valuations.

where g can be social welfare $\sum_i v_i(x_i)$, revenue $\sum_i \tau_i$, or max-min fairness $\min_i v_i(x_i)$.

A key theoretical result establishes when adaptive mechanisms outperform static ones.

Theorem 4 (Brero et al. (2021), Propositions 1–4, informal)**.** *Each feature of adaptive mechanisms is necessary for welfare optimality, even in simple settings. Personalized prices are needed with one item and two i.i.d. agents (Proposition 1); adaptive prices are needed with two identical items and three i.i.d. agents (Proposition 2); adaptive ordering is needed with six agents whose valuations are correlated (Proposition 3); both adaptive prices and ordering are needed with four agents whose valuations are independently but non-identically distributed (Proposition 4).*

The policy maps from a sufficient statistic of the observation history to actions. Brero et al. (2021) show that this statistic can be represented compactly. The set of remaining items and agents suffices for independent valuations, while the full allocation matrix is needed for correlated valuations. They train the mechanism policy using Proximal Policy Optimization (PPO),[167] which handles the discrete action space (agent selection) and continuous action space (price setting) of the POMDP.

Experimental results show that the learned SPMs achieve near-optimal welfare across settings with up to 20 agents and 5 items (with similar results noted for up to 30 of each), significantly outperforming static pricing benchmarks. The improvement is largest when agent valuations are correlated, since adaptive prices allow the mechanism to infer information about remaining agents from earlier purchases.

The limitation is that the POMDP formulation requires knowledge of the prior distribution over valuations, which in practice must be estimated from data. The method also does not scale easily to very large numbers of items due to the combinatorial action space.

8.3.2 Fitted Policy Iteration for Combinatorial Auctions

Ravindranath et al. (2024) address revenue-maximizing mechanism design for combinatorial auctions with multiple items and strategic bidders. Their innovation is integrating differentiable auction structure into a fitted policy iteration framework, enabling analytical gradient computation where standard RL methods struggle with high variance.

The mechanism visits agents one at a time in sequence. Each agent i, upon being visited, selects a bundle of items from those still available, given posted prices. Valuations are drawn once from distributions V_i and remain fixed throughout the mechanism. Complementarities arise from the structure of the valuation function over bundles, not from dynamic evolution. The MDP state at step t is $s_t = (i_t, S_t)$, where i_t is the current bidder and S_t is the set of remaining items.

The mechanism's policy maps from state to a price vector over available items. The key technical contribution is making the auction clearing differentiable. In a standard auction, the allocation is an argmax over bids (non-differentiable), and payments depend discontinuously on the allocation. Ravindranath et al. (2024) replace the hard bundle selection with a softmax relaxation,[168] enabling analytical gradient computation through the mechanism. The actor loss is the negative expected revenue, and gradients flow directly through the softmax-relaxed allocation and payment rules. The paper explicitly avoids REINFORCE-style estimators, noting that analytical gradients overcome the sample inefficiency and high variance of score-function methods.

The method follows fitted policy iteration (Bertsekas and Tsitsiklis, 1996), alternating between evaluating the current policy (computing expected revenue) and improving it via gradient

[167] Proximal Policy Optimization (Schulman et al., 2017) is a policy gradient algorithm that constrains each update to a trust region, preventing large destabilizing policy changes. It is described in Chapter 2.

[168] The softmax relaxation replaces the hard allocation $argmax_i\, b_i$ with a soft allocation $\exp(b_i/\tau)/\sum_j \exp(b_j/\tau)$, which is differentiable and approaches the hard allocation as $\tau \to 0$.

ascent on the actor loss. This differs from model-free RL approaches (PPO, SAC) that the paper uses as baselines.

Experiments on settings with additive and combinatorial valuations show that the learned mechanisms achieve up to 13% higher revenue than item-wise Myerson optimal auctions, with the largest gains in combinatorial settings where bundle complementarities make item-wise pricing suboptimal. Standard RL baselines (PPO, SAC) are also outperformed, confirming the advantage of exploiting differentiable structure.[169]

8.4 Simulation Study: DDC Estimation at Scale

The test bed is a multi-component extension of the Rust (1987) bus engine replacement model. In the original formulation a maintenance superintendent observes discretized mileage $m \in \{0, \ldots, M-1\}$ and makes a binary keep-or-replace decision, facing running cost $c(s;\theta) = \theta_1 x + \theta_2 x^2$ with $x = m/M$, replacement cost RC, and Type I extreme value additive errors that yield logit conditional choice probabilities. We extend the model to K independent wear components, each evolving in $\{0, \ldots, M-1\}$, with aggregate normalized wear $x(s) = \sum_k m_k/M$ entering the same cost function. The state space is $|\mathcal{S}| = M^K$, so increasing K produces a controlled scaling experiment in which the data-generating process is identical across scales but the computational burden grows exponentially.

We compare four estimation methods on a multi-component bus engine replacement problem (Rust, 1987) with $K \in \{1, 2, 3, 4\}$ independent wear components, producing state spaces from 20 to 160,000 states. Panel data consist of $N = 500$ agents observed for $T = 100$ periods. The four methods are NFXP (nested fixed-point MLE), CCP (Hotz-Miller inversion), TD-CCP Linear (semi-gradient TD with polynomial basis), and TD-CCP Neural (approximate value iteration with a two-layer MLP), where the two TD-CCP variants follow Adusumilli and Eckardt (2022).[170] Each configuration is replicated across 10 seeds with the PyTorch optimizer also seeded; Table 15 and Figure 16 report means with seed-level standard errors.

NFXP wall-clock grows by roughly three orders of magnitude across the four scales, reflecting the cost of repeated value iteration inside each likelihood evaluation. CCP becomes infeasible beyond K=2 due to sparse state coverage in the panel data. TD-CCP Neural maintains near-constant runtime across all K levels with RC RMSE comparable to NFXP, consistent with the AVI approach of Adusumilli and Eckardt (2022). TD-CCP Linear runs in well under a second at all scales but suffers from basis misspecification at higher K, with RC RMSE rising monotonically in K; see Table 15 and Figure 16.

[169] DDPG was also tested but found to be unstable. DQN is not applicable to continuous price-setting.

[170] The TD-CCP variants implemented here are simplified relative to Adusumilli and Eckardt (2022). We omit the locally robust PMLE correction (Theorem 5 of that paper, the construction that yields $\sqrt{n}$-consistency); the reported point estimates are for the simpler plug-in PMLE and do not inherit the paper's $\sqrt{n}$-convergence guarantee. We also reformulate the bootstrap target by decomposing $EV(s;\theta)$ into θ-linear pieces and learning each piece separately, rather than fitting $h(a,s)$ directly as in the paper; the two are algebraically related under our binary action with one-period reset, but the change is undocumented in the original. Finally, the marketed advantage of TD-CCP that "no transition density is needed" is qualified in our implementation: the PMLE step still evaluates $v_0 = -c + \gamma P_{\text{keep}} \widehat{EV}$ using the empirical P_{keep} assumed known under the Rust convention. A fully density-free variant that replaces $P_{\text{keep}} \widehat{EV}$ by an out-of-sample target average is left for future work.

Table 15: DDC estimation results across state-space scales. Rows show method, number of components K, state-space size $|\mathcal{S}|$, mean wall-clock time (seconds), RC bias, RC RMSE, and root mean squared error for θ_1 and θ_2. Standard errors in adjacent columns; means over 10 seeds, dashes indicate method failure (sparse state coverage).

				RC Bias		RC RMSE		θ_1 RMSE		θ_2 RMSE	
Method	K	$\|S\|$	Time (s)	Mean	SE	Value	SE	Value	SE	Value	SE
NFXP	1	20	0.2	+0.053	0.027	0.097	0.015	0.307	0.055	0.495	0.111
CCP	1	20	0.1	+0.055	0.027	0.099	0.015	0.316	0.055	0.510	0.112
TD-CCP Linear	1	20	0.1	-0.087	0.017	0.101	0.016	0.255	0.041	0.296	0.044
TD-CCP Neural	1	20	38.4	+0.054	0.026	0.094	0.013	0.260	0.054	0.423	0.082
NFXP	2	400	0.3	+0.027	0.028	0.087	0.018	0.270	0.061	0.344	0.077
CCP	2	400	0.1	+0.089	0.027	0.120	0.021	0.527	0.068	0.797	0.089
TD-CCP Linear	2	400	0.1	-0.171	0.021	0.182	0.021	0.131	0.027	0.888	0.060
TD-CCP Neural	2	400	32.9	+0.012	0.030	0.090	0.021	0.247	0.065	0.334	0.067
NFXP	3	8,000	4.4	+0.011	0.014	0.044	0.009	0.236	0.046	0.364	0.081
CCP	3	8,000	—	—	—	—	—	—	—	—	—
TD-CCP Linear	3	8,000	0.1	-0.246	0.011	0.248	0.010	0.125	0.022	1.178	0.071
TD-CCP Neural	3	8,000	35.9	+0.007	0.017	0.051	0.009	0.215	0.050	0.318	0.081
NFXP	4	160,000	163.5	-0.034	0.017	0.061	0.012	0.153	0.027	0.143	0.024
CCP	4	160,000	—	—	—	—	—	—	—	—	—
TD-CCP Linear	4	160,000	0.2	-0.328	0.014	0.331	0.014	0.226	0.028	1.103	0.027
TD-CCP Neural	4	160,000	40.7	-0.031	0.019	0.066	0.011	0.258	0.042	0.236	0.039

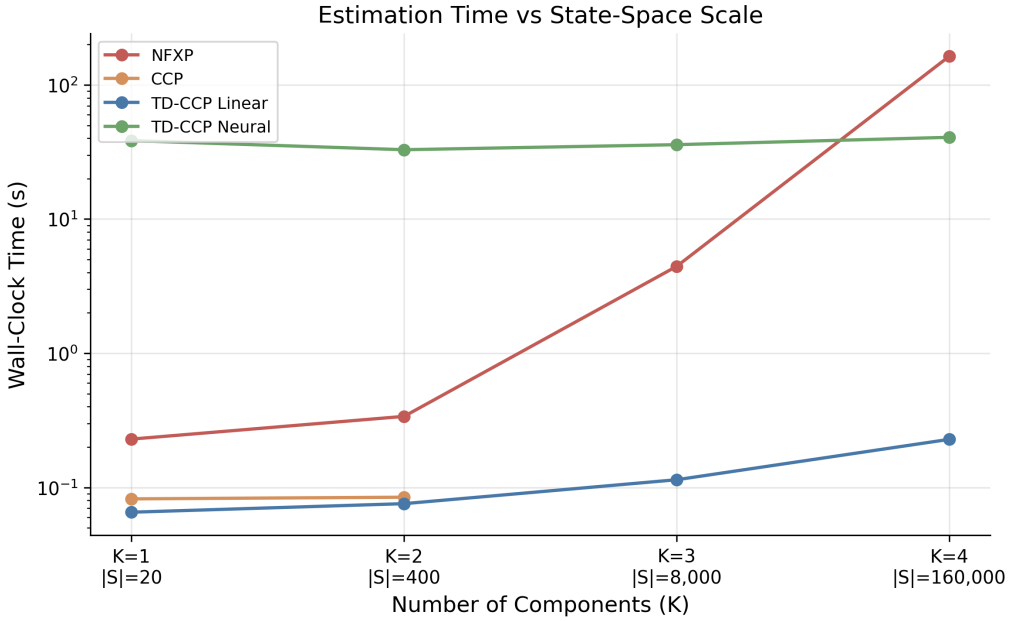

Figure 16: Wall-clock estimation time versus state-space scale for the four DDC estimators. Vertical axis is log-scaled. Each point is the mean over 10 seeds.

9 Reinforcement Learning for Macroeconomic Models

Reinforcement learning is a recent arrival in macroeconomics. Most of the algorithms surveyed in this chapter were published in the past few years, and the literature is small enough that few results have been independently replicated. The arrival is nevertheless of some interest, because RL admits two distinct interpretations in a macro context. As a solver, RL is a computational tool that finds rational-expectations or Nash equilibria for models whose state space breaks dynamic programming. As a behavioural model, RL describes a boundedly-rational agent who learns a policy from realised utility under no a priori knowledge of the economy. The two interpretations share the same training code; the distinction is in what the analyst uses the output to claim.

This chapter is organised around four lenses on what RL does for macroeconomics. Section 9.1 fixes notation and defines the four problem classes the chapter uses. Section 9.2 and Section 9.3 treat RL as a global solver, first for representative-agent macro models that classical dynamic programming already handles, and then for heterogeneous-agent models where DP breaks down. The simulation in Section 9.4 concretises the comparison on a textbook RBC. Section 9.5 takes the continuum limit to mean field games with common noise, in two variants by game structure: symmetric mean-field and hierarchical Stackelberg. Section 9.7 reframes RL as a model of the household itself, using learning dynamics to reproduce empirical anomalies that rational expectations cannot match jointly. Section 9.8 surveys RL for policy design under two methodologically distinct settings, multi-agent ABM in which the planner and the citizens are all RL learners co-adapting, and single-planner RL in which only the planner uses RL and the rest of the macro model is fixed. Section 9.10 closes with a short synthesis.

9.1 Setup and notation

We collect the definitions here so that subsequent sections can map a paper's notation to a single chapter-wide convention without restating the machinery.

9.1.1 The agent's MDP

A Markov decision process (MDP) is a tuple $(\mathcal{S}, \mathcal{A}, P, r, \beta)$ collecting state space, action space, transition rule, reward, and discount. Concretely, the state space is $\mathcal{S}$, the action space is $\mathcal{A}$, the transition kernel $P(s' \mid s, a)$ gives the probability of moving to state s' after taking action a in state s, the reward function is $r(s, a) \in \mathbb{R}$, and the discount factor is $\beta \in (0, 1)$. A policy $\pi(a \mid s)$ maps states to action distributions. The return is $G_t = \sum_{k \geq 0} \beta^k r_{t+k}$, the value is $V^\pi(s) = \mathbb{E}^\pi[G_t \mid s_t = s]$, and the Q-function is $Q^\pi(s, a) = \mathbb{E}^\pi[G_t \mid s_t = s, a_t = a]$. The optimal value satisfies the Bellman equation $V^\star(s) = \max_a \{r(s, a) + \beta \, \mathbb{E}[V^\star(s') \mid s, a]\}$. Reinforcement learning finds an approximately optimal policy from interaction with the environment rather than from a fully specified model. We refer to RL algorithms (value iteration, Q-learning, policy gradient, actor-critic, soft actor-critic, proximal policy optimisation, deep deterministic policy gradient) by name without re-deriving them; the algorithmic taxonomy is covered in Sections 4 and 5 of the survey.

9.1.2 Generalisations used in this chapter

Four generalisations of the MDP appear in the macro literature surveyed below.

A partially observed MDP (POMDP) supplements the MDP with an observation space $\mathcal{O}$ and an emission distribution $U(o \mid s)$. The agent does not see the state s directly; it conditions its policy on the observation history $o_{0:t}$, typically through a recurrent network or a sufficient statistic (a function of the history that preserves all decision-relevant information). POMDPs arise in macro whenever a household sees only prices or summary statistics rather than the full

cross-sectional distribution, that is, the population-wide distribution of individual states such as wealth and productivity. The cross-sectional distribution matters to the household because equilibrium prices, and therefore its own budget constraint, depend on what the rest of the population is doing.

A Markov game extends the MDP to n agents who share the state but choose actions independently. Each agent $i \in \{1, \dots, n\}$ has its own action space $\mathcal{A}_i$ and reward $r_i(s, a)$, where $a = (a_1, \dots, a_n)$ is the joint action. Each agent treats the other agents' policies as part of a non-stationary environment and maximises its own return. Two-level games (a planner and a population of agents) are a special case.

A mean field game with common noise takes the continuum limit $n \to \infty$. A representative agent interacts with the cross-sectional distribution $\mu \in \Delta(\mathcal{S})$ of individual states and with a common noise $z \in \mathcal{Z}$ that captures aggregate shocks. The individual transition is $T(s' \mid s, a, \mu, z)$; the mean field evolves under $\mu_{t+1} = \Phi^{\pi}(\mu_t, z_t)$ for an operator Φ^{π} induced by the policy. The continuum limit makes the otherwise exponential per-agent state space tractable at the cost of losing finite-population effects.

A multi-objective MDP replaces the scalar reward r with a vector $r \in \mathbb{R}^d$. The solution concept is a Pareto front rather than a single optimum.

9.1.3 Equilibrium concepts

A policy $\pi_i^\star$ is a best response to the other agents' policies π_{-i} when $V_i^{(\pi_i^\star, \pi_{-i})}(s) \geq V_i^{(\pi_i', \pi_{-i})}(s)$ for every alternative π_i'. A Nash equilibrium is a profile $(\pi_1^\star, \dots, \pi_n^\star)$ in which every component is a best response to the others. A mean-field Nash equilibrium is the continuum analogue, in which the representative agent's policy is a best response to the cross-sectional distribution it induces. A restricted perceptions equilibrium is a best response within a restricted policy class, for example policies that condition on prices but not on the full distribution; it is weaker than rational expectations and is the equilibrium concept SRL targets in Section 9.3. An ε-equilibrium weakens Nash by an ε slack on each player's incentive. Exploitability, $\sup_{\pi'} V^{\pi'} - V^{\pi}$ evaluated at the current mean field (the gap between the best response and the current policy), is the convergence metric used in MFG-RL; it upper-bounds the gap to a mean-field Nash equilibrium.

9.1.4 Notation summary

Throughout the chapter we use s for state, a for action, r for reward, π for policy, V and Q for values, β for the discount factor, P or T for the transition kernel, $\mu \in \Delta(\mathcal{S})$ for the cross-sectional distribution, $z \in \mathcal{Z}$ for the aggregate shock or common noise, p for the equilibrium price vector when explicit, θ for policy parameters, ϕ for auxiliary parameters when a second network is needed (a critic in single-agent RL, a planner in two-level RL), n for the number of agents in a Markov game, and i for an agent index. Algorithm-specific scalars (training-step counts N, T, trajectory counts E) are introduced inside the relevant algorithm boxes.

The chapter applies RL to macro models under two interpretations, introduced above and used without further comment hereafter. Under the solver interpretation, the policy network is a numerical device for finding an equilibrium that closed-form or DP-based methods cannot reach. Under the behavioural interpretation, the policy network is the household's actual cognition, learning a policy from realised utility under no prior model knowledge. Section 9.7 leans on the behavioural interpretation; Sections 9.2, 9.3, and 9.5 lean on the solver interpretation; the policy-design section uses one or the other depending on whether the planner is the focus.

A growing parallel literature uses deep learning rather than reinforcement learning to solve heterogeneous-agent and DSGE models by training networks to satisfy equilibrium equations as nonlinear regression objectives (**??**Azinovic-Yang and Žemlička, 2026; **?**; **?**; **?**; **?**; **?**; **?**); Fernández-Villaverde et al. (2024) survey this line. The distinction is that those methods

compute equation residuals offline, while RL trains the network from a reward signal under the agent's own policy. The chapter restricts attention to the RL lens and surfaces the equation-residual literature only as a comparison point where relevant.

9.2 RL for single-agent macro models

The simplest macroeconomic environment in which RL can be benchmarked against dynamic programming is the representative-agent stochastic real business cycle. The state space is two-dimensional, the analytical and numerical benchmarks are well understood, and the training dynamics expose how RL behaves on a problem for which DP remains the right tool. The interest is in the comparison rather than in any new claim about the RBC.

Atashbar and Shi (2023) train a representative household with deep deterministic policy gradient on a canonical stochastic RBC with log utility, Cobb-Douglas production, and AR(1) total factor productivity. The state is (K, A) with K capital and A productivity, the action is consumption C, and the transition is the capital accumulation equation together with the AR(1) for productivity. In both a deterministic variant without TFP shocks and a stochastic variant with shocks, the trained agent's consumption and investment policy lies close to the deterministic steady-state benchmark. The paper also documents training runs that fail to converge under high discount factors and suboptimal critic hyperparameters; the instability foreshadows the heterogeneous-agent setting in Section 9.3, where long-horizon credit assignment across an idiosyncratic-shock distribution amplifies the same problem. Section 9.4 returns to the same RBC with a side-by-side comparison across PPO, DDPG, value-function iteration, and a Blanchard-Kahn log-linearisation.

Shi (2022) trains an actor-critic agent on a stochastic optimal-growth environment with no a priori knowledge of preferences or the productivity process. The agent recovers the rational-expectations optimal saving rule asymptotically. Higher exploration accelerates learning at the cost of lower realised welfare during the exploration phase, which gives the explore-vs-exploit trade-off a macroeconomic interpretation that Section 9.7 returns to under the behavioural lens.

9.3 RL as a global solver for heterogeneous-agent macro models

Heterogeneous-agent (HA) macro models replace the representative agent of Section 9.4 with a continuum of households facing uninsurable idiosyncratic shocks on top of an aggregate shock. Canonical examples are Aiyagari (1994), Krusell and Smith (1998), and Kaplan et al. (2018). The state of the agent's problem now includes the cross-sectional distribution $\mu_t \in \Delta(\mathcal{S})$ of individual states, because equilibrium prices depend on μ_t and the distribution itself depends on the policy. Dynamic programming runs on an infinite-dimensional state and the value function lives on $\mathcal{S} \times \Delta(\mathcal{S}) \times \mathcal{Z}$. The traditional macro response summarises μ_t by a handful of moments (Krusell and Smith, 1998) or linearises around a stationary equilibrium (Reiter, 2009; Auclert et al., 2021). Both reductions impose ex ante which features of the distribution matter.

Yang et al. (2025) propose Structural Reinforcement Learning (SRL), which sidesteps the distribution entirely. The structural assumption is that the household knows its own budget constraint, preferences, and idiosyncratic shock process exactly. The only unknown part of the environment from the household's point of view is the joint process for equilibrium prices and aggregate shocks. SRL trains the policy by simulating the full economy and treating prices as exogenous to the agent through a stop-gradient. The sequence-space view of Auclert et al. (2021) justifies replacing μ in the agent state with the equilibrium price sequence: under rational expectations the price path is itself a sufficient statistic for the part of μ that affects forward-looking household decisions, because the household's budget constraint depends on μ only through equilibrium prices. Han et al. (2022a), the immediate predecessor of SRL, kept the full cross-sectional distribution μ in the agent state via a deep-network policy that conditions on μ directly; SRL replaces μ with prices.

Algorithm 1 Structural Policy Gradient (SPG) for HA macro models

Require: initial tabular policy $\theta_0 \in \mathbb{R}^{J\times K\times L}$ on the (s, z, p) grid; step sizes $\{\eta_k\}$; trajectory count N; horizon T; initial distributions ψ_g, ψ_z; convergence tolerance ε

for $k = 0, 1, 2, \ldots$ **do**
  Sample N initial conditions $(\mu_0^n, z_0^n) \sim (\psi_g, \psi_z)$ for $n = 1, \ldots, N$
  Initialise the value-computation distribution $d_0^n = d_0$, uniform over the individual-state grid
  for $n = 1, \ldots, N$ **do**
    for $t = 0, 1, \ldots, T-1$ **do**
      Compute the equilibrium price $p_t^n = P^*(\mu_t^n, z_t^n)$ from market clearing, with a stop-gradient applied to p_t^n
      Roll the cross-sectional distribution: $\mu_{t+1}^n = A_{\pi_\theta}(z_t^n, p_t^n)^\top \mu_t^n$ (used only to update prices)
      Roll the value-computation distribution: $d_{t+1}^n = A_{\pi_\theta}(z_t^n, p_t^n)^\top d_t^n$ (used only for the value)
      Sample the next aggregate shock $z_{t+1}^n \sim \Xi(\cdot \mid z_t^n)$
    end for
  end for
  Form the Monte Carlo value $\widehat{v}^\pi(\theta_k) = \frac{1}{N}\sum_{n=1}^{N}\sum_{t=0}^{T-1}\beta^t \langle d_t^n,\, u^{\pi_\theta}(z_t^n, p_t^n)\rangle$
  Compute the analytic gradient $\nabla_\theta \widehat{v}^\pi(\theta_k)$ by backpropagating through A_{π_θ} with prices held fixed by the stop-gradient
  Update $\theta_{k+1} \leftarrow \theta_k + \eta_k \nabla_\theta \widehat{v}^\pi(\theta_k)$
  if $\|\theta_{k+1} - \theta_k\|_\infty < \varepsilon$ **then**
    return θ_{k+1}
  end if
end for

The SRL household's state is $s_t = (b_t, y_t, z_t, p_t)$, with b_t wealth, y_t the idiosyncratic productivity shock, z_t the aggregate shock, and p_t the equilibrium price vector. The action is $a_t = c_t$, consumption, with savings b_{t+1} determined residually by the budget. The next state is generated by three structural pieces: the budget constraint for b_{t+1}, the exogenous Markov chain for (y_{t+1}, z_{t+1}), and the market-clearing update $p_{t+1} = P^\star(\mu_{t+1}, z_{t+1})$ that the agent treats as exogenous. The reward is $r_t = u(c_t)$. The policy class is restricted to $\pi(s, z, p)$, which means the agent never conditions on μ at any step. Yang et al. (2025) call the resulting fixed point a sequential restricted perceptions equilibrium: given the price process induced by $P^\star$, the policy $\pi^\star$ maximises expected discounted utility within the restricted class, and market clearing holds at $p_t = P^\star(\mu_t, z_t)$ for every t. Agents' price beliefs are statistically consistent with realised prices on the equilibrium path, and the equilibrium is a fixed point in policy space rather than in the perceived-law-of-motion regression of Krusell-Smith.

The policy is parameterised as a tabular vector $\theta \in \mathbb{R}^{J\times K\times L}$ on the discretised (s, z, p) grid, and training uses the structural policy gradient algorithm of Algorithm 1. The cross-sectional distribution rolls forward to compute prices while a separate value-computation distribution rolls forward to score the policy. The Monte Carlo value is differentiated analytically by backpropagating through the known one-step transition matrix $A_\pi(z, p)$, the grid-restricted version of the transition kernel. Convergence is declared when the ℓ_∞ distance between successive policy iterates falls below a tolerance ε.

Yang et al. (2025) report runtimes on a single A100 GPU. Krusell-Smith converges in roughly 55 seconds and the authors report that its dynamics match the deep-learning rational-expectations benchmark. The Huggett model with aggregate risk, which has a nontrivial market-clearing condition for bonds, converges in roughly 75 seconds. A one-account HANK

model with a forward-looking Phillips curve and sticky prices converges in roughly three minutes. Allowing agents to condition on a short price history rather than only the current price does not change the solution materially, which suggests that current prices encode most of the information relevant for behaviour in these calibrations.

9.4 Simulation: representative-agent RBC under DP and DRL

We check that RL matches dynamic programming on a problem for which DP is the right tool. The environment is a standard representative-agent stochastic RBC. The household chooses consumption C_t and saves the residual, with capital evolving as $K_{t+1} = (1-\delta)K_t + A_t K_t^{\alpha} - C_t$ and total factor productivity following $\log A_{t+1} = \rho \log A_t + \varepsilon_{t+1}$ for $\varepsilon_{t+1} \sim \mathcal{N}(0, \sigma^2)$. Utility is $u(C_t) = \log C_t$. Parameters are calibrated as $\beta = 0.96$, $\alpha = 0.36$, $\delta = 0.10$, $\rho = 0.95$, $\sigma = 0.007$, with horizon $T = 200$. The deterministic steady state is $K^{\star} = 4.29$, $C^{\star} = 1.26$.

The state is $s = (K, A)$, the action is $a = C$, the reward is $r = \log C$, and the transition is the capital accumulation equation together with the AR(1) for TFP. Four methods are compared: a Blanchard-Kahn log-linearisation around the deterministic steady state (KPR), value-function iteration on a 400×41 tensor grid with a Tauchen-discretised TFP process (VFI), proximal policy optimisation with a Gaussian actor (PPO), and deep deterministic policy gradient (DDPG).[171]

Table 16 reports mean episode return, standard error, and policy mean-squared error against VFI. KPR matches VFI to within 0.001 in policy MSE and within statistical noise on mean return. PPO converges to within the standard error of VFI with a policy MSE of 0.009. DDPG attains a mean return within 2% of VFI but with roughly double the cross-seed standard error and a policy MSE of 0.037, consistent with the known sensitivity of deterministic policy gradient methods to hyperparameter and initialisation choices. Figure 17 traces the learning curves of PPO and DDPG against the VFI and KPR horizontal benchmarks. Both RL methods approach the analytical benchmark within their respective training budgets. The result is what one would hope on a model this small. It is not a demonstration that RL replaces dynamic programming; it is a check that RL does not fail it on the textbook case.

Method	Mean return	SE	Policy MSE vs VFI	Wall clock
KPR	45.88	0.587	0.0004	–
VFI	45.86	0.589	0.0000	13.5 s
PPO	45.82	0.843	0.0087	–
DDPG	45.17	1.476	0.0365	–

Table 16: Representative-agent RBC under four methods. Mean episode return averaged over 30 evaluation episodes with shared initial conditions and shock paths. Standard error across 10 training seeds for PPO and DDPG, across 30 evaluation episodes for KPR and VFI. Policy mean-squared error reported against VFI consumption on 1000 random states drawn uniformly from the evaluation support.

9.5 Mean Field Games as a large-population RL problem

The mean-field game generalisation of Section 9.1 applies when large populations of identical, anonymous agents interact through aggregate quantities. Direct multi-agent RL does not scale, since the joint state and action spaces grow exponentially in n. Taking the continuum limit

[171] Both RL methods use a 64-64 multilayer perceptron for actor and critic. Each is trained from scratch with 10 independent seeds. Every method is evaluated on a fixed set of 30 initial conditions $(K_0, A_0) \sim \mathcal{U}[0.5, 8] \times \mathcal{U}[0.95, 1.05]$ and shock paths $\{\varepsilon_t\}_{t=1}^{T}$, identical across methods, so cross-method differences are not driven by evaluation noise. The training budgets and convergence-monitoring schedules are recorded in the simulation script.

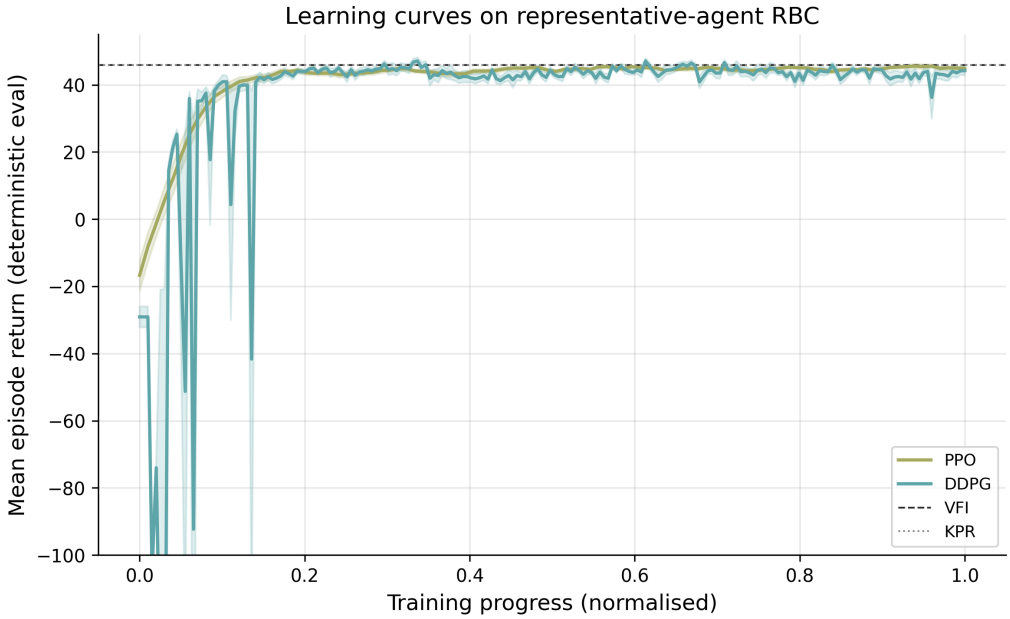

Figure 17: Mean deterministic-evaluation return as a function of normalised training progress for PPO (olive) and DDPG (cyan), shaded by cross-seed standard error across ten training seeds. Horizontal lines mark the VFI benchmark mean return (dashed black) and the KPR log-linear benchmark (dotted gray).

reduces the problem to a single representative agent whose environment includes the population distribution $\mu \in \Delta(\mathcal{S})$ and the common noise z. The RL problem is to find a policy that is a best response to the distribution it induces under the noise process. Convergence to a mean-field Nash equilibrium is monitored by exploitability.

The Krusell-Smith environment appears here under a different lens than in Section 9.3. SRL conditions agents on the current price vector and treats it as a Markovian sufficient statistic; RSPG instead lets agents carry a recurrent memory of past observations, relaxing the Markov assumption and replacing restricted perceptions with a partially observable mean-field Nash equilibrium. The two are alternative solution concepts for the same underlying household problem, not competing claims about the same equilibrium.

9.5.1 Recurrent Structural Policy Gradient

Wibault et al. (2026) propose the Recurrent Structural Policy Gradient (RSPG) algorithm for partially observable mean field games with common noise. In macroeconomic applications agents typically observe equilibrium prices but not the cross-sectional distribution, so the natural formulation is a POMFG. RSPG exploits the fact that individual transitions $T(s' \mid s, a, \mu, z)$ are known to the agent (the household knows its own budget constraint and idiosyncratic shock process), while the aggregate process for (μ, z) must be learned. The setting therefore sits between dynamic programming, which would integrate over both, and model-free RL, which would sample both.

In chapter notation the POMFG with common noise is a tuple $(p_{\mu_0}, p_{z_0}, \mathcal{S}, \mathcal{Z}, \mathcal{A}, T, \Xi, R, \beta, U, \mathcal{O})$, collecting initial distributions for the individual state and common noise, individual and aggregate state and action spaces, individual and common-noise transitions, reward, discount, and observation map. It combines the MFG primitives of Section 9.1 with the POMDP observation pair $(U, \mathcal{O})$ and a common-noise transition $\Xi(\cdot \mid z)$. The mean field evolves under the analytic operator $\mu_{t+1} = \Phi^{\pi}(\mu_t, z_t)$ with (s, s') entry $\mathbb{E}_{a\sim\pi}[T(s' \mid s, a, \mu, z)]$. The RSPG architectural choice is to restrict the policy memory to a history of shared aggregate observations $o_{0:t}$, param-

Algorithm 2 Recurrent Structural Policy Gradient (RSPG)

Require: initial recurrent policy θ_0; trajectories E; horizon T

1: **for** $k = 0, 1, 2, \ldots$ until convergence **do**
2: Sample E common-noise trajectories $z^e_{0:T}$ with $z^e_0 \sim p_{z_0}$ and $z^e_{t+1} \sim \Xi(\cdot \mid z^e_t)$
3: Roll forward $\mu^e_{t+1} = \Phi^{\pi_\theta}(\mu^e_t, z^e_t)$ analytically, with policy memory $h^e_{t+1} = \mathrm{RNN}(h^e_t, o^e_t)$
4: Form the sample return $\widehat{J}(\theta_k)$ from (85) averaged over $e = 1, \ldots, E$
5: Update $\theta_{k+1} \leftarrow \theta_k + \eta_k \nabla_\theta \widehat{J}(\theta_k)$ with gradients flowing through T and R only
6: **end for**
7: **return** θ^*

eterised as the hidden state of a recurrent neural network. The reduced policy $\pi(a \mid s, h(o_{0:t}))$ retains history-dependence while keeping the asymptotic cost of the analytic mean-field update independent of the individual state.

For an initial common-noise distribution p_{z_0}, RSPG samples E trajectories in parallel, rolls out the endogenous mean field $\mu^\pi_{0:T}$ via the analytic operator Φ^π, and computes the discounted return

$$J^\pi \;=\; \mathbb{E}_{z_{0:T} \sim \Xi}\Big[\sum_{t=0}^{T-1} \beta^t \,\langle \mu^\pi_t,\; R(\cdot,\, \pi,\, \mu^\pi_t,\, z_t)\rangle\Big]. \tag{85}$$

Gradients flow through the individual transitions T and the expected-reward over actions. Gradients do not flow through the mean-field transitions; the analytic mean-field update is treated as a fixed environment, mirroring SRL's treatment of equilibrium prices.[172]

Wibault et al. (2026) evaluate RSPG on partially observable Linear-Quadratic, Beach-Bar, and Krusell-Smith environments. Across the three, RSPG attains the lowest or second-lowest exploitability among the algorithms compared and converges roughly an order of magnitude faster in wall-clock time than independent and recurrent PPO baselines. Finite-horizon agents under RSPG spend wealth just before episode end, pushing interest rates up and wages down. Memoryless variants of SPG and IPPO do not show this pattern. The paper positions RSPG as the first method to solve a partially observable Krusell-Smith model in which agents observe prices but not the cross-sectional distribution.

9.5.2 Hierarchical Stackelberg mean-field RL

Mi et al. (2025) extend the mean-field formulation of Section 9.5.1 from a symmetric population to a hierarchical setting in which a government (leader) sets policy first and a continuum of heterogeneous individual followers optimises against the announced policy. The framework formalises a Dynamic Stackelberg Mean Field Game (DSMFG) as the tuple $(\mathcal{S}^l, \mathcal{S}^f, \mathcal{A}^l, \mathcal{A}^f, P, r^l, r^f, T)$ collecting leader state space, follower state space, leader action space, follower action space, joint transition, leader reward, follower reward, and horizon. A mean field $L_t \in \Delta(\mathcal{S}^f \times \mathcal{A}^f)$ summarises the population state-action distribution.

The follower's best response to a leader policy π^l is the representative-agent policy

$$\pi^{f,*}(\pi^l) \;\in\; \arg\max_{\pi^f} \;\mathbb{E}\left[\sum_{t=0}^{T} \beta^t \, r^f(s^f_t, a^f_t, L_t, \pi^l_t)\right], \tag{86}$$

under the McKean-Vlasov fixed point that requires L_t to be the distribution induced by $\pi^{f,*}(\pi^l)$

[172] The training loop is parameterised by the common-noise-trajectory batch size E, the rollout horizon T, and the RNN hidden dimension. Convergence is monitored by exploitability, the gap between the value of a best-response policy and the value of the agent's current policy at the current mean field. The exploitability metric requires a separate best-response oracle that Wibault et al. (2026) train as a single-agent RL agent against the frozen mean-field path; reported exploitabilities are therefore upper bounds.

at t. The leader then chooses

$$\pi^{l,*} \in \arg\max_{\pi^l} \mathbb{E}\left[\sum_{t=0}^{T} \beta^t \, r^l(s_t^l, a_t^l, L_t)\right] \tag{87}$$

anticipating the follower best response. The dynamic feedback is critical, as a static Stackelberg formulation cannot capture the temporal coupling that arises when followers observe and respond to the leader's policy over T periods.

The Stackelberg Mean Field Reinforcement Learning (SMFRL) algorithm trains both levels jointly under centralised training with decentralised execution. A Stackelberg mean-field Q-function $\widetilde{Q}^l(s^l, a^l, L)$ scores leader actions given the population state-action distribution; a follower-shared Q-network conditions on the same mean field. Best-response updates alternate: fix the leader, train followers to a ε-best-response and update L; then fix followers, optimise the leader. The mean-field approximation reduces the pairwise complexity of agent-agent and agent-government interactions from $O(N^2)$ to $O(N)$.

Mi et al. (2025) report that DSMFG scales to 1,000 followers where the prior multi-agent benchmark in the same TaxAI environment (Mi et al., 2024) ran at one hundred agents. On the optimal-tax benchmark the learned leader policy achieves a four-fold per-capita GDP gain over the analytical Saez optimum (Saez, 2001) and a nineteen-fold improvement over the static 2022 US federal income tax schedule, both within the calibrated TaxAI environment. Ablations confirm that removing either the Stackelberg hierarchy or the mean-field approximation loses welfare or training stability. Exploitability decreases monotonically over training, indicating convergence to an approximate Stackelberg mean-field equilibrium. DSMFG bridges the symmetric mean-field methods of Section 9.5.1 and the single-planner setting of Section 9.8, in that the leader-follower structure makes the planner an explicit second decision-maker without dropping the continuum-population approximation that keeps the problem tractable.

9.5.3 Offline, asynchronous, and non-stationary MFG-RL

Three recent algorithms extend MFG-RL along orthogonal axes. Brunnbauer et al. (2024) learn an equilibrium policy from a static dataset of past trajectories without online interaction, by adapting Munchausen online mirror descent to offline data with an overestimation-mitigating regulariser; their experiments are on crowd-exploration and navigation, not macro. Yang (2026) drop the synchronous-move assumption by replacing the mean-action statistic $\bar{a}$ with the population distribution $\mu \in \Delta(\mathcal{O})$, derive a TMF Bellman equation, and prove an $O(1/\sqrt{N})$ finite-population exploitability bound that holds for any batch size of acting agents. Magnino et al. (2025) consider non-stationary continuous-space MFGs and report roughly an order-of-magnitude reduction in sampling cost on their benchmarks via a fictitious-play loop with a time-conditional normalising flow on the equilibrium density.[173]

9.6 Simulation: a partially observable linear-quadratic mean field game

To make the RSPG mechanism concrete, we use a compact version of the partially observable Linear-Quadratic environment in Wibault et al. (2026). The individual state is $s \in \{0, \ldots, 98\}$ in the public MFAX implementation, the action is $a \in \{-3, \ldots, 3\}$, the common noise is $z \in \{-1, 1\}$, and the horizon is $T = 30$. The common noise is persistent within an episode. It shifts the whole population down early in the episode, is neutral in the middle, and shifts it in the opposite direction late in the episode. Let $\bar{s}_t = \sum_{s'} \mu_t(s') s'$ denote the population mean. The agent observes its own state and $o_t = \bar{s}_t$, but not the full distribution, the common-noise

[173] Peng and Wang (2025) apply PPO with generalised advantage estimation and target networks to mean-field control games, a hybrid that combines internal coordination as in mean-field control with external competition as in MFGs.

realisation, or the calendar time. The mean field evolves by the analytic population transition operator $\mu_{t+1} = \Phi^{\pi}(\mu_t, z_t)$.

The step reward follows the paper's quadratic form,

$$R_t(s, a, \mu, z) = -c_a a^2 + qa(\bar{s}_{t+1} - s) - \frac{\kappa}{2}(\bar{s}_{t+1} - s)^2, \tag{88}$$

with terminal reward $R_T(s, \mu, z) = -c_{\text{term}}(\bar{s}_T - s)^2/2$. The implementation follows MFAX in evaluating the reward after the population transition, using $\bar{s}_{t+1}$, and using the terminal reward on the final transition. The reward encourages agents to coordinate with the moving population while paying an action cost.

The reported results come from the public MFAX implementation of the paper's two structural policy-gradient algorithms on this environment. SPG uses the analytic mean-field update and a memoryless categorical policy that conditions on current state and current mean observation. RSPG uses the same analytic update, but encodes the sequence of shared mean-state observations with a GRU; the recurrent hidden state is independent of the individual state, as in the paper. Both methods are evaluated with the analytic mean-field path induced by the final policy. Approximate exploitability is computed by backward induction against that path, so lower values mean a smaller gain from deviating to a best response. We do not include the paper's sampled recurrent PPO grid in the table, because that baseline is a much heavier finite-agent RL experiment rather than the structural mean-field update studied here.

Table 17 reports exploitability, expected return, and training time across ten training seeds after sweeping the official learning-rate grid. RSPG has lower exploitability than SPG, which is the partial-observation lesson of the paper's LQ example: history helps infer where the common-noise episode is headed. Expected returns are reported for completeness but should not be read as a welfare ranking across methods, since each method induces a different mean-field path. Figure 18 plots exploitability over training and the final exploitability sensitivity to the learning rate.

Method	LR	Exploitability	Expected return	Train time (s)
SPG	10^{-2}	86.64 ± 16.29	-1021.3 ± 22.3	24.39 ± 0.06
RSPG	10^{-3}	60.37 ± 4.11	-1186.9 ± 10.1	28.79 ± 0.58

Table 17: Partially observable Linear-Quadratic mean field game. Approximate exploitability is computed by a best-response dynamic program on the analytic mean-field path induced by the final policy. Expected return and training time are averaged over 10 training seeds, with standard errors across seeds. Learning rates are selected from the official MFAX grid $\{10^{-4}, 10^{-3}, 10^{-2}\}$ by mean final exploitability for each method.

9.7 RL as a model of bounded rationality

The preceding sections used RL as a numerical device for finding equilibrium policies that closed-form or DP-based methods cannot reach. The interpretation switches here. The household itself learns from realised utility under no a priori knowledge of the income process, and the question is whether the resulting consumption behaviour matches empirical patterns that rational expectations does not.

Kaplowitz (2025) models a household solving a Markovian consumption-savings problem with cash-on-hand $x = y + Ra$, consumption share $\psi \in [0, 1]$, and savings residual $a' = (1 - \psi)x$, where $y \in \{y_e, y_u\}$ denotes employed and unemployed income drawn from a Bernoulli transition matrix $P_{yy'}$. The household does not know $P_{yy'}$ and learns an expected-continuation-value estimator $\widehat{EV}(y, a'; \phi)$ as a neural network with weights ϕ updated by stochastic gradient descent

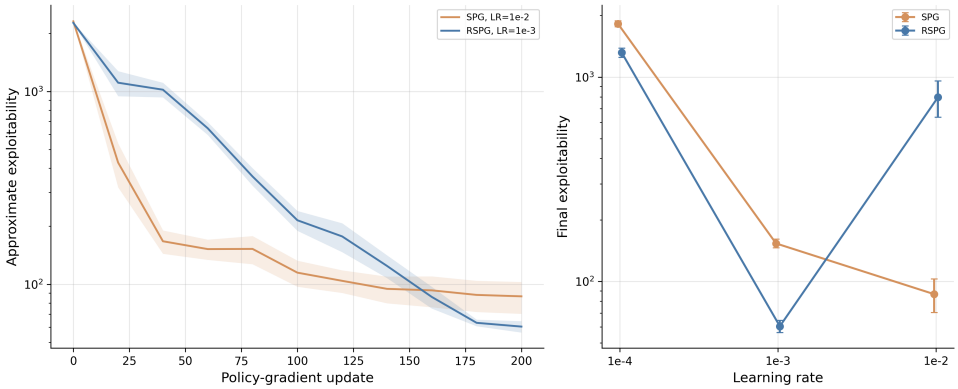

Figure 18: Official MFAX Linear-Quadratic grid. The left panel shows approximate exploitability over policy-gradient updates for the selected SPG and RSPG learning rates, shaded by cross-seed standard error across ten training seeds. The right panel shows final exploitability across the swept learning rates.

under a Robbins-Monro $1/\sqrt{t}$ learning-rate schedule. The Q-function is

$$Q(a, y, a') \;=\; u(\psi x) + \beta\, \widehat{EV}(y, a'; \phi), \tag{89}$$

and the consumption policy is a polynomial fit over Q. The agent is initialised at the rational-expectations solution to the same problem, so any deviation from rational behaviour reflects the learning dynamics rather than initial misspecification. The agent runs for fifty quarters with parameters drawn from the standard household-finance calibration (Krueger et al., 2016; Ganong et al., 2024 as cited in the source).

The same RL mechanism reproduces two empirical patterns that the combination of full-information rational expectations and borrowing constraints cannot generate jointly. First, the Ganong-et-al. marginal propensity to consume out of stimulus transfers is around 0.50 for previously-low-asset unemployed households and around 0.34 for previously-high-asset unemployed households, even when neither group is at a borrowing constraint. The simulated agents reproduce a gap of roughly the same magnitude (approximately 0.20). Second, the Malmendier-Shen scarring effect, in which past unemployment experience persistently lowers consumption controlling for current state, appears with the correct sign in the simulated paths. Both patterns trace to a single mechanism, value-function approximation errors that evolve with experience and generate ex-post heterogeneity from ex-ante identical agents. Belief-updating explanations, in which the household revises beliefs about $P_{yy'}$ after each unemployment spell, can produce the scarring effect but force consumption levels and MPCs to co-move in a way the data do not show.

Shi (2022) (Section 9.2) shows that a single-agent actor-critic on stochastic optimal growth converges to the rational-expectations optimal saving rule asymptotically; KAP's contribution is the non-asymptotic regime in which the agent has not yet converged. Kuriksha (2021) embeds heterogeneous neural-network learners in an Aiyagari-class economy. Each household updates its saving rule from its own realised consumption history, and the resulting wealth distribution is fat-tailed and exhibits the high average MPCs and excess sensitivity documented in the empirical literature, even without any single agent matching specific anomalies. The two papers together support the reading that RL households fail to satisfy rational expectations during learning in ways that reproduce stylised micro and macro facts, and that the mechanism is generic to value-function approximation rather than a peculiarity of any one architecture.

9.8 Multi-agent reinforcement learning for policy design

The first variant of RL for policy design treats the planner and the citizens as RL learners co-adapting in a shared environment. The planner's tax schedule, monetary rule, or climate-cooperation protocol is the action of one RL agent; the citizens' consumption, labour, or mitigation responses are the actions of other RL agents. Both sides learn from realised payoffs, so the planner sees the emergent strategic responses of the citizens to its policy choices. This setting is a Markov game in the sense of Section 9.1, with n heterogeneous agents whose actions are coordinated either by market clearing or by explicit search-and-match protocols. The contribution of RL is the discovery of mechanism designs and the surfacing of emergent behaviours that classical optimisation cannot reach. The single-planner variant in Section 9.9 drops the multi-agent training and treats the rest of the macroeconomy as a fixed environment.

9.8.1 MARL on canonical macro environments

Gabriele et al. (2026) build MARL-BC, an n-household RBC in which the representative-agent RBC and the Krusell-Smith mean-field economy appear as limit cases. Each household chooses a consumption fraction and labour supply at every period. Production is Cobb-Douglas in productivity-weighted aggregate capital and labour. In the chapter's Markov-game notation, the state is shared across agents and contains aggregate capital, labour, and the per-agent productivity draws; each agent's action is its consumption fraction and labour choice; each agent's reward is the log-plus-leisure utility

$$r_t^i \;=\; \log c_t^i + b\,\log(1-\ell_t^i). \tag{90}$$

The authors benchmark PPO, SAC, DDPG, and TD3 and report that the off-policy methods are more sample-efficient than PPO on the configurations they test.[174] With $n = 1$ the framework reproduces the textbook RBC solution. With a large population of ex-ante identical agents (in their largest run, on a 23×23 productivity grid yielding $n = 529$) it reproduces the Krusell-Smith mean-field wealth distribution and aggregate dynamics. With a population of heterogeneous agents it produces wealth and consumption distributions that mean-field methods cannot represent.

Curry et al. (2022) consider a richer micro-founded RBC with 100 worker-consumers, 10 firms, and a tax-setting government, formulated as a partially observable Markov game in which firms set prices and wages and goods are rationed proportionally if demand exceeds supply. Each agent type is trained by independent PPO with parameter sharing within type, accelerated by the WarpDrive GPU framework.[175] In the open-economy variant with an external export market, the model exhibits multiple qualitatively distinct ε-meta-equilibria.

9.8.2 RL agents inside macro ABMs

Brusatin et al. (2024) extend an existing macroeconomic ABM with capital and credit, in which 1,000 worker-households, 20 capital-goods firms, 100 consumption-goods firms, and one bank interact through search-and-match consumption markets, a labour market, and a credit market. The consumption-goods firms in the original ABM follow a hand-coded trend-following rule for setting prices and target quantities. The authors replace N of these 100 firms with tabular Q-learners.

[174] Sample efficiency is reported in terms of environment steps to reach a fixed return threshold. The exact threshold and the number of seeds vary across the three population configurations, $n = 1$, $n = 20$ ex-ante identical, $n = 20$ heterogeneous. The paper does not provide a single hyperparameter table for the four algorithms.

[175] Without a curriculum, independent learners fail to recover non-trivial behaviour. A multi-phase curriculum that stages consumer, firm, and government training recovers non-trivial ε-meta-equilibria. The ε metric is evaluated by training single-agent best-response oracles against frozen others, so reported values are upper bounds.

Each RL firm i has state $s_{i,t}$ consisting of its log price-delta and log stock-delta relative to market averages, both binned on a finite grid; the action $a_{i,t}$ is a discrete multiplicative adjustment to next-period price and quantity; the reward $r_{i,t}$ is profit normalised by assets, with a bankruptcy penalty. The Q-table update is the standard tabular Q-learning rule

$$Q_{i,t+1}(s_{i,t}, a_{i,t}) \;=\; (1-\alpha)\, Q_{i,t}(s_{i,t}, a_{i,t}) + \alpha \left[\, r_{i,t} + \beta \max_{a'} Q_{i,t}(s_{i,t+1}, a') \,\right] \tag{91}$$

with learning rate α and ε-greedy exploration.[176]

Brusatin et al. (2024) report that RL firms learn three distinct profit-maximising strategies depending on competition level and the share of rational firms. Independent learners with separate Q-tables segregate into distinct strategic groups, raising aggregate market power and profits. Raising the share of RL firms always raises total output in the configurations tested, sometimes at the cost of higher output volatility.

9.8.3 Two-level deep RL for optimal taxation

Zheng et al. (2022) propose the AI Economist, a two-level deep RL framework for optimal taxation. The inner level is a population of agents who maximise discounted post-tax utility; the outer level is a social planner who sets a tax schedule to maximise a social-welfare function. Both levels are parameterised by deep neural networks and trained with proximal policy optimisation.

In the chapter's Markov-game notation, agent i has policy π_θ and solves

$$\max_{\pi_\theta} \; \mathbb{E} \sum_{t=0}^{H} \beta^t \, u_i(C_{i,t}, L_{i,t}), \tag{92}$$

where u_i is isoelastic in post-tax income $C_{i,t}$ with curvature $\eta > 0$ and linear in cumulative labour $L_{i,t}$; post-tax income depends on a bracketed tax schedule $\mathcal{T}(z;\tau)$. The planner has policy π_ϕ and solves

$$\max_{\pi_\phi} \; \mathbb{E} \sum_{t=0}^{H} \beta^t \, \mathrm{swf}_t, \tag{93}$$

with swf_t either inverse-income-weighted utilitarian welfare or the product of equality and productivity.

The two-level formulation is unstable because each side's effective MDP is non-stationary in the other side's policy. Zheng et al. (2022) stabilise training with two mechanisms. A curriculum gradually introduces labour costs and then taxes, so that early in training the agents experience low costs and explore freely. Entropy regularisation on π_ϕ keeps the tax schedule diffuse, so that agents experience a wide range of tax rates and learn to condition on them. These mechanisms stage training in two phases: agents adapt to random taxes first; the planner then enters and the two levels co-adapt.[177]

Zheng et al. (2022) test the framework in two environments. In a one-step economy with closed-form Saez optimum (Saez, 2001), the AI Economist's learned tax schedule matches the Saez optimum to within statistical noise. In the four-agent Open-Quadrant variant of the Gather-Trade-Build spatial environment, the learned schedule reportedly outperforms a Saez

[176] The grid discretisation, the exploration-rate schedule, and the training-step budget per firm are left as configurable inputs in the published code. The shared-policy variant uses a single Q-table across all RL firms; the independent-policy variant maintains a separate Q-table per firm. The market-competition level is controlled by the number of firms a consumer compares prices across.

[177] Both networks are LSTM-based and trained with PPO; the agent and planner have separate replay buffers $\mathcal{D}$ and $\mathcal{D}_p$. The planner acts every T environment steps at a tax-period boundary; agents act every step. Curriculum and entropy schedules are advanced between training iterations and reported in the paper's experimental appendix. The reported wall-clock cost is several thousand GPU-hours for the Gather-Trade-Build runs, the largest training budget of the anchor papers in this chapter.

Algorithm 3 Two-level Reinforcement Learning, after Zheng et al. (2022)

Require: initial agent parameters θ, planner parameters ϕ; sampling horizon H; tax-period length T; inner learner $\mathcal{A}$ (PPO in this work)

Initialise curriculum and planner-entropy schedules
while not converged **do**
  Reset world state s_0 and replay buffers $\mathcal{D}$ (agents), $\mathcal{D}_p$ (planner)
  for $t = 0, \ldots, H$ **do**
    if $t \bmod T = 0$ **then**
      Planner samples tax rates $\tau \sim \pi_\phi(\cdot \mid o_{p,t}, h_{p,t})$
    end if
    Each agent samples action $a_{i,t} \sim \pi_\theta(\cdot \mid o_{i,t}, h_{i,t})$ in parallel
    Step the environment; compute post-tax rewards; deliver the planner reward at tax-period boundaries
    Append transitions to $\mathcal{D}$ and $\mathcal{D}_p$
  end for
  Update θ via $\mathcal{A}$ on $\mathcal{D}$; update ϕ via $\mathcal{A}$ on $\mathcal{D}_p$
  Advance curriculum and entropy schedules
end while
return (θ, ϕ)

baseline calibrated to the same environment by 8% on utilitarian welfare and 12% on the product of equality and productivity. The schedule is non-monotonic, with highest marginal rates on middle income brackets. Emergent behaviour includes specialisation by skill into gatherers and builders and intertemporal tax avoidance by high-skill agents. The result is suggestive rather than decisive: the two-level RL planner discovers tax schedules that the analytical Saez framework does not, but the welfare ranking depends on the choice of SWF and on the absence of behavioural responses outside the modelled action space.[178]

9.8.4 Scaling MARL-based optimal taxation

Mi et al. (2024) extend the AI Economist agenda to a Bewley-Aiyagari environment calibrated to US data, scaling from the four-to-ten-agent Gather-Trade-Build sandbox of Zheng et al. (2022) to ten thousand heterogeneous households. Households face idiosyncratic labour-income shocks and choose consumption, saving, and labour supply; firms produce with Cobb-Douglas technology; a financial intermediary intermediates household savings to firm investment; the government sets income-tax brackets and lump-sum transfers. The framework benchmarks seven multi-agent RL algorithms (DDQN, MAAC, MADDPG, BiCNet, and centralised-training/decentralised-execution variants) against a genetic-algorithm baseline and a dynamic-programming single-planner baseline. The seven MARL methods uniformly outperform the classical baselines on welfare metrics in the configurations tested. Households trained under MARL exhibit emergent tax-avoidance strategies, a strategic-response phenomenon that the smaller-scale spatial environment of Zheng et al. (2022) cannot surface. The finding is a scale-up rather than a methodological alternative; AIE remains the conceptual anchor for two-level RL policy design.

[178] Tacchetti et al. (2025) survey the deep mechanism design landscape, recommending bilevel optimisation as the central stabilisation strategy and noting that human-response models built from imitation data are not robust to strategic adversaries who anticipate being modelled.

9.8.5 Multi-region RL on RICE-N

Climate-economy policy combines multiple regions interacting through global temperature and trade, no central authority enforcing cooperation, and a multi-decade horizon. The benchmark family of models is the integrated assessment model, which couples a climate block to an economic block and informs IPCC reports. The canonical IAMs are DICE (Nordhaus, 1992) and its regional extension RICE. RICE-N replaces the optimal-control solver inside RICE with a multi-agent system in which each region is a strategic learner.

In chapter notation RICE-N is a Markov game with n regions. The state $s_t = (s_t^{\text{nat}}, s_t^{\text{soc}})$ has a natural component, the atmospheric carbon mass and global mean temperature, and a social component collecting each region's capital, technology, population, trade balance, and the negotiation state. Region i's action $a_{i,t}$ is a tuple of a savings rate, mitigation rate, import-demand and import-tariff vectors, and a vector of negotiation actions. The transition factorises into a natural block (carbon mass and temperature evolve by the carbon-cycle and energy-balance equations inherited from DICE, with aggregate emissions $\sum_i E_{i,t}$ as the only social-to-natural channel) and a social block (Cobb-Douglas output scaled by a damage factor $1 - \Omega(T_t)$, with capital accumulating in the standard way and the negotiation state updating under the protocol in force). Region i's reward is the isoelastic utility of per-capita consumption, where consumption is an Armington aggregate of domestic and imported goods net of foreign tariffs.

RICE-N uses RL rather than DP because the game has no central planner, is non-stationary from any one region's perspective, has a combinatorial negotiation action space, and is only partially observed; classical RICE sidesteps all four by computing a single-planner Nash equilibrium. The agents themselves remain utility-maximisers; the RL is the method that finds their strategic policies, not a claim that regions are boundedly rational. The cost of the RL solution is the loss of the optimality certificate and the seed-independent reproducibility that the classical RICE optimum provides.

The contribution of RICE-N is to formalise the negotiation stage that precedes economic activity at each timestep, with one timestep representing five years. Zhang et al. (2025) compare two negotiation protocols. Bilateral Negotiation has each region propose a pair $(\hat{m}_i, \hat{m}_j)$ to every other region j, stating its own promised mitigation rate and the rate it requests; regions accept or reject; each commits to the maximum mitigation rate across accepted proposals. The Basic Club protocol (Nordhaus, 2015) proposes a club rate $\hat{m}_i$; regions accept or reject; a club forms with a common minimum rate, and non-members face a tariff proportional to the gap. Each region's policy is a neural network with weights shared across regions and region-specific inputs, trained with advantage actor-critic.[179]

Three findings carry policy interpretation that the classical DICE and RICE optimisations cannot produce. A climate club enforced by border tariffs is self-sustaining in the configurations tested. The negotiation institution is itself a policy lever, in that Bilateral Negotiation and Basic Club reach materially different climate and economic outcomes from the same underlying economy. Both protocols lower the cross-region inequality of abatement cost and mitigation rate relative to the no-negotiation baseline, and the authors caution that a uniform border tariff can still operate as a de facto carbon tax on developing regions absent redistribution or technology transfer.

9.8.6 Multi-objective MARL for equitable policy

Biswas et al. (2025) keep the multi-region structure of RICE-N but change the reward. JUS-

[179] Both negotiation protocols reduce temperature growth at the cost of a small drop in production, while distributing emission-reduction costs more equitably than the no-negotiation baseline in the published runs. Zhang et al. (2025) are deliberately non-committal on what equilibrium concept the learners reach; the framework produces a family of outcomes indexed by protocol design rather than converging to a single solution. The AAAI workshop predecessor (Zhang et al., 2022) introduced the modelling framework without the Basic Club protocol; Heitzig et al. (2023) proposes a Conditional Commitments mechanism intended to enhance participation stability.

TICE incorporates the economy, damage, and abatement modules from the 57-region RICE50+ integrated assessment model and couples them with the FAIR climate emulator. It is a multi-objective multi-agent Markov decision process in which the regions share a team reward that is a two-component vector, the negative of global mean temperature and global net economic output, both to be maximised. Training uses a multi-objective multi-agent actor-critic that decomposes the vector-reward problem into scalarised single-objective problems via weighted-sum scalarisation.[180] JUSTICE and RICE-N together frame the question of whether RL converges to one equilibrium or many. RICE-N reaches one realisation at a time from a family of possible equilibria; JUSTICE enumerates the Pareto frontier directly. A single-objective RICE optimiser collapses the climate-equity trade-off into one weighted-sum welfare function, fixing the trade-off before the policymaker sees it; the Pareto-front output leaves it open.

9.9 Single-planner reinforcement learning for policy design

The second variant of RL for policy design treats the planner as the sole RL agent. The rest of the macroeconomic environment is a fixed DSGE, a fitted VAR, or an empirically calibrated structural model populated by rational households whose responses are taken as given. The contribution of RL is the ability to learn nonlinear and state-dependent reaction functions in environments where closed-form optimal control is intractable. The setting is closer to optimal control theory than to mechanism design, with RL in the role that nonlinear model predictive control plays in classical engineering applications.

9.9.1 Single-planner regime learning in a monetary DSGE

Chen et al. (2023b) apply deep RL to the Benhabib et al. (2001) model of monetary-fiscal interaction. A representative household solves

$$\max \mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t \left[u(c_t) + v(m_t) - w(n_t) \right] \quad \text{s.t.} \quad c_t + m_t + b_t = w_t n_t - \tau_t + \frac{R_{t-1} b_{t-1} + m_{t-1}}{\pi_t}, \tag{94}$$

where $m_t = M_t/P_t$ is real money, $b_t = B_t/P_t$ is real bonds, τ_t is a lump-sum tax, w_t is the real wage, $\pi_t = P_t/P_{t-1}$ is gross inflation, and output is linear in labour, $y_t = \varepsilon_t^y n_t$, with ε_t^y an i.i.d. technology shock. Fiscal policy follows a Leeper-style rule $\tau_t = \gamma_0 + \gamma\, b_{t-1} + \varepsilon_t^\tau$ and monetary policy follows a global nonlinear interest-rate rule $R_t = f(\pi_t)\, \varepsilon_t^R$ with f strictly positive and strictly increasing. The deterministic Fisher equation $R = \pi/\beta$ intersects $f(\pi)$ at the inflation target $\pi^\star$ and at a liquidity trap π_L.[181]

The household's state is $s_t = (m_{t-1}, b_{t-1}, \pi_{t-1}, c_{t-1}, n_{t-1}, \varepsilon_t^\tau, \varepsilon_t^R, \varepsilon_t^y)$, the action $a_t = (c_t, b_t, n_t)$ is a continuous tuple of consumption, bond holdings, and labour, the reward $r_t = u(c_t) + v(m_t) - w(n_t)$ is the instantaneous utility, and the transition $P(s_{t+1} \mid s_t, a_t)$ is induced by the household's intertemporal Euler equation

$$u'(c_t) \;=\; \beta\, R_t\, \mathbb{E}_t \left[\frac{u'(c_{t+1})}{\pi_{t+1}} \right] \tag{95}$$

together with the equilibrium fiscal-monetary rules and the exogenous shock processes. The

[180] Biswas et al. (2025) sample one hundred weight vectors, train one policy per weight, and collect the non-dominated policies into the solution set. The output is a frontier of policies rather than a single policy. A Gini-based equity index across regions is reported alongside the climate-output frontier.

[181] Each policy rule is locally active or passive depending on the sign of a coefficient. Fiscal policy is active when $\gamma < \beta^{-1} - 1$. Monetary policy is locally active at inflation π when $f'(\pi) > \beta^{-1}$. The cross-product of the two stances generates four regimes, of which the doubly-passive case is locally indeterminate under rational expectations and the doubly-active case is locally explosive. Evans and Honkapohja (2005) show that recursive-least-squares adaptive learning reaches only the determinate regimes.

policy is learned by soft actor-critic (Haarnoja et al., 2018).[182]

The substantive claim is that the set of monetary-fiscal regimes in which a household can converge to a stationary policy is strictly larger under deep RL than under the recursive-least-squares adaptive-learning benchmark of Evans and Honkapohja (2005). RL extends learnability to the indeterminate and explosive regimes that adaptive learning cannot reach.

9.9.2 Benchmarking RL on monetary policy

Wang et al. (2025b) compare nine RL algorithms on a monetary-policy MDP fitted to US Federal Reserve quarterly data (1955–2025). In chapter notation the state is $s_t = (\pi_t, U_t, \text{gap}_t, R_t)$ comprising inflation, unemployment, the output gap, and the policy rate; the action a_t is a discrete adjustment to the policy rate; the reward encodes the Fed's dual mandate as a quadratic loss in deviations from a 2% inflation target and a 4.5% natural unemployment rate. The transition is a linear-Gaussian VAR fitted to historical transitions.[183] Standard tabular Q-learning achieves the best mean discounted return, outperforming both the enhanced RL methods and a Taylor-rule baseline. The dynamics are linear-Gaussian rather than a structural DSGE and the paper has not been peer-reviewed, so the finding is preliminary. It suggests, no more strongly than that, that sophisticated RL machinery does not dominate the combination of simple rules and tabular RL in low-dimensional monetary-policy MDPs.

9.9.3 Single-planner RL on data-grounded monetary models

Hinterlang and Taenzer (2024) train a central-bank RL agent on a neural-network surrogate of the US economy estimated from quarterly data 1987Q3–2007Q2. The pipeline first fits transition equations from data, in linear (SVAR) and nonlinear (feedforward ANN) variants, and then trains an RL reaction function inside the fitted environment with the zero lower bound, asymmetric central-bank preferences, and the dual mandate baked into the reward. The RL-optimised linear rule reduces the central-bank loss by more than thirty-five per cent relative to common Taylor-style rules and the realised federal funds rate over the sample window. The nonlinear ANN reaction function extends this to a forty-three per cent reduction. A cross-validation exercise feeds the learned rules into eleven alternative DSGE models from the Macroeconomic Model Database and finds smaller unconditional inflation and output variances under the RL rules than under the Taylor benchmarks, a generalisation result absent from the MPU comparison above.

Khundadze and Semmler (2025) extend the single-planner setup to the Eurozone, using soft actor-critic to control inflation, the output gap, and debt in a two-bloc (North-South) macro environment with asymmetric shocks. The RL policy is benchmarked against a nonlinear Model Predictive Control (NMPC) baseline. SAC achieves comparable or better stabilisation on average and is more robust under asymmetric shocks because the learned feedback policy generalises where NMPC must re-solve an open-loop trajectory each period. Cross-bloc transfers emerge as a learned component of the joint fiscal-monetary policy.

[182] SAC is an off-policy maximum-entropy actor-critic. The Gaussian actor π_θ and the Q-function Q_ϕ are deep neural networks trained by stochastic gradient descent on minibatches drawn from a replay buffer. The critic update follows the temporal-difference loss with a target network and the actor update is the reparameterised maximum-entropy gradient. Chen et al. (2023b) report that the trained policy converges to a stationary outcome in all four monetary-fiscal regimes, including the doubly-passive case (locally indeterminate under rational expectations) and the doubly-active case (locally explosive). The interpretation is that the deep policy is constrained by the global utility-maximisation problem rather than by the linearised dynamics around any one steady state.

[183] The nine algorithms span tabular Q-learning variants, SARSA, actor-critic, deep Q-networks, Bayesian Q-learning, and a POMDP particle-filter variant. The training and evaluation protocols are aligned across algorithms, with each agent trained on the same fitted VAR dynamics and evaluated on a held-out window.

9.10 Discussion

9.10.1 Four roles, one toolbox

Reinforcement learning enters macroeconomics in four distinct roles covered above. As a global solver, structural policy gradient methods solve heterogeneous-agent models on timescales that classical projection and perturbation cannot reach. As a game-theoretic formalism, mean-field RL handles partial observability and hierarchical Stackelberg structure. As a model of the household, RL learning dynamics reproduce empirical anomalies that rational expectations with borrowing constraints cannot generate jointly. As a planner, RL discovers tax schedules, climate-cooperation mechanisms, and monetary-fiscal reaction functions that closed-form optimisation does not. The four roles share an algorithmic toolbox but answer different questions, and the empirical and theoretical literatures around each are at different stages of maturity. The current state is best read as early progress rather than as a settled methodology.

9.10.2 Common methodological issues

Reading across the lenses surfaces issues that no single section makes explicit. The first concerns convergence theory. Of the algorithms surveyed in this chapter, only the temporal mean-field construction of Yang (2026) carries a classical guarantee, with existence and uniqueness of the equilibrium under a discrete monotonicity condition, finite-population $O(1/\sqrt{N})$ Nash approximation regardless of the per-step batch size, and provable convergence of the policy-gradient iterate (the sequence of policy updates produced by the algorithm). The remaining methods, including Yang et al. (2025), Wibault et al. (2026), Kaplowitz (2025), Zheng et al. (2022), Chen et al. (2023b), Mi et al. (2025), Han et al. (2022a), Brunnbauer et al. (2024), and Magnino et al. (2025), monitor convergence empirically by exploitability, L^∞ norms of policy updates, or Bellman residuals against application-specific tolerances. This is not a failure of rigour. The macro equilibrium concepts these algorithms target, including restricted perceptions equilibrium, mean-field Nash, Stackelberg mean-field, and co-adaptation (joint training of planner and agents against each other's current policy), live in spaces where classical fixed-point theorems (existence results such as Brouwer or Kakutani) either do not apply or do not yield computable bounds. The honest reading of the literature is that convergence here means training stopped changing the policy, not that the iterate provably reaches a model equilibrium.

The four lenses also target four different equilibrium concepts, and papers within a lens diverge further (Section 9.1). The restricted perceptions equilibrium of Yang et al. (2025) assumes agents condition on the current price vector as if it were Markov, which is not satisfied by true Krusell-Smith equilibrium prices but is what makes structural policy gradient tractable. Wibault et al. (2026) extends this to a partially observable mean-field Nash with a shared observation and recurrent memory. Mi et al. (2025) targets a Stackelberg mean-field with a single leader and a continuum of homogeneous followers. Zheng et al. (2022) trains planner and citizens by PPO against each other's current policy without naming the equilibrium the joint iterate is supposed to reach. Chen et al. (2023b) accept whichever DSGE steady state the SAC iterate stabilises at, including the indeterminate and explosive regimes that adaptive learning cannot reach. Kaplowitz (2025) abandons equilibrium altogether in favour of single-agent Q-learning calibrated to micro-panel moments. A claim that RL solves Krusell-Smith therefore means different things under Yang et al. (2025) than under Han et al. (2022a), where the same model is solved as a recursive equilibrium in which μ is summarised by a finite set of moments (moment closure), with neural-network value and policy functions.

Several recurring pitfalls follow from these design choices. Multiplicity is acknowledged but resolved silently by initialisation, through the warm-up phase of Yang et al. (2025), the two-phase entropy schedule of Zheng et al. (2022), and the regime-by-regime training of Chen et al. (2023b); none of these papers tests initialisation robustness. Curriculum learning is empirically

necessary but theoretically unmotivated, with Curry et al. (2022) showing independent multi-agent RL fails to recover non-trivial behaviour without one and Zheng et al. (2022) confirming the same pattern in two-level policy design. Hyperparameter sensitivity is severe and unevenly characterised, with Wibault et al. (2026) reporting that off-the-shelf IPPO and recurrent-IPPO require 81 to 243 configurations on partially observable Krusell-Smith while structural policy gradient methods converge with three; no paper offers transferable tuning guidance. Discount-factor sensitivity is essentially unexamined, with Atashbar and Shi (2023) flagging instability at high β but no paper sweeping β over the macro-realistic range. The single-agent to equilibrium gap is open, since the household learning rule of Kaplowitz (2025) reproduces marginal propensity to consume heterogeneity and scarring at the household level but has not been tested for consistency once embedded in an Aiyagari-style equilibrium where its policy would shift the cross-sectional distribution.

Three cross-cutting findings deserve to be surfaced separately from their parent sections. The first is that tabular and grid-based RL competes with or beats deep RL on macro problems. Wang et al. (2025b) benchmarks nine algorithms on US Federal Reserve quarterly data and finds tabular Q-learning beats DQN, soft actor-critic, and Bayesian variants on a quadratic-loss monetary objective, and Yang et al. (2025) reports grid policies training faster than deep networks on Krusell-Smith. The ordering familiar from robotics and games does not transfer mechanically to macro, whose state spaces are low-dimensional once correctly compressed. The second finding is that RL reaches equilibrium regions adaptive learning cannot. Chen et al. (2023b) shows deep RL stabilises in indeterminate and explosive DSGE regimes that least-squares learning rules out by E-stability (the expectational-stability condition under which a perceived law of motion is learnable by least-squares updating), suggesting the same toolbox extends the equilibrium set under bounded rationality rather than merely reproducing it. The third is sociological. The three methodological clusters covered in this chapter, structural solvers (Yang et al., 2025; Wibault et al., 2026; Han et al., 2022a; Azinovic-Yang and Žemlička, 2026), behavioural single-agent learners (Kaplowitz, 2025; Kuriksha, 2021; Shi, 2022), and policy designers (Zheng et al., 2022; Mi et al., 2024; Chen et al., 2023b; Hinterlang and Taenzer, 2024; Khundadze and Semmler, 2025; Wang et al., 2025b), have evolved in parallel with limited cross-citation. The solver papers do not engage the behavioural ones, the behavioural paper of Kaplowitz (2025) cites no RL-macro work, and the two monetary policy papers (Hinterlang and Taenzer, 2024; Wang et al., 2025b) appear unaware of each other. The risk is that solver-driven equilibrium claims, behavioural calibration evidence, and policy-design exercises drift apart without methodological reconciliation.

10 Reinforcement Learning in Games

With multiple agents adapting simultaneously, each agent's environment includes the others' changing policies, so the stationary-transition assumption behind single-agent convergence results fails. Shoham et al. (2007) enumerate five desiderata for multi-agent learning: (1) convergence to a stationary strategy in self-play; (2) rationality (best-responding against stationary opponents); (3) equilibrium attainment; (4) safety (guaranteeing at least the Nash-value payoff); (5) social welfare. No existing algorithm satisfies all five in general games.

Two paradigms emerged. Value-based methods generalize the Bellman operator to games, replacing the max with game-theoretic solution concepts (minimax, Nash), targeting stochastic games with simultaneous moves and observable payoffs. Regret-based methods (CFR) accumulate counterfactual regrets and let the time-averaged strategy converge, targeting extensive-form games with sequential moves and private information. Computing Nash equilibria is PPAD-complete (Daskalakis et al., 2009), so neither approach escapes the fundamental hardness, but both achieve convergence in the game classes they target.

10.1 Stochastic Games and Equilibrium Learning

10.1.1 The Stochastic Game Framework

An n-player stochastic game $\Gamma = (n, \mathcal{S}, \mathcal{A}_1, \dots, \mathcal{A}_n, P, r_1, \dots, r_n, \gamma)$ consists of a finite state space $\mathcal{S}$; finite action sets $\mathcal{A}_i$ for each player i; a transition function $P : \mathcal{S} \times \mathcal{A}_1 \times \cdots \times \mathcal{A}_n \to \Delta(\mathcal{S})$; reward functions $r_i : \mathcal{S} \times \mathcal{A}_1 \times \cdots \times \mathcal{A}_n \to \mathbb{R}$; and a common discount factor $\gamma \in [0, 1)$. At each stage, all players simultaneously choose actions, receive individual rewards, and the game transitions to a new state.[184]

A Markov decision process is a stochastic game with $n = 1$; a matrix game is a stochastic game with $|\mathcal{S}| = 1$. Each player i seeks a policy $\pi_i : \mathcal{S} \to \Delta(\mathcal{A}_i)$ maximizing discounted return $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_i(s_t, a_{1,t}, \dots, a_{n,t})]$.

Standard Q-learning convergence (Watkins and Dayan, 1992a) requires stationary transition and reward dynamics; with multiple learners, this assumption fails. Bowling and Veloso (2002) formalized two properties a learning algorithm should satisfy. It is *rational* if, when all other players converge to stationary policies, it converges to a best response; it is *convergent* if, in self-play, it converges to a stationary policy.

If all players use rational, convergent algorithms, the resulting profile is a Nash equilibrium by construction. The challenge is achieving both properties simultaneously.

10.1.2 Minimax-Q Learning

Littman (1994) proposed the first Q-learning algorithm for stochastic games, targeting two-player zero-sum games. The key modification replaces the max operator in the standard Q-learning backup with a minimax operator. Each agent maintains $Q_i(s, a_i, a_{-i})$ over the joint action space. The update rule is

$$Q_i(s, a_i, a_{-i}) \leftarrow (1 - \alpha)\, Q_i(s, a_i, a_{-i}) + \alpha \left[r_i + \gamma\, V_i(s') \right], \tag{96}$$

where the value backup solves a linear program:

$$V_i(s) = \max_{\pi_i \in \Delta(\mathcal{A}_i)} \min_{a_{-i} \in \mathcal{A}_{-i}} \sum_{a_i \in \mathcal{A}_i} \pi_i(a_i)\, Q_i(s, a_i, a_{-i}). \tag{97}$$

This is the RL analogue of Shapley's value iteration for zero-sum stochastic games (Shapley, 1964). The resulting policy $\pi_i(s)$ is generally a mixed strategy, since deterministic policies are exploitable in adversarial settings.

Minimax-Q converges to the minimax Q-values under Robbins-Monro learning rates ($\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$) and infinite exploration of all state-action tuples.[185] However, the algorithm sacrifices rationality: it plays the equilibrium strategy even against exploitable opponents.[186]

10.1.3 Nash-Q Learning

Hu and Wellman (2003) extended the framework to general-sum stochastic games, where players may have aligned, opposed, or mixed incentives. Each agent i maintains a Q-function over the

[184] Shapley (1953) introduced stochastic games in 1953 for the two-player zero-sum case, proving existence of the value via a contraction argument on the Bellman operator. The general-sum extension to n players is due to Fink (1964) and Takahashi (1964).

[185] The convergence proof extends the contraction argument for standard Q-learning. Because the zero-sum minimax operator is a contraction with modulus γ under the ℓ^∞ norm, the stochastic approximation converges to the fixed point. See Littman (1994) and the general treatment in Szepesvári and Littman (1999).

[186] In the soccer game of Littman (1994), minimax-Q won 53.7% against a hand-built opponent versus 26.1% for Q-learning. Against an adversarial challenger, Q-learning won 0% (its deterministic policy was fully predictable); minimax-Q won 37.5% through mixed strategies.

joint action space $Q_i(s, a_1, \ldots, a_n)$ and updates via

$$Q_i(s, \mathbf{a}) \leftarrow (1 - \alpha)\, Q_i(s, \mathbf{a}) + \alpha \left[r_i + \gamma \, \text{Nash}_i(Q_1(s'), \ldots, Q_n(s')) \right], \tag{98}$$

where $\text{Nash}_i(\cdot)$ denotes player i's payoff under a Nash equilibrium of the stage game defined by the current Q-values $(Q_1(s'), \ldots, Q_n(s'))$. At each backup, the algorithm treats the Q-values as payoff matrices, computes a Nash equilibrium of this matrix game, and uses the equilibrium payoffs for the value estimate.

Theorem 5 (Hu and Wellman (2003))**.** *Nash-Q converges to Nash Q-values under Robbins-Monro learning rates and infinite exploration, provided every stage game encountered during learning has either (a) a global optimal point (all agents receive their highest payoff at the same joint action) or (b) a unique saddle-point Nash equilibrium.*

These conditions are restrictive. When stage games have multiple Nash equilibria, agents may select different equilibria for their backups, causing divergence.[187] Nash-Q reduces to standard Q-learning in the single-agent case.

10.1.4 The Convergence Problem

Table 18 summarizes the trade-offs. No single algorithm achieves both rationality and convergence in general games.

Table 18: Multi-agent Q-learning algorithms: convergence and information requirements

Algorithm	Rational	Convergent	Game class	Information
Indep. Q-learning	Yes	No	Any	Own reward
Minimax-Q	No	Yes	Zero-sum	Joint actions
Nash-Q	Cond.	Cond.	General-sum	All rewards
WoLF-PHC	Yes	Yes (2×2)	General-sum	Own reward

WoLF-PHC (Win or Learn Fast, Policy Hill-Climbing) of Bowling and Veloso (2002) maintains a policy $\pi_i(a|s)$ and a running average policy $\bar{\pi}_i(a|s)$, updating Q-values as in standard Q-learning. The policy moves toward the greedy action at a variable rate:

$$\delta = \begin{cases} \delta_l & \text{if } \sum_a \pi_i(a|s)\, Q_i(s,a) < \sum_a \bar{\pi}_i(a|s)\, Q_i(s,a) \quad \text{(losing)} \\ \delta_w & \text{otherwise} \quad \text{(winning)} \end{cases} \tag{99}$$

with $\delta_l > \delta_w$. When the current policy underperforms the historical average (losing), the agent adapts quickly; when outperforming (winning), it adapts slowly to avoid destabilizing the opponent. Bowling and Veloso (2002) proved that WoLF-IGA (the infinitesimal gradient ascent variant) converges to Nash equilibrium in all two-player, two-action games. The trajectory traces piecewise ellipses around the equilibrium, shrinking by a factor of $\ell^4 < 1$ per orbit, where $\ell = \sqrt{\delta_w / \delta_l}$. WoLF-PHC requires only own-reward observations, the same information as independent Q-learning.

Two further algorithms deserve mention. Friend-or-Foe Q-learning (Littman, 2001) decomposes agents as cooperative (friend) or adversarial (foe), using max for friends and minimax

[187] In the experiments of Hu and Wellman (2003), a grid game with a unique equilibrium Q-function converged in 100% of trials under Nash-Q versus 20% under independent Q-learning; a game with three equilibrium Q-functions converged in only 68–90% of trials. Nash-Q requires each agent to observe all other agents' rewards and to maintain Q-values over the joint action space $\mathcal{A}_1 \times \cdots \times \mathcal{A}_n$, with storage $O(n|\mathcal{S}| \prod_i |\mathcal{A}_i|)$, exponential in the number of players.

for foes; it always converges but requires knowing the relationship type a priori.[188] The evolutionary perspective of Börgers and Sarin (1997) provides a deeper lens: reinforcement learning dynamics in matrix games converge to the replicator equation from evolutionary game theory. The cycling of Q-learning in games such as matching pennies is structurally identical to the cycling of replicator dynamics in Rock-Paper-Scissors games.

10.1.5 Simulation Study: Cournot and Bertrand Duopoly

Two canonical games from industrial organization serve as benchmarks. In Cournot duopoly, two firms choose quantities $q_i \in \{0, 1, \ldots, 9\}$ with inverse demand $P(Q) = 10 - Q$ and marginal cost $c = 1$; the continuous Nash equilibrium is $q^* = 3$ with profit $\pi^* = 9$. In Bertrand duopoly with differentiated products, two firms choose prices $p_i \in \{0, 1, \ldots, 9\}$ with demand $d_i = 10 - 2p_i + p_j$ and marginal cost $c = 1$; the symmetric first-order condition gives $p^* = (a + bc)/(2b - e) = 4$ with profit $\pi^* = 18$. Bertrand admits a single pure-strategy Nash on the integer grid at $(p_0, p_1) = (4, 4)$, while Cournot admits three pure-strategy Nash equilibria on the integer grid, $(2, 4)$, $(3, 3)$, and $(4, 2)$, only the symmetric of which coincides with the continuous solution.[189] Three algorithms compete: independent Q-learning (IQL), Nash-Q, and WoLF-PHC. The framing connects to the recent literature on Q-learning collusion in IO duopoly (Calvano et al., 2020); the stateless design used here has no memory of past actions, so collusive trigger strategies are not available and the equilibria reached are one-shot Nash.

Table 19: Convergence to Nash equilibrium in Cournot and Bertrand duopoly

Game	Algorithm	Action	Profit	$\lvert a - a^* \rvert$
Cournot	IQL	2.95 ± 0.05	9.1 ± 0.0	0.05
	Nash-Q	2.89 ± 0.33	8.8 ± 1.3	0.17
	WoLF-PHC	3.00 ± 0.00	9.0 ± 0.0	0.00
Bertrand	IQL	4.00 ± 0.00	18.0 ± 0.0	0.00
	Nash-Q	4.00 ± 0.00	18.0 ± 0.0	0.00
	WoLF-PHC	3.95 ± 0.05	17.8 ± 0.2	0.05

Notes: Action and Profit report mean ± standard error across 20 seeds over the final 5,000 iterations. $|a - a^*|$ is the mean distance from the symmetric continuous Nash action ($q^* = 3$ for Cournot, $p^* = 4$ for Bertrand).

Table 19 and Figure 19 report the results. All three algorithms settle near the symmetric Nash in both games. IQL, which lacks any game-theoretic computation, matches the game-aware methods in these well-structured games. The advantage of Nash-Q and WoLF-PHC emerges in games requiring mixed strategies, where IQL's deterministic policy limit prevents convergence.

10.2 Counterfactual Regret Minimization

Extensive-form games require a different approach. CFR bypasses equilibrium selection: instead of computing Nash equilibria at each step, it minimizes cumulative regret and lets the time-averaged strategy converge to equilibrium.

An extensive-form game consists of a game tree with information sets $\mathcal{I}_i$ partitioning player i's decision nodes. An information set groups nodes where the player cannot distinguish between

[188] Correlated-Q learning (Greenwald and Hall, 2003) generalizes both Nash-Q and Minimax-Q by using correlated equilibrium, a probability distribution over joint actions enforced by a correlating device. The set of correlated equilibria contains all Nash equilibria. Correlated-Q converges under conditions analogous to Nash-Q.

[189] Each configuration runs for 50,000 iterations across 20 seeds. The Nash-Q implementation here selects the joint-payoff-maximizing equilibrium when multiple pure Nash exist, a deviation from the canonical Hu-Wellman 2003 backup that pins down a single value function; in this game it picks $(3, 3)$ for Cournot.

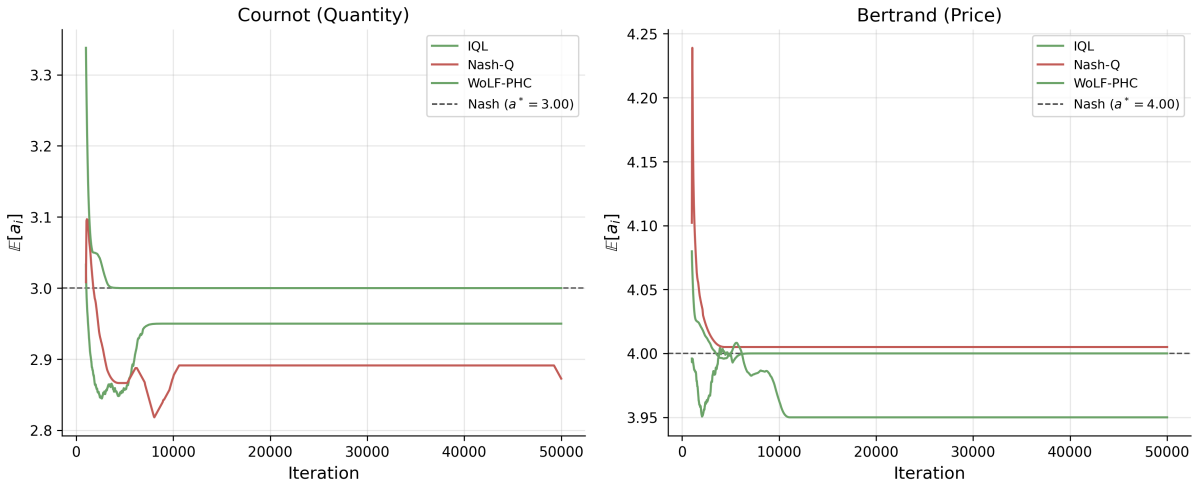

Figure 19: Convergence of expected actions to Nash equilibrium. Left: Cournot duopoly. Right: Bertrand duopoly. Smoothed over 1,000-iteration windows, averaged across 20 seeds.

them due to hidden opponent actions or private information. A behavioral strategy $\sigma_i : \mathcal{I}_i \to \Delta(\mathcal{A})$ assigns action probabilities at each information set.

The *counterfactual value* of action a at information set I is

$$v_i^\sigma(I, a) = \sum_{h \in I} \pi_{-i}^\sigma(h) \sum_{z \sqsupseteq ha} \pi^\sigma(ha, z) u_i(z) \tag{100}$$

where $\pi_{-i}^\sigma(h)$ is the probability opponents reach h, and $\pi^\sigma(ha, z)$ is the probability of reaching terminal z from ha. The counterfactual formulation weights by opponent reach $\pi_{-i}^\sigma(h)$ rather than joint reach $\pi^\sigma(h)$, ensuring non-zero updates even at rarely-visited information sets. Cumulative regret for action a is $R^T(I, a) = \sum_{t=1}^T [v^{\sigma^t}(I, a) - v^{\sigma^t}(I)]$. CFR updates via *regret matching*:[190]

$$\sigma^{T+1}(I, a) = \frac{[R^T(I, a)]^+}{\sum_{a'} [R^T(I, a')]^+}, \quad [x]^+ = \max(x, 0). \tag{101}$$

The average strategy $\bar{\sigma}^T$ converges to ε-Nash with $\varepsilon = O(|\mathcal{I}|\sqrt{|\mathcal{A}|}\Delta/\sqrt{T})$, where Δ is the range of payoffs (Zinkevich et al., 2008).[191]

CFR+ (Tammelin, 2014) replaces standard regret matching with Regret Matching+, which truncates cumulative regrets to zero after each update rather than only at action selection. The update becomes $R^{T+1}(I, a) = \max(R^T(I, a) + r^{T+1}(I, a), 0)$, where $r^{T+1}(I, a) = v^{\sigma^{T+1}}(I, a) - v^{\sigma^{T+1}}(I)$ is the instantaneous counterfactual regret. CFR+ also weights iteration t by t when computing the average strategy. While vanilla CFR converges at $O(1/\sqrt{T})$, CFR+ empirically converges at $O(1/T)$.[192] CFR+ enabled Bowling et al. (2015) to essentially solve heads-up limit Texas hold'em, a game with 3.16×10^{17} states, the first non-trivial imperfect-information game played competitively by humans to be essentially solved. Their program Cepheus achieved exploitability below 1 mbb/g (milli-big-blind per game).

[190] Regret matching is an action selection rule where the probability of choosing action a is proportional to the cumulative regret for not having chosen a in the past (truncated at zero). Actions with high regret receive higher probability; actions with negative regret (having performed worse than average) receive zero probability.

[191] An ε-Nash equilibrium is a strategy profile where no player can improve her expected payoff by more than ε through unilateral deviation. As $\varepsilon \to 0$, this converges to an exact Nash equilibrium.

[192] The $O(1/T)$ rate for CFR+ is empirically observed but not yet proven in full generality. Tammelin (2014) conjectured this rate based on extensive experiments across poker variants.

10.3 Neural Extensions

Tabular CFR stores regrets at every information set, infeasible when $|\mathcal{I}| > 10^{14}$. Two neural approaches scale to large games.

10.3.1 Deep CFR

Brown et al. (2019) approximate cumulative regrets with a neural network $V_\theta : \mathcal{I} \times \mathcal{A} \to \mathbb{R}$ trained on sampled (information set, iteration, regret) tuples:

$$L_V(\theta) = \mathbb{E}_{(I,t',r)\sim\mathcal{M}}\left[t' \cdot (V_\theta(I,a) - r)^2\right]. \tag{102}$$

Weighting by iteration index t' gives more recent regret estimates higher importance. A separate network Π_ϕ approximates the average strategy, trained via weighted MSE on strategy samples with the same iteration weighting. The current strategy derives from regret matching, with $\sigma(I,a) \propto [V_\theta(I,a)]^+$.

10.3.2 Neural Fictitious Self-Play

NFSP (Heinrich and Silver, 2016) combines *fictitious play*[193] (Brown, 1951) with deep Q-learning.[194] Each player maintains two networks: a best-response network Q_θ trained via DQN, and an average strategy network $\bar{\pi}_\phi$ trained via supervised learning on the best-response actions:

$$L_{\text{RL}}(\theta) = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q_{\theta^-}(I',a') - Q_\theta(I,a)\right)^2\right], \quad L_{\text{SL}}(\phi) = \mathbb{E}\left[-\log \bar{\pi}_\phi(a|I)\right]. \tag{103}$$

During play, agents follow $\bar{\pi}_\phi$ with probability $1-\eta$ and Q_θ with probability η.

10.3.3 Poker Results

These methods achieved superhuman performance in poker. Deep CFR attained *exploitability* of 37 mbb/g[195] in heads-up flop hold'em (Brown et al., 2019). Libratus defeated top human professionals in heads-up no-limit hold'em using nested subgame solving with CFR (Brown and Sandholm, 2018). Pluribus extended to six-player no-limit hold'em (Brown and Sandholm, 2019). The game tree of heads-up no-limit hold'em contains $\sim 10^{161}$ states; these results demonstrate that CFR-based methods scale to practically relevant game sizes.

10.4 The Coase Conjecture in a Durable Goods Monopoly

The durable goods monopoly is a canonical extensive-form bargaining problem with private information: the seller does not know the buyer's valuation.

Coase (1972) conjectured that a durable goods monopolist[196] loses market power when buyers are patient. Unable to commit to future prices, the seller competes with her future

[193] Fictitious play (Brown, 1951) is a classical learning rule where each player best-responds to the empirical distribution of opponents' past actions. Under certain conditions (e.g., two-player zero-sum games), the time-average strategies converge to Nash equilibrium.

[194] Deep Q-Network (DQN) (Mnih et al., 2015) approximates the Q-function with a neural network, using experience replay and a target network to stabilize training. The target network θ^- in the loss function is a delayed copy of the main network, updated periodically.

[195] Exploitability measures how far a strategy is from Nash equilibrium, defined as the maximum expected gain an adversary could achieve by best-responding. Zero exploitability means the strategy is unexploitable (Nash). In poker, exploitability is measured in milli-big-blinds per game (mbb/g).

[196] A durable good provides utility over multiple periods (e.g., a car, appliance, or software license) rather than being consumed immediately. Unlike non-durable goods, durable goods create intertemporal competition, as the seller at time t competes with her own future self at $t+1$.

self, eroding rents. As the inter-offer interval shrinks ($\delta \to 1$), price collapses to marginal cost and all surplus is extracted by buyers. Stokey (1981) showed that a monopolist facing rational consumers cannot price discriminate intertemporally; Bulow (1982) proved that in the no-gap case the monopolist prefers renting to selling. Gul et al. (1986) formalized the Coase conjecture for the gap case, where the seller's cost is strictly below all buyer valuations, establishing that the unique stationary Markov-perfect equilibrium price collapses to marginal cost as the period length shrinks. Ausubel and Deneckere (1989) and the survey of Ausubel et al. (2002) extend the analysis to the no-gap case and to bargaining with two-sided incomplete information.

The Coase conjecture is asymptotic in T and δ. A two-period game cannot exhibit it; the surplus erosion only operates when the seller can re-optimize many times and buyers can credibly wait. The simulation below uses backward induction on the seller's recursive problem with a continuum of buyers, varying T and δ on a grid, and compares the Markov-perfect no-commitment value against the commitment benchmark. A separate two-period simulation in Section 10.5 illustrates the screening-versus-pooling equilibrium structure that underlies the durable-goods game; the Coase conjecture proper is delivered by the asymptotic sweep here.

10.4.1 Model and Dynamic Programming

A monopolist with marginal cost $c = 0$ faces a continuum of buyers with valuations v drawn from the uniform distribution on $[0, 1]$. The mass of buyers is normalized to one. In each period $t = 1, \ldots, T$, the seller posts a price p_t; each remaining buyer of type v accepts iff $v - p_t \geq \delta\, \mathbb{E}_t[v - p^*_{t+1}]$, where p^*_{t+1} is the equilibrium price the seller will post at $t + 1$. The seller's per-period revenue is p_t times the mass of buyers who accept.

Define the state at the start of period t as the cutoff $v_t \in [0, 1]$: remaining buyers have valuations uniformly distributed on $[0, v_t]$, with mass v_t. The seller's action is the next-period cutoff $w \in [0, v_t]$; buyers with type in $[w, v_t]$ accept at price p_t and exit, while those in $[0, w]$ remain. Marginal-buyer indifference at w pins down the posted price,

$$p_t(w) = (1 - \delta)w + \delta\, p^*_{t+1}(w), \tag{104}$$

where $p^*_{t+1}(w)$ is the seller's equilibrium price at state w in period $t + 1$. The seller's Bellman equation is

$$V_t(v) = \max_{w \in [0,v]} \left\{ p_t(w)\,(v - w) + \delta\, V_{t+1}(w) \right\}, \tag{105}$$

with terminal condition $V_T(v) = \max_{w \in [0,v]} w(v - w) = v^2/4$ at $w = v/2$.

With uniform F on $[0, v]$, the value, equilibrium price, and equilibrium cutoff functions are homogeneous in v: $V_t(v) = \beta_t v^2$, $p^*_t(v) = \mu_t v$, $w^*_t(v) = \lambda_t v$. Substituting into (104)–(105) reduces the dynamic program to a scalar recursion in $(\mu_t, \lambda_t, \beta_t)$ with terminal values $\mu_T = \lambda_T = 1/2$ and $\beta_T = 1/4$.[197]

The commitment benchmark is the optimal pre-committed price path. With forward-looking buyers, uniform F, and $c = 0$, the optimal commitment is the static-monopoly price $p^* = 1/2$ in every period. Buyers with $v \geq 1/2$ buy in period 1; nobody else buys (any later discount would induce strategic waiting and lower expected revenue). Commitment value: $V^{\text{com}} = p^*(1 - p^*) = 1/4$ independent of (T, δ).

10.4.2 Results

Table 20 reports the sweep. At $T = 200$, $\delta = 0.99$ the no-commitment monopolist captures $V^{\text{nc}} = 0.058$ against the commitment benchmark $V^{\text{com}} = 0.25$, a ratio of 0.23; the terminal

[197] Define $A_{t+1} = 1 - \delta + \delta\mu_{t+1}$. The first-order condition for the seller's choice of w in (105) gives $\lambda_t = A_{t+1}/(2(A_{t+1} - \delta\beta_{t+1}))$, after which $\mu_t = \lambda_t A_{t+1}$ and $\beta_t = \lambda_t(1 - \lambda_t)A_{t+1} + \delta\beta_{t+1}\lambda_t^2$. The recursion is closed-form per period; the entire backward induction over $T = 200$ periods runs in under a millisecond.

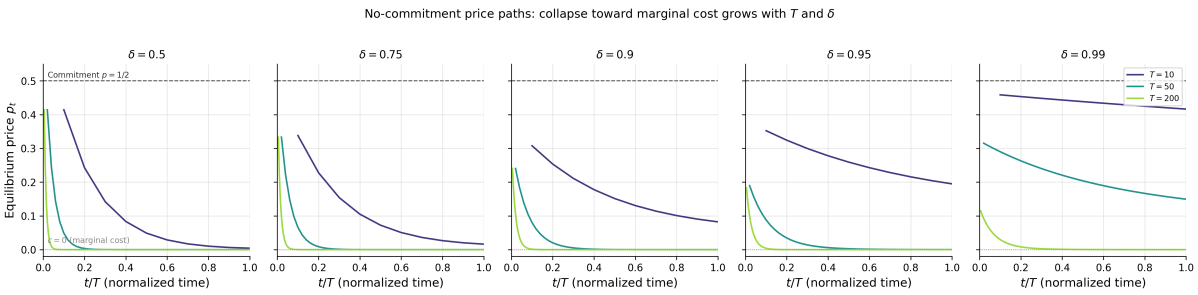

Figure 20: No-commitment price paths p_t versus normalized period t/T for representative horizons $T \in \{10, 50, 200\}$ at $\delta \in \{0.5, 0.75, 0.9, 0.95, 0.99\}$. Dashed line: commitment price $p = 1/2$. Dotted line: marginal cost $c = 0$.

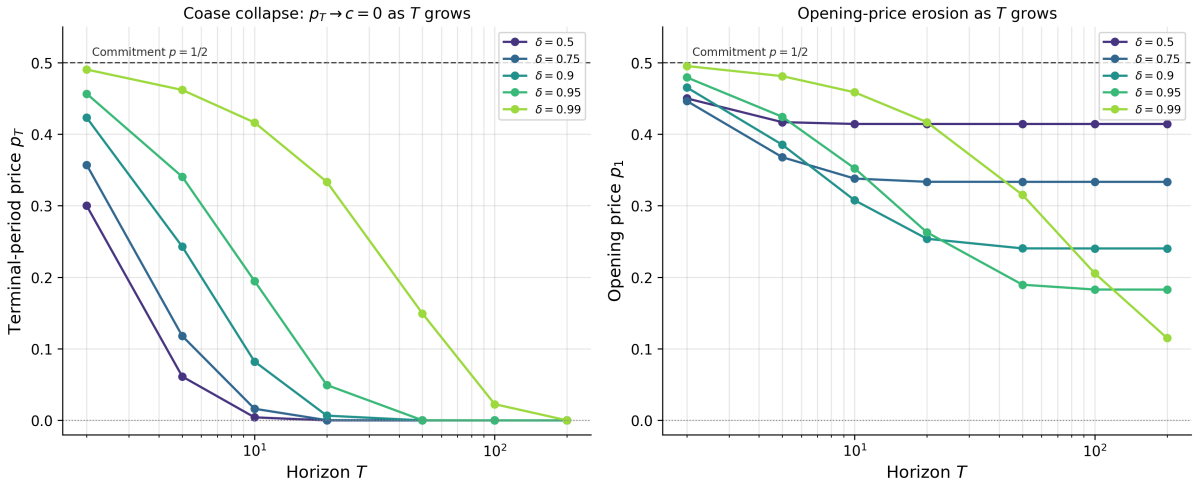

Figure 21: Coase collapse. Left: terminal-period price p_T versus T on a log-T axis. Right: opening price p_1 versus T, same axis. Each curve fixes δ.

Table 20: Commitment value V^{com}, no-commitment Markov-perfect value V^{nc}, ratio $V^{\mathrm{nc}}/V^{\mathrm{com}}$, opening price p_1, and terminal price p_T across the (T, δ) grid. Uniform valuations on $[0, 1]$, $c = 0$.

T	δ	V^{com}	V^{nc}	$V^{\mathrm{nc}}/V^{\mathrm{com}}$	p_1	p_T
2	0.50	0.2500	0.2250	0.900	0.4500	0.3000
2	0.75	0.2500	0.2232	0.893	0.4464	0.3571
2	0.90	0.2500	0.2327	0.931	0.4654	0.4231
2	0.95	0.2500	0.2397	0.959	0.4793	0.4565
2	0.99	0.2500	0.2476	0.990	0.4952	0.4903
5	0.50	0.2500	0.2084	0.834	0.4168	0.0613
5	0.75	0.2500	0.1839	0.736	0.3678	0.1180
5	0.90	0.2500	0.1926	0.770	0.3852	0.2428
5	0.95	0.2500	0.2121	0.849	0.4243	0.3405
5	0.99	0.2500	0.2405	0.962	0.4811	0.4618
10	0.50	0.2500	0.2071	0.828	0.4142	0.0042
10	0.75	0.2500	0.1690	0.676	0.3379	0.0162
10	0.90	0.2500	0.1538	0.615	0.3076	0.0823
10	0.95	0.2500	0.1761	0.705	0.3523	0.1948
10	0.99	0.2500	0.2293	0.917	0.4585	0.4163
20	0.50	0.2500	0.2071	0.828	0.4142	0.0000
20	0.75	0.2500	0.1667	0.667	0.3334	0.0003
20	0.90	0.2500	0.1269	0.508	0.2538	0.0066
20	0.95	0.2500	0.1314	0.526	0.2628	0.0494
20	0.99	0.2500	0.2084	0.834	0.4168	0.3333
50	0.50	0.2500	0.2071	0.828	0.4142	0.0000
50	0.75	0.2500	0.1667	0.667	0.3333	0.0000
50	0.90	0.2500	0.1202	0.481	0.2404	0.0000
50	0.95	0.2500	0.0948	0.379	0.1896	0.0002
50	0.99	0.2500	0.1576	0.630	0.3152	0.1494
100	0.50	0.2500	0.2071	0.828	0.4142	0.0000
100	0.75	0.2500	0.1667	0.667	0.3333	0.0000
100	0.90	0.2500	0.1201	0.481	0.2403	0.0000
100	0.95	0.2500	0.0914	0.366	0.1828	0.0000
100	0.99	0.2500	0.1028	0.411	0.2056	0.0227
200	0.50	0.2500	0.2071	0.828	0.4142	0.0000
200	0.75	0.2500	0.1667	0.667	0.3333	0.0000
200	0.90	0.2500	0.1201	0.481	0.2403	0.0000
200	0.95	0.2500	0.0914	0.365	0.1827	0.0000
200	0.99	0.2500	0.0576	0.230	0.1151	0.0000

price p_T rounds to zero to four decimals, and the opening price $p_1 = 0.115$ is well below the static monopoly price $1/2$. Holding δ at 0.95 and varying T, the value ratio drops monotonically from 0.96 at $T = 2$ to 0.37 at $T = 200$, and p_T from 0.46 to numerical zero. Holding T at 200 and varying δ, the ratio drops monotonically from 0.83 at $\delta = 0.5$ to 0.23 at $\delta = 0.99$. As a cross-check, the finite-horizon values at $T = 200$ for $\delta \leq 0.95$ match the analytical stationary Markov-perfect equilibrium values β^* and μ^* to five decimals.[198]

Figure 20 traces the equilibrium price path: long horizons and patient buyers force the seller to start lower and end near marginal cost. Figure 21 summarizes the collapse rate: $p_T \to 0$ at all $\delta < 1$ once T is large enough, and p_1 falls toward the stationary level $\mu^*(\delta)$ that itself tends to zero as $\delta \to 1$.

The Coase conjecture is delivered numerically: in the absence of commitment power, the durable-goods monopolist captures only a fraction of the commitment surplus, and that fraction shrinks toward zero as T and δ jointly grow. Section 10.5 below presents a two-period precursor that focuses on the screening-versus-pooling equilibrium structure.

10.5 Screening versus Pooling in the Durable Goods Monopoly

The two-period analogue of the model in Section 10.4 reduces to an extensive-form game with a discrete buyer type and a discrete seller action set; it is small enough to admit equilibrium computation via counterfactual regret minimization (CFR) and serves as a methodological precursor to the dynamic programming sweep above. Replacing the continuum of valuations with a two-point distribution $v \in \{v_L, v_H\}$ and restricting the seller to two prices $\{v_L, P^*(\delta)\}$ recovers the screening-versus-pooling threshold $\pi^* = 1/2$ in closed form; CFR finds this threshold without being told the analytical solution.

10.5.1 Model

A seller with zero cost faces a buyer with private valuation $v \in \{v_L, v_H\}$, where $\Pr(v = v_H) = \pi$. In each period $t = 1, 2, \ldots$, the seller posts price p_t; the buyer accepts or rejects. Upon acceptance at t, the seller receives $\delta^{t-1} p_t$ and the buyer receives $\delta^{t-1}(v - p_t)$, where $\delta \in (0, 1)$ is the common discount factor. The game ends upon acceptance or after T periods. Parameters: $v_L = 100$, $v_H = 200$, $T = 2$ periods.

10.5.2 Equilibrium Analysis

The screening price $P^*(\delta)$ makes the high-type buyer indifferent between accepting now and waiting.

$$v_H - P^* = \delta(v_H - v_L) \implies P^*(\delta) = v_H - \delta(v_H - v_L). \tag{106}$$

The seller's optimal strategy depends on π. Let $\Pi_{\text{screen}} = \pi P^* + (1 - \pi)\delta v_L$ and $\Pi_{\text{pool}} = v_L$. The seller screens if $\Pi_{\text{screen}} > \Pi_{\text{pool}}$, yielding threshold

$$\pi^* = \frac{v_L(1 - \delta)}{P^* - \delta v_L} = \frac{1}{2} \quad \text{when } v_H = 2v_L. \tag{107}$$

For $\pi < \pi^*$, the seller pools (offers v_L immediately). For $\pi > \pi^*$, the seller screens (offers P^*, then v_L if rejected).[199] As $\delta \to 1$, the screening price $P^*(\delta) \to v_L$ and the seller cannot extract

[198] The stationary fixed point of the recursion in the footnote above satisfies $\mu = \lambda(1 - \delta)/(1 - \delta\lambda)$, $\beta = \lambda(1 - \lambda)A/(1 - \delta\lambda^2)$, and $\lambda = A/(2(A - \delta\beta))$. At $\delta = 0.9$ this gives $\lambda \approx 0.760$, $\mu \approx 0.240$, $\beta \approx 0.120$, matching the table's $T = 200$ entries. At $\delta = 0.99$ the stationary $\mu \approx 0.091$, $\beta \approx 0.045$, so $T = 200$ is still slightly above the asymptotic limit; the residual gap closes as T grows further.

[199] In a screening equilibrium, the seller uses price to separate buyer types: high-value buyers accept immediately at a high price, while low-value buyers reject and receive a lower offer. In a pooling equilibrium, the seller offers a single price that all types accept. The gap case ($0 = c < v_L$) guarantees trade in equilibrium.

a screening premium from high types, but in a two-period game with action set $\{v_L, P^*(\delta)\}$ the asymptotic price-collapse mechanism of the Coase conjecture does not operate.

10.5.3 Computational Results

I model the bargaining game as an extensive-form game and apply CFR.[200]

Table 21: CFR strategy versus analytical equilibrium: π-sweep at $\delta = 0.5$, $n = 10$ seeds per cell.

π	P^*	P(Screen)	SE	Theory	$\lvert\Delta\rvert$	NashConv	SE	Eq. Type
0.10	150	0.000	0.000	0.0	0.000	4.04	0.59	Pooling
0.15	150	0.000	0.000	0.0	0.000	6.04	0.88	Pooling
0.20	150	0.000	0.000	0.0	0.000	8.04	1.17	Pooling
0.25	150	0.000	0.000	0.0	0.000	10.05	1.46	Pooling
0.30	150	0.001	0.000	0.0	0.001	12.05	1.75	Pooling
0.35	150	0.001	0.000	0.0	0.001	14.06	2.04	Pooling
0.40	150	0.001	0.001	0.0	0.001	16.58	2.37	Pooling
0.45	150	0.202	0.133	0.0	0.202	23.36	4.80	Pooling
0.50	150	0.351	0.150	0.5	0.149	23.85	4.32	Indifferent
0.55	150	0.401	0.163	1.0	0.599	23.59	3.97	Screening
0.60	150	0.598	0.144	1.0	0.402	23.20	3.78	Screening
0.65	150	0.801	0.132	1.0	0.199	22.41	3.11	Screening
0.70	150	0.900	0.100	1.0	0.100	19.41	2.75	Screening
0.75	150	0.900	0.100	1.0	0.100	16.16	2.29	Screening
0.80	150	1.000	0.000	1.0	0.000	14.92	1.27	Screening
0.85	150	1.000	0.000	1.0	0.000	11.17	0.96	Screening
0.90	150	1.000	0.000	1.0	0.000	7.42	0.65	Screening

Notes: P(Screen) is the probability the seller offers $P^* = 150$ in period 1, averaged across seeds. SE columns report the across-seed standard error. Theory gives the analytical pure-strategy prediction (0 for pooling, 1 for screening, 0.5 at the indifference point $\pi^* = 1/2$). $|\Delta| = |\text{P(Screen)} - \text{Theory}|$. NashConv is the sum of both players' exploitabilities at iteration 5000, on the same utility scale as the payoffs.

Table 21 reports results from varying π at fixed $\delta = 0.5$. Away from the indifference threshold ($\pi \leq 0.40$ and $\pi \geq 0.80$), CFR's average strategy matches the analytical step function: $|\Delta| \leq 0.001$ in the pooling region and $|\Delta| = 0.000$ in the deep screening region. In the transition window $\pi \in [0.45, 0.65]$, CFR returns mixed strategies with across-seed standard errors of 0.13–0.16, reflecting the seller's economic indifference where both pooling and screening yield identical expected profit. The empirical transition CFR finds is slightly delayed relative to the analytical $\pi^* = 1/2$: P(Screen) ≈ 0.60 at $\pi = 0.60$ and reaches 0.90 only at $\pi = 0.70$. Residual NashConv after 5,000 iterations is in the range 4–24 across π, equivalent to up to 12% of the maximum single-player payoff; CFR is not at ε-Nash for small ε here, and reporting NashConv on a linear utility scale rather than a log scale makes this gap visible.

A δ-sweep at $\pi = 0.7$ verifies the screening price formula $P^*(\delta) = 200 - 100\delta$ with zero error across $\delta \in [0.1, 0.9]$. The CFR average P(Screen) drops from 1.0 at $\delta = 0.1$ to 0.50 ± 0.17 at $\delta \geq 0.75$. The analytical column predicts pure screening (P(Screen) = 1) throughout this range

[200]The game tree has two information sets for the seller (indexed by rejection history) and two for the buyer (indexed by private type). CFR runs for 5,000 iterations per parameter configuration. Vanilla CFR is deterministic given zero initial regrets, so seed-level variability is obtained by perturbing the initial regret table with small Gaussian noise (standard deviation 0.05); ten seeds are run per (π, δ) cell. The reported P(Screen) and NashConv are means across seeds; SE columns report the standard error of the mean. NashConv is the sum of both players' exploitabilities, reported on the utility scale where the maximum single-player payoff is $v_H = 200$.

because $\pi = 0.7 > \pi^* = 1/2$, so this drift is not a Coase regime switch. Instead, as δ grows, the screening price $P^*(\delta)$ shrinks toward v_L and the seller's screening profit approaches the pooling profit; the seller becomes near-indifferent and CFR's average strategy spreads across the two corners under seed-perturbed initial regrets.[201]

10.6 Discussion

Stochastic-game Q-learning and CFR target complementary game classes: simultaneous-move games with observable payoffs and extensive-form games with private information, respectively. The simulations confirm convergence to known equilibria in both settings without encoding domain structure into the algorithms.

11 Bandits and Dynamic Pricing

Dynamic pricing is the canonical in-field reinforcement-learning problem for economics. Unlike simulator-based training, the agent learns from real customers, so *exploration* is not an abstract sampling cost: a bad price loses revenue today and may affect the customer relationship. A seller faces T customers in sequence, sets one price for each customer, and observes only whether the customer bought.[202] The seller does not observe the customer's willingness to pay.

Regret, the total revenue gap between the seller's policy and the best benchmark price or pricing rule, is the central measure. If the benchmark earns r^* per customer, a policy with regret $R(T)$ earns $Tr^* - R(T)$ in total. The chapter asks what economic structure turns pricing from a slow arm-learning problem into a fast demand-learning problem. With no useful structure, regret is polynomial in T. With enough structure—well-separated parametric demand, sparse contextual demand, revealed-preference bounds, or smooth monotone demand curves—learning can be much faster. But the assumptions matter: an unknown noise distribution, strategic buyers, or fairness constraints can restore high regret even when the demand model looks rich.

11.1 Foundations

11.1.1 No Structure on Demand

Kleinberg and Leighton (2003) study the benchmark case where the seller knows almost nothing. Customers arrive with valuations drawn i.i.d. from an unknown distribution on $[0, 1]$, and the seller posts a price from a continuous set. The demand curve $D(p) = \Pr(v \geq p)$ is unknown; the only feedback is whether each customer bought. The seller's goal is to minimize regret against the single price p^* that maximizes $p \cdot D(p)$. Kleinberg and Leighton prove that the minimax regret is $\Theta(\sqrt{T})$.[203]

[201] Four stress tests were conducted: (1) awkward primes $(v_L, v_H) = (37, 83)$, converging to $P^* = 55$ (theory: 55.4); (2) information leak with a single seller information set at the root; (3) grid shift with 150 removed, recovering 145 (99.4%); (4) 3-period game, finding $P_1 = 120$ versus theoretical 136. The 3-period result reflects a tie-breaking equilibrium: at $P_1 = 136$ the high buyer is exactly indifferent, so the seller prefers $P_1 = 120$ (revenue 108 versus 86.4 if rejected).

[202] Rothschild (1974) posed pricing under demand uncertainty as a two-armed bandit problem. His insight was that a *myopic* seller can get stuck at a suboptimal price forever, because exploiting the currently best-looking price generates no information about alternatives.

[203] The upper bound, $O(\sqrt{T \log T})$, discretizes $[0, 1]$ into $K = \lceil (T/\log T)^{1/4} \rceil$ prices and runs the UCB1 algorithm of Auer et al. (2002a). The lower bound, $\Omega(\sqrt{T})$, constructs a family of demand curves parameterized by the location of the optimal price $p^* \in [0.3, 0.4]$. The key tension: posting prices far from p^* is informative about demand but costly in revenue; posting prices near p^* is cheap but uninformative. Resolving this tension costs at least $\Omega(\sqrt{T})$ in cumulative revenue. UCB1 (Auer et al., 2002a) selects the price maximizing $\hat{\mu}_{p_k}(t) + \sqrt{2 \ln t / N_{p_k}(t)}$, where $\hat{\mu}_{p_k}(t)$ is the empirical mean profit and $N_{p_k}(t)$ the number of trials; the second term is an exploration bonus that shrinks as a price is tried more, implementing the principle of optimism in the face of uncertainty.

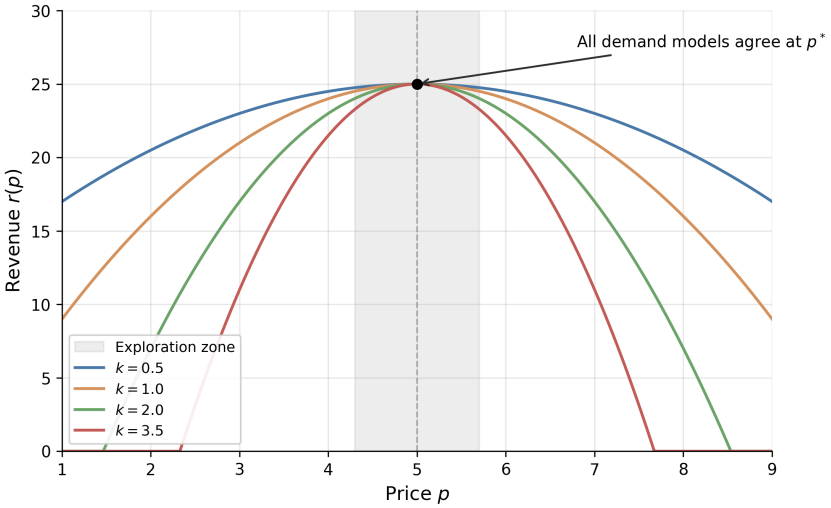

Figure 22: Revenue curves $r(p) = r^* - k(p - p^*)^2$ for four demand curvatures $k \in \{0.5, 1.0, 2.0, 3.5\}$. All models agree at the optimal price $p^* = 5$. Within the shaded exploration zone, the curves are nearly indistinguishable, so playing prices near p^* is uninformative about the demand parameter.

The rate is a useful baseline. After 10,000 customers, $\sqrt{T}$ regret is roughly 100 customers' worth of revenue lost to learning. No algorithm can improve on this worst-case rate without adding structure to D. For adversarial valuations, where a worst-case opponent rather than a fixed distribution chooses customer values, the minimax regret rises to $\Theta(T^{2/3})$, achieved by the Exp3 algorithm[204] on a discretized price grid. I focus on the stochastic setting throughout this chapter.

11.1.2 Parametric Demand

Broder and Rusmevichientong (2012) show that parametric demand is not automatically enough. Let $d(p; z) = \Pr(V \geq p)$, where $z \in \mathbb{R}^n$ is an unknown parameter governing the demand curve. The seller observes binary purchase decisions and updates a maximum likelihood estimate of z. Under standard regularity conditions (bounded demand, unique optimal price, smooth revenue function), the minimax regret is again $\Theta(\sqrt{T})$.[205] The issue is not whether the model has a finite-dimensional parameter; it is whether profitable prices are also informative about that parameter.

The picture changes under a "well-separated" condition requiring that every price in the feasible set is informative about the demand parameter. Formally, the Fisher information $I(p, z)$ is bounded below by $c_f > 0$ for all prices p and parameters z.[206] Under well-separation, a pure greedy policy (MLE-Greedy, pricing at $p^*(\hat{z}_t)$ each period) achieves $O(\log T)$ regret (Theorem 4.8), because every price is informative and dedicated exploration rounds become unnecessary.

[204] *Exp3* (Auer et al., 2002b) maintains a weight $w_k(t)$ for each price, selecting p_k with probability proportional to $w_k(t)$ and updating the chosen price's weight by $\exp(\eta \hat{r}_{k,t})$ where $\hat{r}_{k,t}$ is the revenue importance-weighted by the selection probability; because no model of demand is assumed, the guarantee holds against an adversary who chooses valuations after observing the algorithm.

[205] The lower bound (Theorem 3.1 of Broder and Rusmevichientong (2012)) constructs a linear demand family where all demand curves pass through the same point at the optimal price $p^*(z_0)$. Observing purchases at this price provides no information about z. An MLE-Cycle policy that interleaves dedicated exploration rounds with greedy pricing achieves the matching upper bound $O(\sqrt{T})$ (Theorem 3.6).

[206] This means no two demand curves $d(p; z)$ and $d(p; z')$ cross on the pricing interval, so every purchase observation distinguishes the two hypotheses. In the lower-bound family of Theorem 3.1, the curves all cross at $p = 1$, which is why the $\sqrt{T}$ floor emerges.

11.1.3 High-Dimensional Features with Sparsity

Javanmard and Nazerzadeh (2019) extend the setting to products described by d-dimensional feature vectors x_t. The market value is $v_t = x_t^\top \theta_0 + z_t$, where $\theta_0 \in \mathbb{R}^d$ is unknown and z_t is i.i.d. noise from a known log-concave distribution F.[207] Only s_0 of the d coordinates of θ_0 are nonzero, but the seller does not know which ones.

Their RMLP algorithm (Regularized Maximum Likelihood Pricing) operates in episodes whose lengths double (1, 2, 4, 8, ...). At each episode boundary, the seller fits a LASSO-penalized maximum likelihood estimate of θ_0 using data from the previous episode, then prices greedily throughout the current episode. The regret is $O(s_0 \log d \cdot \log T)$ (Theorem 4.1), with a matching lower bound of $\Omega(s_0(\log d + \log T))$ (Theorem 5.1). The reason this beats $\sqrt{T}$ connects back to the Broder lower bound. In the parametric model of Section 11.1.2, all demand curves can cross at the optimal price, making that price uninformative. Here, customer features vary across periods, so the aggregate demand function at any fixed price changes in proportion to the estimation error in θ_0. Every price is informative, and dedicated exploration rounds become unnecessary.[208] Even with hundreds of features, if only a handful matter, learning is fast.

11.2 Revealed Preference and Partial Identification

11.2.1 WARP-Based Elimination

Misra et al. (2019) bring economic theory into the bandit pricing framework. Their model has K discrete prices, S consumer segments (segment membership observed, but valuations unknown), and within-segment heterogeneity δ. A consumer in segment s has valuation $v_i = v_s + n_i$, where v_s is the segment midpoint and $n_i \in [-\delta, \delta]$ is idiosyncratic noise. The key structural assumption is WARP (Weak Axiom of Revealed Preference): if a consumer buys at price p, she would buy at any lower price $p' < p$.

WARP turns a binary purchase into more information than a single arm reward. If a customer in segment s buys at \$80, then prices below \$80 are also feasible for that customer; if she refuses \$120, prices above \$120 are also ruled out. Aggregating these inequalities gives partial identification of each segment's valuation. Suppose the seller has offered several prices to segment s. Define $p_s^{\min} = \max\{p_k : \text{all observed segment-}s \text{ customers bought at } p_k\}$ and $p_s^{\max} = \min\{p_k : \text{no observed segment-}s \text{ customer bought at } p_k\}$. Then the segment midpoint lies in $[p_s^{\min}, p_s^{\max}]$, and within-segment heterogeneity satisfies $\hat{\delta}_s \le (p_s^{\max} - p_s^{\min})/2$. These bounds propagate to aggregate demand: for each price p_k, the seller constructs upper and lower bounds on total profit $\pi(p_k) = p_k \cdot D(p_k)$. When the profit upper bound at some price falls below the best profit lower bound across all prices, that price is dominated and permanently eliminated from consideration.

The UCB-PI algorithm combines dominance elimination with a price-scaled exploration bonus:

$$I_{p_k}(t) = \hat{\mu}_{p_k}(t) + p_k \sqrt{\frac{2 \ln t}{N_{p_k}(t)}} \tag{108}$$

if p_k is not dominated, and $I_{p_k}(t) = 0$ otherwise, where $\hat{\mu}_{p_k}(t)$ is the average profit observed at price p_k and $N_{p_k}(t)$ is the number of trials.[209] Two design choices drive the improvement

[207] Log-concavity of F and $1 - F$ is satisfied by the normal, logistic, uniform, Laplace, and exponential distributions. It ensures that expected revenue $p \cdot [1 - F(p - x_t^\top \theta_0)]$ is strictly quasi-concave in p, giving a unique optimal price.

[208] If some feature directions are rarely observed, the seller cannot learn all coordinates of θ_0 quickly, and regret degrades to $O(\sqrt{\log(d) \cdot T})$ (Theorem 4.2). If the noise distribution belongs to a known parametric family but its scale parameter is unknown, regret reverts to $\Omega(\sqrt{T})$ (Theorem 7.1), foreshadowing the result of Xu and Wang (2021) discussed in Section 11.3.

[209] Standard UCB1 uses an exploration bonus of $\sqrt{2 \ln t / N_{p_k}(t)}$, which assumes rewards in $[0, 1]$. Since profit at price p_k is bounded by p_k, scaling the bonus by p_k tightens exploration for cheap prices that cannot contribute

over standard UCB1. First, dominance elimination reduces the effective number of arms the algorithm must explore. Second, the p_k scaling focuses exploration on prices where uncertainty actually matters for profit. These gains come from pruning dominated arms and calibrating arm-level exploration; they do not fully exploit smoothness or monotone dependence across nearby prices. Together, within the partial-identification model, these yield $O(\log T)$ regret.

$$\mathbb{E}[R_T(\text{UCB-PI})] \leq \sum_{k \neq k^*} \frac{8 p_k \log T}{\Delta_k} + \left(1 + \frac{\pi^2}{3}\right) \sum_{k=1}^{K} \Delta_k \tag{109}$$

where $\Delta_k = \mu_{k^*} - \mu_k$ is the gap between arm k and the optimal arm.[210]

Misra et al. (2019) calibrate the model to an in-field deployment at ZipRecruiter, a B2B online recruiting platform. With 7,870 customers per month, 1,000 segments, and 10 price points from \$19 to \$399, UCB-PI achieves 98% of oracle profit and produces 43% higher profits during the first month of testing compared to a learn-then-earn alternative. The algorithm has both higher mean profit and lower variance, reflecting the value of eliminating dominated prices early.[211]

11.2.2 Demand-Curve Learning

Weaver et al. (2025) clarify what UCB-PI still leaves on the table. After dominated prices have been removed, the remaining prices are still learned mostly as separate arms. But pricing arms are not independent pulls: they lie on one demand curve. Misra et al. (2019) use revealed-preference bounds to eliminate dominated prices; Weaver et al. (2025) use smoothness and monotonicity to create informational externalities across prices.[212]

GP-UCB and GP-TS place a nonparametric prior on demand $D(p)$ rather than on each price's reward separately. An observation at one price therefore updates beliefs about nearby prices. The monotone variants add the economic restriction that demand weakly decreases with price by modeling both $D(p)$ and its derivative $D'(p)$ and constraining the derivative to be nonpositive. This is a stronger form of economic structure than arm elimination. It does not merely say which prices can be discarded; it says how evidence at one price should move beliefs about the rest of the price grid. That is why the incremental gains from partial identification can be small once a curve-learning bandit is already transferring information efficiently across prices.

11.3 The Value of Knowing the Noise Distribution

Xu and Wang (2021) ask how much it helps to know the shape of demand uncertainty. In their model, a feature vector $x_t \in \mathbb{R}^d$ describes each sales session, the customer's valuation is $w_t = x_t^\top \theta^* + N_t$ where θ^* is unknown and N_t is zero-mean i.i.d. noise with CDF F, and the seller observes only whether the customer bought at the posted price. The question is whether

much profit regardless.

[210] The first sum is the leading term, scaling as $O(\log T)$. The second sum is a constant that does not grow with T. Replacing p_k with 1 recovers the standard UCB1 bound, which is looser.

[211] For multi-product settings, Mueller et al. (2019) impose low-rank structure on the price-sensitivity matrix, achieving regret $O(T^{3/4}\sqrt{d})$ that scales with the latent demand dimension d rather than the number of products. Badanidiyuru et al. (2013) extend the bandit framework to handle inventory constraints ("bandits with knapsacks"), relevant when the seller faces limited stock alongside the pricing decision.

[212] For example, if demand at \$40 is estimated around 60%, then a curve-learning bandit treats demand at \$41 as nearby rather than unknown from scratch; a monotone variant also rules out demand at \$41 being systematically higher than demand at \$40. Price scaling matters because the same five-percentage-point demand uncertainty creates \$0.50 of profit uncertainty at a \$10 price but \$5.00 at a \$100 price, so whether the optimal price is low, middle, or high on the grid changes the exploration problem.

the seller knows F. This is not a cosmetic modeling choice: the error distribution determines how purchase probabilities translate into valuations.[213]

If F is known (the seller knows that demand shocks are, say, normally distributed with known variance), the EMLP algorithm (Epoch-based Maximum Likelihood Pricing) achieves regret $O(d \log T)$ (Theorem 3).[214] After 10,000 customers with $d = 5$ features, the revenue loss is roughly 50 customers' worth, an improvement over the $\sqrt{T} \approx 100$ customers' worth that Kleinberg's lower bound imposes without structural knowledge.

If F is unknown (even if only the variance of a Gaussian is unknown, with everything else known), the regret is at least $\Omega(\sqrt{T})$ (Theorem 12).[215] The seller is back to the Kleinberg baseline: additional features alone do not buy logarithmic regret.

For fixed d, the gap between $O(d \log T)$ and $\Omega(\sqrt{T})$ grows like $\sqrt{T}/\log T$. This is not a constant-factor tuning issue; it is a change in the learning rate. When a modeler specifies a logit or probit demand model, the assumed noise distribution can buy logarithmic regret. Semiparametric approaches that leave the error distribution unspecified pay a concrete cost, reverting from logarithmic to polynomial regret.[216]

11.4 Strategic Buyers

Liu et al. (2024a) introduce a different economic friction: buyers may respond strategically to the learning rule itself. At time t, a buyer arrives with true covariates $x_t^0 \in \mathbb{R}^d$ and valuation $v_t = \theta_0^\top x_t^0 + z_t$. The seller announces a pricing rule $p_t = g(\hat{\theta}_k^\top x_t)$, where $\hat{\theta}_k$ is the current parameter estimate. Crucially, the buyer observes this rule and can distort her reported features. She solves a cost-minimization problem:

$$\min_{\tilde{x}} \; (p - v_t) + \frac{1}{2}(\tilde{x} - x_t^0)^\top A(\tilde{x} - x_t^0) \tag{110}$$

where A is a positive definite matrix governing manipulation costs.[217] The first-order condition yields a systematic bias: the buyer shifts her features to make the seller's model predict a lower valuation, securing a lower price. The seller observes only the distorted features $\tilde{x}_t$, not the true x_t^0.

Theorem 6 (Theorem 1 of Liu et al. (2024a))**.** *Under standard regularity conditions, any pricing policy that treats reported features as truthful accumulates regret* $\Omega(T)$.

[213]The regret benchmark here differs from Section 11.1.1. Kleinberg and Leighton (2003) and Broder and Rusmevichientong (2012) measure regret against the best fixed price; Xu and Wang (2021) and Liu et al. (2024a) measure regret against the clairvoyant contextual policy that sets the optimal price p_t^* for each customer's features x_t. The contextual benchmark is harder.

[214]EMLP runs in doubling epochs of length $\tau_k = 2^{k-1}$. At each epoch boundary, the seller fits a maximum likelihood estimate $\hat{\theta}_k$ using data from the previous epoch, then prices greedily at $p_t = J(x_t^\top \hat{\theta}_k)$ throughout the epoch, where $J(u) = \arg\max_v \, v[1 - F(v - u)]$ is the revenue-maximizing price function. The key technical insight is that the negative log-likelihood is strongly convex (Lemma 7 of Xu and Wang (2021)), so MLE concentrates at rate $O(d/\tau_k)$. Since regret is quadratic in the parameter estimation error (Lemma 5) and there are $O(\log T)$ epochs, the total regret is $O(d \log T)$.

[215]The lower bound constructs two noise variances $\sigma_1 = 1$ and $\sigma_2 = 1 - T^{-1/4}$. Any algorithm that performs well under both must spend $\Omega(\sqrt{T})$ revenue distinguishing the two cases. This extends the "uninformative price" phenomenon of Broder and Rusmevichientong (2012): when the seller does not know F, there exist prices at which observed purchase behavior is nearly identical under different demand parameters.

[216]Tullii et al. (2024) establish the tightest known bound under minimal distributional assumptions: if the noise distribution (c.d.f.) is merely Lipschitz continuous, the minimax regret is $\Theta(T^{2/3})$, strictly between the $\log T$ rate with known F and the $\sqrt{T}$ rate with unknown F. Fan et al. (2024) consider a semiparametric setting where the noise density is smooth and connect the pricing problem to the econometrics of semiparametric estimation.

[217]The matrix A captures how costly it is for the buyer to distort each feature dimension. High eigenvalues of A mean manipulation is expensive. This is the standard model of strategic classification (Hardt et al., 2016), adapted to pricing.

The regret is linear in T: every standard dynamic pricing algorithm, including EMLP and RMLP, systematically underprices because it bases decisions on manipulated features, and the bias does not shrink with more data because the manipulation is endogenous to the pricing rule.

The fix is to jointly estimate demand parameters and manipulation behavior. Liu et al. (2024a) propose an episodic algorithm with two phases per episode. During the exploration phase, the seller posts uniform random prices that do not depend on features. Since the price is independent of $\tilde{x}_t$, buyers have no incentive to manipulate, and the seller observes true features x_t^0. During the exploitation phase, the seller uses the corrected pricing rule that anticipates the manipulation:

$$p_t = g\left(\hat{\theta}_k^\top x_t + \hat{\beta}_k^\top A^{-1}\hat{\beta}_k \cdot g'(\hat{\theta}_k^\top x_t)\right) \tag{111}$$

where $\hat{\beta}_k$ is the estimated coefficient on the manipulable features and g is the optimal pricing function. This correction adds a markup that offsets the anticipated feature distortion.

Theorem 7 (Theorem 2 of Liu et al. (2024a))**.** *With known manipulation cost matrix A, the strategic pricing algorithm achieves regret $O(d\sqrt{T})$.*

When A is unknown, the seller can still recover $O(d\sqrt{T/\tau})$ regret by tracking repeat buyers across exploration and exploitation phases, where τ is the fraction of buyers who appear in both phases (Theorem 3). Higher repeat rates mean more matched pairs for estimating manipulation behavior.

The lesson is that demand learning is also mechanism design. A pricing rule changes buyer incentives, and those incentives change the data the seller sees. Incentive compatibility matters even in settings where the seller is "just" learning demand.[218]

11.5 Comparison of Regret Rates

Table 22 collects the regret rates discussed above. The dominant pattern is that stronger assumptions yield faster learning, but only when the assumptions make profitable actions informative about demand. For fixed dimension, the gap between $\log T$ and $\sqrt{T}$ grows without bound, so the distinction is not just a constant factor. Strategic behavior is the outlier: it produces linear regret that no amount of data can overcome without explicit correction. Figure 23 plots each rate on a log-log scale.

Table 22: Regret rates in dynamic pricing under progressively stronger assumptions. T is the number of customers, d the feature dimension, s_0 the sparsity level. The last column translates asymptotic rates into concrete terms for $T = 10{,}000$ with $d = 5$, setting constants to 1.

Paper	Demand	Noise	Regret	Per 10K (d=5)
Kleinberg and Leighton (2003)	none	none	$\sqrt{T}$	∼100 lost
Broder and Rusmevichientong (2012)	parametric	known family	$\sqrt{T}$	∼100 lost
Broder and Rusmevichientong (2012)	param., well-sep.	known family	$\log T$	∼9 lost
Javanmard and Nazerzadeh (2019)	linear, s_0-sparse	known, log-conc.	$s_0 \log d \log T$	fast if s_0 small
Xu and Wang (2021)	linear, contextual	known	$d \log T$	∼46 lost
Xu and Wang (2021)	linear, contextual	unknown var.	$\geq \sqrt{T}$	∼100 lost
Tullii et al. (2024)	linear, contextual	Lipschitz only	$T^{2/3}$	∼464 lost
Misra et al. (2019)	WARP	–	$\log T$	∼9 lost
Liu et al. (2024a)	linear + strategic	known, naïve	T	never improves
Liu et al. (2024a)	linear + strategic	known, corrected	$d\sqrt{T}$	∼500 lost

[218] Agrawal and Tang (2024) document a related phenomenon in pricing with reference effects. If consumers anchor on past prices, a static pricing policy that ignores reference dependence accumulates linear regret $\Omega(T)$. Chen et al. (2025) show that imposing fairness constraints (requiring similar prices for similar customers) raises the regret floor to $\Theta(T^{2/3})$, a social cost of equitable treatment.

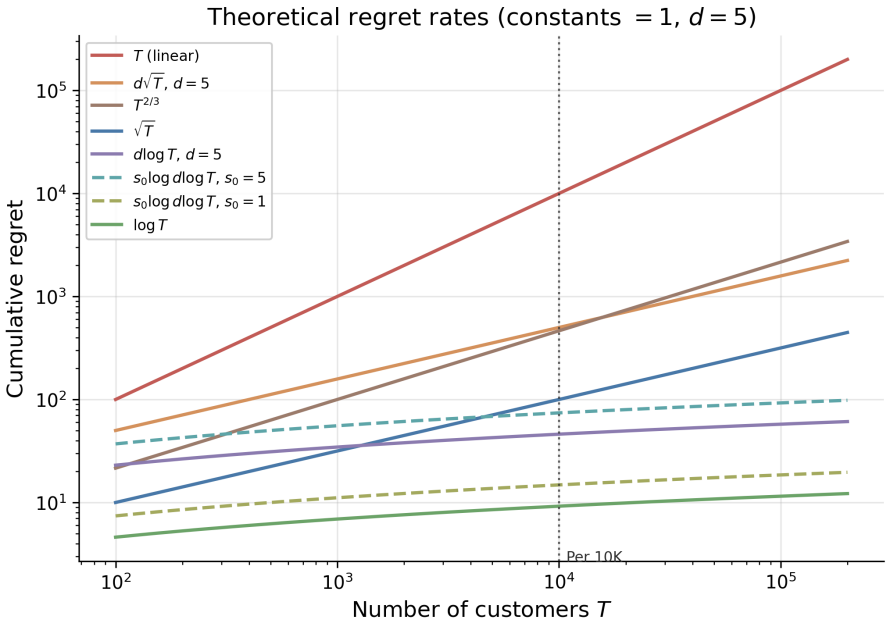

Figure 23: Theoretical regret rate functions at constants equal to 1, $d = 5$. Two cases for the $s_0 \log d \log T$ rate are shown: $s_0 = 1$ (very sparse, ≈ 15 lost at $T = 10{,}000$) and $s_0 = 5$ (moderate sparsity, ≈ 74 lost). The vertical line marks $T = 10{,}000$, matching the "Per 10K" column of Table 22.

11.6 Applications

The same pattern appears beyond single-product posted pricing. Practical pricing systems are useful when they exploit structure that would be invisible to a generic bandit: low-rank substitution patterns across products, common elasticity parameters, capacity constraints, or shared demand forecasts.

11.6.1 Joint Assortment and Pricing at Scale

Cai et al. (2023) tackle the joint assortment-pricing problem, where a retailer must simultaneously choose which products to display and at what prices. With a large catalog and limited shelf space, the number of possible assortments is combinatorially vast; for a Chinese instant noodle producer with 176 products and 30 display slots, there are $\binom{176}{30} \approx 6.4 \times 10^{33}$ possible assortments before prices are even set. Customer demand also depends on market context such as region and season. In each period t, the retailer selects an assortment-pricing vector $a_t \in \mathbb{R}^{d_a}$ (encoding which products to display and at what prices), observes a context vector $x_t \in \mathbb{R}^{d_x}$ (encoding customer demographics and market conditions), and earns revenue Y_t. Cai et al. model expected revenue as $\mathbb{E}[Y_t \mid x_t, a_t] = a_t^\top \Theta^* x_t$, where $\Theta^* \in \mathbb{R}^{d_a \times d_x}$ is an unknown matrix assumed to have low rank $r \ll \min\{d_a, d_x\}$. The low-rank assumption is the demand-side analogue of factor models in asset pricing; a small number of latent dimensions (flavor preferences, seasonal effects, price sensitivity) explain most of the variation in purchasing behavior.

Their Hi-CCAB algorithm estimates Θ^* via penalized least squares, where the penalty is the nuclear norm (the sum of singular values) of Θ. This penalty encourages low-rank solutions, playing the same role for matrices that the LASSO penalty plays for sparse vectors. The time-averaged regret is $\tilde{O}(T^{-1/6})$ with dimension dependence $r(d_a + d_x)$ rather than $d_a \cdot d_x$, so the effective parameter count scales with the number of latent factors, not the full product-by-context matrix. In simulation, Hi-CCAB achieves nearly four times the cumulative sales of the noodle producer's historical assortment strategies, averaged over 100 replications.

Ganti et al. (2018) deploy Thompson Sampling in-field for dynamic pricing at Walmart.com. Their MAX-REV-TS algorithm models demand for each item i on day t via a constant-elasticity function $d_{i,t}(p) = f_{i,t}(p/p_{i,t-1})^{\gamma_i^*}$, where $d_{i,t}(p)$ is unit sales at price p, $f_{i,t}$ is a baseline demand forecast at the previous day's price $p_{i,t-1}$, and $\gamma_i^* < -1$ is the unknown price elasticity. The structural assumption, that demand responds to price through a single elasticity parameter per item, reduces the learning problem from estimating a full demand curve to estimating one scalar per item. MAX-REV-TS places a Gaussian prior over the elasticity vector γ^* and draws posterior samples at each period to solve a constrained revenue maximization problem. In a five-week in-field experiment on a basket of roughly 5,000 items, Thompson Sampling produced a statistically significant increase in per-item revenue relative to the passive pricing baseline.

11.7 Simulation Study: Structural Knowledge and Curve Learning

I use two simulations to make the chapter's taxonomy concrete. The first is a WARP and partial-identification ladder based on Misra et al. (2019). The second is a Weaver-style curve-learning exercise that isolates the informational-externality point.

In the first simulation, I run six algorithms on the Misra et al. (2019) demand environment to trace how cumulative regret responds to increasing structural knowledge.[219] This is a WARP and partial-identification demonstration, not a horse race against GP-UCB, GP-TS, or monotone GP demand-curve methods. The six algorithms, ordered by the structural knowledge they exploit, are: (0) ε-greedy with $\varepsilon = 0.1$, which never adapts its exploration rate; (1) Learn-Then-Earn (LTE), which explores uniformly for the first 5% of rounds and then commits to the empirical best price; (2) UCB1 (Auer et al., 2002a), which adapts exploration via confidence bounds but ignores demand structure; (3) Thompson Sampling (Thompson, 1933), which maintains a Bayesian posterior over purchase rates[220]; (4) UCB-PI (Misra et al., 2019), which uses WARP to eliminate dominated prices and scales the exploration bonus by the price level; and (5) UCB-PI-tuned, which adds a variance-based refinement to the exploration bonus.

Within this WARP-only comparison, Figure 24 reports cumulative regret at four checkpoints. The finite-sample pattern is mixed. UCB-PI-tuned achieves the lowest regret by $T = 200{,}000$, but plain UCB-PI is worse than Thompson Sampling and LTE in this run. The diagnostics also show that $R/\log T$ for plain UCB-PI keeps rising over the plotted checkpoints, so the simulated finite-sample behavior is not visibly logarithmic. The right conclusion is narrower: WARP-based elimination helps relative to plain UCB1, but good finite-sample performance depends on the variance-tuned implementation, and these results do not compare against curve-learning GP methods.

The second simulation makes the Weaver et al. (2025) critique operational. It is a small independent replication of the ten-price design in their simulations. Customer willingness to pay is drawn from Beta$(2, 9)$, Beta$(2, 2)$, or Beta$(9, 2)$, giving low-, middle-, and high-optimal-price cases. The tested prices are $p_a = a/10$ for $a = 1, \ldots, 10$, the firm updates prices every 10 customers, each run lasts $T = 2{,}500$ customers, and the table averages over 1,000 simulation seeds. The algorithms are independent-arm Thompson Sampling, GP-UCB, GP-TS, GP-UCB-M, and GP-TS-M.[221]

[219] The environment has $K = 100$ prices on a grid from \$0.01 to \$1.00, $S = 1{,}000$ consumer segments with equal weights, within-segment heterogeneity $\delta = 0.1$, and segment midpoints $v_s \sim \text{Uniform}(0.1, 0.9)$. A consumer purchases if and only if $v_i \geq p$ (WARP). Each algorithm runs across 10 seeds with $T = 200{,}000$ rounds.

[220] At each period the algorithm draws one sample $\tilde{\mu}_k$ from each arm's posterior over its purchase rate and selects the arm with the highest sampled expected profit $p_k\tilde{\mu}_k$; arms with uncertain posteriors have high-variance draws and are selected frequently, while well-estimated arms are selected in proportion to how likely they are optimal.

[221] The GP code is an independent finite-grid implementation of the same demand-GP and derivative-sign ideas in Weaver et al. (2025), not the authors' replication code. Demand, not reward, is the latent GP object. A batch sale rate is treated as a noisy demand observation with variance $0.25/n_a$. GP-UCB and GP-TS choose prices from posterior upper bounds or posterior draws of $D(p)$. The monotone variants form a joint RBF-GP over $D(0)$,

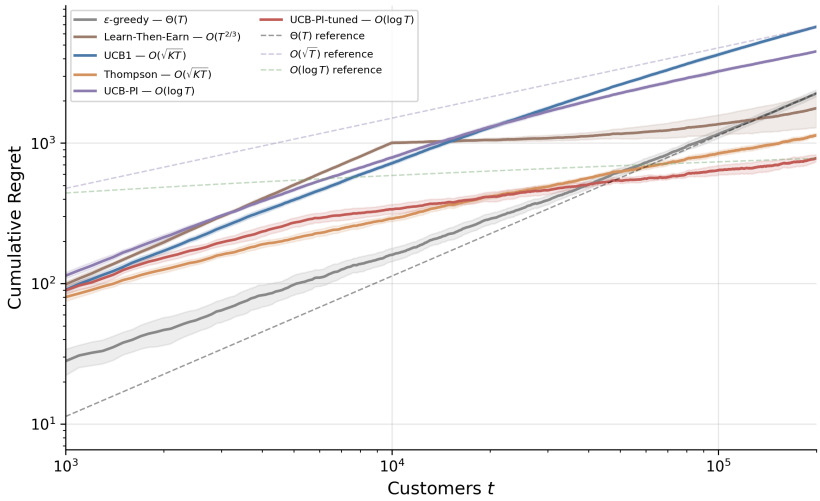

Figure 24: Cumulative regret on log-log axes ($K = 100$, $S = 1{,}000$, 10 seeds). Each algorithm's legend entry includes its theoretical regret rate. Dashed lines show $\Theta(T)$, $O(\sqrt{T})$, and $O(\log T)$ reference rates. Shaded regions are ± 2 standard errors.

Figure 25 plots cumulative profit as a percentage of the price-set oracle Π_P^*, and Table 23 reports the final values. The low-optimal-price case is the clearest diagnostic. With $v \sim \text{Beta}(2, 9)$, Thompson Sampling earns 83.6% of the grid oracle by $T = 2{,}500$, while GP-TS-M earns 97.5% and GP-UCB-M earns 98.3%. In the middle case, TS reaches 93.5% and the monotone GP variants reach about 97%. In the high-price case, TS, GP-TS, and GP-UCB are already near oracle, while the monotone variants are slightly lower. The conclusion is therefore narrower and closer to Weaver et al. (2025): curve-level informational externalities are most valuable when independent-arm bandits are pulled toward high-price uncertainty even though the true optimum is low; when the optimal price is high, the same uncertainty is less damaging because the uncertain high-price region is also close to the profitable region.

WTP	p_P^*	TS	GP-TS	GP-TS-M	GP-UCB-M
$B(2, 9)$	0.2	83.6%	77.2%	97.5%	98.3%
$B(2, 2)$	0.4	93.5%	94.8%	97.0%	97.5%
$B(9, 2)$	0.7	98.2%	98.2%	95.8%	96.1%

Table 23: Final cumulative profit as a percentage of the price-set oracle by $T = 2{,}500$ in the Weaver-style curve-learning simulation.

$D(p)$, and $D'(p)$, enforce $D'(p) \leq 0$ through truncated-normal derivative draws, and reconstruct demand by integrating the derivative path on the price grid. GP-UCB-M uses upper quantiles of these constrained posterior curves.

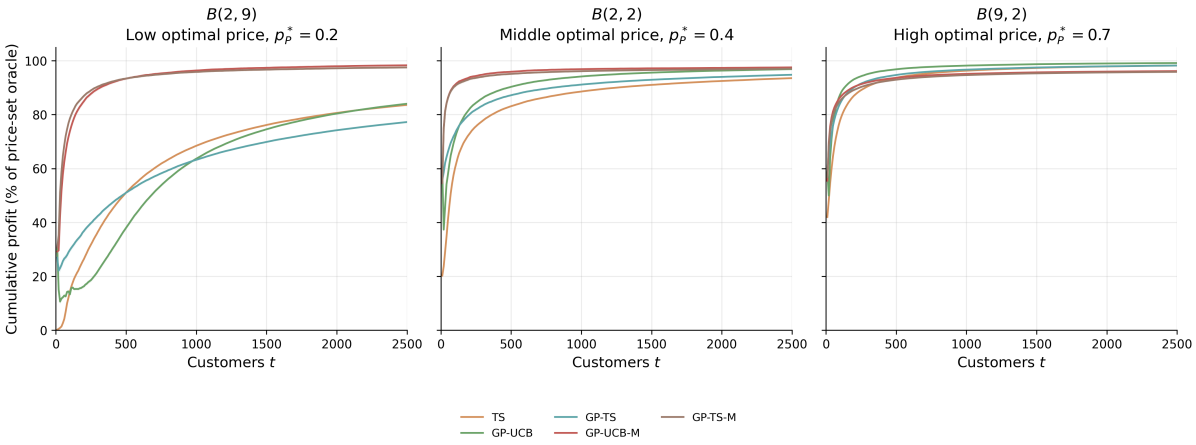

Figure 25: Weaver-style curve-learning pricing simulation across low, middle, and high optimal-price locations ($K = 10$, $T = 2{,}500$, 1,000 seeds, price updates every 10 customers). The y-axis is cumulative profit divided by the price-set oracle. GP-UCB and GP-TS share information across nearby prices; GP-UCB-M and GP-TS-M additionally impose $D'(p) \leq 0$ through derivative-constrained GP curves. Shaded regions are ± 2 standard errors.

12 Offline Reinforcement Learning

In the preceding chapters, the agent interacts with its environment while learning: it tries a price, observes a purchase, and updates its belief. Online learning is natural in digital markets where experimentation is cheap and feedback is instant. In many economically important settings, however, experimentation is impossible or prohibitively costly. A hospital cannot randomly assign treatments to learn optimal dosing. A central bank cannot experiment with interest rate schedules to discover optimal monetary policy. A firm inheriting a decade of transaction logs wants to improve its pricing rule without conducting new experiments during the transition. In each case, the agent has access to a fixed dataset of past decisions and outcomes, collected under some historical policy, and must learn the best possible new policy from this data alone.

This is the problem of *offline reinforcement learning*, also called *batch reinforcement learning*.[222] The agent never queries the environment. All learning happens from a fixed dataset $\mathcal{D} = \{(s_i, a_i, r_i, s_i')\}_{i=1}^n$ collected by some behavioral policy π_b. The goal is to find a policy $\hat{\pi}$ whose value $V^{\hat{\pi}}$ is as close to the optimal V^* as possible.

Online RL algorithms (Q-learning, SARSA, policy gradient methods from Section 4) can in principle be applied to offline data by treating the dataset as a replay buffer. In practice, this fails catastrophically. The reason is *distributional shift*: the learned policy $\hat{\pi}$ inevitably queries state-action pairs that the behavioral policy π_b never visited, and the Q-function at these unseen pairs is pure extrapolation. Because the Bellman backup propagates these extrapolation errors through the max operator, errors compound geometrically across the planning horizon, producing arbitrarily poor policies.[223]

[222] The term "batch RL" was standard in the earlier literature (Ernst et al., 2005; Lange et al., 2012). "Offline RL" became dominant after Levine et al. (2020), who distinguished it from off-policy RL (which still collects new data, just under a different policy than the target). I use "offline RL" throughout.

[223] This failure mode was first demonstrated empirically by Fujimoto et al. (2019b), who showed that standard off-policy algorithms (DDPG, SAC) trained purely from a static dataset performed worse than the behavioral policy itself, even when that behavioral policy was a partially-trained, mediocre agent. The gap widened with dataset size, the opposite of what one expects from more data.

12.1 The Pessimism Principle

The overestimation failure has a clean theoretical characterization. Consider tabular Q-learning with a fixed dataset. At any state-action pair (s, a) not in the dataset, the empirical Bellman backup is undefined, but the max in $\max_{a'} \hat{Q}(s', a')$ may still select it if the randomly-initialized Q-value happens to be high. With function approximation, the problem is subtler but identical in spirit: the function approximator can assign high values to out-of-distribution inputs without any corrective signal.

The solution, established simultaneously by several groups, is the *pessimism principle*: construct a lower confidence bound on the Q-function and optimize against it. Formally, given a dataset $\mathcal{D}$ and a confidence parameter δ, construct a penalty function $\Gamma(s, a)$ that is large where data coverage is poor, and define the pessimistic Q-function

$$\tilde{Q}(s, a) = \hat{Q}(s, a) - \Gamma(s, a) \tag{112}$$

where $\hat{Q}$ is the standard empirical Bellman solution. The policy $\hat{\pi}(s) = \arg\max_a \tilde{Q}(s, a)$ selects actions that are both high-value *and* well-supported by data.

Definition D1 (Pessimistic Value Iteration, PEVI (Jin et al., 2021))**.** *Given dataset $\mathcal{D}$, penalty function $\Gamma_h(s, a)$ for each stage h, and horizon H, PEVI computes*

$$\hat{Q}_h(s, a) = r_h(s, a) + \hat{P}_h \hat{V}_{h+1}(s, a) - \Gamma_h(s, a) \tag{113}$$

$$\hat{V}_h(s) = \max\{\max_a \hat{Q}_h(s, a),\, 0\} \tag{114}$$

$$\hat{\pi}_h(s) = \arg\max_a \hat{Q}_h(s, a) \tag{115}$$

where $\hat{P}_h$ is the empirical transition operator estimated from $\mathcal{D}$.

Jin et al. (2021) show that with $\Gamma_h(s, a) = c \cdot \sqrt{H^3 / N_h(s, a)}$, where $N_h(s, a)$ counts visits to (s, a) at stage h and c is an absolute constant, PEVI achieves

$$V^* - V^{\hat{\pi}} \leq \tilde{O}\left(\sum_{h=1}^{H} \mathbb{E}_{\pi^*} \left[\sqrt{\frac{1}{N_h(s_h, a_h)}} \right] \right) \tag{116}$$

with high probability. The bound depends on the data coverage at states and actions visited by the *optimal* policy π^*, not the full state-action space. This is the key advantage of pessimism over uniform coverage requirements.

12.1.1 Concentrability and Coverage

The bound in (116) is instance-dependent, scaling with how well π_b covers π^*. The classical way to formalize this is through *concentrability coefficients* (Munos and Szepesvári, 2008).

Definition D2 (Single-policy concentrability)**.** *The single-policy concentrability coefficient of π^* with respect to π_b is*

$$C^* = \max_{h \in [H]} \left\| \frac{d_h^{\pi^*}}{d_h^{\pi_b}} \right\|_\infty \tag{117}$$

where $d_h^\pi(s, a)$ is the state-action occupancy measure of π at stage h.

When C^* is finite, the optimal policy visits only states and actions that π_b also visits with non-negligible probability, and the $1/\sqrt{N_h(s_h, a_h)}$ terms in (116) remain controlled. Rashidinejad et al. (2021) prove that single-policy concentrability is both necessary and sufficient for offline learning: if $C^* < \infty$, then $O(|\mathcal{S}| H^2 C^* / \epsilon^2)$ samples suffice for an ϵ-optimal policy; if $C^* = \infty$, no algorithm can guarantee suboptimality better than the behavioral policy.

12.1.2 Impossibility Results

Zanette (2021) establish fundamental limits on offline RL by showing that it can be exponentially harder than online RL. Specifically, there exist MDPs with S states and horizon H where online RL finds an ϵ-optimal policy in $\text{poly}(S, H, 1/\epsilon)$ episodes, but any offline algorithm requires $\Omega(2^H)$ samples unless the dataset covers all reachable states. The construction uses a binary tree MDP where the optimal path visits a unique leaf, and any dataset that misses this leaf provides no information about the optimal action at the root.

The practical implication is that offline RL is not a universal replacement for online experimentation. When the behavioral policy is far from optimal, especially in long-horizon problems, offline methods provably cannot recover the optimal policy without exponentially large datasets. Pessimistic algorithms are the best one can do, but they are still fundamentally constrained by what the data contains.

12.2 Algorithms

I present four practical algorithms that instantiate different approaches to the distributional shift problem. All four can be understood as modifications of standard Q-learning (Section 4) that prevent the agent from overvaluing actions outside the data support.

12.2.1 Fitted Q-Iteration

Fitted Q-Iteration (FQI, Ernst et al., 2005) is the simplest offline RL algorithm, predating the modern pessimism framework. FQI applies the standard Bellman backup iteratively using supervised regression on the fixed dataset, as described in Section 4. It does not include any explicit pessimism mechanism. When state-action coverage is poor, the max in the Bellman backup selects the highest Q-value among all actions at s', including actions never observed in the data. If the function approximator generalizes poorly at these unseen actions, targets become noisy and biased upward, causing the overestimation cascade. FQI works well when coverage is good but degrades as coverage gaps grow.[224]

12.2.2 Conservative Q-Learning

Conservative Q-Learning (CQL, Kumar et al., 2020) adds an explicit penalty that pushes down Q-values at actions not well-represented in the data. The key idea is to add a regularizer to the Bellman error objective that minimizes Q-values under a broad distribution over actions while maximizing Q-values at the actions actually taken in the dataset.

Definition D3 (Conservative Q-Learning)**.** *CQL modifies the standard Bellman error objective by adding a conservative regularizer. At each iteration, CQL solves*

$$\hat{Q}_{k+1} = \arg\min_Q \; \alpha \left(\mathbb{E}_{s\sim\mathcal{D}} \left[\log \sum_a \exp Q(s,a) \right] - \mathbb{E}_{(s,a)\sim\mathcal{D}}[Q(s,a)] \right) + \frac{1}{2}\mathbb{E}_{\mathcal{D}} \left[\left(Q(s,a) - \hat{\mathcal{B}}^{\pi_k} \hat{Q}_k(s,a) \right)^2 \right] \tag{118}$$

where $\alpha > 0$ is a hyperparameter controlling the degree of conservatism, and $\hat{\mathcal{B}}^{\pi_k}$ is the empirical Bellman operator.

The first term in the regularizer, $\log \sum_a \exp Q(s,a)$, is a soft maximum over all actions, pushing down Q-values uniformly. The second term, $\mathbb{E}_{\mathcal{D}}[Q(s,a)]$, pulls Q-values back up at the data actions. The net effect is a penalty on Q-values for actions that appear infrequently relative to their softmax contribution. Kumar et al. (2020) prove that the resulting Q-function is a

[224] Munos and Szepesvári (2008) prove finite-time error bounds for FQI under approximate Bellman completeness and all-policy concentrability. Both assumptions are strong, and violation of either leads to divergence in practice.

pointwise lower bound on the true Q-function (Theorem 3.2), making it a concrete instantiation of the pessimism principle from (112) with an implicit, data-adaptive penalty Γ.

12.2.3 Implicit Q-Learning

Implicit Q-Learning (IQL, Kostrikov et al., 2022) avoids querying Q-values at unseen actions entirely. Instead of computing $\max_{a'} Q(s', a')$ in the Bellman backup (which requires evaluating Q at potentially out-of-distribution actions), IQL learns a separate value function $V(s)$ that approximates the in-sample maximum through *expectile regression.*[225]

Definition D4 (Implicit Q-Learning)**.** *IQL maintains three functions: $Q_\theta(s, a)$, $V_\psi(s)$, and a policy $\pi_\phi(a|s)$. The value function is trained via expectile regression*

$$L_V(\psi) = \mathbb{E}_{(s,a)\sim\mathcal{D}}\left[L_2^\tau(Q_{\bar{\theta}}(s, a) - V_\psi(s))\right] \tag{119}$$

where $L_2^\tau(u) = |\tau - \mathbb{1}\{u < 0\}| \cdot u^2$ is the asymmetric squared loss with expectile parameter $\tau \in (0.5, 1)$, and $Q_{\bar{\theta}}$ uses a target network. The Q-function is trained with V_ψ as the continuation value

$$L_Q(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma V_\psi(s') - Q_\theta(s, a)\right)^2\right] \tag{120}$$

The expectile parameter τ controls the degree of optimism within the data support. As $\tau \to 1$, the expectile converges to the in-sample maximum; as $\tau \to 0.5$, it converges to the in-sample mean. Setting $\tau = 0.7$ balances exploiting the best observed actions against the noise in finite samples. The critical property is that the Q-function is never evaluated at actions outside the dataset: the max operation is implicit in the expectile regression of V.

12.2.4 Batch-Constrained Q-Learning

Batch-Constrained Q-Learning (BCQ, Fujimoto et al., 2019b) takes a different approach: rather than modifying the Q-function objective, it restricts the policy to actions similar to those in the dataset. BCQ first learns a generative model $G_\omega(s)$ of the behavioral policy, then constrains the policy to only select actions with high likelihood under G_ω.[226]

Definition D5 (Batch-Constrained Q-Learning)**.** *BCQ learns a behavioral model $G_\omega(a|s)$ from the dataset via maximum likelihood, and constrains action selection*

$$\hat{\pi}(s) = \arg \max_{a: G_\omega(a|s) \geq \tau \cdot \max_{a'} G_\omega(a'|s)} Q_\theta(s, a) \tag{121}$$

where $\tau \in (0, 1]$ is a threshold parameter. The Q-function is trained with standard Bellman backups, but the max in the target computation is also constrained to the feasible action set.

BCQ's constraint is hard rather than soft: actions with behavioral probability below τ times the most likely action are simply excluded from consideration. This prevents the Q-function from ever being queried at truly out-of-distribution actions, addressing the distributional shift

[225] The simulation below uses the variant labelled IQL-argmax in Table 24. The expectile-V learning step follows Kostrikov et al. (2022) verbatim. The policy-extraction step in the original paper is advantage-weighted regression on $\exp(\beta(Q-V))$; for the discrete 10-price action set used here, that step is replaced with $\arg\max_a Q(s, a)$, which is equivalent in the deterministic limit and avoids the additional temperature hyperparameter. The substitution does not change which actions get high Q-values; it only changes how the policy network is fit.

[226] The original Fujimoto et al. (2019b) BCQ targets continuous action spaces: G_ω is a variational autoencoder and a perturbation network adjusts sampled actions toward higher Q. For the discrete pricing action set used in the simulation below, the discrete BCQ-D variant of Fujimoto et al. (2019a) is the more natural choice. BCQ-D drops the VAE and perturbation network and uses an MLP classifier of the behavioral action distribution, then thresholds: only actions a with $G_\omega(a \mid s) \geq \tau \cdot \max_{a'} G_\omega(a' \mid s)$ are admissible. The pessimism mechanism is the same in both variants; the architecture differs because the action space differs. Table 24 reports BCQ-D.

problem at the policy level rather than the value function level. The tradeoff is that BCQ's performance is bounded by the quality of actions in the dataset. If the behavioral policy never takes the optimal action at some state, BCQ cannot discover it regardless of sample size.

12.3 Trajectory Models and Return-Conditioned Supervised Learning

Pessimism-based offline RL keeps the value function as the central object, with the policy emerging from a Q-function constructed to be conservative outside the data. An alternative inverts that relationship. Train a supervised or generative model on the logged trajectories themselves, and treat the model as the policy. The agent forecasts what to do, and the forecast is the action. Two members of this family bracket the design space, a transformer-based trajectory model and a stripped-down MLP that isolates the operative ingredient.

12.3.1 Decision Transformer

An offline reinforcement-learning dataset is a collection of trajectories $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_T, a_T, r_T)$ generated by some behavioural policy. A trajectory generative model treats τ as a sequence and trains a generative model on those sequences. The Chen et al. (2021) Decision Transformer represents each trajectory as a flat sequence

$$\tau = (\widehat{R}_0, s_0, a_0, \widehat{R}_1, s_1, a_1, \ldots, \widehat{R}_T, s_T, a_T), \tag{122}$$

where $\widehat{R}_t = \sum_{t' \geq t} r_{t'}$ is the return-to-go from time t onwards. A causally masked GPT-style transformer is trained to predict the next action token given the last K context tokens. At test time, the operator specifies a desired return $R^\star$ and primes the model with $\widehat{R}_0 = R^\star$ and the current state s_0. The model generates the corresponding action, the environment returns a reward and the next state, the operator decrements $R^\star$ by the realised reward, and the loop repeats. No value function, no policy gradient, no critic appears in training. The conditioning variable plays the role of a behavioural target rather than an estimated value function.[227]

The empirical case rests on D4RL and Atari. On the 1% DQN-replay slice of Atari, the Decision Transformer attains gamer-normalised scores of 267.5 on Breakout against the 211.1 of Conservative Q-Learning, the incumbent offline-RL baseline. On the D4RL locomotion benchmark, the Decision Transformer averages 74.7 across nine task-dataset combinations against CQL's 54.2 and behavioural cloning's 47.7.[228]

12.3.2 Return-Conditioned Supervised Learning

The Decision Transformer combines two ideas, a transformer architecture over trajectory tokens and a return-conditioning protocol at deployment. Emmons et al. (2022) isolate the second from the first. Their return-conditioned supervised learning baseline drops the transformer entirely. A small multilayer perceptron is trained by maximum likelihood to predict the action given the current state and a desired return-to-go,

$$\pi_\theta(a \mid s, \widehat{R}) = \mathrm{softmax}\big(f_\theta(s, \widehat{R})\big). \tag{123}$$

[227] The simulation below implements a fused-token simplification: each timestep is represented as a single token whose embedding sums the return, state, and previous-action contributions, rather than as three distinct tokens at adjacent positions. This shrinks the context window by a factor of three at the cost of departing from the strict autoregressive token order of Chen et al. (2021). The fused form coincides with the architecture used by Emmons et al. (2022), who report that the trajectory-as-sequence framing is what matters; the three-token ordering is not the load-bearing component.

[228] The same trajectory-as-sequence reframing was independently proposed by Janner et al. (2021), with beam search over tokenised states and actions replacing return conditioning. Subsequent diffusion variants (Janner et al., 2022; Ajay et al., 2023) replace autoregressive generation with iterative denoising of an entire trajectory and dispense with dynamic programming entirely. The methodological move is the same, plan by sampling from a generative model of structured objects rather than by maximising a learned value function.

At deployment time the same protocol applies. Specify a target return $R^\star$, condition on it, execute the predicted action, decrement $R^\star$ by the realised reward, and repeat. Across the D4RL locomotion suite RvS matches the Decision Transformer on most task-dataset pairs at a small fraction of the parameter count, suggesting that return-conditioning rather than the transformer architecture is the operative ingredient. The result raises a caveat that both methods share. Conditioning on a return that is far above any value observed in the training data is an extrapolation request, and the policy's behaviour out of distribution is not constrained by the training objective.[229]

12.4 Simulation Study: Offline RL for Dynamic Pricing

The simulation study evaluates seven offline learners on a perishable inventory pricing problem with demand regime switching. The pessimism family appears as FQI, CQL, IQL-argmax, and BCQ-D; the supervised-conditioning family appears as behavioral cloning, the Decision Transformer, and return-conditioned supervised learning. A retailer with $I_{\max} = 30$ units of perishable inventory must set prices over $H = 20$ periods. The state (i, d, t) consists of current inventory $i \in \{0, \ldots, 30\}$, demand regime $d \in \{1, 2, 3, 4\}$, and time remaining $t \in \{1, \ldots, 20\}$. The action is a price $p \in \{1, \ldots, 10\}$. Demand follows $Q \sim \text{Poisson}(\lambda_0[d] \cdot e^{-0.15p})$ with base rates $\lambda_0 = (1.5, 3.0, 5.0, 8.0)$, and the reward is $r = p \cdot \min(Q, i)$. Demand regimes follow a 4-state Markov chain with diagonal persistence 0.6. Unsold inventory at the terminal period incurs a spoilage cost of \$2.00 per unit, making clearance pricing valuable near the deadline.[230] The behavioral policy represents a conservative pricing team that always sets the maximum price ($p = 10$) regardless of demand regime, inventory, or time remaining, with probability 0.85 and randomizes uniformly over all prices with probability 0.15.[231] All episodes start at full inventory ($i = 30$). All offline methods train on 500 episodes and are evaluated over 1,000 episodes against the DP optimal policy computed by backward induction.[232] Results are averaged over 20 independent seeds.

FQI achieves 81.2% of the DP optimal, substantially below the behavioral cloning baseline of 88.0% (Table 24). Without any mechanism to control extrapolation error, the $\max_{a'} Q(s', a')$ operator in FQI's Bellman backup selects overestimated Q-values at out-of-distribution actions, and these overestimates compound across 200 iterations of fitted Q-iteration. CQL and IQL-argmax both exceed the behavioral baseline, achieving 91.9% and 92.0% respectively.[233] CQL's

[229] Brandfonbrener et al. (2022) formalise the failure mode. In stochastic environments, conditioning on a high return induces a policy that selects actions whose value distribution has a high upper tail rather than a high expectation, which is a different optimisation problem. The Decision Transformer and RvS recover the optimal policy only when the environment is near-deterministic or when the target return matches what an optimal policy would actually achieve. The reduction-to-supervised-learning that makes the family attractive comes with this stochastic-environment caveat.

[230] The spoilage penalty creates distributional shift. The optimal policy adapts prices to inventory level and time remaining, using lower prices near the deadline when inventory is high. The behavioral policy ignores these state variables and prices at the maximum, so the Q-function at state-adapted pricing actions is extrapolation from sparse data. The \$2.00 penalty is a deliberate design choice: under harsher penalties (e.g., \$10 per unit), all methods collapse to 48–53% of optimal regardless of algorithmic sophistication, confirming that no offline correction can overcome severe distributional shift when the penalty regime amplifies consequences of the behavioral policy's suboptimality.

[231] With 500 episodes (10,000 transitions) on a state-action space of 24,800 pairs, the 85% concentration at price 10 ensures that the behavioral state-action occupancy diverges significantly from the optimal policy's occupancy, while the 15% uniform component provides sparse off-policy coverage.

[232] FQI uses the standard Bellman backup with $\max_{a'} Q(s', a')$, deliberately without a target network, to isolate the overestimation cascade as a pedagogical baseline. Adding target networks to FQI mitigates but does not eliminate extrapolation error. CQL and IQL include target networks following their original implementations (Kumar et al., 2020; Kostrikov et al., 2022); for CQL, target networks proved essential, as the conservative penalty amplifies bootstrap instability without them.

[233] CQL uses $\alpha = 0.1$, the result of a search over $\alpha \in \{5.0, 2.0, 0.5, 0.1\}$. Larger values push Q-values down too aggressively, collapsing the learned policy to the behavioral action at most states; the right α is problem-specific

Table 24: Policy value for each offline RL method, expressed as mean return and percentage of the DP optimal. Standard errors computed over 20 seeds.

Method	Mean Return	% of Optimal
DP Oracle	192.41 ± 0.33	100.0%
IQL-argmax	176.98 ± 0.56	92.0%
CQL	176.73 ± 1.13	91.9%
BC	169.27 ± 0.60	88.0%
BCQ-D	169.27 ± 0.60	88.0%
DT	169.27 ± 0.60	88.0%
RvS	169.27 ± 0.60	88.0%
FQI	156.18 ± 1.68	81.2%

conservative penalty suppresses Q-values at actions not well-represented in the data while preserving relative ordering among data-supported actions, allowing the policy to discover lower prices that clear inventory near the deadline. IQL-argmax achieves the same improvement through a different mechanism: by replacing $\max_{a'} Q(s', a')$ with the expectile-regressed value function $V(s')$ as the Bellman continuation, it avoids querying Q at unseen actions during training while still extracting a policy that improves on the behavioral at states where the 15% noise component revealed better pricing actions.

Four of the trained methods, BC, BCQ-D, DT, and RvS, all report 169.27 ± 0.60, identical to four decimal places (Table 24). The coincidence is not numerical accident but a property of the behavioral distribution. With 85% of the dataset mass on the single action $p = 10$, BC's cross-entropy minimiser is the constant policy $\hat{\pi}_{BC}(s) = 10$ at every state; BCQ-D's threshold $\tau \cdot \max_{a'} G_\omega(a' \mid s)$ exceeds the empirical probability of every other action and admits only $a = 10$ into the constrained set; DT and RvS, conditioned on the high target return $R^\star = V^*(s_0) \approx 184$, see an extrapolation request well outside the training distribution and default to the modal training action, which is again $p = 10$. The four resulting policies are pointwise identical, so under the shared evaluation RNG they trace bit-identical reward streams and produce bit-identical per-seed means. This is the expected behaviour of supervised-conditioning offline methods on heavily concentrated behavioral data: none of them has a mechanism to extrapolate beyond what the behavioral policy did, and the constraint binds at the same action for all four (Levine et al., 2020; Fujimoto et al., 2019b). Distinguishing among them requires a behavioral that spreads probability across multiple actions, which the coverage experiment below begins to examine.

These results validate several theoretical predictions from the preceding sections. FQI's degradation below the behavioral baseline (81.2% versus 88.0%) demonstrates the overestimation cascade described in Section 12.2: without pessimism, the $\max_{a'}$ operator selects overestimated Q-values at out-of-distribution actions, and 200 iterations of bootstrapping compound these errors geometrically (Fujimoto et al., 2019b). CQL and IQL-argmax both exceeding BC confirms the pessimism principle (Section 12.1): CQL's conservative penalty produces a pointwise lower bound on the true Q-function (Definition D3, Theorem 3.2 of Kumar et al. 2020), while IQL's expectile regression (Definition D4) avoids querying Q at unseen actions entirely (Kostrikov et al., 2022). BCQ-D matching BC exactly illustrates the action-constraint tradeoff formalized in Definition D5: performance is bounded by the quality of actions in the data, and when the behavioral policy concentrates on a single action, no amount of Q-learning can overcome the constraint.

Figure 26 varies the behavioral noise parameter $\epsilon_b \in \{0.05, 0.3, 0.9\}$. CQL and IQL-argmax remain above the BC baseline across all coverage levels, confirming that both pessimism mechanisms provide robustness to the data distribution. FQI exhibits non-monotone behavior: it

and can vary by orders of magnitude.

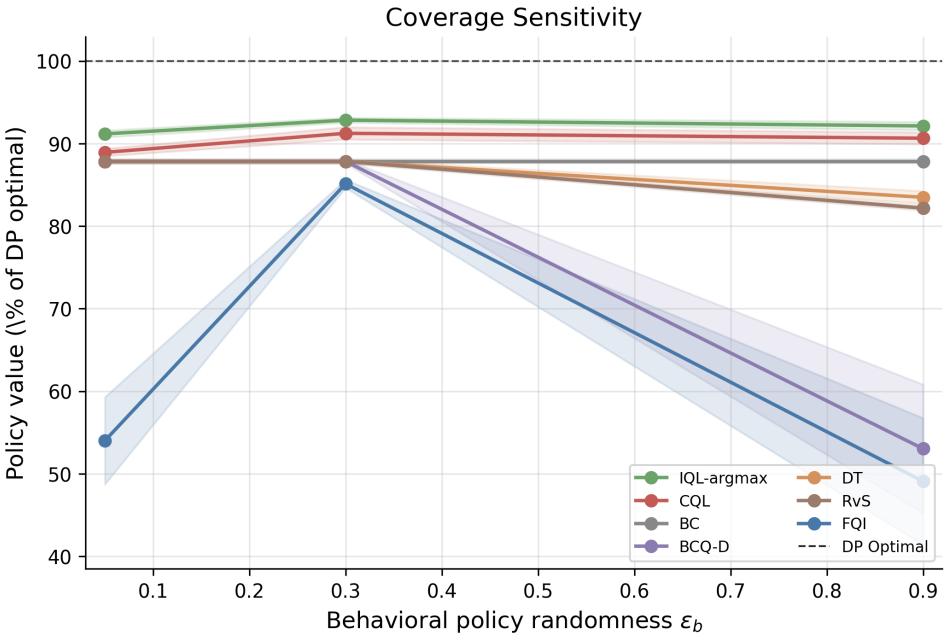

Figure 26: Policy value (as % of DP optimal) versus behavioral policy randomness ϵ_b for the pessimism family (FQI, CQL, IQL-argmax, BCQ-D), the supervised-conditioning family (DT, RvS), and the behavioral cloning baseline (BC). Higher ϵ_b increases data coverage.

peaks at moderate coverage ($\epsilon_b = 0.3$) where partial exploration controls the overestimation cascade, but collapses at both extremes.[234] BCQ-D collapses at $\epsilon_b = 0.9$ because a nearly uniform behavioral policy renders the action constraint vacuous, reducing BCQ-D to unconstrained FQI. DT and RvS sit at the BC line for $\epsilon_b \in \{0.05, 0.3\}$ and drift below it at $\epsilon_b = 0.9$; with broader coverage the return-conditioning extrapolation has more room to pick out-of-distribution actions, and the supervised objective offers no mechanism to penalise them. These coverage patterns connect directly to the concentrability framework (Definition D2): as ϵ_b decreases, the concentrability coefficient C^* grows because the optimal policy's state-action occupancy diverges from the behavioral policy's, and the $1/\sqrt{N_h(s_h, a_h)}$ terms in (116) become large at states the optimal policy visits. CQL and IQL-argmax remain robust because their conservative adjustments scale with data sparsity, while FQI lacks this adaptive correction.

The chapter's object is therefore fixed-data policy improvement when scalar rewards are observed. This is distinct from RLHF, where the reward itself must be inferred from preference comparisons and then interpreted as an object for aggregation, incentives, and welfare. The next chapter takes up that separate problem.

13 RLHF and AI Alignment

Every chapter so far assumes access to a scalar reward signal: dynamic programming requires $r(s, a)$, model-free RL observes r_t after each transition, and the Bellman optimality equation presupposes that rewards are known or observable. When they are neither, the DP/RL machinery cannot be applied directly.

In many domains, scalar rewards are unavailable but ordinal preferences over trajectories are easy to elicit. A human evaluator cannot assign a meaningful numerical score to a paragraph of text, but can reliably say "response A is better than response B." The raw data is a set of

[234] At high ϵ_b, the near-uniform behavioral policy provides Q-function targets across all actions, giving the unconstrained $\max_{a'}$ operator more opportunities to select overestimated values rather than fewer.

trajectory pairs with binary preference labels. Reinforcement learning from human feedback (RLHF) uses these ordinal comparisons to learn a proxy reward function r_θ, which then serves as the scalar signal for standard RL optimization. This is not an inverse problem in the IRL sense; the goal is not to rationalize observed behavior, but rather a two-stage forward problem in which the analyst first learns a reward from human judgments and then solves the resulting MDP. Christiano et al. (2017) demonstrated that this approach could train agents without an explicit reward function. RLHF has since become the predominant method for aligning large language models.

13.1 Learning Rewards from Preferences

The canonical RLHF framework is built on a formal model of human preference. The observed data consists of tuples (s, y_w, y_l), where s is a context, and y_w and y_l are two outputs, with y_w being the "winner" preferred by a human. Assuming preferences follow a latent utility model, the Bradley-Terry model (Bradley and Terry, 1952) gives the probability that y_w is preferred: $P(y_w \succ y_l|s) = \sigma(r_\theta(s, y_w) - r_\theta(s, y_l))$, where $r_\theta : \mathcal{S} \times \mathcal{Y} \to \mathbb{R}$ is a learned *reward model* parameterized by θ, trained to approximate human preferences (not the ground-truth reward, which is unobserved), and $\sigma(\cdot)$ is the logistic function. This formulation is a binary logit model (Section 2, Equation 1).[235] The reward model parameters θ are estimated by minimizing the negative log-likelihood of the observed human choices:

$$\mathcal{L}(\theta) = -\mathbb{E}_{(s,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(r_\theta(s, y_w) - r_\theta(s, y_l)\right)\right], \tag{124}$$

In the LLM setting, the "outputs" y_w and y_l are token sequences, i.e., trajectories of the autoregressive policy[236], so preferences are over trajectories rather than single actions. The preference loss in Equation (124) is MLE of a choice model where the "alternatives" are trajectory segments and the "choice" is the human-preferred one.

This logit foundation is also a social-choice assumption once labelers are heterogeneous. Pooling pairwise comparisons from many evaluators does not recover a neutral "human preference"; it estimates a single score whose odds ratios summarize the aggregation rule induced by the sampling scheme, label model, and loss function. Recent alignment work makes this point explicit: standard Bradley-Terry-style reward learning can behave like a hidden social welfare function, can fail natural axioms for aggregating rankings, and can create strategic reporting incentives when preferences differ across contexts or groups (Conitzer et al., 2024; Siththaranjan et al., 2024; Ge et al., 2024; Park et al., 2024). DPO inherits the same aggregation problem because it replaces the explicit reward model with an implicit reward difference, not with a theory of whose preferences count.

13.2 The RLHF Pipeline and Direct Optimization

The learned r_θ then serves as a proxy objective for policy optimization. Building on the reward-learning and RL fine-tuning framework of Ziegler et al. (2019) and Stiennon et al. (2020), the canonical three-stage pipeline was formalized by Ouyang et al. (2022). First, a base pre-trained model is initialized via supervised fine-tuning (SFT)[237] on a small dataset of high-quality

[235] Iskhakov et al. (2020) discuss the contrasts and synergies between machine learning and structural econometrics, including the shared reliance on logit-based choice models that underlies both RLHF and dynamic discrete choice estimation.

[236] An *autoregressive* language model generates text token by token: at each step it outputs a distribution over the vocabulary conditioned on all preceding tokens, then samples the next token; the full response is therefore a trajectory in token space and the model acts as a sequential policy over a vocabulary-sized action set.

[237] *Pretraining* optimizes a language model to predict the next token across a massive text corpus, producing broad linguistic knowledge with no behavioral objective. *Supervised fine-tuning* (SFT) continues training on a small curated dataset of (prompt, ideal-response) pairs to specialize the model toward the desired task and

demonstrations, yielding an initial policy π^{SFT}. This grounds the model in the desired style and format. The second step is the reward model training as described, using preference data generated from this π^{SFT}.

In the third step, the SFT policy π_ϕ is fine-tuned via PPO (Section 5.5) to maximize the frozen reward model r_θ,[238] with a KL-divergence penalty preventing the policy from drifting into regions where r_θ is unreliable. The objective is

$$J(\phi) = \mathbb{E}_{s\sim\mathcal{D}, y\sim\pi_\phi(\cdot|s)}[r_\theta(s,y)] - \lambda_{KL}\mathbb{E}_{s\sim\mathcal{D}}[D_{KL}(\pi_\phi(\cdot|s)\,||\,\pi^{SFT}(\cdot|s))], \tag{125}$$

where D_{KL} is the Kullback-Leibler divergence and λ_{KL} controls the penalty strength.[239] Without this constraint, the policy exploits inaccuracies in r_θ to achieve high proxy scores with degenerate behavior ("reward hacking"), the RLHF analogue of divergence under function approximation (Section 5.3).

The KL-regularized objective in Equation (125) admits a Bayesian interpretation (Korbak et al., 2022). In this view, the reference policy π^{SFT} acts as a prior distribution over plausible responses. The reward model r_θ provides evidence, specifying which responses are more desirable. The goal of alignment is to find the posterior distribution π^* that optimally combines the prior with this evidence. This ideal posterior policy takes the form of Equation (126):

$$\pi^*(y|s) \propto \pi^{SFT}(y|s)\exp\left(\frac{r_\theta(s,y)}{\lambda_{KL}}\right). \tag{126}$$

The reward function scaled by λ_{KL} defines the log-likelihood, so the KL-regularized objective $J(\phi)$ is equivalent (up to an additive constant) to the Evidence Lower Bound (ELBO) for this Bayesian inference problem. Maximizing the RLHF objective via PPO is therefore variational inference: finding the policy π_ϕ that minimizes KL divergence to π^*. This reframes the KL penalty as a structural component of the inference problem rather than an ad-hoc regularizer.

Despite this closed-form characterization, the three-stage RLHF pipeline is complex to implement, requiring training multiple large models and a computationally expensive RL loop. Direct Preference Optimization (DPO), introduced by Rafailov et al. (2023), collapses the pipeline into a single supervised learning objective by reparameterizing the reward function in terms of π^* and π^{SFT}, as in Equation (127):

$$r(s,y) = \lambda_{KL}\log\left(\frac{\pi^*(y|s)}{\pi^{SFT}(y|s)}\right) + \lambda_{KL}\log Z(s). \tag{127}$$

When this analytical expression for the reward is substituted into the Bradley-Terry preference loss from Equation (124), the unknown partition function $Z(s)$ cancels out. This yields a loss function that depends only on the policy π_ϕ being optimized and the fixed reference policy π^{SFT}. The DPO objective, given in Equation (128), is thus a simple binary cross-entropy loss over policy likelihoods:

$$\mathcal{L}_{DPO}(\phi;\pi^{SFT}) = -\mathbb{E}_{(s,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\lambda_{KL}\log\frac{\pi_\phi(y_w|s)}{\pi^{SFT}(y_w|s)} - \lambda_{KL}\log\frac{\pi_\phi(y_l|s)}{\pi^{SFT}(y_l|s)}\right)\right]. \tag{128}$$

This objective is optimized on ϕ using standard supervised learning with a static preference dataset. The gradient increases the likelihood of preferred responses y_w and decreases the likelihood of dispreferred responses y_l, relative to the reference policy.

The growing family of DPO variants is best read economically by asking what part of the

establish the reference policy π^{SFT} from which the KL penalty is measured.

[238] After fine-tuning concludes, the reward model may be reused for evaluation or for later alignment iterations, but it is not updated inside the PPO optimization loop.

[239] λ_{KL} denotes the KL penalty weight, reserving β for model parameters and γ for discount factors. The standard RLHF literature, including Rafailov et al. (2023), uses β for this parameter.

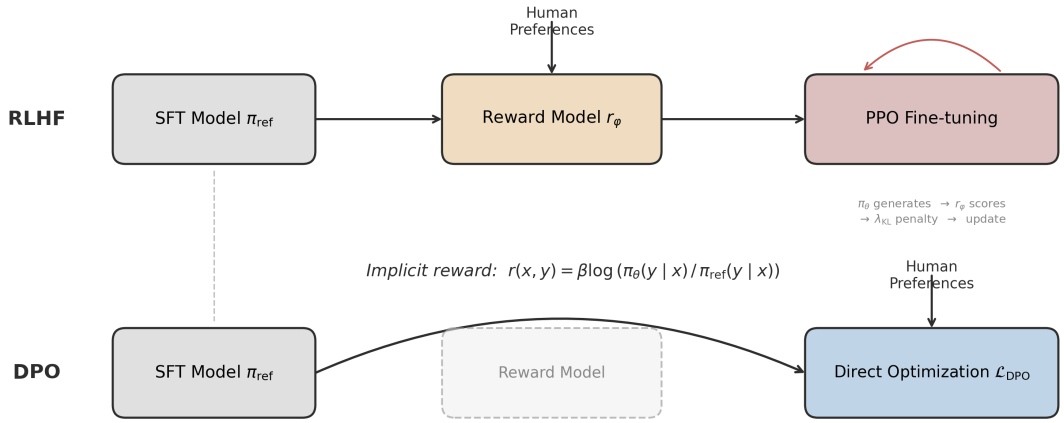

Figure 27: **RLHF versus DPO pipelines.** Top row: the three-stage RLHF pipeline trains a reward model from human preferences, then uses PPO to fine-tune the policy with a KL penalty. Bottom row: DPO collapses the pipeline into a single supervised learning objective over preference pairs, eliminating the explicit reward model (ghosted box).

preference problem changes. IPO modifies the link/objective structure to avoid some DPO overfitting behavior (Azar et al., 2024). KTO replaces pairwise comparisons with a prospect-theory-style objective over desirable and undesirable outputs (Ethayarajh et al., 2024). Nash-LHF and SPPO treat preference optimization as a game over pairwise comparisons, which is closer to social choice when preferences are cyclic or non-transitive (Munos et al., 2023; Wu et al., 2024). MaxMin-RLHF makes the aggregation rule explicitly egalitarian (Chakraborty et al., 2024). ORPO, SimPO, and robust DPO mainly change implementation, reference-model dependence, or noisy-label bias rather than the underlying welfare question (Hong et al., 2024a; Meng et al., 2024; Chowdhury et al., 2024). Self-rewarding loops move some feedback generation from humans to model judges, which lowers data-collection costs but makes the evaluator itself part of the mechanism (Yuan et al., 2024).

13.3 AI Alignment as Economics

RLHF turns alignment into an economics problem: elicit preferences, aggregate them across people, design incentives for truthful feedback, and interpret the learned object. The phrase "alignment tax" appears in the assistant-alignment literature as a warning that optimizing for helpfulness, harmlessness, and honesty can trade off against other capabilities (Askell et al., 2021); Ouyang et al. (2022) provide concrete InstructGPT evidence of benchmark regressions on tasks such as SQuAD, DROP, and HellaSwag. That evidence should not be read as a law of nature. It is an empirical frontier of a particular training recipe, model class, and evaluation set.

13.3.1 Social Choice and Whose Preferences

The central welfare question is whose preferences are being aligned to. If one reward model is trained on pooled labels from a non-representative group of contractors, the resulting policy optimizes an implicit aggregation rule, not society's preferences. Social choice supplies the vocabulary for this problem: alternatives, voters, ranking rules, impossibility theorems, strategic

reports, minority protection, and interpersonal aggregation (Conitzer et al., 2024; Mishra, 2023). Distributional preference learning and axiomatic analyses show that standard BTL aggregation can encode Borda-like rules or fail desirable fairness and consistency conditions (Siththaranjan et al., 2024; Ge et al., 2024). Alternatives such as personalization, MaxMin-RLHF, Nash learning from human feedback, and maximal-lottery alignment make the aggregation rule more explicit, though some of these connections remain preprint-level rather than settled practice (Park et al., 2024; Chakraborty et al., 2024; Munos et al., 2023; Maura-Rivero et al., 2025).

13.3.2 Revealed Preference Tests of LLMs

A related literature treats language models themselves as economic agents and asks whether their choices satisfy revealed-preference restrictions. Budget-allocation experiments, consumer-preference replications, moral-choice tasks, and Bayesian decision problems find that LLMs can look rational in some narrow domains and fragile in others (Chen et al., 2023c; Goli and Singh, 2024; Seror, 2024; Mu et al., 2025). These tests are useful because they translate vague alignment claims into falsifiable restrictions, such as GARP consistency or proximity to a Bayes rule. But the results are prompt-, model-, version-, and domain-sensitive. They should be used as diagnostics of a trained system, not as evidence that an LLM has a stable utility function in the welfare-economics sense.

13.3.3 Mechanism Design for Feedback

Feedback is not passively observed; it is elicited through a mechanism. Labelers may be inattentive, strategic, ideologically motivated, or selected by the platform's contracting process. Mechanism design therefore enters before the reward model is trained. VCG-style fine-tuning formulations ask how to aggregate multiple reward models when participants can benefit from manipulating the training rule (Sun et al., 2024). Strategyproof RLHF studies the same issue for preference reports, with a tradeoff between incentive compatibility and the quality of the final policy (Kleine Buening et al., 2025). Older peer-prediction mechanisms such as Bayesian truth serum and output-agreement methods show how payments can elicit truthful subjective signals when no ground truth is immediately observable (Prelec, 2004; Miller et al., 2005). For RLHF, this means the data-generating process is not a nuisance detail; it determines which preferences are observed and how costly truthful feedback is.

13.3.4 Structural Identification

Finally, preference optimization is not welfare analysis unless the learned reward has a structural interpretation. Bradley-Terry and DPO identify reward differences only through observed pairwise comparisons, up to normalizations and only on the covered comparison support. IRL identification results show that many rewards can rationalize the same behavior or preferences, and that entropy regularization, extra environments, or downstream invariance restrictions are needed to recover sharper objects (Cao et al., 2021; Skalse et al., 2023; Knox et al., 2022). Economically, the genealogy runs from McFadden logit choice (McFadden, 1974b), through Rust-style dynamic discrete choice (Rust, 1994, 1996), to maximum-entropy IRL (Ziebart et al., 2008), and finally to DPO's log-odds reparameterization (Rafailov et al., 2023). Recent econometric work makes this bridge explicit, but still as an emerging working-paper literature rather than a settled textbook synthesis (van der Laan et al., 2025). The practical lesson is conservative: RLHF can produce useful policies, but the reward object is partially identified, aggregation-dependent, and limited by the offline support of the comparison data.

13.4 Simulation Study: Preference Learning in Job Search

A worker searches for jobs in a labor market with compensating differentials, following a McCall (1970)-style search model. Wages are measured in thousands and take values $w \in \{20, 28, 38, 50, 65, 82, 100, 125\}$, while amenities take values $z \in \{0, 1, \ldots, 6\}$ and capture commute quality, flexibility, and job security. The state space has 112 states: 56 searching states in which the worker observes a pending offer (w_i, z_j) and decides to accept or reject, plus 56 employed states (w_i, z_j) in which the worker decides to stay or quit. The offer distribution exhibits compensating differentials, with wage rank and amenity rank negatively correlated ($\rho = -0.74$): high-wage offers cluster with low amenities and vice versa. The worker's true per-period utility is $u(w, z) = \alpha \log(w) + (1 - \alpha)z$ with $\alpha = 0.6$, but this function is unobserved; the worker can only compare career trajectories ("I prefer path A to path B"), exactly as in stated-preference surveys in labor economics. While searching, the worker receives the unemployment benefit $u_b = \alpha \log(b)$ where $b = 28$. Layoffs occur with probability $p = 0.05$ per period, and the discount factor is $\gamma = 0.95$. Dynamic programming gives $V^*(s_0) = 74.13$; the optimal policy accepts 25 of 56 offer types and stays employed at 25 of 56 job types. Preference data is generated by rolling out a uniform random policy from a random searching state. Each rollout produces a career segment of $L = 15$ periods recording states and actions. For each of K comparisons, two independent career segments are generated; the segment with higher cumulative discounted utility under the true (unobserved) utility function is labeled as preferred via the Bradley-Terry model. Figure 28 shows the optimal accept/reject and stay/quit boundaries.

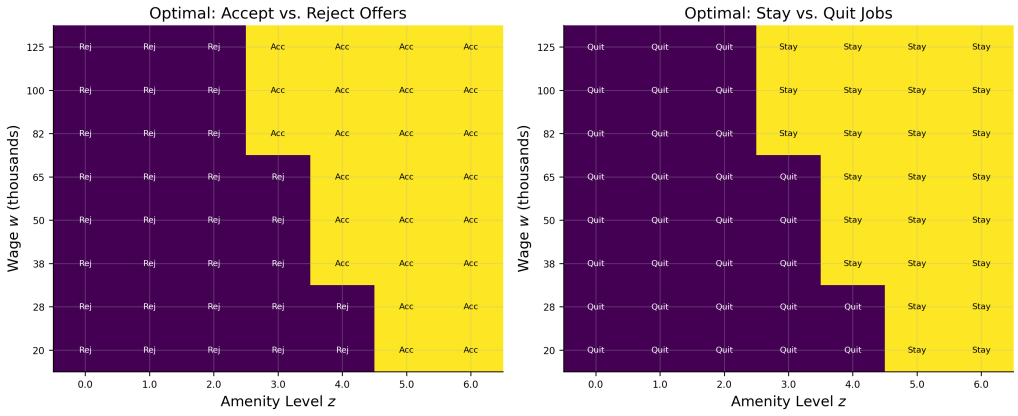

Figure 28: **Optimal accept/reject and stay/quit boundaries in wage-amenity space.** Left: the optimal accept/reject boundary for searching states. Right: the optimal stay/quit boundary for employed states. Both boundaries cut diagonally, reflecting the tradeoff between wage and amenity quality under compensating differentials.

Six methods are compared across $K \in \{25, 50, 100, 200, 500, 1{,}000, 2{,}000, 5{,}000\}$, averaged over 30 seeds. The neural network RLHF reward model is a two-layer MLP trained by Bradley-Terry MLE on the K segment pairs; per-transition rewards are discount-weighted and summed to obtain a segment score, and the resulting 112-state reward table is solved by value iteration.[240] The correctly specified structural model parameterizes utility as $\hat{u} = \hat{\alpha} \log(w) + (1 - \hat{\alpha})z$, estimating the single parameter $\hat{\alpha}$ via Bradley-Terry MLE, then solves the MDP. The misspecified model uses $\hat{u} = \hat{\alpha} \log(w) + (1 - \hat{\alpha})\bar{z}$, where $\bar{z} = 3$ is the mean amenity level; this model ignores amenity variation across jobs, treating all amenities as identical. DPO trains a tabular softmax policy directly from trajectory comparisons, bypassing reward modeling entirely.[241] Tabular

[240] The neural network has 4 inputs (normalized log-wage, normalized amenity, employment indicator, action), 32 hidden units per layer, and ∼1,200 parameters. The logistic loss from Equation (124) is applied to discount-weighted segment reward sums.

[241] DPO uses 112 logit parameters ϕ_s, one per state, trained via the DPO loss (Equation 128) with Adam

Q-learning (10,000 episodes, $\varepsilon = 0.15$, learning rate 0.1) and exact DP provide scalar-reward baselines. All four preference methods receive identical comparison data per seed; DPO receives a same-state variant generated from the same seed.

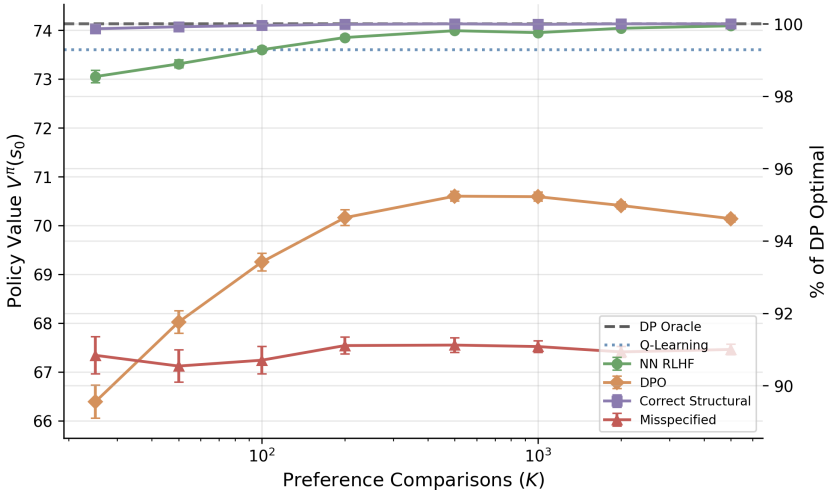

Figure 29: **Policy value $V^{\pi}(s_0)$ versus number of preference comparisons K for all six methods (30 seeds, $L = 15$).** The right axis shows the percentage of DP-optimal value.

Figure 29 reports the main results. Three findings stand out. First, the correctly specified structural model dominates: even at $K = 25$ it achieves 99.9% of DP-optimal, illustrating the power of correct specification when the model has a single free parameter. Second, the neural network converges more slowly but reaches 99.9% by $K = 5{,}000$, reflecting the higher sample complexity of a flexible model relative to a one-parameter structural specification. Third, DPO plateaus at approximately 95% by $K = 500$ and does not improve further.[242] This plateau is qualitatively different from the gridworld failure (−118%): DPO recovers a reasonable policy, but the gap does not close with additional data.[243] The misspecified constant-amenity model plateaus at 91% regardless of K, because additional preference data cannot correct the omitted variable.

Table 25: **Diagnostics at $K = 5{,}000$ (single seed): policy agreement with π^*, value-function correlation, and mean accepted wage and amenity for each method.**

Method	Policy agree. (%)	V^{π} corr.	Mean amenity	Mean wage	$\hat{\alpha}$
NN RLHF	96.4	1.000	4.67	73	—
DPO	57.1	0.777	3.38	63	—
Correct	100.0	1.000	4.84	71	0.597
Misspecified	50.0	0.889	3.00	70	—
Optimal	100.0	1.000	4.84	71	—

Table 25 provides state-level diagnostics at $K = 5{,}000$. The structural model and neural

optimization over a sweep of $\lambda_{KL} \in \{0.01, 0.05, 0.1, 0.5, 1.0, 5.0\}$, selecting the λ_{KL} that minimizes training loss. The reference policy is uniform: $\pi^{SFT}(a|s) = 0.5$. To match the LLM setup where both completions condition on the same prompt, DPO comparison pairs start from the same initial state.

[242] DPO learns only from (s, a) pairs visited in training trajectories generated by the random behavioral policy. It cannot propagate value to undervisited states the way value iteration does after learning a reward model, so states poorly covered by the behavioral policy remain suboptimal regardless of K.

[243] DPO fails catastrophically in gridworld because transitions are stochastic (10% slip probability) and rewards are transition-dependent. The same (s, a) pair yields different rewards depending on whether the agent slipped, so the DPO loss conflates policy quality with transition luck. In the job search model, accept/reject deterministically changes employment status, and only the 5% layoff probability introduces stochastic transitions.

network achieve near-perfect policy agreement with π^* (100% and 96% respectively). DPO agrees on only 57% of states with value-function correlation 0.78, systematically underselecting on both wage and amenity dimensions.[244] The misspecified model agrees on 50%, with disagreements concentrated where amenity variation matters.[245]

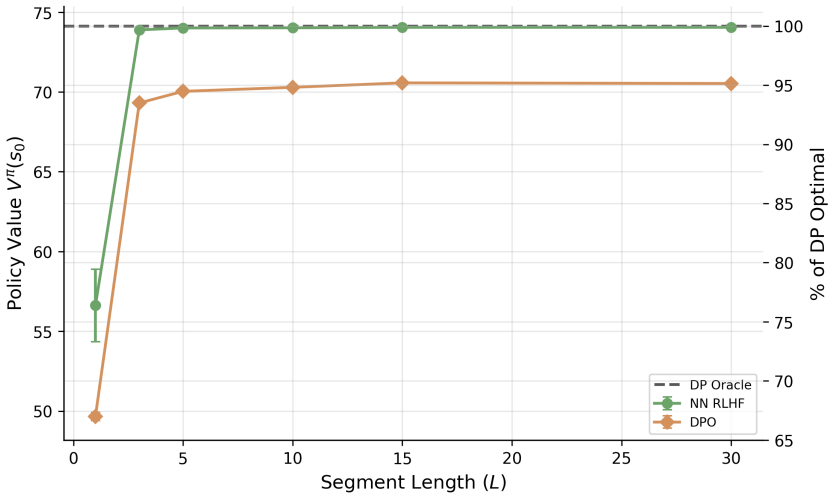

Figure 30: **Policy value $V^{\pi}(s_0)$ versus segment length L at $K = 2{,}000$ for the neural network and DPO (20 seeds).** The right axis shows the percentage of DP-optimal value.

Figure 30 reports the segment length ablation at $K = 2{,}000$. At $L = 1$, both methods perform poorly because single-step comparisons carry minimal information about long-run value. The neural network recovers rapidly (by $L = 3$) because the reward model aggregates per-transition estimates over longer segments and value iteration propagates them through the full transition structure; DPO improves monotonically but plateaus at its ∼95% ceiling, as it must learn the policy directly from trajectory comparisons without access to the transition model.

Two-stage RLHF separates preference estimation (a static econometric problem) from dynamic programming (which exploits the known transition model); DPO conflates the two and forfeits the transition structure that structural modelers typically have access to.[246] The DPO plateau in the job-search study is the computational version of the identification point above. More comparisons tighten the empirical binary choice model on the support of observed trajectories, but they do not reveal counterfactual value in poorly covered states, nor do they identify a welfare object independent of the aggregation rule used to construct the labels.

14 Causal Inference for Reinforcement Learning

Reinforcement learning algorithms solve Markov decision processes by estimating value functions or optimizing policies from sampled transitions. Two assumptions underlie the standard formulation. First, the agent's action at time t is determined solely by the observed state s_t and the agent's policy $\pi(a \mid s_t)$; there is no unobserved variable (confounder) simultaneously

[244] DPO's mean accepted amenity of 3.4 parallels the misspecified structural model's 3.0, though the mechanisms differ: the misspecified model ignores amenity variation by construction, while DPO underweights amenities because the random behavioral policy underrepresents high-amenity employed states in the training data.

[245] An online versus offline ablation for the neural network at $K = 1{,}000$ (20 seeds) shows comparable performance: online 73.92 ± 0.05, offline 73.99 ± 0.03 ($p = 0.09$). The random behavioral policy already provides diverse career trajectories covering the full wage-amenity space.

[246] This advantage is specific to settings where the transition model is known or estimable; in domains without a tractable transition model, DPO's single-stage approach avoids compounding errors from reward model estimation.

influencing both the action and the reward or state transition.[247] Second, the observed state s_t is sufficient for prediction, so that conditioning on s_t renders future states independent of past history. Both assumptions are the Markov property restated in causal language. Both fail routinely in applied settings.

This chapter studies what happens when logged sequential decisions are observational rather than experimental. The central object is the value of a target policy under the interventional law induced by that policy, not merely under the observational law generated by past behavior. The literature now covers confounded MDPs, partial identification and sensitivity analysis, proximal and negative-control methods for POMDPs, instrumental variables, mediator-based identification, and dynamic double machine learning for off-policy evaluation (OPE). The survey articles by Deng et al. (2023) and da Costa Cunha et al. (2025) provide broader coverage of causal representation learning, counterfactual policy optimization, transportability, and fairness.

Table 26 summarizes the main identification strategies. The purpose is not to create a checklist of papers. It is to show economists that many causal RL questions are familiar econometric problems once the estimand is written as a policy value rather than as a static treatment effect.

Table 26: Econometric taxonomy for causal reinforcement learning

RL problem	Econometric analogue	Identifying object
Observed state blocks all confounding	Backdoor adjustment, sequential unconfoundedness, g-formula	Interventional transition and reward from conditional models given observed controls
Exogenous variation shifts actions	Instrumental variables and conditional moment restrictions	Structural transition or reward function satisfying moments conditional on instruments
Latent state is not observed but proxies are available	Proximal causal inference and negative controls	Bridge functions that recover policy value from proxy distributions
Actions operate through observed intermediate variables	Front-door, mediation, and g-methods	Mediated transition law and decomposed direct or indirect dynamic effects
No credible point-identifying source exists	Rosenbaum and Tan sensitivity, partial identification	Bounds over policy values under restricted hidden-bias models
High-dimensional nuisance functions are needed	DML, orthogonal scores, efficient OPE	Neyman-orthogonal moments for value or dynamic treatment parameters

14.1 From Partial Observability to Causal Structure

A partially observable MDP (POMDP) augments the standard MDP with an observation function. The POMDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{O}, P, O, r, \gamma)$, where $\mathcal{O}$ is a finite observation space and O is the observation function.

$$O(o \mid s', a) = P(O_t = o \mid S_t = s', A_{t-1} = a). \tag{129}$$

The agent does not observe s_t directly but instead receives $o_t \sim O(\cdot \mid s_t, a_{t-1})$ and maintains a belief state $b_t \in \Delta(\mathcal{S})$

$$b_t(s) = P(S_t = s \mid o_1, a_1, \ldots, o_t), \tag{130}$$

[247] Throughout this chapter, I reserve "outcome" for its causal inference meaning and use the specific RL quantity (reward, state, return, value) elsewhere.

which is updated via Bayesian filtering at each step. The belief MDP, whose state space is $\Delta(\mathcal{S})$, is itself a fully observable continuous-state MDP, so standard value iteration applies in principle, though computation is intractable in general.

da Costa Cunha et al. (2025) organize sequential decision problems with hidden variables into a hierarchy: standard MDP (full observability, no confounding), POMDP (partial observability, no confounding), confounded MDP (full observability, confounding), and causal POMDP (both). The key distinction is epistemic versus identificational. In a POMDP, the hidden state is a modeling challenge: the agent acknowledges incomplete information and plans accordingly via the belief state, analogous to Kalman or Hamilton filtering in econometrics. In a confounded MDP, the hidden variable is an identification challenge: standard estimators silently produce biased results, analogous to endogeneity and omitted variable bias.

This distinction matters because the same observed sequence can support different econometric interpretations. A hidden patient severity state may be a state variable the physician partially observes, a confounder that drives treatment and prognosis, a source of invalid support for a target policy, or a latent variable for which laboratory histories provide negative controls. Causal RL is the study of which interpretation is credible enough to identify, bound, or estimate the value of a counterfactual policy.

14.2 The Confounded MDP

When unobserved confounders influence both the behavioral policy and the transitions or rewards, the MDP is confounded. This formalization, developed by Zhang and Bareinboim (2019), Zhang and Bareinboim (2020), and Kallus and Zhou (2020), provides the foundation for causal reasoning in sequential decision problems. The key tool is Pearl's do-operator. The interventional distribution $P(Y \mid \mathrm{do}(X = x))$ is the distribution that arises when X is set externally rather than observed passively, severing all incoming causal influences on X while leaving the remaining data-generating process intact.[248]

Definition D6 (Confounded MDP (Zhang and Bareinboim, 2020))**.** *A confounded MDP is a tuple* $(\mathcal{S}, \mathcal{A}, \mathcal{U}, P, r, \gamma)$ *where* $\mathcal{S}$ *is a finite state space,* $\mathcal{A}$ *is a finite action space,* $\mathcal{U}$ *is a space of unobserved confounders,* $\gamma \in [0, 1)$ *is a discount factor, and the dynamics are governed by structural equations*

$$U_t \sim P_U(\cdot \mid S_t), \tag{131}$$
$$A_t \sim \mu(\cdot \mid S_t, U_t), \tag{132}$$
$$R_t = f_R(S_t, A_t, U_t) + \epsilon_t, \tag{133}$$
$$S_{t+1} \sim f_S(\cdot \mid S_t, A_t, U_t). \tag{134}$$

The behavioral (logging) policy μ *depends on the unobserved confounder* U_t *through Equation* (132)*. An evaluation policy* $\pi(a \mid s)$ *depends only on the observed state.*

Unlike the online algorithms discussed earlier in the survey, where behavior and target policies were identical or related by a known exploration mechanism, here μ is an unknown function of unobserved variables.

Because μ depends on U_t, conditioning on $\{A_t = a\}$ carries information about the confounder, so the observational and interventional transitions diverge:

$$P(s' \mid s, a) \neq P(s' \mid s, \mathrm{do}(a)). \tag{135}$$

[248]The do-operator is formalized within the structural causal model (SCM) framework of Pearl (2009). An SCM specifies endogenous variables $\mathbf{V}$, exogenous variables $\mathbf{U}$, structural equations $V_i = f_i(\mathrm{pa}(V_i), U_i)$, and a distribution $P(\mathbf{U})$. The intervention $\mathrm{do}(X = x)$ replaces the structural equation for X with a constant, producing the interventional distribution. See Pearl (2009) for the complete framework, including the causal hierarchy (association, intervention, counterfactual) and general identification theory.

The Bellman equation for policy evaluation must use interventional, not observational, transition probabilities. Define the causal Bellman operator for a *target policy* π:

Definition D7 (Causal Bellman Operator (Zhang and Bareinboim, 2020))**.** *The causal Bellman operator $\mathcal{T}_c^\pi$ for policy π in a confounded MDP is*

$$(\mathcal{T}_c^\pi V)(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} P(s' \mid s, \mathrm{do}(a)) \big[r(s,a) + \gamma V(s') \big], \tag{136}$$

where $r(s,a) = \mathbb{E}[R_t \mid S_t = s, \mathrm{do}(A_t = a)]$ is the interventional expected reward.

Lemma L1 (Bias of Naive Off-Policy Evaluation (Kallus and Zhou, 2020))**.** *Let $\hat{V}^\pi_{naive}$ denote the value function obtained by solving the Bellman equation with observational transitions $P(s' \mid s,a)$, and let V^π denote the true value function under interventional transitions $P(s' \mid s, \mathrm{do}(a))$. In a confounded MDP where $P(s' \mid s,a) \neq P(s' \mid s, \mathrm{do}(a))$ for some (s,a,s'), the naive estimator is biased.*

$$\hat{V}^\pi_{naive}(s) \neq V^\pi(s). \tag{137}$$

The importance-sampling estimator is also biased because the propensity $\mu(a \mid s)$ is not the true behavioral propensity $\mu(a \mid s,u)$:

$$\hat{V}^\pi_{IS} = \frac{1}{N} \sum_{i=1}^{N} \prod_{t=0}^{T} \frac{\pi(a_t^{(i)} \mid s_t^{(i)})}{\mu(a_t^{(i)} \mid s_t^{(i)})} G^{(i)}, \tag{138}$$

$$G^{(i)} = \sum_{t=0}^{T} \gamma^t R_t^{(i)}. \tag{139}$$

Here $G^{(i)}$ is the discounted return of trajectory i and T is the trajectory length.[249]

The backdoor criterion (Pearl, 2009) provides a path to identification. Zhang and Bareinboim (2020) apply it to the confounded MDP setting.

Theorem 8 (Backdoor Identification in Confounded MDPs (Pearl, 2009; Zhang and Bareinboim, 2020))**.** *Suppose a set of observed variables $\mathbf{Z}_t$ satisfies the backdoor criterion relative to (A_t, S_{t+1}) in the causal graph of the confounded MDP, meaning that $\mathbf{Z}_t$ blocks all backdoor paths from A_t to S_{t+1} and no element of $\mathbf{Z}_t$ is a descendant of A_t.*[250] *Then the interventional transition probability is identified:*

$$P(s' \mid s, \mathrm{do}(a)) = \sum_{\mathbf{z}} P(s' \mid s, a, \mathbf{z})\, P(\mathbf{z} \mid s). \tag{140}$$

Substituting Equation (140) into the causal Bellman operator (Equation 136) yields an identified, consistent estimator of V^π.

14.3 Backdoor-Adjusted Off-Policy Evaluation

Off-policy evaluation under confounding is an average treatment effect estimation problem in a dynamic setting (Bannon et al., 2020): μ is the treatment assignment mechanism, π the counterfactual regime, importance sampling corresponds to inverse probability weighting, and doubly robust OPE corresponds to the AIPW estimator of Robins et al. (1994).

[249] This is the sequential analogue of the omitted variable bias in linear regression. In the static case, regressing Y on X without controlling for a confounder U yields a biased coefficient. In the sequential case, the bias propagates through the Bellman recursion and can amplify over the horizon.

[250] A backdoor path from A_t to S_{t+1} is any path in the causal graph that begins with an arrow into A_t, that is, a non-causal path. In the confounded MDP, $A_t \leftarrow U_t \rightarrow S_{t+1}$ is a backdoor path: U_t causes both A_t and S_{t+1}, creating a spurious association. Blocking all such paths by conditioning on appropriate variables eliminates the confounding bias. See Pearl (2009), Chapter 3.

Theorem 8 yields a concrete estimation procedure. Given logged data D_N collected under behavioral policy μ, with elements $(s_t, a_t, z_t, r_t, s_{t+1})$ and observed proxy z_t, the econometrician estimates $\hat{P}(s' \mid s, a, z)$ and $\hat{P}(z \mid s)$ from the logged data, computes

$$\hat{P}(s' \mid s, \mathrm{do}(a)) = \sum_z \hat{P}(s' \mid s, a, z)\, \hat{P}(z \mid s), \tag{141}$$

and solves the causal Bellman equation using the estimated interventional transitions to obtain $\hat{V}^\pi$. The doubly robust variant combines the fitted action-value function $\hat{Q}(s, a)$ with backdoor-adjusted propensities, achieving consistency if either $\hat{Q}$ or the propensity model is correctly specified.

This is the cleanest case because the analyst observes enough variables to repair the state. The rest of the chapter asks what can still be learned when that repair is unavailable, only partially credible, or statistically high-dimensional.

14.4 Alternative Identification Strategies

When no backdoor variable is available, causal RL moves from adjustment to sensitivity analysis, proxies, instruments, mediators, or orthogonal scores. Figure 31 displays the causal graph for three point-identification strategies used in the simulation study.

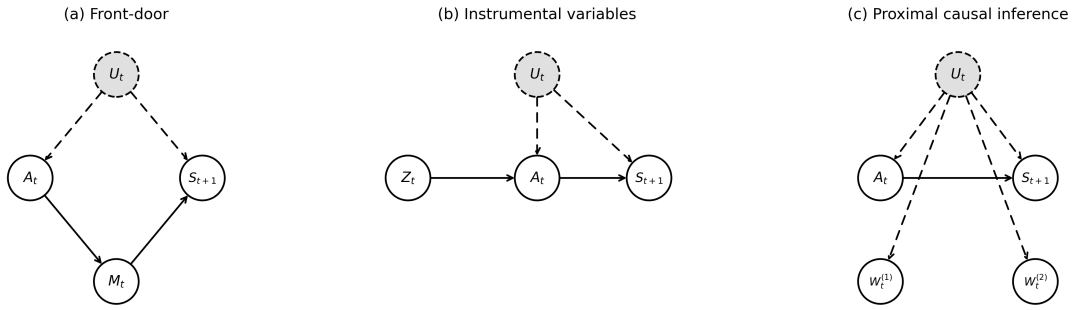

Figure 31: Causal graphs for three identification strategies in confounded MDPs. Gray dashed nodes are unobserved; dashed edges involve unobserved variables. (a) Front-door criterion with mediator M_t. (b) Instrumental variables with exogenous instrument Z_t. (c) Proximal causal inference with proxies $W_t^{(1)}, W_t^{(2)}$.

14.4.1 Sensitivity Analysis and Partial Identification

When point identification is not credible, the estimand becomes a set rather than a number. In static econometrics, this is the logic of Rosenbaum bounds, marginal sensitivity models, and partial identification (Rosenbaum, 2002; Tan, 2006; Manski, 2003). The sequential version asks how much a hidden variable could change the target policy value, allowing the hidden bias to enter each decision epoch. This subsection covers the three results that make this practical for off-policy evaluation in confounded MDPs.

Namkoong et al. (2020) provide the foundational result for finite-horizon sequential OPE under a Rosenbaum-style sensitivity model. Their identification assumption bounds the odds-ratio influence of any unobserved confounder by a parameter $\Gamma \geq 1$ at each decision epoch:

$$\Gamma^{-1} \;\leq\; \frac{\pi_t(a_t \mid H_t, u)/\pi_t(a'_t \mid H_t, u)}{\pi_t(a_t \mid H_t, u')/\pi_t(a'_t \mid H_t, u')} \;\leq\; \Gamma. \tag{142}$$

Under this Γ-bound, their Theorem 2 derives a lower bound on the target policy value $\mathbb{E}[Y(\bar{A}_{1:T})]$ as the solution to a tractable loss-minimization problem with weighted quadratic loss $\ell_\Gamma(z) = \frac{1}{2}(\Gamma z_-^2 + z_+^2)$, and the empirical plug-in estimator is shown consistent in Theorem 3. The paper's central practical warning is sharp. Allowing the Γ-bound to bite at every decision epoch ("per-decision confounding") yields bounds that grow exponentially in the horizon T, becoming uninformative for any realistic Γ when T is more than a handful of periods. The tractable regime is "one-decision confounding," where only a single nominated decision is potentially confounded. Their applied demonstrations on sepsis ICU management and a SMART trial for autistic children show the one-decision setting certifies policy contrasts up to design sensitivity Γ above five, while per-decision confounding loses informativeness already at Γ near 1.5.

Kallus and Zhou (2020) lift this analysis to the infinite-horizon setting and exchange per-step importance sampling for stationary-density ratios. Their Lemma 2 characterizes the partially identified set of state-occupancy ratios as solutions to an estimating equation involving the propensity ratio and an unobserved memoryless confounder, and their Theorem 1 establishes the sharpness of the resulting policy-value bound under a Tan-style marginal sensitivity model. The key structural assumption is that the unobserved confounder is conditionally independent of the prior history given the current state; with that, the bound problem reduces to a non-convex optimization that can be solved by alternating projections.

Bruns-Smith (2021) provide the practitioner-facing estimator that ties this sensitivity machinery back to the chapter's causal Bellman operator (Definition D7). Their Proposition 4 establishes a robust Bellman equation under the same memoryless-confounder assumption: with $\tau = \Lambda/(1+\Lambda)$ where Λ is the Tan-style odds-ratio bound,

$$\bar{Q}_t^\pi(s,a) \;=\; \mathbb{E}\Big[r_t(s,a,s') + \gamma \bar{V}_{t+1}^\pi(s') \,\big|\, s,a\Big] \;-\; \alpha_t(s,a)\cdot \mathrm{CVaR}_{1-\tau}\Big(Y_t(\bar{Q}_{t+1}) \,\big|\, s,a\Big)\,. \tag{143}$$

The robust Q-function is the conditional expected shortfall of the Bellman target, and their orthogonalized loss yields a Neyman-orthogonal pseudo-outcome that relaxes the required nuisance rate from $o_p(n^{-1/2})$ to $o_p(n^{-1/4})$. Their MIMIC-III sepsis application picks a calibrated Λ between 1.4 and 2.5 and shows the robust policy shifts from fluid-heavy toward vasopressors, in line with prior clinical meta-analyses. The bridge to the existing chapter machinery is exact: Equation (183) is a Bellman operator with a CVaR penalty term that subtracts off the worst-case effect of an unobserved confounder under the Tan-style sensitivity bound.

For applied economics the practical hierarchy is the following. One claim is statistical: the logged data support a precise estimate under a maintained model. The other is identificational: the maintained model is strong enough to recover the interventional value. Sensitivity methods relax the second claim and report how quickly a policy recommendation changes as hidden bias grows. Namkoong et al. (2020) establishes that this can be done at all in the sequential setting and characterizes when it fails (per-decision confounding); Kallus and Zhou (2020) extends to infinite horizons; Bruns-Smith (2021) delivers a practitioner-ready estimator that plugs into standard fitted-Q pipelines. Recent conformal sensitivity work (Yin et al., 2024) shows the same instinct in static individualized treatment effects, although the RL setting is harder because per-period uncertainty is recursively priced through continuation values.

14.4.2 Proximal and POMDP Bridge Methods

When confounders are latent but proxy variables, noisy correlates of the confounder, are available, proximal causal inference provides identification. Bennett and Kallus (2023) adapt this logic to sequential settings by treating the problem as OPE in a POMDP. The analyst observes two proxies: a treatment-side proxy $W_t^{(1)}$ and an outcome-side proxy $W_t^{(2)}$, both conditionally independent given the latent confounder U_t. Identification proceeds through a bridge function h

that solves a conditional moment equation linking the two proxies to the interventional quantity:

$$\mathbb{E}[V(S_{t+1}) \mid W_t^{(1)}, S_t, A_t] = \mathbb{E}[h(W_t^{(2)}, S_t, A_t) \mid W_t^{(1)}, S_t, A_t]. \tag{144}$$

The bridge function h is estimated by solving this integral equation, which reduces to a linear system in the discrete case, and the causal effect is recovered by marginalizing over the outcome proxy distribution.

$$P(s' \mid s, \mathrm{do}(a)) = \sum_{w_2} h(w_2, s, a)\, P(W^{(2)}{=}w_2 \mid s). \tag{145}$$

Two bridge functions, analogous to inverse propensity scores and Q-functions, yield a doubly robust estimator of the policy value that is $\sqrt{n}$-consistent without ever observing U_t. Bennett and Kallus (2023) demonstrate this idea in a sepsis management simulator in which the physician observes a true diabetes status that is only partially recorded in the analyst's data. Prior clinical observations serve as treatment-side proxies, current clinical observations serve as outcome-side proxies, and the proximal estimator ranks evaluation policies correctly in between 82% and 100% of test cases where naive estimators fail.

The POMDP bridge literature generalizes this insight beyond tabular examples. Shi et al. (2022) formulate OPE in confounded POMDPs as a minimax learning problem over bridge functions. Uehara et al. (2023) introduce future-dependent value functions that use future proxies as inputs and history proxies as instruments. Hong et al. (2024b) move from evaluation to learning by identifying history-dependent policy gradients in confounded POMDPs. Wang et al. (2025a) show a related proximal route in which past human or AI actions themselves can carry information about latent decision-relevant states.

14.4.3 Instrumental Variables and Orthogonal Scores

When neither backdoor nor front-door variables are available, instrumental variables can identify causal effects. An instrument Z_t must satisfy relevance, meaning it shifts the action A_t, and an exclusion restriction, meaning its effect on S_{t+1} is channeled entirely through A_t. Liao et al. (2024) formalize this as a Confounded MDP with Instrumental Variables (CMDP-IV), where transitions take the form $S_{t+1} = F^*(S_t, A_t) + \epsilon_t$ with unobserved confounders ϵ_t affecting both the behavior policy and transitions. The transition function F^* is recovered from a conditional moment restriction:

$$\mathbb{E}[S_{t+1} - F^*(S_t, A_t) \mid Z_t, S_t] = 0. \tag{146}$$

For a binary action and binary instrument, the Wald estimator provides a closed-form solution. Let β denote the causal effect of promoting (action $a = 0$) on the transition probability.

$$\beta = \frac{P(s' \mid s, Z{=}1) - P(s' \mid s, Z{=}0)}{P(A{=}0 \mid s, Z{=}1) - P(A{=}0 \mid s, Z{=}0)}, \tag{147}$$

where the numerator is the reduced-form effect of the instrument on the transition and the denominator is the first-stage effect on treatment uptake. The interventional transition is then recovered from any instrument value z via $P(s' \mid s, \mathrm{do}(a{=}0)) = P(s' \mid s, Z{=}z) + \beta \cdot (1 - P(A{=}0 \mid s, Z{=}z))$. Their IV-aided Value Iteration algorithm applies this moment restriction at each state to estimate F^*, then runs standard value iteration on the estimated model.

Liao et al. (2024) illustrate with neonatal intensive care unit (NICU) assignment. A hospital must decide whether to admit each newborn to the NICU, and this decision is confounded by unobserved severity indicators that affect both admission and infant health. Differential travel time to a specialty care provider serves as an instrument. Relevance holds because travel time shifts referral probabilities. The exclusion restriction requires that travel time affect infant health only through admission, not through any other channel.

Orthogonal-score methods solve a different problem. They do not by themselves remove hid-

den confounding, but they make causal OPE statistically stable when the identifying assumptions reduce the value to nuisance functions that must be learned flexibly. Liao et al. (2021) estimate long-term average policy values for mobile health applications. Lewis and Syrgkanis (2021) extend double machine learning to dynamic treatment effects through sequential residualization and Neyman-orthogonal moments. Kallus and Uehara (2022) derive efficient double reinforcement learning estimators for Markovian OPE using q-functions and stationary density ratios. The common econometric message is that identification and estimation are separate: first specify why the policy value is identified, then use orthogonal scores to keep first-stage machine learning error from dominating the target estimate.

14.4.4 Mediator-Based Identification

The front-door criterion applies when A_t affects S_{t+1} only through an observed mediator M_t. Three conditions are required: M_t intercepts all directed paths from A_t to S_{t+1}, no unblocked backdoor path exists from A_t to M_t, and A_t blocks all backdoor paths from M_t to S_{t+1}. When satisfied (Pearl, 2009),

$$P(s' \mid s, \mathrm{do}(a)) = \sum_{m} P(m \mid a) \sum_{a'} P(s' \mid s, m, a')\, P(a' \mid s). \tag{148}$$

The first factor is unconfounded; the inner sum adjusts for confounding on the $M_t \to S_{t+1}$ link by averaging over the observational action distribution. This is the sequential analogue of mediation analysis.

Mediator-based RL is most attractive when the action changes an observable channel before it changes the long-run state. In mobile health, an app prompt may affect cortisol through supplement adherence. In retail pricing, a promotion may affect conversion through a marketing follow-up or browsing response. Shi et al. (2024b) use mediator variables to identify and construct confidence intervals for policy values in confounded MDPs. Ge et al. (2023) develop an RL framework for dynamic mediation analysis, decomposing the long-run effect into direct and mediated components. The policy question shifts from "does the action work" to "through which sequential channel does the action work, and is that channel stable enough to transport across policies?"

14.5 Structural Counterfactual Off-Policy Evaluation

The point-identification strategies of the preceding subsections recover the target-policy value $V(\widetilde{\pi})$ from observational data under explicit graphical assumptions. A complementary route, originating in Buesing et al. (2019), operates one level deeper. Cast the partially observable Markov decision process as a structural causal model, infer a posterior over its exogenous noise from the observed trajectory, and roll out under the counterfactual policy with that posterior held fixed. The output is not just a value estimate but a counterfactual trajectory the agent would have produced, sample by sample.

The transition mechanism is $s_{t+1} = f^s(s_t, a_t, u_t^s)$ with exogenous noise u_t^s, the observation mechanism is $o_t = f^o(s_t, u_t^o)$, and the action mechanism is $a_t = f^\pi(h_t, u_t^a)$ given the history h_t. Given an observed trajectory $\hat{\tau}$, the counterfactual rule proceeds in three steps.

Lemma 1 of Buesing et al. (2019) establishes that the marginal of the counterfactual distribution over $\hat{\tau}$ equals the interventional distribution, so the counterfactual estimator is unbiased under correct SCM specification. On the partially-observable grid-world experiment of the paper, the counterfactual estimator's evaluation error decreases monotonically with the amount of off-policy data, while the model-based estimator's error remains stuck at the bias of the dynamics model.

The categorical-action case raises an identification problem. Oberst and Sontag (2019) show that multiple structural causal models can be consistent with the same interventional distribu-

Algorithm 4 Counterfactual off-policy evaluation (Buesing et al., 2019)

1: *Abduction.* Infer the posterior $P(U \mid \hat{\tau})$ over the exogenous noise from the observed trajectory.
2: *Action.* Replace the behaviour-policy mechanism f^{π} with the target-policy mechanism $f^{\widetilde{\pi}}$ to form the modified SCM $\mathcal{M}_{\hat{\tau}}^{\mathrm{do}(\widetilde{\pi})}$ with the same posterior on U.
3: *Prediction.* Sample $U \sim P(U \mid \hat{\tau})$ and roll out the modified SCM to obtain counterfactual trajectories τ^{cf} and their returns.

tion yet imply different counterfactual outcomes, and they resolve the ambiguity by assuming a Gumbel-max sampling mechanism that satisfies a counterfactual-stability condition.[251] The shared identifying assumption across this strand is that the structural causal model is correctly specified. The Lucas critique applies in its sharpest form when the agent's policy itself enters the data-generating process, and identification by instruments or exogeneity restrictions in the spirit of Chernozhukov et al. (2018) is the econometric remedy that composes naturally with the backdoor, front-door, proximal, and mediator-based identification strategies of the preceding subsections.

14.6 The Broader Causal RL Landscape

Two further directions sit outside the chapter's central scope but deserve naming. Pace et al. (2024) respond to the case where no observed control, instrument, mediator, or proxy is strong enough to point-identify the policy value with Delphic offline RL, which treats hidden confounding as nonidentifiable uncertainty rather than pretending it has been solved. Mesnard et al. (2021) extends the structural-counterfactual machinery of Section 14.5 from policy evaluation to per-trajectory counterfactual policy gradients, using constrained future-information conditioning to keep the gradient estimator unbiased. Transportability theory (Pearl and Bareinboim, 2014; Bareinboim and Pearl, 2016; Li et al., 2024c) asks which mechanisms remain invariant across source and target environments, governing sim-to-real transfer and curriculum learning under environment shift.

14.7 Simulation Study: Confounded Retail Pricing MDP

I construct a 5-state engagement funnel $\mathcal{S} = \{0, 1, 2, 3, 4\}$ with two actions (promote, hold price) and an absorbing conversion state at $s = 4$. A retailer manages customers through engagement stages, deciding whether to offer a promotional discount. The data-generating process embeds four distinct sources of causal variation, enabling simultaneous validation of four point-identification strategies from a single DGP.

Figure 32 displays the complete causal graph. Market conditions $Z_t \sim \text{Bernoulli}(0.5)$ are observed and affect both the latent confounder and transitions. Consumer sentiment U_t is an unobserved confounder strongly correlated with Z_t.[252] An independent cost shock $\text{IV}_t \sim \text{Bernoulli}(0.5)$ serves as an instrument. The behavioral pricing policy depends on both U_t and IV_t: $\mu(\text{promote} \mid s, U_t, \text{IV}_t) = 0.55 + \rho \cdot 0.25 \cdot (2U_t - 1) + 0.15 \cdot (\text{IV}_t - 0.5)$, where $\rho \in \{0, 0.2, \ldots, 1.0\}$ controls confounding strength. Promotions trigger marketing follow-ups with M_t serving as a mediator.[253] Two noisy proxies of U_t are available: a CRM score $W_t^{(1)}$ and browsing behavior

[251] The Pearl monotonicity condition resolves the binary case. The Gumbel-max SCM is the natural extension to k-way categorical outcomes. Tang and Wiens (2023) combine SCM-based counterfactual outcomes with importance sampling to obtain variance reductions while preserving unbiasedness under correct human annotation, with a sepsis-simulator demonstration that delivers an RMSE of 0.013 against 0.113 for vanilla per-decision IS.
[252] $P(U_t{=}1 \mid Z_t{=}1) = 0.9$ and $P(U_t{=}1 \mid Z_t{=}0) = 0.1$.
[253] $M_t \sim \text{Bernoulli}(0.8)$ when the retailer promotes and $\text{Bernoulli}(0.2)$ otherwise.

$W_t^{(2)}$.[254]

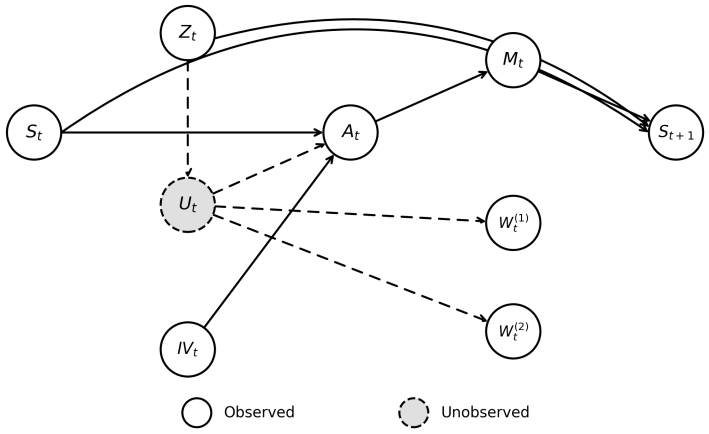

Figure 32: Complete causal graph of the simulation DGP. Gray dashed node (U_t) is unobserved; dashed edges involve U_t.

The action affects the next state only through the mediator: $P(s{+}1 \mid s, M_t, Z_t)$ depends on M_t and Z_t but not on A_t or U_t directly. This enables all four identification strategies simultaneously: Z_t satisfies the backdoor criterion, M_t satisfies the front-door criterion, IV_t satisfies relevance and exclusion, and $W_t^{(1)}, W_t^{(2)}$ satisfy the proximal conditions.[255]

I compare six estimators: oracle, naive (biased per Lemma L1), backdoor (Equation 141), front-door (Equation 148), Wald IV (Equation 147), and proximal (Equations 144 and 145).[256] These estimators correspond to the point-identification rows of Table 26. The simulation intentionally leaves out sensitivity bounds and orthogonal-score refinements so that the figure isolates the identification logic from higher-order statistical efficiency.

Figure 33 reports bias and RMSE across confounding strengths (20 seeds, 2,000 trajectories per seed). The naive estimator's bias grows monotonically with ρ because observational transitions overestimate promotion success: the promote action is more likely when $U_t = 1$, which correlates with favorable conditions, so $\hat{P}_{\text{obs}}$ exceeds the true interventional probability, and this per-step bias compounds through the Bellman recursion. The backdoor and front-door estimators eliminate bias at all ρ, validating Theorem 8 and Equation (148). The IV estimator maintains low bias but higher variance due to the Wald ratio's sensitivity to instrument strength; panel (c) illustrates this classic bias-variance tradeoff. The proximal estimator achieves low bias with moderate variance, confirming that bridge functions recover causal effects from noisy proxies.

14.8 Simulation Study: Counterfactual OPE under a Misspecified SCM

A second synthetic instance illustrates the counterfactual machinery of Section 14.5 on a contextual-bandit substrate that strips the temporal complication and isolates the model-

[254] $W_t^{(1)} \sim \text{Bernoulli}(0.85 \cdot U_t + 0.15 \cdot (1 - U_t))$ and $W_t^{(2)} \sim \text{Bernoulli}(0.75 \cdot U_t + 0.25 \cdot (1 - U_t))$.

[255] Rewards are $r(s, a) = -1$ for $s < 4$, $\gamma = 0.9$. The target policy always promotes. The true interventional transition probability is $P(s{+}1 \mid s, \text{do}(\text{promote})) = 0.615$.

[256] Each configuration uses 2,000 trajectories averaged over 20 seeds.

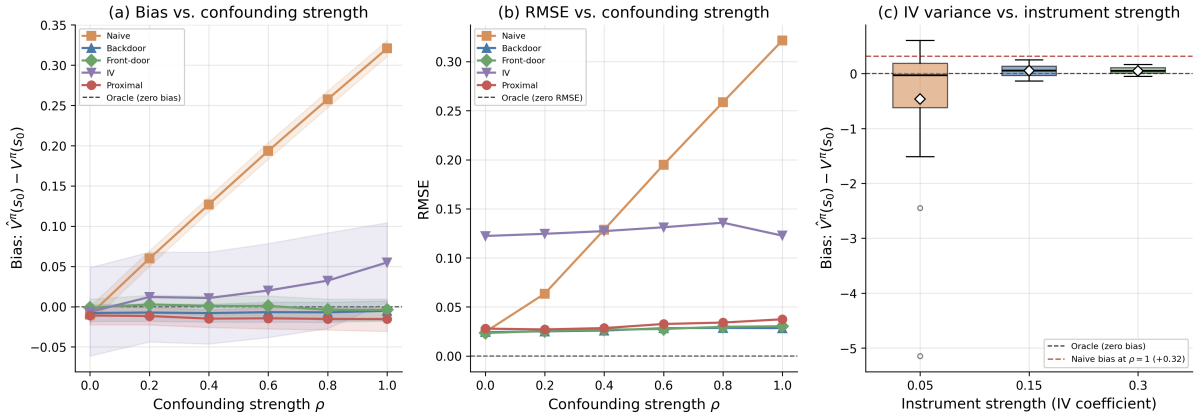

Figure 33: (a) Bias of five OPE estimators as a function of confounding strength ρ. (b) RMSE of five estimators as a function of ρ. (c) IV estimator bias distribution vs. instrument strength at $\rho = 1$; dashed red line is naive estimator bias.

misspecification question. The DGP is a linear structural causal model in two features,

$$y = 1 + 0.5\,x_1 - 0.3\,x_2 + 2\,a + a \cdot x_1 + u, \qquad u \sim \mathcal{N}(0,\, 0.5^2), \tag{149}$$

with $x \sim \mathcal{N}(0, I_2)$, a binary action $a \in \{0, 1\}$, and a heterogeneous treatment effect $\tau(x) = 2 + x_1$. The behaviour policy $\pi_{\text{obs}}(a = 1 \mid x) = \sigma(0.5 + x_1 - 0.5\,x_2)$ is confounded with x. The target policy $\tilde{\pi}(a \mid x) = \mathbf{1}\{x_1 + x_2 > 0\}$ is a deterministic threshold rule. The oracle value $V(\tilde{\pi}) = \mathbb{E}_x[y \mid a = \tilde{\pi}(x)]$ is computed by Monte Carlo at 10^6 realisations.

Three estimators are compared on $n \in \{200, 500, 1000, 2000\}$ logged samples, 20 seeds per cell. Per-decision importance sampling is $\widehat{V}_{\text{IS}} = n^{-1} \sum_i \rho_i y_i$ with $\rho_i = \mathbf{1}\{a_i = \tilde{\pi}(x_i)\}/\pi_{\text{obs}}(a_i \mid x_i)$. The model-based plug-in $\widehat{V}_{\text{MB}} = n^{-1} \sum_i \hat{f}(x_i, \tilde{\pi}(x_i))$ uses an OLS fit $\hat{f}$ on the logged data. The counterfactual estimator follows the abduction-action-prediction rule of Algorithm 4 and combines $\hat{f}$ with a residual correction weighted by the importance ratio,

$$\widehat{V}_{\text{CF}} = n^{-1} \sum_i \hat{f}(x_i, \tilde{\pi}(x_i)) + n^{-1} \sum_i \rho_i (y_i - \hat{f}(x_i, a_i)), \tag{150}$$

which is the doubly-robust form of the abduction-action-prediction rule and is unbiased under correct propensity *or* correct outcome model.[257] The well-specified scenario uses the feature set $\{1, x_1, x_2, a, a \cdot x_1\}$ in $\hat{f}$, matching the DGP. The misspecified scenario drops the interaction $a \cdot x_1$, the source of the heterogeneous treatment effect.

Table 27 reports bias, standard deviation, and root mean squared error at $n = 1000$. Under well-specification the model-based estimator attains bias -0.001 and RMSE 0.056, the counterfactual estimator attains bias 0.003 and RMSE 0.065, and the importance-sampling estimator attains bias 0.017 and RMSE 0.099. The MB and CF estimators are statistically indistinguishable and both improve on IS. Under misspecification the MB estimator's bias jumps to -0.072 and its RMSE to 0.093, while the CF estimator's bias remains at 0.002 and its RMSE at 0.076. The CF estimator's RMSE is 18% lower than MB's, and the bias is two orders of magnitude smaller. The IS estimator is unchanged across scenarios because it does not use an outcome model. Figure 34 plots RMSE against n on log-log scale. The MB curve flattens under misspecification at the population bias of the OLS projection, the CF curve continues to decline

[257] The naive average of the per-observation counterfactual outcome $y'_i = \hat{f}(x_i, \tilde{\pi}(x_i)) + (y_i - \hat{f}(x_i, a_i))$ collapses to $\widehat{V}_{\text{MB}}$ under OLS with an intercept because the OLS residuals sum exactly to zero. The importance-weighted residual correction in (150) restores the bias-cancellation property in finite samples. The estimator coincides with the classical doubly-robust estimator of Robins-Rotnitzky-Zhao adapted to off-policy evaluation.

Table 27: Counterfactual off-policy evaluation under a linear SCM, $n = 1000$, 20 seeds. The CF estimator's residual correction cancels the MB bias under misspecification while inheriting low variance from the model.

Scenario	Estimator	Bias	Std	RMSE
Well-specified	IS	+0.017	0.100	0.099
	MB	−0.001	0.057	0.056
	CF	+0.003	0.067	0.065
Misspecified	IS	+0.017	0.100	0.099
	MB	−0.072	0.061	0.093
	CF	+0.002	0.078	0.076

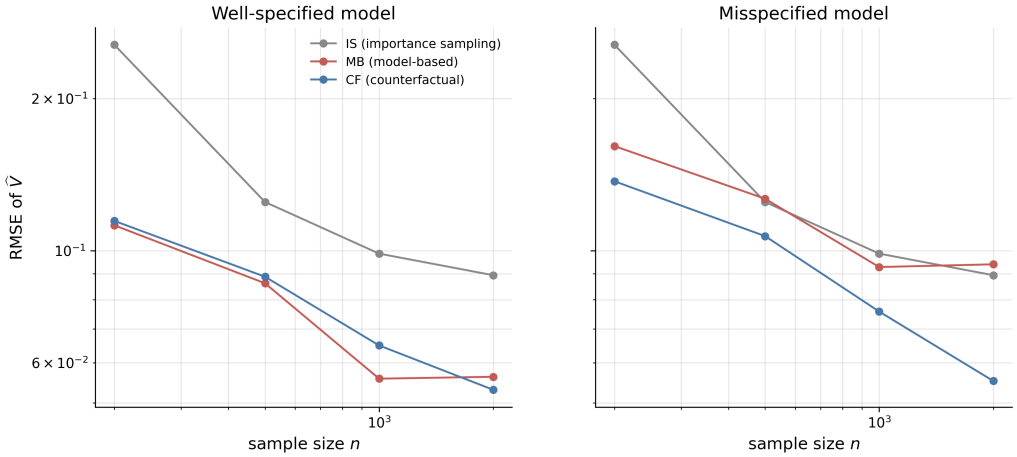

Figure 34: Root mean squared error of three OPE estimators against the sample size, log-log scale, two panels. Left panel, well-specified outcome model. Right panel, misspecified model with the interaction omitted. The MB estimator's RMSE plateaus under misspecification as the bias dominates the variance, while the CF estimator's RMSE continues to decline.

at the parametric rate, and the IS curve traces the higher-variance parametric rate that the importance-weighted estimator delivers without any model assistance.

The companion Chapter 15 inverts the direction of this one, asking how RL machinery imports into econometric causal inference rather than the reverse. The bridge there runs through the equivalence between Murphy's backward Bellman recursion for optimal dynamic treatment regimes and Watkins's Q-learning recursion on a history-augmented state. Once that translation is fixed, the dynamic-DML and policy-learning results of that chapter complement the identification techniques of this one: this chapter deals with what to do when sequential ignorability fails; the companion chapter deals with how far you can push the rate of convergence when it holds.

15 Reinforcement Learning for Causal Inference

15.1 Dynamic Treatment Regimes

In many treatment and policy settings the analyst makes a sequence of decisions, the right choice at each step depends on the subject's evolving state, and the estimation target is the decision rule itself rather than a fixed treatment level. At each of K decision points the analyst observes the subject's state S_k (covariates, past treatments, past responses), chooses a treatment A_k, and observes the next state; after the final decision a scalar outcome Y is realised. The motivating

example in Murphy (2003), used throughout this section, is the Fast Track prevention program, a randomized study targeted at children flagged for elevated behavioral problems in kindergarten,[258] in which the home-visiting intensity each semester was set adaptively from a six-item family-functioning score S_j, with low scores indicating greater need. The design team rejected a uniform schedule because "providing too many home visits might be detrimental, increasing risk for family dependency, pejorative labeling (by self and others) and cause attrition," and adopted an adaptive rule. The estimation target is the rule $\pi^* = (\pi_1^*, \dots, \pi_K^*)$ that maximises the expected outcome; a standard randomized-trial analysis recovers the marginal effect of a fixed protocol, not a rule.

Notation and identification

Write $\bar{A}_k = (A_1, \dots, A_k)$ for the treatment history through decision k, $\bar{S}_k = (S_1, \dots, S_k)$ for the state history, and $\bar{H}_k = (\bar{S}_k, \bar{A}_{k-1})$ for the observed information just before decision A_k. For each potential treatment path $\bar{a}_K$, the potential outcome $Y^*(\bar{a}_K)$ denotes the response that would have been observed under that path (Rubin, 1978; Robins, 1986). A dynamic treatment regime $\pi = (\pi_1, \dots, \pi_K)$ is a sequence of maps $\pi_k : \bar{H}_k \mapsto A_k$, with value $V(\pi) = \mathbb{E}[Y^*(\pi)]$ and optimization target $\pi^* \in \arg\max_\pi V(\pi)$.

Identification rests on three assumptions; Chapter 14 treats failure of the third. *Consistency* states that $Y = Y^*(\bar{A}_K)$ and $S_k = S_k^*(\bar{A}_{k-1})$, so observed outcomes equal realized potential outcomes. *Positivity* requires every action of interest to receive strictly positive probability conditional on every history that arises in the population. *Sequential ignorability* requires $A_k \perp \{Y^*, S_{k+1}^*, \dots\} \mid \bar{H}_k$ for every k, meaning the decision A_k may depend on the observed history but not on unobserved factors that also affect future states and outcomes.

The backward recursion

Under these three assumptions, the optimal value functions satisfy the finite-horizon backward recursion

$$V_{K+1}(\bar{h}_{K+1}) = Y, \tag{151}$$

$$Q_k(\bar{h}_k, a_k) = \mathbb{E}\Big[V_{k+1}(\bar{H}_{k+1}) \,\Big|\, \bar{H}_k = \bar{h}_k,\ A_k = a_k\Big], \tag{152}$$

$$V_k(\bar{h}_k) = \max_{a_k} Q_k(\bar{h}_k, a_k), \qquad \pi_k^*(\bar{h}_k) = \arg\max_{a_k} Q_k(\bar{h}_k, a_k). \tag{153}$$

Murphy calls V_k the *optimal benefit-to-go*, inverting the engineering "optimal cost-to-go" of Bertsekas (1996) since the response is to be maximized rather than minimized. The recursion is, in her words, "a finite time version of Bellman's equation (Bellman, 1957)."

What separates the setting from textbook backward induction is identification. Engineering dynamic programming is given the joint distribution of states and outcomes indexed by every possible treatment path; for any candidate path the analyst can compute or simulate the resulting distribution and take expectations. Murphy's data are different. The cohort has N subjects each observed once along the single realized path that the treating physician chose. For any other path that the subject did not follow, the relevant $(\bar{S}_K, Y)$ distribution is counterfactual and unobserved. Worse, the empirical conditional distribution of S_{k+1} given S_k and $A_k = a$ in the cohort is not generally the distribution of S_{k+1} that would arise if every subject were assigned a at stage k, because the decision A_k may be correlated with unobserved factors that also affect S_{k+1}. Running the empirical Bellman backup on the cohort directly would absorb this bias into every $\hat{Q}_k$.

Sequential ignorability rules out the bias. Conditional on $\bar{H}_k$, the action A_k is independent of future potential outcomes, so any factor that influences both A_k and future states or

[258] The program was designed to prevent the emergence of conduct disorders and drug use in at-risk children.

outcomes must be measured in the history. The treating physician may consult anything in the chart, including past treatment responses, but does not condition on private information unavailable to the analyst. Under this assumption Robins's G-computation formula identifies the counterfactual outcome distribution from the observed-data distribution, and the recursion in (152)–(153) runs empirically. Take the $K = 2$ case with binary actions to see what this looks like in practice. At stage two, $Q_2(\bar{h}_2, a_2) = \mathbb{E}[Y \mid \bar{H}_2 = \bar{h}_2, A_2 = a_2]$ is estimated by regressing Y on $(\bar{h}_2, a_2)$ across the cohort, giving $\hat{Q}_2$ and $\hat{V}_2(\bar{h}_2) = \max_{a_2} \hat{Q}_2(\bar{h}_2, a_2)$ at every history. At stage one, $\hat{V}_2(\bar{H}_2)$ is then treated as a synthetic outcome and regressed on $(\bar{h}_1, a_1)$, yielding $\hat{Q}_1$. The optimal rule at each stage is the argmax of the corresponding regression. The whole estimator is two regressions executed in sequence. In the offline-RL literature this backward-induction-with-regression procedure is known as Fitted Q-Iteration (Ernst et al., 2005), and it is precisely the batch form of the Bellman backup that Q-learning implements online. Sections 3 and 4 of Murphy (2003) refine the procedure by reparameterising in terms of regret contrasts, which lets the estimator place smoothness assumptions only on the quantities directly relevant for the optimal rule.

Connection to Q-learning and the dictionary

The recursion in (152)–(153) is the same Bellman backup that drives Q-learning, covered in Section 4. Watkins and Dayan (1992b) estimates Q^* from a stream of (s, a, r, s') transitions, with each observation nudging the current estimate by stochastic approximation. Murphy estimates the same conditional expectation in batch, with one regression per decision stage on the full cohort. The role that successive transitions play in Watkins's setting is played by successive subjects in Murphy's. Enrolling another patient adds a row to each per-stage regression and sharpens $\hat{Q}_k$, the way an arriving transition adjusts $\hat{Q}$ in Q-learning. A Murphy cohort replayed one subject at a time, with each $\hat{Q}_k$ updated by a temporal-difference step rather than refit, would converge to the same Q^*.

Three questions ground the equivalence. First, do Murphy's batch backward regression and online Q-learning recover the same optimal regime on a tractable two-stage DTR as the cohort size grows? Second, Q-learning solves the recursion by stochastic approximation rather than by one-shot regression; how does its recovered regime improve as the cohort is replayed more times at a fixed cohort size? Third, when the state is high-dimensional and tabular Q is infeasible, do the neural-network analogues of the two estimators – Neural Fitted Q-Iteration and DQN – still recover the same optimal regime?

The tractable setting is a synthetic two-stage DTR with a five-level ordinal status $S_k \in \{1, \ldots, 5\}$, a binary action, a logistic behavior policy, and an outcome that rewards treatment when status is low and penalizes it when status is high. The high-dimensional setting holds the structural form fixed but replaces the five-level state with a $p = 10$-dimensional continuous one, an autoregressive transition with treatment effect on the first coordinate, and the same threshold interaction in the outcome.[259] Both DGPs respect sequential ignorability by construction.

Figure 35 reports the value of the recovered regime, $V(\hat{\pi})/V^*$, across the three questions. Panel (Q1) shows the tabular sample-size sweep. Murphy and online Q-learning curves overlap throughout, both approaching V^* as N grows; the small residual gap at large N is the constant-α bias of Q-learning. Panel (Q2) shows the tabular training-budget sweep at $N = 300$. Q-learning at one replay epoch sits below the converged Murphy reference; by three epochs it has caught up. Panel (Q3) shows the high-dimensional sample-size sweep. Neural Fitted Q-Iteration and DQN both rise from the behavior-policy baseline toward V^* as the cohort grows, with Neural-

[259] Implementation source: `ch10b_rl_for_ci/sims/dtr_qlearning_vs_murphy.py`. Tabular case: Q over the (history, action) grid for both estimators, oracle V^* by exact backward induction, 50 Monte Carlo seeds. High-dimensional case: two-layer MLPs with 64 hidden units for both estimators (Neural Fitted Q-Iteration fits Q stage-by-stage by full-batch MSE; DQN trains both Q networks jointly by minibatch TD), oracle V^* by Monte Carlo on the known DGP, 20 Monte Carlo seeds.

FQI slightly more sample-efficient at moderate N because it trains the stage-two regression to convergence before bootstrapping the stage-one regression. The equivalence survives the transition to function approximation.

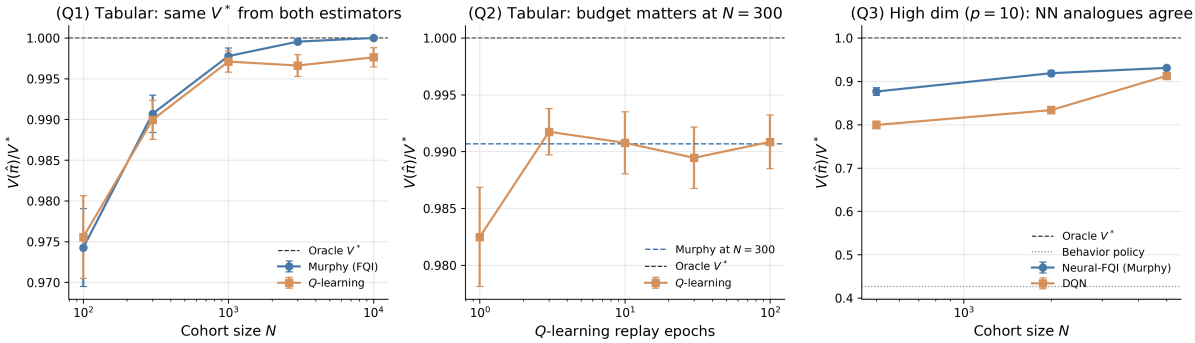

Figure 35: Murphy's batch backward regression vs online Q-learning on a synthetic two-stage DTR under sequential ignorability. (Q1) Tabular sample-size sweep at fixed Q-learning budget. (Q2) Tabular training-budget sweep at fixed $N = 300$, with Murphy's converged value as the dashed reference. (Q3) High-dimensional sweep ($p = 10$ continuous state), comparing Neural Fitted Q-Iteration to DQN with the same MLP architecture; the dotted line is the behavior-policy value. Error bars are 95% confidence intervals over Monte Carlo seeds.

Table 28: Recovered policy value $V(\hat{\pi})/V^*$ at the largest cohort size on the synthetic two-stage DTR (rank-ordered by performance). Tabular setting: $N = 10{,}000$ subjects, 50 Monte Carlo seeds. High-dimensional setting: $p = 10$ continuous state, $N = 5{,}000$, 20 seeds.

Method	$V(\hat{\pi})/V^*$	SE
Murphy / FQI (tabular, $N = 10000$)	1.0000	(0.0000)
Oracle V^* (tabular)	1.0000	–
Oracle V^* (high-dim, $p = 10$)	1.0000	–
Q-learning (tabular, $N = 10000$, 100 replays)	0.9976	(0.0006)
Neural-FQI / Murphy (high-dim, $N = 5000$)	0.9310	(0.0024)
DQN (high-dim, $N = 5000$)	0.9126	(0.0030)

Once the Bellman recursion is shared, the rest of the vocabulary aligns predictably. Table 29 translates between the two traditions following Schulte et al. (2014).[260]

Two caveats apply to the translation. First, the A-learning contrast $C_k(\bar{h}_k) = Q_k(\bar{h}_k, 1) - Q_k(\bar{h}_k, 0)$ is anchored at the reference action $a_k = 0$, while the RL advantage $A_k^\pi(\bar{h}_k, a_k) = Q_k(\bar{h}_k, a_k) - V_k^\pi(\bar{h}_k)$ is anchored at the policy mean, and the two agree only when the baseline policy is constant at $a_k = 0$. Second, sequential ignorability and the offline-RL convention of a known behavior policy play the same identification role but are not literally the same statement, the former a property of the unobserved confounder structure and the latter a property of the data-collection protocol. The current chapter takes sequential ignorability as given throughout, and Chapter 14 catalogues the proxy, instrumental, sensitivity, and mediator strategies that apply when it fails.

The Murphy-Watkins bridge is now being run at scale on real electronic health records. Conservative Q-learning has been applied to vasopressor and intravenous-fluid recommendations on

[260]Throughout the table, the RL state s is read as the full history $\bar{h}_k = (\bar{s}_k, \bar{a}_{k-1})$ at decision k, the horizon is finite and equal to K, all immediate rewards are zero except $R_{K+1} = Y$, and $\gamma = 1$. The Markov property of standard RL is replaced by history augmentation.

Table 29: Translation between dynamic treatment regimes and the g-methods of Robins (1986); Robins et al. (2000); Murphy (2003); Schulte et al. (2014) on the left, and reinforcement learning after Watkins and Dayan (1992b); Sutton and Barto (2018) on the right.

Dynamic treatment regimes / g-methods	Reinforcement learning
$Q_k(\bar{h}_k, a_k)$, conditional mean of V_{k+1}	$Q(s,a)$, state-action value function
$V_k(\bar{h}_k) = \max_{a_k} Q_k(\bar{h}_k, a_k)$	$V(s) = \max_a Q(s,a)$, state value function
$d_k^{\text{opt}}(\bar{h}_k) = \arg\max_{a_k} Q_k(\bar{h}_k, a_k)$	Greedy policy $\pi^*(s) = \arg\max_a Q(s,a)$
Propensity $e_k(\bar{h}_k, a_k) = \mathbb{P}(A_k = a_k \mid \bar{H}_k = \bar{h}_k)$	Behavior / logging policy $\mu_k(a_k \mid \bar{h}_k)$
G-computation (sequential conditional regression)	Fitted-Q evaluation (Ernst et al., 2005)
IPTW for marginal structural models (Robins et al., 2000)	Off-policy importance sampling (Precup et al., 2000)
G-estimation for structural nested mean models (Robins, 2004)	Orthogonal-score off-policy evaluation (Lewis and Syrgkanis, 2021)
A-learning / contrast (blip) estimation (Schulte et al., 2014; Robins, 2004)	Advantage function (Baird, 1993)
Sequential ignorability: $A_k \perp \{Y^*, S^*_{k+1}, \ldots\} \mid \bar{H}_k$	Known behavior policy $\mu_k(\cdot \mid \bar{h}_k)$ (offline RL setting)
Positivity: $e_k(\bar{h}_k, a) > 0$ for every action of interest	Coverage: $\mu_k(a \mid \bar{h}_k) > 0$ for all $a \in \operatorname{supp} \pi^*$

MIMIC-III sepsis cohorts (Kaushik et al., 2022) and to mechanical-ventilation parameter selection on MIMIC-III intensive-care patients (Kondrup et al., 2023). Cross-site policy transfer between clinical environments has been formalised as a counterfactual problem and handled with structural causal models on a sepsis simulator (Killian et al., 2022). Methodological-rigor work emphasises that naïve deployment is insufficient. Cross-off-policy evaluation across reward weightings identifies policies that generalise rather than overfit on intubated COVID-19 patients in the Dutch ICU Data Warehouse (Roggeveen et al., 2024), and benchmark studies show that standard offline-RL algorithms degrade markedly under realistic noise, missingness, and pharmacokinetic variability (Luo et al., 2024). Inverse-constrained methods extend the framework to infer an implicit safety specification from clinician demonstrations while the survival reward is taken as given (Fang et al., 2024). The deconfounding actor-critic of Yin et al. (2022) bakes a balancing module directly into the actor-critic learning loop, anticipating themes that recur in Chapter 14.

15.2 Dynamic Double Machine Learning for Structural Nested Mean Models

Section 15.1 established that the recursion at the heart of g-methods and the recursion at the heart of Q-learning are the same object on history-augmented data. The natural follow-up question is what happens when the nuisance functions inside that recursion (the outcome regression, the propensity score) are estimated by modern machine learning. Lewis and Syrgkanis (2021) answer this question for the structural nested mean model. A Neyman-orthogonal version of Robins's g-estimation, combined with cross-fitting, delivers $\sqrt{n}$-consistent asymptotically normal estimates of the dynamic structural parameters, and those parameters in turn deliver off-policy evaluation at parametric rates for any target dynamic policy.

For the Fast Track cohort the structural parameter ψ_t has a direct interpretation as the marginal causal effect of an additional home visit at semester t, holding future visits at a fixed reference level. The question of this section is what valid $\sqrt{n}$ inference on ψ_1 and ψ_2 requires when the outcome and propensity nuisance functions are estimated by an off-the-shelf machine

learner on the longitudinal cohort, and how to convert those structural parameters into the value of any counterfactual home-visiting regime.

Structural nested mean models

Following Robins (1986, 2004); Schulte et al. (2014), the conditional mean of the outcome given history admits a period-by-period decomposition. For each $t = 1, \dots, T$, the *blip function* is the marginal causal effect of the period-t treatment under a fixed reference future,

$$\gamma_t(\bar{h}_t, a_t) = \mathbb{E}\Big[Y \mid \bar{H}_t = \bar{h}_t,\, A_t = a_t,\, A_{t+1} = \cdots = A_T = 0\Big] - \mathbb{E}\Big[Y \mid \bar{H}_t = \bar{h}_t,\, A_t = 0,\, A_{t+1} = \cdots = A_T = 0\Big]. \quad (154)$$

A structural nested mean model parameterizes the blip as $\gamma_t(\bar{h}_t, a_t) = \psi_t' B_t(\bar{h}_t, a_t)$ for known basis functions B_t and unknown low-dimensional parameter $\psi_t \in \mathbb{R}^d$. The full structural parameter vector $\psi^* = (\psi_1^*, \dots, \psi_T^*)$ identifies the value of any target dynamic policy via the g-formula applied to the partialled-out outcome (Robins, 2004).

Sequential residualisation

Lewis and Syrgkanis (2021) construct cross-fitted residuals via two nuisance regressions per period, one for the outcome and one for the treatment given history. Let $\hat{q}_t(\bar{h}_t) = \widehat{\mathbb{E}}[Y \mid \bar{H}_t]$ and $\hat{p}_{j,t}(\bar{h}_t) = \widehat{\mathbb{E}}[A_j \mid \bar{H}_t]$ for $j = t, t+1, \dots, T$, fitted by any sufficiently fast machine learner on a held-out fold. Define the residualised outcome $\tilde{Y}_t = Y - \hat{q}_t(\bar{H}_t)$ and the residualised treatments $\tilde{A}_{j,t} = A_j - \hat{p}_{j,t}(\bar{H}_t)$. The main result of Lewis and Syrgkanis (2021), their Theorem 2, is that the structural parameter ψ_t^* satisfies the conditional Neyman-orthogonal moment restriction

$$\mathbb{E}\left[\Big(\tilde{Y}_t - \sum_{j=t}^{T} \psi_j^{*\prime}\, \tilde{A}_{j,t}\Big)\, \tilde{A}_{t,t} \,\Big|\, \bar{H}_t\right] = 0, \qquad t = T, T-1, \dots, 1, \quad (155)$$

and the resulting system is solved recursively from $t = T$ downward. The algorithm estimates $\hat{\psi}_T$ from the period-T moment, peels off the estimated period-T effect from the outcome to define the calibrated outcome $\tilde{Y}_{T-1} - \hat{\psi}_T' \tilde{A}_{T,T-1}$, regresses against $\tilde{A}_{T-1,T-1}$ to obtain $\hat{\psi}_{T-1}$, and continues backward to ψ_1. Each stage solves an ordinary least squares regression on residualised data.

Asymptotic normality at $\sqrt{n}$ rates

The Neyman-orthogonal moment in Equation (155) has the crucial property that small errors in the nuisance estimates $\hat{q}_t$ and $\hat{p}_{j,t}$ enter the estimating equation at second order rather than first. Theorem 4 of Lewis and Syrgkanis (2021) shows that if each cross-fitted nuisance achieves mean-squared-error rate $o_p(n^{-1/4})$ in the population norm, then the recursive estimator $\hat{\psi}$ is $\sqrt{n}$-consistent and asymptotically normal,

$$\sqrt{n}\,(\hat{\psi} - \psi^*) \xrightarrow{d} \mathcal{N}(0,\, V), \quad (156)$$

with a consistently estimable covariance matrix V. The $o_p(n^{-1/4})$ condition is satisfied by Lasso regression under standard sparsity conditions, by random forests under regularity conditions on the population regression function, and by any other learner that achieves the corresponding excess-risk rate. The history $\bar{H}_t$ may itself be high-dimensional; only the structural parameter $\psi^* \in \mathbb{R}^{Td}$ is required to be low-dimensional. The resulting confidence intervals are asymptotically uniformly valid over the corresponding class of data-generating processes.

The off-policy evaluation bridge

The structural parameters ψ^* identify the value of any target dynamic policy π, because the SNMM blip representation gives a closed-form expression for the policy value $V(\pi) = \mathbb{E}[Y^*(\pi)]$ that depends only on ψ^* and the marginal action distribution under π (Lewis and Syrgkanis, 2021, Section 6). The headline claim from the paper's abstract states the bridge directly:

> *These structural parameters can be used for off-policy evaluation of any target dynamic policy at parametric rates, subject to semi-parametric restrictions on the data generating process.*

This is the bridge promised at the close of Section 15.1. Murphy's Q-recursion estimated under semiparametric efficiency gives parametric-rate inference on policy value, even when the controls inside the recursion are estimated by black-box machine learning. The simulation study in Section 15.6 demonstrates the empirical $\sqrt{n}$ coverage on a two-stage SNMM. At $n = 4000$, Dynamic DML attains 96.5% empirical coverage of nominal 95% confidence intervals on ψ_1 and 93.0% on ψ_2, with RMSE shrinking at the rate predicted by Theorem 4 of Lewis and Syrgkanis (2021). The naive panel regression that omits the time-varying confounder carries a bias of -0.19 on ψ_1 and $+0.93$ on ψ_2 that does not shrink with the sample size, and its confidence intervals cover the truth in 1% and 0% of replications respectively. MSM with stabilised inverse-probability weighting is consistent but slow; at the same sample size its coverage is 88% on ψ_1 and 73% on ψ_2.

Recursive Riesz representers

The Neyman-orthogonal moment of Lewis and Syrgkanis (2021) is hand-derived. The residual-on-residual construction in Equation (155) achieves orthogonality by inspection, because residualising both sides of a regression makes the moment locally insensitive to small perturbations in the nuisance regressions. Chernozhukov et al. (2023) automate this step. Rather than write the orthogonal moment in closed form, they characterize the de-biasing correction as a sequence of *Riesz representers* solving a system of recursive linear functional equations, and they show that these representers can be learned by minimising a sequence of quadratic Riesz losses without ever writing the orthogonal moment in closed form. Formally, define $f_{T+1}(\bar{h}_{T+1}) = Y$ and the nested mean regressions $f_t(\bar{h}_t, a_t) = \mathbb{E}\Big[f_{t+1}(\bar{H}_{t+1}, \pi_{t+1}(\bar{H}_{t+1})) \mid \bar{H}_t, A_t = a_t\Big]$ for $t = T, \dots, 1$. The policy value $\theta(\pi) = \mathbb{E}[Y^*(\pi)]$ admits the debiased representation

$$\theta(\pi) = \mathbb{E}\left[f_1(\bar{H}_1, \pi_1(\bar{H}_1)) + \sum_{t=1}^{T} \alpha_{t-1}(\bar{H}_{t-1}, A_{t-1}) \left(f_{t+1}(\bar{H}_{t+1}, \pi_{t+1}) - f_t(\bar{H}_t, A_t)\right)\right], \tag{157}$$

where $\alpha_0 \equiv 1$ and each $\alpha_t : \mathcal{H}_t \times \mathcal{A}_t \to \mathbb{R}$ is the Riesz representer of the linear functional $L_t(g) = \mathbb{E}\big[\alpha_{t-1}(\bar{H}_{t-1}, A_{t-1})\, g(\bar{H}_t, \pi_t)\big]$. Each α_t is estimated by minimising the quadratic Riesz loss $\mathbb{E}_n\big[\alpha^2(\bar{H}_t, A_t) - 2\,\hat{\alpha}_{t-1}(\bar{H}_{t-1}, A_{t-1})\,\alpha(\bar{H}_t, \pi_t)\big]$, with no closed-form inverse-propensity-weight expression required. The two constructions are complementary. Lewis and Syrgkanis (2021) target the SNMM blip parameters at parametric rates; Chernozhukov et al. (2023) target the policy value $\theta(\pi)$ non-parametrically. Both achieve the mixed-bias product structure under which $o_p(n^{-1/4})$ nuisance error suffices for the target oracle rate.[261]

In offline-RL vocabulary, the Lewis-Syrgkanis estimator is the orthogonal-score version of *fitted-Q evaluation*, with the residual-on-residual moment in Equation (155) replacing the ordinary mean-squared-error loss that FQE uses to fit the period-t value function. The recursive

[261] The reason this works at the population-risk level is the orthogonal statistical learning theory of Foster and Syrgkanis (2023), whose Theorem 1 shows that under prediction-space strong convexity of the target loss and Neyman orthogonality of the loss derivative, the excess risk of a two-stage cross-fitted estimator has nuisance error entering at second order, so $n^{-1/4}$ mean-squared error on each nuisance suffices for the target oracle rate n^{-1}.

Riesz representer construction of Chernozhukov et al. (2023) is *automatic debiased off-policy evaluation*, applicable to any RL OPE estimator that targets a smooth functional of a sequence of conditional means, including fitted-Q evaluation, model-based OPE, and marginalised importance sampling. The bridge in this section is what lets those RL OPE methods inherit the $\sqrt{n}$ rate that semiparametric efficiency theory establishes for the SNMM target.

15.3 Dynamic Offline Policy Learning

Section 15.2 delivers $\sqrt{n}$ confidence intervals on dynamic structural parameters under sequential ignorability, which in turn deliver off-policy evaluation at parametric rates for any target dynamic policy. This subsection turns from evaluation to optimization. Given observational data on n trajectories, can we *learn* an optimal dynamic regime $\pi^* \in \arg\max_{\pi \in \Pi} V(\pi)$ at the same $\sqrt{n}$ regret rate, using modern machine learning for the nuisances? Sakaguchi (2024) answers yes, by combining the doubly-robust AIPW machinery of Athey and Wager (2021) with fitted-Q-evaluation in a backward-induction loop.

For the Fast Track cohort, a natural restricted policy class is the family of threshold rules in the family-functioning score, $\pi_t(\bar{h}_t) = \mathbb{1}\{S_{t,j} < c_t\}$ for thresholds (c_1, c_2) to be learned from data. The question of this section is the welfare-maximising thresholds with a $\sqrt{n}$-rate welfare-regret guarantee, when the cohort is observational and the nuisances are estimated by machine learning.

The static $T = 1$ precursor

In the single-period treatment-choice problem the offline policy-learning literature is well-developed. Kitagawa and Tetenov (2018) introduced empirical welfare maximization with known propensities and proved a matching minimax bound of $\Theta(\sqrt{\mathrm{VC}(\Pi)/n})$ on welfare regret, applied to the National JTPA Study. Athey and Wager (2021) generalized to unknown propensities via cross-fitted doubly-robust scores (their Equation 14), maintaining the $O_p(\sqrt{\mathrm{VC}(\Pi)/n})$ rate under product-rate nuisance conditions and extending to instrumental-variable identification. Zhou et al. (2023) extended further to multi-action treatments with an exact decision-tree-search algorithm. All three are the $T = 1$ special case of what follows.

Backward-induction AIPW under sequential ignorability

For each stage $t = 1, \ldots, T$, fix a policy class Π_t over which to optimize the stage-t rule, and let $\Pi = \Pi_1 \times \cdots \times \Pi_T$. Define the action-value function for any candidate future policy $\pi_{(t+1):T} = (\pi_{t+1}, \ldots, \pi_T)$ recursively by

$$Q_t^{\pi_{(t+1):T}}(\bar{h}_t, a_t) = \mathbb{E}\Big[Y_t + Q_{t+1}^{\pi_{(t+2):T}}(\bar{H}_{t+1}, \pi_{t+1}(\bar{H}_{t+1})) \,\Big|\, \bar{H}_t = \bar{h}_t, A_t = a_t\Big], \tag{158}$$

with terminal condition $Q_T(\bar{h}_T, a_T) = \mathbb{E}[Y_T \mid \bar{H}_T = \bar{h}_T, A_T = a_T]$. The recursion estimates Q in the form of *fitted-Q-evaluation* (Ernst et al., 2005) applied under sequential ignorability. Let $e_t(\bar{h}_t, a_t) = \mathbb{P}(A_t = a_t \mid \bar{H}_t = \bar{h}_t)$ denote the stage-t propensity.

Sakaguchi (2024) estimates the optimal regime $\hat{\pi} = (\hat{\pi}_1, \ldots, \hat{\pi}_T)$ by backward induction with cross-fitted nuisances. Randomly partition the data into K folds, and let $\hat{Q}_t^{-k}$ and $\hat{e}_t^{-k}$ denote the nuisance estimators fitted on the complement of the k-th fold (any sufficiently fast machine learner). Starting at $t = T$, construct the cross-fitted AIPW score for any candidate stage-T policy π_T,

$$\hat{\Gamma}_{i,T}(a) = \hat{Q}_T^{-k(i)}(\bar{H}_{i,T}, a) + \frac{\mathbb{1}\{A_{i,T} = a\}}{\hat{e}_T^{-k(i)}(\bar{H}_{i,T}, a)} \left(Y_{i,T} - \hat{Q}_T^{-k(i)}(\bar{H}_{i,T}, A_{i,T})\right), \tag{159}$$

and select $\hat{\pi}_T = \arg\max_{\pi_T \in \Pi_T} \frac{1}{n} \sum_{i=1}^{n} \hat{\Gamma}_{i,T}(\pi_T(\bar{H}_{i,T}))$. Given $\hat{\pi}_T$, refit $\hat{Q}_{T-1}^{\hat{\pi}_T}$ by fitted-Q-evaluation against the calibrated next-stage value, build the stage-$T-1$ AIPW score from Equation (159) with the new Q-function in the position of $\hat{Q}_T^{-k(i)}$, and select $\hat{\pi}_{T-1}$. Continue backward to $\hat{\pi}_1$. Each stage solves a single classification-style policy optimization on cross-fitted scores; the nuisances at stage t depend only on the previously estimated future policies $\hat{\pi}_{(t+1):T}$, never on the candidate π_t being optimized.

The $\sqrt{n}$ welfare regret bound

Sakaguchi (2024, Theorem 4.3) establishes that under sequential ignorability, overlap (Assumption 2.3 of the paper), a correct-specification condition on the policy classes (Assumption 3.1), uniform MSE convergence of $o_p(n^{-1/4})$ on both the Q-functions for the estimated future policies and the propensity scores, and an entropy-integral complexity bound on Π, the welfare regret of the estimated regime satisfies

$$V(\pi^*) - V(\hat{\pi}) = O_p\Big(\kappa(\Pi)\, n^{-1/2}\Big), \tag{160}$$

where $\kappa(\Pi)$ is the entropy integral of the DTR class. This matches the $\sqrt{n}$ minimax-optimal rate that Kitagawa and Tetenov (2018) established for the static $T = 1$ case, and is the first such result for the multi-stage problem with purely nonparametric ML nuisances. A companion result (their Theorem 5.1) shows that the alternative simultaneous-optimization approach attains the same rate without requiring correct specification of the policy class, at the cost of computational tractability that degrades quickly in T.

Empirical application

Sakaguchi (2024, Section 7) applies the method to the Tennessee Project STAR class-size experiment. The treatment at each of two stages (kindergarten, first grade) is the choice between a regular-size class with a full-time teacher aide and a small-size class without one; the welfare objective is the percentile rank of combined reading and mathematics test scores at the end of first grade. Decision trees of depth one and two are used for the stage-specific policy classes, with cross-fitted random-forest nuisances. The estimated optimal dynamic regime improves academic achievement by approximately 8% relative to uniformly assigning all students to classes with a teacher aide, and by approximately 1% relative to uniformly assigning to small classes, both estimated via five-fold cross-validation. A companion field experiment in Japan (Ida et al., 2024) applies a related dynamic-targeting estimator to energy-saving rebate programs and confirms empirically that two-period dynamic targeting outperforms one-shot static targeting on realized welfare gains.[262]

In offline-RL vocabulary, the Sakaguchi estimator is the doubly-robust analog of *fitted-Q iteration* for policy optimisation. The action-value recursion in Equation (158) is fitted-Q evaluation under sequential ignorability; the augmentation in Equation (159) adds the importance-weighted residual correction that makes the score estimator doubly robust. The connection extends downstream. Shi et al. (2024a) target the advantage function directly rather than the action-value function and obtain a faster minimax rate, in the same spirit that *advantage-weighted regression* and *implicit Q-learning* replace the Q-function with smoother surrogates for offline policy improvement. The DR-AIPW scoring is the principal innovation that distinguishes

[262] A parallel literature targets related aspects of the same problem. Nie et al. (2021) develop an advantage doubly-robust estimator for the "when-to-treat" subclass of dynamic regimes and prove the first dynamic analog of the Athey and Wager (2021) static regret bound. Shi et al. (2024a) extend to infinite-horizon offline reinforcement learning, showing that the advantage function is smoother than the Q-function and yielding a strictly faster minimax rate than fitted-Q-iteration. Luckett et al. (2020) introduce V-learning for infinite-horizon mobile-health regimes, bypassing Q-estimation by directly optimizing the value function via the Bellman estimating equation.

the Sakaguchi guarantee from those purely-RL offline methods, which lack the doubly-robust property and so require correct specification of either the outcome model or the propensity to attain the parametric rate.

15.4 Causal Bandits

Sections 15.2 and 15.3 consider multi-stage problems where the state evolves under the influence of past treatments. This subsection collapses the horizon to a single stage but expands the structure of the action set, replacing the opaque arms of a standard multi-armed bandit with interventions on a known causal graph. The resulting "causal bandit" setting was introduced by Bareinboim et al. (2015) for the case of unobserved confounders between the agent's choice and the reward, and formalized into a regret theorem by Lattimore et al. (2016).

Bandits under unobserved confounding

The motivating example of Bareinboim et al. (2015) is a "greedy casino" with two slot machines M_1 and M_2. An unobserved binary variable D (the player is drunk) and an unobserved binary variable B (the chosen machine is blinking) jointly determine the gambler's natural arm choice $X \in \{M_1, M_2\}$ via a structural equation $X = f(D, B)$. Both confounders also affect the reward Y. The casino parameters are chosen so that the observational distribution and the interventional distribution are both flat,

$$\mathbb{P}(Y = 1 \mid X = M_j) = \mathbb{P}(Y = 1 \mid \mathrm{do}(X = M_j)) = \tfrac{1}{2}, \qquad j \in \{1, 2\}. \tag{161}$$

Bareinboim et al. (2015) run ϵ-greedy, UCB1, Thompson Sampling, and EXP3 on this environment and observe that all four are statistically indistinguishable from random guessing, with linear cumulative regret. The reason is that both the observational and interventional distributions average out the contribution of the confounders, leaving no signal at all in the average payoff per arm.

The fix proposed in Bareinboim et al. (2015) is to refine the decision objective from the marginal interventional value to the *effect of treatment on the treated*, that is, to condition on the agent's own natural intuition $X = x$ and ask what the payoff would have been under an alternative action,

$$a^* = \arg\max_a \mathbb{E}\big[Y_{X=a} = 1 \,\big|\, X = x\big]. \tag{162}$$

The authors call this the *Regret Decision Criterion*. Equation (162) is generally not identified from observational and interventional data alone, but in the binary case it can be recovered through algebraic combinations of $\mathbb{P}(Y \mid X)$ and $\mathbb{P}(Y \mid \mathrm{do}(X))$, and in the general case through intention-specific randomization that interrupts the agent before action selection. Plugging this counterfactual quantity into Thompson Sampling yields Causal Thompson Sampling (TS_C), which seeds Beta posteriors from observational data via the consistency axiom and biases sampling via the relative-difference-of-confounders (RDC) weighting that suppresses arms whose interventional value varies sharply across contexts. In the greedy casino, TS_C achieves bounded cumulative regret while standard Thompson Sampling grows linearly. The simulation in Section 15.6 implements both the full TS_C of Bareinboim et al. (2015) (Algorithm 1, with consistency-axiom seeding of the off-intuition arm at fractional pseudo-count $c = 0.5$ and RDC weighting $w_a = \mathrm{clip}(1 - |\hat{Q}_1(a) - \hat{Q}_2(a)|, 0.01, 1)$ on running cell means $\hat{Q}_x(a)$) and a minimal context-conditional Thompson sampling (CCTS) baseline that retains only the (x, a) posterior without seeding or weighting. The two-algorithm comparison lets the reader isolate the contribution of the additional Bareinboim components on this specific MDP.

Causal-graph-aware exploration

Lattimore et al. (2016) formalize the gain from exploiting causal structure as a regret theorem. They study the parallel-bandit special case: N binary parents $X_1, \ldots, X_N$ of a binary reward Y, with action set $\mathcal{A} = \{\text{do}()\} \cup \{\text{do}(X_i = j) : i \in [N], j \in \{0, 1\}\}$. After each action the learner observes both Y and the realized values of all non-intervened parents. Their Algorithm 1 spends half the budget on do() to estimate the marginal probabilities $q_i = \mathbb{P}(X_i = 1)$ and uses the post-action observations to credit-assign across all arms. The remaining budget is allocated to the actions whose direct-observation probability is smallest, with the cutoff τ chosen to optimize the worst-case regret. Define the graph-derived hardness

$$m(q) = \min_{\tau \in \{2,\ldots,N\}} \max\left(\tau, \left|\{i : \min(q_i, 1 - q_i) < 1/\tau\}\right|\right) \in [2, N]. \tag{163}$$

Theorem 1 of Lattimore et al. (2016) gives the simple-regret guarantee for Algorithm 1,

$$R_T(\text{Algorithm 1}) = \tilde{O}\left(\sqrt{m(q)/T}\right), \tag{164}$$

and Theorem 2 establishes the matching lower bound $R_T = \Omega(\sqrt{m(q)/T})$ over all algorithms. Standard best-arm-identification procedures that ignore the graph achieve $\Omega(\sqrt{N/T})$ (Lattimore et al., 2016, Theorem 4), so the graph-aware algorithm strictly improves whenever $m(q) < N$ — equivalently, whenever the parent distribution has at least one extreme coordinate. In the balanced case $q = (\frac{1}{2}, \ldots, \frac{1}{2})$ the hardness collapses to $m(q) = 2$ regardless of N; in the degenerate case $q = (0, \ldots, 0)$ the graph provides no help and $m(q) = N$. For general directed acyclic graphs Algorithm 2 of Lattimore et al. (2016) uses a truncated importance-weighted estimator with a mixing distribution η over interventions, achieving simple regret $\tilde{O}(\sqrt{m(\eta)/T})$ where $m(\eta)$ is the analogous graph-derived hardness; for the parallel bandit case $m(\eta) = m(q)$.

In reinforcement-learning vocabulary, the MABUC instance is *causal Thompson sampling*, in which the Beta posterior is indexed not by arm alone but by the agent's natural intuition $X = x$ that conditions the counterfactual reward distribution, and the Regret Decision Criterion is the Bayesian-bandit analog of an advantage estimate under unobserved confounding. The parallel-bandit Algorithm 1 is *graph-aware UCB*, a deterministic budget split that exploits the post-action observations of non-intervened parents as additional samples for arms that were not pulled. The dictionary entry for *Bandits with Unobserved Confounders* in Table 29 translates these primitives one-for-one with their RL counterparts.

The simulation study in Section 15.6 reproduces the $\sqrt{m/T}$ rate for Algorithm 1 on the monotone segment of the hardness grid $m \in \{2, 8, 24\}$ at $N = 50$, contrasted with the asymptotic $\sqrt{N/T}$ rate lower bound for a graph-blind best-arm-identification baseline, and includes the greedy-casino instance of Bareinboim et al. (2015) with both the full TS_C algorithm (consistency-axiom seeding plus RDC weighting) and a minimal context-conditional Thompson sampling baseline to demonstrate the value of causal reasoning under unobserved confounders.

15.5 Adaptive Experimentation and Post-Experiment Inference

The causal-bandit setting of Section 15.4 fixes the data-generating process and the action set and asks how to learn the best intervention. This subsection considers the dual concern. When the data themselves are collected adaptively by a Thompson-sampling-style assignment rule that the experimenter designs, what does optimal design look like, and how should the analyst conduct valid statistical inference on the resulting data? Kasy and Sautmann (2021) answer the design question; Hadad et al. (2021) answer the inference question. The two papers address different parts of the same applied workflow: design a multi-wave adaptive experiment that ends in a policy choice, then deliver confidence intervals on the welfare gain.

Design: exploration sampling for policy choice

Kasy and Sautmann (2021) consider a finite-horizon adaptive experimental design problem with K candidate treatments, T waves of approximately equal size, and a welfare objective equal to the average outcome of the policy chosen at the end of the experiment. Unlike the in-sample welfare objective targeted by classical bandit algorithms, this objective values only the post-experiment policy, not the participants' realizations during the experiment. The authors prove this distinction matters. Standard Thompson sampling, which assigns each treatment d at wave t with probability equal to the posterior probability p_t^d that d is optimal given history, is not rate-optimal for policy choice, because Thompson sampling concentrates on a single top treatment fast enough to starve the suboptimal treatments of the samples needed to verify their suboptimality.

The proposed correction is *exploration sampling*. Replace the Thompson assignment probability p_t^d with a concave transformation that flattens at the top,

$$q_t^d = S_t \cdot p_t^d \cdot (1 - p_t^d), \tag{165}$$

where S_t is a normalizing constant. Two consequences follow. First, no treatment ever receives more than 50% of the sample at any wave, regardless of how confident the posterior becomes. Second, in large samples the assignment shares converge to a non-random vector ρ^d for which the posterior probability of suboptimality decays at the same exponential rate for every suboptimal treatment; power for rejection is equalized. Theorem 1 of Kasy and Sautmann (2021) establishes that exploration sampling attains the best exponential rate of expected policy regret subject to the constraint that the top treatment receives at most half the sample, a constraint that the paper's Proposition 2 shows loses at most a factor of two relative to the fully optimal allocation.

The empirical anchor is a field experiment with Precision Agriculture for Development (PAD) in India, deploying exploration sampling across 17 waves of approximately 600 farmers each, comparing six recruitment strategies for an agricultural extension service. The realized assignment trajectory matches the theoretical prediction. The top treatment receives close to but not above half the sample, and posterior probabilities for suboptimal treatments converge to zero at comparable rates. The full experiment identifies the optimal recruitment strategy with posterior probability above 0.75 after roughly 10,000 participants.

Inference: adaptively-weighted AIPW after the experiment

Once the experiment in Kasy and Sautmann (2021) has run, the analyst still needs valid confidence intervals on the welfare of the chosen policy and on the contrasts between treatments. This is harder than it looks. Hadad et al. (2021) show that the obvious estimators all fail. The sample mean of Y_t within each arm is biased downward whenever a temporary downward fluctuation in the early phase of the experiment causes that arm to be sampled less in later phases. The inverse-propensity-weighted estimator $\hat{Q}^{\text{IPW}}(w) = T^{-1}\sum_t \mathbb{1}\{W_t = w\}Y_t/e_t(w)$ is unbiased but heavy-tailed, because the inverse weights $1/e_t(w)$ grow without bound as the bandit algorithm down-weights an arm. The vanilla augmented inverse-propensity score

$$\hat{\Gamma}_t^{\text{AIPW}}(w) = \hat{m}_t(w) + \frac{\mathbb{1}\{W_t = w\}}{e_t(w)}\big(Y_t - \hat{m}_t(w)\big) \tag{166}$$

inherits the same heavy-tailed behavior when averaged uniformly across time, because the conditional variances of its components depend on $1/e_t(w)$ and can diverge as the bandit assignment probabilities approach zero.

The solution proposed in Hadad et al. (2021) is to non-uniformly average the AIPW scores using history-adapted evaluation weights $h_t(w)$ that compensate for the propensity decay. The

adaptively-weighted AIPW estimator is

$$\hat{Q}^{\mathrm{AW}}(w) = \frac{\sum_{t=1}^{T} h_t(w)\,\hat{\Gamma}_t^{\mathrm{AIPW}}(w)}{\sum_{t=1}^{T} h_t(w)}. \tag{167}$$

Their Theorem 2 establishes that for any history-adapted weights satisfying an infinite-sampling condition, a variance-convergence condition, and a Lyapunov-type moment condition, the studentized statistic is asymptotically standard normal. The simplest choice that satisfies these conditions and approximately minimizes asymptotic variance is the constant-allocation-rate weight $h_t(w) \propto \sqrt{e_t(w)}$, which compensates for propensity decay at exactly the right rate. The empirical demonstration in their Section 4 reports near-nominal 95% coverage of confidence intervals constructed from the studentized statistic under Thompson-sampling data collection, while the sample mean and the IPW estimator both fail.[263]

Together, Kasy and Sautmann (2021) and Hadad et al. (2021) close the loop for adaptive experimentation. The first paper says what assignment rule maximizes the welfare of the post-experiment policy choice; the second paper says how to construct valid confidence intervals on the resulting welfare estimates. Both treat the assignment mechanism as a Thompson-family bandit, and both deliver formal optimality results: rate-optimality for the design (Kasy and Sautmann, 2021, Theorem 1) and asymptotic normality for the inference (Hadad et al., 2021, Theorem 2). The use of an RL algorithm at the design stage of a field experiment, with causal-inference guarantees at the inference stage, is the cleanest example in this chapter of the RL-for-CI narrative bridging from theory to deployed practice.

In reinforcement-learning vocabulary, exploration sampling is a *welfare-objective Thompson-sampling variant* that trades in-sample cumulative-regret optimality for post-experiment power equalisation across suboptimal arms. The adaptively-weighted AIPW estimator is the *propensity-decay-robust off-policy-evaluation estimator* that any reinforcement-learning pipeline collecting adaptive data needs in order to recover valid inference from its own logged data, whether the adaptive collection is a contextual bandit, an online RL algorithm with an off-policy correction, or a preference-elicitation loop for RLHF.

15.6 Simulation Studies

This subsection presents two computational experiments that ground the chapter's central claims. The first targets Section 15.2: dynamic double machine learning on a two-stage structural nested mean model delivers $\sqrt{n}$ confidence intervals on the dynamic treatment effect parameters, while a naive panel regression and a marginal-structural-model with stabilized inverse-probability-of-treatment weights both fail. The second targets Section 15.4: on the parallel-bandit family the graph-aware causal-bandit algorithm of Lattimore et al. (2016) attains $\tilde{O}(\sqrt{m(q)/T})$ simple regret on the monotone region $m(q) \in \{2, 8, 24\}$ of the hardness grid while a graph-blind best-arm-identification baseline stays well above the asymptotic $\sqrt{N/T}$ rate lower bound, and on the greedy-casino MABUC instance of Bareinboim et al. (2015) both the full TS_C algorithm and a minimal context-conditional Thompson sampling baseline achieve bounded cumulative regret while standard Thompson sampling grows linearly.

[263] Bibaut et al. (2021) extend the adaptively-weighted AIPW framework to contextual adaptive experiments, where the assignment rule may depend on per-unit covariates. Their CADR estimator inversely weights each canonical-gradient term by an adaptively estimated conditional standard deviation, producing the first asymptotically-normal post-bandit estimator that works with contextual bandit data. The construction requires no outcome-model consistency, only that the conditional-standard-deviation estimator be measurable with respect to past data.

Sim 1: dynamic DML on a 2-stage SNMM

The data-generating process follows a two-stage Markovian setup with high-dimensional state, confounded propensities, and treatment-confounder feedback. State dimension $p = 20$ with sparse outcome coefficients on $s = 5$ coordinates. At stage one, $X_1 \sim \mathcal{N}(0, I_p)$ and the binary treatment $T_1 \sim \text{Bernoulli}(\sigma(\gamma^\top X_1))$ with $\|\gamma\|_2 = 1.5$ supported on the first s coordinates. The period-1 state shock $\eta_1 \sim \mathcal{N}(0, \sigma_\eta^2 I_p)$ with $\sigma_\eta = 0.6$ drives the next state $X_2 = BX_1 + \alpha T_1 + \eta_1$ with $\|B\|_{\text{op}} = 0.5$ and $\alpha = e_1$, after which $T_2 \sim \text{Bernoulli}(\sigma(\gamma^\top X_2))$. The outcome

$$Y = \psi_1^* T_1 + \psi_2^* T_2 + \mu^\top X_1 + \nu^\top \eta_1 + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, 0.5^2), \tag{168}$$

has true structural parameters $(\psi_1^*, \psi_2^*) = (1.0, 0.5)$ and the outcome-coupling vector $\nu = 2.0 \cdot \gamma/\|\gamma\|$ is aligned with the propensity direction, so that η_1 enters Y and also drives the period-2 propensity through X_2. This is the canonical treatment-confounder-feedback channel: conditional on (X_1, T_1), T_2 is correlated with Y's residual through η_1.

Three estimators are compared. *Naive OLS* regresses Y on (T_1, T_2, X_1), omitting the time-varying confounder X_2. *MSM with stabilized IPTW* estimates the propensity at each stage by L1-regularized logistic regression, constructs stabilized weights as the product of marginal-over-conditional propensity ratios at each stage, and fits a weighted least-squares regression of Y on (T_1, T_2). *Dynamic DML* implements Algorithm 1 of Lewis and Syrgkanis (2021): at each stage cross-fitted Lasso outcome regressions and L1-logistic propensity models on five folds, residualized at the appropriate history, with $\hat{\psi}_2$ recovered from the period-2 residual-on-residual moment and $\hat{\psi}_1$ from the peeled-off period-1 moment per Theorem 2 of the paper. Standard errors use the Z-estimator influence function $\hat{\sigma}^2(\hat{\psi}_t) = \text{Var}(\tilde{T}_{t,t}\, \hat{r}_t)/\hat{J}_{t,t}^2$. Two hundred Monte Carlo replications per configuration over the sample size grid $n \in \{250, 500, 1000, 2000, 4000\}$.

Table 30 and Figure 36 report the results.[264] The naive panel regression carries a bias of approximately 0.93 on $\hat{\psi}_2$ that does not shrink with n, and its 95% confidence intervals never cover the truth. The MSM estimator is asymptotically consistent but converges slowly: its bias on $\hat{\psi}_2$ falls from 0.68 at $n = 250$ to 0.15 at $n = 4000$, but coverage of stabilized-weight sandwich confidence intervals stays well below nominal across the grid. Dynamic DML achieves the asymptotic guarantee. Bias falls from 0.06 to less than 0.005 on $\hat{\psi}_1$ and from 0.03 to 0.001 on $\hat{\psi}_2$. RMSE shrinks at the $\sqrt{n}$ rate predicted by Theorem 4 of Lewis and Syrgkanis (2021). Coverage of the influence-function confidence intervals is uniformly close to the nominal 95% level across the sample-size grid.

Table 30: Sim 1, dynamic DML on a two-stage SNMM. Bias, root-mean-square error, and 95%-CI coverage for each estimator at $n = 4000$, over 200 Monte Carlo replications. True parameters $\psi_1^* = 1.0$, $\psi_2^* = 0.5$.

	$\psi_1^* = 1.00$			$\psi_2^* = 0.50$		
Method	Bias	RMSE	Cov	Bias	RMSE	Cov
Naive OLS	-0.191	0.197	0.01	+0.933	0.934	0.00
MSM/IPTW	+0.019	0.103	0.88	+0.146	0.192	0.73
Dynamic DML	-0.005	0.046	0.96	-0.001	0.018	0.93

Sim 2: causal bandits on the parallel-bandit family

The data-generating process matches the parallel-bandit setting of Lattimore et al. (2016, §3). The reward Y depends on $N = 50$ binary parents $X_1, \ldots, X_N$ through a single high-leverage

[264] Sim 1 source: `ch10b_rl_for_ci/sims/dynamic_dml_snmm.py`.

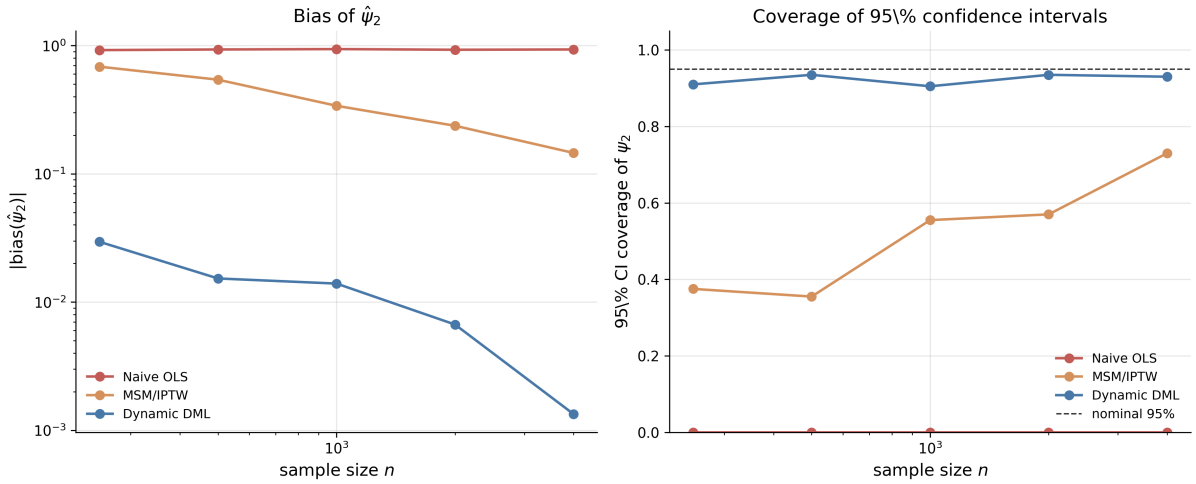

Figure 36: Sim 1, dynamic DML on a two-stage SNMM. Left panel: absolute bias of $\hat{\psi}_2$ versus sample size n (log-log). Right panel: empirical coverage of nominal 95% confidence intervals on ψ_2 versus n. Dotted reference line is the nominal coverage level. Each point averages 200 Monte Carlo replications under the data-generating process in Equation (168).

coordinate: $\mathbb{E}[Y \mid X] = 0.5 + \epsilon$ if $X_w = 1$ and $0.5 - \epsilon$ otherwise, with $\epsilon = 0.3$ and the index w drawn uniformly at random per replication. The action set has $2N + 1 = 101$ arms: the empty intervention do() and each binary intervention do($X_i = j$). Under the observational distribution each parent X_i is Bernoulli with mean q_i; the graph-derived hardness

$$m(q) = \min_{\tau \in \{2,\ldots,N\}} \max(\tau,\ |\{i : \min(q_i, 1 - q_i) < 1/\tau\}|) \in [2, N] \tag{169}$$

is controlled by setting exactly m^* coordinates of q to the extreme value 0.05 and the remainder to 0.5, giving $m(q) = m^*$ by construction.

Four algorithms are compared. *Successive Reject* (Audibert and Bubeck 2010, as in Lattimore et al., 2016) is the graph-blind best-arm-identification baseline that allocates the budget across all $2N + 1$ arms using the standard logarithmic schedule. *Lattimore Algorithm 1* spends $T/2$ rounds on do() to estimate the parent propensities and the conditional means $\hat{\mathbb{P}}(Y \mid X_i = j)$ that identify each direct-causal arm in the parallel graph, then allocates the remaining $T/2$ rounds uniformly across the unbalanced arms whose direct-observation probability falls below $1/\hat{\tau}^*$. *Context-conditional Thompson sampling* (CCTS) is the minimal causal baseline: it maintains a Beta posterior indexed by (x, a), seeded along the on-intuition diagonal $a = x$ from observational data, and at each step samples from the intuition-conditional posterior with straight argmax over arms. *Full* TS_C of Bareinboim et al. (2015) Algorithm 1 augments CCTS in two ways. First, each observational sample $(X = x, Y = y)$ also contributes a fractional pseudo-count $c = 0.5$ to the off-intuition cell $(x, a \neq x)$ via the consistency axiom of Pearl (2009) §3.6.[265] Second, the agent maintains running empirical means $\hat{Q}_x(a)$ for each (x, a) cell and weights each posterior sample θ_a by the RDC factor $w_a = \text{clip}(1 - |\hat{Q}_1(a) - \hat{Q}_2(a)|, 0.01, 1)$, suppressing arms with high cross-context disagreement. The vanilla Thompson sampling baseline ignores intuition entirely and tracks a single Beta posterior per arm.

For the simple-regret experiments two thousand Monte Carlo replications are run on a grid of hardness values $m^* \in \{2, 8, 24, 48\}$ at horizon $T = 400$, and on a grid of horizons $T \in \{100, 200, 400, 800, 1600\}$ at fixed hardness $m^* = 2$. For the MABUC experiment five

[265]The paper seeds the on-intuition cell $(x, a = x)$ at full count and leaves the off-intuition cell to be learned online. The fractional off-intuition seeding $c = 0.5$ used here is one operationalization that extends the seeding step to the off-intuition cell under a weak consistency assumption; the choice of c is disclosed but not optimized.

hundred replications are run at $T = 1000$ using the greedy-casino payoffs of Bareinboim et al. (2015, Table 1): under each of two unobserved-confounder configurations $X \in \{0, 1\}$, following the agent's natural intuition yields payoff 0.1 and going against it yields 0.5. The observational distribution and the experimental distribution both average out to $0.5 \times 0.1 + 0.5 \times 0.5 = 0.3$ per arm, making standard Thompson sampling unable to distinguish the arms in expectation.

Table 31 and Figure 37 report the results.[266] The graph-blind Successive Reject algorithm's simple regret is essentially flat in m^* at approximately 0.28, well above the asymptotic rate lower bound $\sqrt{N/T} \cdot \epsilon \approx 0.106$ for graph-blind algorithms; with $T = 400$ and $K = 2N + 1 = 101$ arms the finite-T constants dominate the rate since $T \ll K \log K$, so the baseline is bottlenecked by the cost of exploring all $2N + 1$ opaque arms. The graph-aware algorithm achieves regret 0.024 at $m^* = 2$ and rises approximately as $\sqrt{m^*/T}$ for $m^* \in \{2, 8, 24\}$, in line with Theorem 1 of Lattimore et al. (2016); the trend reverses at $m^* = 48$, where regret drops to 0.071 rather than continuing to grow. The non-monotonicity is a finite-sample artefact of the single-coordinate reward construction: at $m^* = 48$ nearly all 50 parents are unbalanced, so the optimal-coordinate arm is included in the phase-2 allocation with probability close to one and is estimated directly, whereas at intermediate m^* the optimal coordinate falls into the balanced set roughly half the time and is recovered only through the noisier phase-1 regression on observational data. In the horizon-sweep panel at fixed $m^* = 2$, Lattimore Algorithm 1's regret falls from 0.13 at $T = 100$ to less than 0.001 at $T = 1600$, while Successive Reject's regret falls from 0.31 to only 0.14 over the same range. In the greedy-casino MABUC panel, vanilla Thompson sampling accumulates cumulative regret 200.5 at $T = 1000$ (linear in T, as predicted), CCTS accumulates only 0.66 over the same horizon (a 305× improvement), and the full TS_C algorithm of Bareinboim et al. (2015) accumulates 4.49 (a 45× improvement over vanilla). On this specific MDP, CCTS dominates the full TS_C by a factor of approximately 7× in final cumulative regret. The pattern is consistent with the structure of the greedy casino: the off-intuition arm's true payoff (0.50) exceeds the on-intuition payoff (0.10), so the fractional consistency-axiom seed transferred from on-intuition observations attaches a pessimistic prior to the (high-value) off-intuition cell that the agent must then unlearn through experimental pulls. The RDC weighting compounds the effect by suppressing both arms symmetrically once the running $\hat{Q}_x(a)$ estimates reveal the cross-context flip, which slows convergence but does not change the argmax ordering. The simpler context-conditional posterior dominates here because the (x, a) cells are independently identifiable from experimental data alone, and the additional Bareinboim components do not exploit information beyond what the (x, a) posterior already captures on this two-context, two-arm instance.

Table 31: Sim 2, causal bandits on the parallel-bandit family. Simple regret (mean, standard error in parentheses) for the graph-blind Successive Reject baseline and the graph-aware Lattimore Algorithm 1 across the graph-hardness grid, at horizon $T = 400$, $N = 50$ parents, gap $\epsilon = 0.3$, two thousand Monte Carlo replications per cell.

Method	$m = 2$	$m = 8$	$m = 24$	$m = 48$
Successive Reject	0.281 (0.002)	0.281 (0.003)	0.295 (0.005)	0.318 (0.006)
Lattimore Alg 1	0.024 (0.002)	0.073 (0.003)	0.125 (0.004)	0.071 (0.004)

Both simulations corroborate the theoretical claims of the underlying papers. Sim 1 shows that the Lewis-Syrgkanis recursive residualisation moment delivers the $\sqrt{n}$ asymptotic normality predicted by Theorem 4 of Lewis and Syrgkanis (2021), even though the state is high-dimensional, the propensities are confounded, and naive panel regression with the same nuisance learners is biased. Sim 2 shows that the $\sqrt{m(q)/T}$ trend of Lattimore et al. (2016, Theorems 1–2) holds on the monotone segment $m^* \in \{2, 8, 24\}$ of the hardness grid (the trend reverses at

[266] Sim 2 source: `ch10b_rl_for_ci/sims/causal_bandit_parallel.py`.

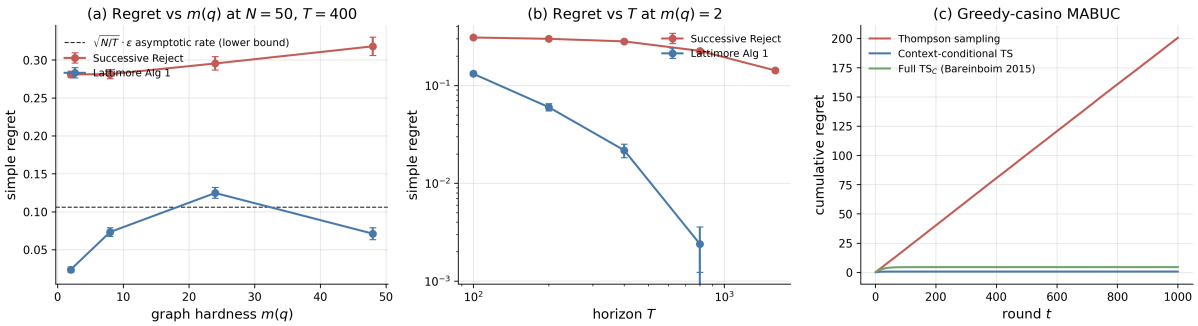

Figure 37: Sim 2, causal bandits. (a) Simple regret versus graph hardness $m(q)$ at fixed horizon $T = 400$ and parent count $N = 50$; the dotted reference line is the theoretical regret lower bound $\sqrt{N/T} \cdot \epsilon$ on the asymptotic rate achievable by any graph-blind algorithm, not an upper bound on the Successive Reject baseline at finite T. Observed Successive Reject regret sits well above this line because $T = 400 \ll K \log K$ at $K = 101$, so finite-T constants dominate the rate. (b) Simple regret versus horizon T at fixed hardness $m(q) = 2$, log-log axes. (c) MABUC greedy-casino panel under the unobserved-confounder instance of Bareinboim et al. (2015, Table 1): vanilla Thompson sampling grows linearly because the observational and interventional marginals are both flat; context-conditional Thompson sampling (CCTS) and the full TS$_C$ algorithm of Bareinboim et al. (2015) both remain bounded by conditioning on the agent's natural intuition, with CCTS slightly ahead on this instance. Error bars and shaded bands are 95% confidence intervals; two thousand Monte Carlo replications for (a)–(b), five hundred for (c).

$m^* = 48$ due to a finite-sample artefact of the single-coordinate reward construction discussed above), and that the MABUC counterfactual reasoning of Bareinboim et al. (2015) converts a hopeless bandit problem into an almost-trivial one for any context-conditional posterior. The Bareinboim-Forney-Pearl TS$_C$ refinements (consistency-axiom seeding of the off-intuition arm and RDC bias weighting on running cross-context disagreement) introduce a mild slowdown on this specific two-arm two-context instance because the off-intuition fractional seed transfers a low-payoff prior to a high-payoff cell, but the substantive linear-versus-bounded gap relative to vanilla Thompson sampling holds for both context-conditional variants. The chapter's broader thesis is that these are the same machine learning machinery applied at the design stage and the inference stage of dynamic causal-inference problems, brought to bear under the identification assumptions catalogued in Section 15.1.

15.7 Discussion

The five sections of this chapter share one underlying claim. Dynamic causal inference and reinforcement learning are not separate enterprises that happen to use similar notation; they are the same enterprise applied to different data sources. The recursion in Section 15.1 is Bellman's. The Lewis-Syrgkanis sequential residualisation is Robins's g-estimation with cross-fitting added. The Sakaguchi backward-induction AIPW is the offline-RL Q-learning update with doubly-robust scoring. The Lattimore causal bandit is the multi-armed bandit with the action set replaced by interventions on a known graph. The Kasy-Sautmann exploration sampling is Thompson sampling reshaped for a different objective function. In each case the move is not to invent a new algorithm but to recognize that an algorithm developed in one tradition addresses a question that arose in the other tradition.

Three implications follow for the empirical worker.

First, the rate of convergence of an estimator is determined by the orthogonality of the moment that defines it, not by the field in which the moment was first written down. The

orthogonal statistical learning theory of Foster and Syrgkanis (2023) unifies the dynamic-DML rate calculations of Section 15.2 with the policy-learning regret bounds of Section 15.3 and the post-bandit-inference asymptotic normality of Section 15.5. The common requirement is that nuisance estimation errors enter the target estimator at second order; the common consequence is that any sufficiently fast machine learner can be used for the nuisance components without sacrificing the parametric rate on the structural target. This is the modern semiparametric efficiency story, transported into reinforcement learning territory.

Second, identification is prior to estimation. The recursions in this chapter all rely on sequential ignorability, positivity, and consistency. When those assumptions hold by design (a sequential multiple-assignment randomized trial, an adaptive field experiment with a known assignment mechanism, a known causal graph), the chapter's techniques deliver $\sqrt{n}$ inference on dynamic causal quantities. When they fail, the recursions still run but they no longer estimate causal quantities; the proxy, instrument, sensitivity, and mediator strategies of Chapter 14 are the recovery routes. The economist's instinct to ask first what identifies the parameter of interest, then what estimates it, applies unchanged here.

Third, the bridge runs in both directions. Several of the most useful results in modern reinforcement learning are descendants of biostatistical and econometric reasoning rather than purely computer-scientific. The advantage doubly-robust estimator of Nie et al. (2021) is the dynamic generalization of the Athey-Wager policy-learning estimator. The adaptively-weighted AIPW of Hadad et al. (2021) is Robins-style augmented inverse-propensity weighting reweighted for adaptive data collection. The recursive Riesz representers of Chernozhukov et al. (2023) are an automation of the locally-robust correction that Robins introduced in g-estimation. The traffic of ideas is two-way, and a chapter framed as "reinforcement learning for causal inference" could equally be read as "causal inference recovers, validates, and extends reinforcement learning under identification."

Open problems remain in several directions. The dynamic-DML rate of Lewis and Syrgkanis (2021) is established for parametric structural nested mean models with linear blips; extensions to fully nonparametric blip functions, to the high-stake combination of large state spaces and adaptively collected data, and to settings where the policy class itself is learned from the data (rather than pre-specified) are active research topics. The matching minimax bound of Lattimore et al. (2016) applies to the parallel-bandit setting; for general directed acyclic graphs the upper bound is established but the lower bound is not. The adaptively-weighted AIPW of Hadad et al. (2021) assumes a known assignment mechanism; relaxing this to allow the analyst to recover valid inference from contextual bandit data collected by a black-box system is the contribution of Bibaut et al. (2021), but the corresponding practical guidance remains to be developed. Each of these open problems is, at the methodological level, a question about how to import additional machine-learning machinery into a causal-inference recursion without losing the orthogonality property that makes the rate work.

A final remark on practice. The PAD India field experiment of Kasy and Sautmann (2021) and the Tennessee Project STAR re-analysis of Sakaguchi (2024) are the chapter's clearest demonstrations that the theory has practical purchase: a reinforcement-learning algorithm at the design stage with a causal-inference estimator at the inference stage produces valid welfare gains on real participants in real settings. The chapter's broader message is that this combination is no longer exotic. It is the natural next step once one accepts that dynamic causal inference and reinforcement learning are doing the same thing under different names, and that the modern machine-learning toolkit, applied carefully, can serve both at once.

16 Quantile, Robust and Constrained Reinforcement Learning

The preceding chapters optimized expected returns. This chapter relaxes that assumption in three directions: tracking full return distributions rather than means (distributional RL),

imposing constraints on secondary objectives (constrained MDPs), and hedging against model misspecification (robust MDPs).

16.1 Distributional Reinforcement Learning and Risk Measures

Standard reinforcement learning characterizes a policy π by the expected return $V^\pi(s) = \mathbb{E}^\pi[\sum_{t=0}^{\infty} \gamma^t R_t \mid S_0 = s]$. Distributional reinforcement learning instead maintains the full random variable $Z^\pi(s,a) = \sum_{t=0}^{\infty} \gamma^t R_t$ whose expectation is $Q^\pi(s,a)$. Bellemare et al. (2017) showed that the Bellman equation has a distributional counterpart that propagates entire distributions rather than expectations, and that this distributional operator contracts in the Wasserstein metric.

Let $\mathcal{Z}$ denote the space of return-distribution functions mapping state-action pairs to distributions over $\mathbb{R}$. The distributional Bellman operator $\mathcal{T}^\pi$ is defined by

$$\mathcal{T}^\pi Z(s,a) \overset{D}{=} R(s,a) + \gamma Z(S', A'), \quad S' \sim P(\cdot|s,a),\ A' \sim \pi(\cdot|S'), \tag{170}$$

where $\overset{D}{=}$ denotes equality in distribution. Bellemare et al. (2017) proved that $\mathcal{T}^\pi$ is a γ-contraction in the Wasserstein distance (informally, the minimum cost of reshaping one distribution into another) between distributions,

$$\bar{d}_p(Z_1, Z_2) = \sup_{s,a} d_p(Z_1(s,a),\ Z_2(s,a)), \tag{171}$$

where d_p is the p-Wasserstein distance between univariate distributions. Since $\mathcal{T}^\pi$ is a contraction, iterating it converges to a unique return distribution Z^π under policy π. For quantile representations, minimizing the quantile regression loss (172) is equivalent to minimizing the 1-Wasserstein distance to the Bellman target (Dabney et al., 2018b), so QR-DQN and IQN directly inherit this convergence guarantee.[267]

A standard DQN network outputs a single scalar $Q_\theta(s,a)$ per action and minimizes the squared TD error $(r + \gamma \max_{a'} Q_{\bar{\theta}}(s',a') - Q_\theta(s,a))^2$, where r is the observed reward, s' is the next state, and $\bar{\theta}$ denotes a slowly-updated target network.

Dabney et al. (2018b) introduced Quantile Regression DQN (QR-DQN), whose network instead outputs N values $\theta_1(s,a), \dots, \theta_N(s,a)$ representing quantile locations of the return distribution, each with equal probability $1/N$. The quantile regression loss for a single quantile level $\tau \in (0,1)$ is $\rho_\tau(u) = u(\tau - \mathbb{1}\{u < 0\})$, which penalizes underestimates by a factor of τ and overestimates by $1-\tau$. The full QR-DQN loss sums this over all pairs of current quantiles i and target quantiles j:

$$L_{\mathrm{QR}} = \frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{N} \rho_{\hat{\tau}_i}\big(\underbrace{r + \gamma\, \theta_j^{\mathrm{target}}(s', a^*)}_{\text{target quantile } j} - \underbrace{\theta_i(s,a)}_{\text{current quantile } i}\big), \tag{172}$$

where $\hat{\tau}_i = (2i-1)/(2N)$ is the midpoint of the i-th quantile interval, $\theta_j^{\mathrm{target}}$ comes from the target network, and $a^* = \arg\max_{a'} \frac{1}{N}\sum_j \theta_j(s',a')$ is the greedy action under the mean return.[268]

Dabney et al. (2018a) extended this to Implicit Quantile Networks (IQN). Instead of outputting a fixed set of N quantiles, the IQN network takes a quantile level $\tau \in [0,1]$ as an additional input and outputs the corresponding return value $F_Z^{-1}(\tau; s, a)$. The IQN loss has the

[267] $\mathcal{T}^\pi$ is not a contraction in total variation or KL divergence. The optimality operator $\mathcal{T}^*$ is not a contraction in any distribution metric, though expected values still converge (Bellemare et al., 2017).

[268] The resulting projected operator is a γ-contraction in the ∞-Wasserstein metric (Dabney et al., 2018b).

same pairwise structure, but with quantile levels sampled continuously:

$$L_{\text{IQN}} = \frac{1}{KK'} \sum_{i=1}^{K} \sum_{j=1}^{K'} \rho_{\tau_i}\big(r + \gamma\, F_Z^{-1}(\tau'_j; s', a^*) - F_Z^{-1}(\tau_i; s, a)\big), \tag{173}$$

where $\tau_1, \ldots, \tau_K$ and $\tau'_1, \ldots, \tau'_{K'}$ are independent draws from $U([0,1])$, and K, K' are the number of samples per update. Because the quantile levels are sampled rather than fixed, IQN learns the full continuous quantile function rather than a discrete approximation. This continuous representation is the key bridge to risk-sensitive control.

An agent that maximizes expected return is indifferent between a certain payoff of 100 and a coin flip paying 0 or 200. In many applications this is inadequate: a portfolio manager, a robot near a cliff, or a firm facing bankruptcy all care about the shape of the return distribution, not just its mean. Distributional RL provides the return distribution; what remains is a principled way to convert that distribution into a scalar objective that encodes the desired risk attitude. Yaari (1987) showed that this can be done with a distortion function $h : [0,1] \to [0,1]$ (increasing, $h(0) = 0$, $h(1) = 1$) applied to the cumulative distribution before integrating:

$$\rho_h(Z) = \int_0^1 F_Z^{-1}(\tau)\, h'(1-\tau)\, d\tau. \tag{174}$$

This class includes the expected value ($h(\tau) = \tau$), Conditional Value-at-Risk at level α ($h(\tau) = \min(\tau/\alpha, 1)$), and the Cumulative Prospect Theory (Tversky and Kahneman, 1992), which overweights tail outcomes relative to their true probabilities. Since IQN can evaluate $F_Z^{-1}(\tau)$ at any τ, each of these risk measures reduces to choosing which quantiles to average over and how to weight them. The network itself does not change; only the sampling distribution of τ at decision time does. Sampling τ from the non-uniform distribution $\beta(\tau) = h'(1-\tau)$ rather than uniformly yields policies that maximize the distortion risk measure ρ_h.

CVaR at level α (the expected return in the worst α-fraction of outcomes) is the most widely used case. In IQN, sampling $\tau \sim U([0, \alpha])$ instead of $U([0,1])$ yields a CVaR-optimal policy. CVaR can also be optimized exactly via dynamic programming on an augmented state space (Bäuerle and Ott, 2011), or through its equivalence to robust MDPs with adversarial transition reweighting (Chow et al., 2015).[269]

16.1.1 Simulation Study: Risk-Sensitive Inventory Management

A newsvendor MDP with fat-tailed demand illustrates the mechanism. The agent manages inventory over a 10-period horizon with state $s \in \{0, \ldots, 15\}$ (current stock) and action $a \in \{0, \ldots, 5\}$ (order quantity). Demand follows a mixture distribution, $D \sim 0.8 \cdot \text{Poisson}(3) + 0.2 \cdot \text{Poisson}(10)$, creating occasional demand spikes that make risk preferences matter.[270] A single IQN network is trained for 50,000 episodes with $\tau \sim U([0,1])$, then evaluated under two τ-sampling distributions: $U([0,1])$ for risk-neutral and $U([0, 0.05])$ for CVaR_{95}. Table 32 reports results over 50,000 evaluation episodes alongside the exact DP oracle.

The IQN-Neutral policy recovers 93% of the DP oracle's mean return. Switching to CVaR_{95} at decision time trades 1.6 points of mean return for 2.8 points of improved tail performance, achieved by ordering more safety stock (3.30 versus 2.44 units per period).

[269] Risk-averse dynamic programming requires the risk measure to satisfy a recursive nestedness property (the multi-period risk decomposes into nested single-period evaluations, as the Bellman equation does for expected value) (Ruszczyński, 2010). Hau et al. (2023) showed that popular decompositions for CVaR are suboptimal regardless of discretization, correcting earlier claims. Tamar et al. (2015) extended the policy gradient theorem to coherent risk objectives as an alternative to the distributional approach. Prashanth et al. (2016) proved consistency of CPT-value estimation from sampled returns.

[270] Per-step reward: $5 \cdot \min(s + a, D) - 2a - 1 \cdot (s + a - D)^+ - 8 \cdot (D - s - a)^+$. The stockout penalty (8) exceeds the holding cost (1), so risk-averse agents should carry more safety stock.

Table 32: Risk-sensitive inventory management.

Policy	Mean Return	CVaR_{95}	CVaR_{99}	Avg. Order
DP Oracle	40.4	-52.6	-98.3	2.56
IQN-Neutral	37.4	-57.8	-104.0	2.44
IQN-CVaR_{95}	35.8	-55.0	-97.5	3.30

16.2 Constrained Markov Decision Processes

A constrained MDP augments the standard MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ with K auxiliary cost functions $c_k : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ and constraint thresholds q_k. The agent maximizes the primary objective subject to

$$\max_{\pi} \ \mathbb{E}^{\pi}\Big[\sum_{t=0}^{\infty} \gamma^t r(S_t, A_t)\Big] \quad \text{subject to} \quad \mathbb{E}^{\pi}\Big[\sum_{t=0}^{\infty} \gamma^t c_k(S_t, A_t)\Big] \le q_k, \quad k = 1, \dots, K. \tag{175}$$

Altman (1999) showed that this problem becomes a linear program when reformulated over *occupation measures* (the discounted fraction of time spent in each state-action pair). The occupation measure of policy π is

$$\nu^{\pi}(s, a) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(S_t = s, A_t = a \mid \pi), \tag{176}$$

which satisfies the flow conservation constraints (probability flowing into each state from transitions equals probability flowing out via actions)

$$\sum_{a} \nu(s, a) = (1 - \gamma)\mu_0(s) + \gamma \sum_{s', a'} P(s|s', a')\nu(s', a') \quad \text{for all } s \in \mathcal{S}. \tag{177}$$

Since both the objective and constraints are linear in ν, and the feasible set forms a polytope,[271] the CMDP becomes a standard LP. By Carathéodory's theorem,[272] optimal CMDP policies mix at most $K + 1$ deterministic policies for K constraints.

The LP formulation yields *strong duality* under Slater's condition.[273] The dual problem is

$$\min_{\lambda \ge 0} \max_{\pi} \ L(\pi, \lambda) = \mathbb{E}^{\pi}\Big[\sum_{t=0}^{\infty} \gamma^t (r(S_t, A_t) - \sum_{k=1}^{K} \lambda_k c_k(S_t, A_t))\Big] + \sum_{k=1}^{K} \lambda_k q_k, \tag{178}$$

and the optimal dual variable λ_k^* is the *shadow price* of the k-th constraint. By the envelope theorem, $\partial V^* / \partial q_k = \lambda_k^*$: the marginal change in optimal value per unit relaxation of constraint q_k. This is the same shadow price that appears in the linear programming dual of resource allocation problems.

Paternain et al. (2019) extended this result beyond the LP formulation. Despite the non-convexity of the objective in policy parameters, they proved that constrained RL has *zero duality gap*: the optimal dual value equals the primal. The proof exploits the convexity of the occupancy measure set and applies a minimax theorem. This means the CMDP can be solved

[271] The feasible occupation measures form a bounded convex set (polytope) defined by non-negativity, the Bellman flow conservation equations, and the K linear constraint inequalities.

[272] Carathéodory's theorem states that any point in the convex hull of a set in $\mathbb{R}^d$ can be written as a convex combination of at most $d + 1$ extreme points. Adding K constraint inequalities introduces up to K additional dimensions along which the optimum may lie in the interior, so the optimal solution is a convex combination of at most $K + 1$ vertices.

[273] Slater's condition requires the existence of a feasible policy π' satisfying all constraints with strict inequality: $\mathbb{E}^{\pi'}[\sum_t \gamma^t c_k(S_t, A_t)] < q_k$ for all k.

exactly via dual ascent on the Lagrange multipliers, even when using parametric policy classes, with an approximation error bounded by the expressiveness of the parametrization.

Achiam et al. (2017) gave the foundational trust-region instantiation. At iteration k, the policy update solves

$$\max_{\pi} \ \mathbb{E}_{s\sim d^{\pi_k}, a\sim\pi}\big[A^{r}_{\pi_k}(s,a)\big] \quad \text{s.t.} \quad J_{C_i}(\pi_k) + \frac{1}{1-\gamma}\mathbb{E}_{s\sim d^{\pi_k}, a\sim\pi}\big[A^{C_i}_{\pi_k}(s,a)\big] \le d_i, \quad \bar{D}_{\mathrm{KL}}(\pi\|\pi_k) \le \delta, \tag{179}$$

where $A^{r}_{\pi_k}$ and $A^{C_i}_{\pi_k}$ are the advantage functions for the reward and the i-th cost, d^{π_k} is the discounted state visitation distribution, and $\bar{D}_{\mathrm{KL}}(\pi\|\pi_k) = \mathbb{E}_{s\sim\pi_k}[D_{\mathrm{KL}}(\pi(\cdot|s)\|\pi_k(\cdot|s))]$. For the exact solution to (179), monotone reward improvement and per-iterate constraint satisfaction are guaranteed.[274,275]

In practice, the dominant method is PPO-Lagrangian, which replaces the trust-region constraint in (179) with the PPO clipped surrogate objective applied to the Lagrangian advantage $\hat{A}^{\lambda}_t = \hat{A}^{r}_t - \lambda \hat{A}^{c}_t$:

$$\max_{\theta} \ \mathbb{E}_{s,a\sim\pi_{\theta_k}}\Big[\min\big(\rho_t \hat{A}^{\lambda}_t,\ \mathrm{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\,\hat{A}^{\lambda}_t\big)\Big], \qquad \rho_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}, \tag{180}$$

where $\hat{A}^{r}_t$ and $\hat{A}^{c}_t$ are generalized advantage estimates computed from two separate critics $V^{r}_{\phi}(s)$ and $V^{c}_{\psi}(s)$ via $\hat{A}_t = \sum_{l=0}^{T-t}(\gamma\lambda_{\mathrm{GAE}})^l \delta_{t+l}$ with TD residuals $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ (analogously for cost). After each policy update, the multiplier is adjusted by projected gradient ascent on the constraint violation:

$$\lambda_{t+1} = \big[\lambda_t + \eta(\hat{J}_C(\pi_\theta) - d)\big]_+. \tag{181}$$

Stooke et al. (2020) refined this update with a PID controller that dampens the multiplier oscillations characteristic of naive dual ascent.[276]

16.2.1 Simulation Study: Carbon-Constrained Production

A factory maximizes manufacturing profit subject to a carbon emissions budget. The state is (inventory level, demand regime), where inventory $\in \{0, \dots, 8\}$ and demand switches between low and high regimes via a Markov chain. The action is (production level, energy source), where production $\in \{0, 1, 2, 3\}$ and energy is dirty (cheap, 3.0 tons CO_2 per unit) or clean (expensive, 0.5 tons per unit). The reward is daily profit; the cost is CO_2 emitted. The carbon budget d is set to 30% of the unconstrained optimum's discounted emissions, and the LP dual yields an analytical shadow price $\lambda^* = 1.20$.[277] The CMDP is

$$\max_{\pi} \ \mathbb{E}^{\pi}\Big[\sum_{t=0}^{\infty}\gamma^t \underbrace{\big(p\min(I_t + a^{\mathrm{prod}}_t,\ D_t) - \kappa_{e_t} a^{\mathrm{prod}}_t - h\,(I_t + a^{\mathrm{prod}}_t - D_t)^+\big)}_{r(S_t,A_t)}\Big] \quad \text{s.t.} \quad \mathbb{E}^{\pi}\Big[\sum_{t=0}^{\infty}\gamma^t \underbrace{\xi_{e_t}\, a^{\mathrm{prod}}_t}_{c(S_t,A_t)}\Big] \le d, \tag{182}$$

[274] Achiam et al. (2017) bound the worst-case constraint degradation at $O(\sqrt{\delta}\,\gamma\varepsilon/(1-\gamma)^2)$, where ε bounds the cost advantage, via Pinsker's inequality. The implemented Constrained Policy Optimization (CPO) algorithm solves a quadratic approximation to (179); the hard guarantee applies to the exact solution.

[275] Subsequent theoretical work sharpened convergence rates for primal-dual CMDP methods: Tessler et al. (2019) proved almost-sure convergence of a three-timescale actor-critic-multiplier scheme (RCPO) to a local Lagrangian optimum; Ding et al. (2020) established the first non-asymptotic $O(1/\sqrt{T})$ rate for both optimality gap and constraint violation (dimension-free, via Fisher information geometry); Liu et al. (2022) improved this to $O(\log(T)/T)$ using policy mirror descent; and Ying et al. (2022) achieved $O(1/T)$ with entropy regularization.

[276] The FSRL library (Liu et al., 2024b) provides a pip-installable implementation of PPO-Lagrangian alongside CPO and TRPO-Lagrangian.

[277] The constrained LP over occupation measures (18 states $\times$ 8 actions = 144 variables) is solved exactly with HiGHS. The shadow price λ^* is the dual variable on the carbon constraint.

where $p = 10$ is the unit price, $\kappa_e \in \{2, 5\}$ is the production cost for dirty and clean energy respectively, $h = 1$ is the holding cost, $\xi_e \in \{3.0, 0.5\}$ is the emission rate, D_t is stochastic demand, and $\gamma = 0.95$.

Table 33 and Figure 38 compare three methods: the constrained LP oracle, unconstrained Q-learning, and Lagrangian Q-learning with dual ascent on (181). Unconstrained Q-learning converges near the DP optimum (return 255 versus 273) but violates the carbon budget by a factor of three (cost 96 versus $d = 31$). The learned multiplier rises from zero, overshoots to $\lambda \approx 3.2$ before settling at $\lambda \approx 1.40$, near the analytical shadow price $\lambda^* = 1.20$. The overshoot is the characteristic oscillation of naive dual ascent that Stooke et al. (2020)'s PID controller is designed to dampen. Because the transient spike drives the policy toward clean energy more aggressively than necessary, the final cost (27) undershoots the budget (31), leaving the Lagrangian agent slightly too conservative (return 180 versus the LP optimum of 186). The LP optimal policy is stochastic, mixing dirty and clean energy at a single state; Q-learning recovers a deterministic approximation.

Table 33: Carbon-constrained production results. Budget $d = 31.35$.

Method	Return	Cost	Budget	λ
LP Oracle	186.4	31.35	Y	1.20
Unconstrained Q-learning	255.2	95.83	N	–
Lagrangian Q-learning	180.3	26.88	Y	1.40

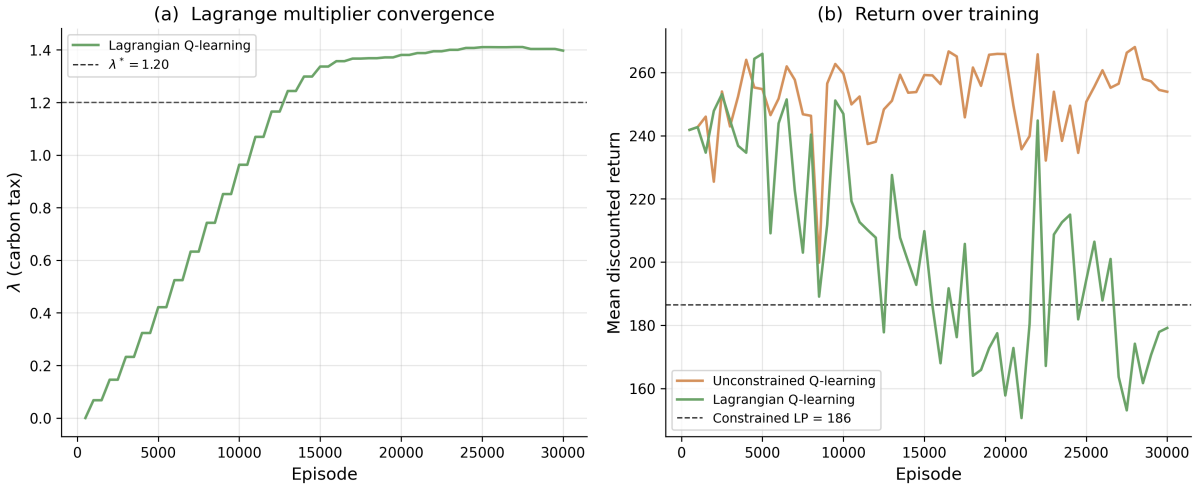

Figure 38: (a) Lagrange multiplier λ over training episodes, with the LP shadow price λ^* as the dashed reference. (b) Mean discounted return for unconstrained and Lagrangian Q-learning, with the constrained LP optimum as the dashed reference.

16.3 Robust MDPs and Ambiguity Aversion

Iyengar (2005) defined the robust Bellman operator

$$(TV)(s) = \max_a \min_{p \in \mathcal{P}(s,a)} \{r(s,a) + \gamma\, p \cdot V\}, \tag{183}$$

where $\mathcal{P}(s,a)$ is an uncertainty set of transition distributions for each state-action pair and $p_0(\cdot|s,a)$ is the nominal transition kernel.[278] Under the rectangularity assumption (the uncer-

[278]The nominal model $p_0(\cdot|s,a)$ is the agent's best estimate of the transition kernel, typically estimated from data or specified by a simulator.

tainty sets are independent across state-action pairs, so the joint set is a Cartesian product), T is a γ-contraction in the sup-norm, so robust value iteration and policy iteration converge with the same guarantees as their standard counterparts (recall Section 4); the only change is substituting T for the standard Bellman operator.[279] For KL balls $\mathcal{P}(s,a) = \{p : \mathrm{KL}(p\|p_0) \le \kappa\}$,[280] the inner minimization has the closed-form solution $q^*(s') \propto p_0(s'|s,a)\exp(-\gamma V(s')/\theta)$, an exponential tilting of nominal transitions toward low-value states, where θ is the Lagrange multiplier on the KL constraint (Nilim and El Ghaoui, 2005). Hansen and Sargent (2001) independently derived the same operator from decision theory as "multiplier preferences," in which the agent maximizes expected utility penalized by the KL cost of distorting beliefs away from a reference model. Petersen et al. (2000) proved that KL-robust MDPs, multiplier preferences, and risk-sensitive control with exponential utility $\mathbb{E}_P[\exp(-\text{cost}/\theta)]$ are three descriptions of the same problem.[281]

16.3.1 Algorithms for Robust RL

The robust Bellman operator (183) is a drop-in replacement for the standard Bellman target. Robust Q-learning replaces the TD target with its worst-case counterpart:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma \min_{p \in \mathcal{P}(s,a)} \sum_{s'} p(s') \max_{a'} Q(s',a') - Q(s,a)\Big], \tag{184}$$

where the inner minimization uses the exponential tilting for KL balls or bisection for TV and chi-squared sets.[282]

For high-dimensional problems where tabular methods and explicit uncertainty sets are infeasible, Pinto et al. (2017) introduced Robust Adversarial Reinforcement Learning (RARL):

$$\max_{\pi_{\text{agent}}} \min_{\pi_{\text{adv}}} \mathbb{E}\Big[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t, a_t^{\text{adv}})\Big], \tag{185}$$

where a_t^{adv} is the adversary's action. Training alternates between updating the agent policy π_{agent} via TRPO or PPO with the adversary fixed, then updating the adversary π_{adv} with the agent fixed. The adversary's action space is problem-specific: joint torques for locomotion, force perturbations for manipulation. The resulting agent policies are more conservative, with wider stability margins; in locomotion tasks, robust agents adopt lower center-of-gravity gaits. Pinto et al. (2017) demonstrated that policies trained against an adversary in simulation transferred to physical robots more reliably than standard PPO policies.

Derman et al. (2021) proved that entropy regularization provides implicit robustness. The

[279] Rectangularity means $\mathcal{P} = \prod_{s,a} \mathcal{P}(s,a)$. Without it, the problem becomes NP-hard (Wiesemann et al., 2013).

[280] A KL ball of radius κ around the nominal contains all distributions "close" to p_0 in an information-theoretic sense. For example, if $p_0 = (0.1, 0.3, 0.4, 0.2)$ across four states, a ball with $\kappa = 0.1$ allows distributions like $(0.15, 0.35, 0.35, 0.15)$ but not $(0.5, 0.5, 0, 0)$, which would be too far from the nominal. Barillas et al. (2009) proposed calibrating κ (equivalently θ) via detection error probabilities. When KL sets are insufficient because p_0 assigns zero probability to relevant outcomes, Wasserstein balls $\{Q : W_p(Q, P_0) \le \epsilon\}$ are the alternative (Mohajerin Esfahani and Kuhn, 2018; Grand-Clément and Kroer, 2021; Yu et al., 2023).

[281] See also Whittle (1981), Jacobson (1973), Fleming and McEneaney (1995). Maccheroni et al. (2006) axiomatized variational preferences, in which the agent evaluates each act under the worst-case belief penalized by a divergence cost: $V(f) = \min_p\{\int u(f)\,dp + c(p)\}$, nesting maxmin EU (Gilboa and Schmeidler, 1989), Hansen-Sargent, and mean-variance as special cases. Strzalecki (2011) showed KL is the unique penalty satisfying a natural invariance axiom; Hansen and Sargent (2024) provided a recent comprehensive treatment.

[282] Sample complexity for learning robust policies scales polynomially in $|\mathcal{S}||\mathcal{A}|$: Panaganti and Kalathil (2022) established the first bounds under (s,a)-rectangular uncertainty for TV, KL, and chi-squared sets; Clavier et al. (2024) improved the rates for model-based approaches.

soft Bellman equation used in SAC,

$$V(s) = \tau \log \sum_a \exp(Q(s,a)/\tau), \tag{186}$$

is equivalent to solving a robust MDP with reward uncertainty set $\{r' : \mathrm{KL}(r' \| r) \le \kappa\}$, where the entropy temperature τ maps directly to the robustness radius κ. Adding a value-regularization term extends this to robustness against transition misspecification.[283]

These three approaches offer different trade-offs. Robust Q-learning requires an explicit uncertainty set but provides exact minimax guarantees for tabular problems. RARL avoids specifying an uncertainty set by learning the adversary, making it practical for continuous control, but provides no formal robustness certificate. Entropy regularization provides implicit robustness at zero additional implementation cost for practitioners already using SAC, but ties the robustness radius to the temperature parameter.

16.3.2 Simulation Study: Consumption-Savings Under Model Mismatch

A consumption-savings agent with constant relative risk aversion (CRRA) utility $u(c) = c^{1-\sigma}/(1-\sigma)$, $\sigma = 2$, receives stochastic income $y \in \{1, \dots, 5\}$ each period and chooses how much to consume versus save at gross return $R = 1.02$, with $\gamma = 0.95$. The robust Bellman equation for this problem is

$$V(w) = \max_{c \in \{0,\dots,w\}} \Big\{ u(c) + \gamma \min_{q:\, \mathrm{KL}(q \| p_0) \le \kappa} \sum_y q(y)\, V\big(R(w-c)+y\big) \Big\}, \tag{187}$$

where w is wealth, c is consumption, and the inner minimization tilts income probabilities toward low-income states via $q^*(y) \propto p_0(y) \exp(-\gamma V(R(w-c)+y)/\theta)$. Seven policies are computed: standard DP under the nominal income distribution $p_0 = (0.05, 0.10, 0.20, 0.30, 0.35)$, robust DP at $\theta = 5$ (moderate) and $\theta = 2$ (high robustness), an oracle that knows the true perturbed distribution $\tilde{p} = (0.30, 0.30, 0.20, 0.10, 0.10)$, and three model-free counterparts trained via tabular Q-learning under the nominal model.[284] All seven policies are then evaluated under both the nominal and perturbed income models.

Table 34 reports discounted returns under both models. Under the nominal model, all policies perform similarly. Under the perturbed model, standard DP degrades by 76% while the $\theta = 2$ robust policy degrades by 72%. The Q-learning and robust Q-learning policies match their DP counterparts, confirming that the model-free algorithms recover the same precautionary savings behavior. Figure 39 shows the mechanism: robust agents consume less at each wealth level, building a precautionary savings buffer against adverse income realizations.

[283] The "Twice Regularized MDP" (R^2-MDP) combines policy regularization (reward robustness) with value regularization (transition robustness). Standard SAC training already implements the reward-robust component; transition robustness requires an additional penalty on the divergence between learned and nominal dynamics models.

[284] Standard Q-learning uses the usual TD target $r + \gamma \max_{a'} Q(s', a')$; robust Q-learning replaces this with the worst-case target (184), using the nominal kernel for the inner minimization (generative model setting). Visit-count learning rates $\alpha = C/(C + N(s,a))$ with $C = 100$ and ε-greedy exploration decaying from 1.0 to 0.05 over 10^5 episodes.

Table 34: Discounted returns under nominal and perturbed income. DP policies are computed via value iteration; Q-learning policies are trained under the nominal model.

Method	Nominal	Perturbed	Degradation (%)
Standard DP	−4.94	−8.69	−76.0
Q-learning	−4.94	−8.70	−76.0
Robust DP (θ=5.0)	−4.94	−8.67	−75.5
Robust Q-learning (θ=5)	−4.95	−8.68	−75.5
Robust DP (θ=2.0)	−4.95	−8.49	−71.6
Robust Q-learning (θ=2)	−4.95	−8.49	−71.5
Oracle	−5.46	−7.60	−39.1

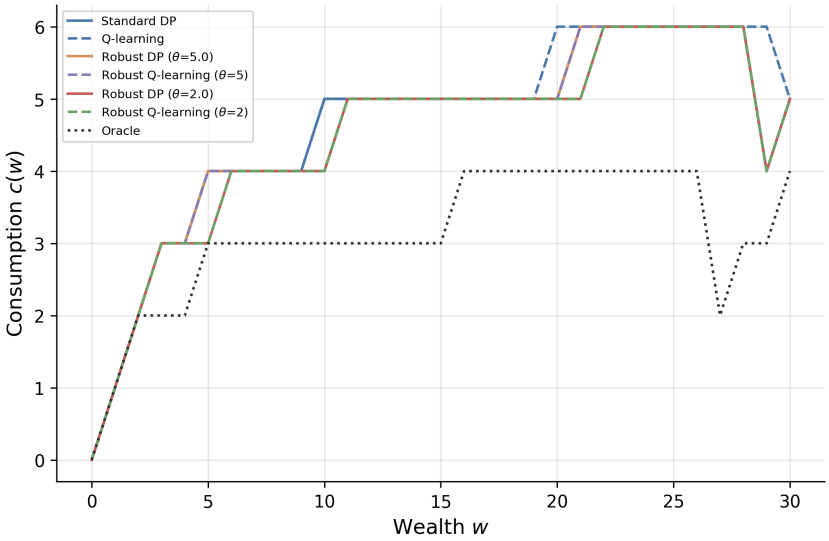

Figure 39: Consumption policy $c(w)$ for each method. Dashed lines show Q-learning policies; solid lines show DP.

17 World Models and Model-Based Reinforcement Learning

17.1 Learning Without Rational Expectations

This chapter studies world models in reinforcement learning, learned forecasters of state transitions and rewards used by an agent to plan its actions. In the language of economics a world model is a perceived law of motion that the agent estimates from data and then plans against, an object that economics has studied since at least the 1980s under names such as adaptive learning, fictitious play, and Berk-Nash equilibrium. The subsection that follows briefly catalogues these analogues before the chapter develops the reinforcement-learning line in detail.

17.1.1 A family of non-rational-expectations learning approaches

The first approach is belief-based learning from game theory, in which an agent maintains a representation of opponents' play or of the environment and refines it as new observations arrive. Fictitious play (Brown, 1951), its smoothed variant (**?**), and stochastic fictitious play under global convergence conditions (**?**) all share with model-based reinforcement learning the basic move of starting from a possibly wrong belief and updating it from data, and they ask the same kinds of convergence question (whether beliefs concentrate, whether they diverge, whether they select among multiple equilibria). The textbook treatments in **?** and **?** carry the main results, and the parallels collected below are in that same family rather than formal

equivalences.

The second is recursive least squares adaptive learning, developed by Marcet and Sargent (1989) and synthesized in **?**. In this approach the agent holds a perceived linear law of motion and updates its coefficients from realized data; convergence to a rational-expectations equilibrium holds when an associated ordinary differential equation is locally stable at the fixed point and fails otherwise. The construction is single-rule and on-policy, and it contains no explicit exploration mechanism, which sharply constrains the regions in which it converges.

The third is genetic algorithm learning (**??**). A population of binary-encoded decision rules evolves under fitness-proportional selection, crossover, mutation, and an election operator that admits offspring only when they would have outperformed their parents on the previous environment. The augmented genetic algorithm converges to the rational-expectations equilibrium of the cobweb in both stable and unstable parameter regions, including those in which recursive least squares diverges, with exploration provided by population diversity rather than by a noise term on a single policy.

The fourth is evolutionary heuristic switching (**?**), in which agents choose among a finite set of forecasting heuristics by a logit on past performance with intensity-of-choice parameter β. As β rises from zero the joint dynamics trace a period-doubling cascade from a stable steady state to high-dimensional attractors, and the source of complexity is the discrete selection rule itself rather than the model class or any exploration operator.

The fifth is misspecified Bayesian learning, formalized as Berk-Nash equilibrium by **?**. The agent's subjective model class need not contain the truth; beliefs concentrate on parameters minimizing Kullback-Leibler divergence to the data-generating process, and at equilibrium the strategy is optimal given the belief while the belief is best-fit given the strategy. Misspecification is permanent rather than a transient phase as in the four approaches above.

The parallel collected here is architectural rather than an equivalence claim; adaptive learning asks whether beliefs converge while model-based reinforcement learning asks whether a learned transition model improves control, and the success criterion that follows the two literatures is correspondingly different.

The five approaches share a common formal vocabulary, drawn from stochastic approximation and fixed-point analysis, and they address related but distinct convergence questions. Single-agent model-based reinforcement learning, the line this chapter touches on next, asks whether a policy approaches optimality on the true environment as the learned model and the policy are updated together. Multi-agent learning in agent-based models, including **?**, Krusell and Smith (1998), and Calvano et al. (2020), asks which rational-expectations equilibrium the joint dynamics select when several exist, and whether the selected equilibrium matches the lab-experimental or empirical attractor. Both questions concern the limiting behavior of non-rational-expectations learning dynamics, the machinery is similar, and the objects differ; one is the optimality gap of a single policy, the other is the basin of attraction of a system of interacting learners. The machinery for both traces back to the algorithmic background developed in §4. With this brief catalogue in place, the chapter turns next to the two 1990 origin papers that opened the model-based line, one from the reinforcement-learning tradition and one from the recurrent-neural-network tradition.

17.2 Two 1990 Origins: Dyna and Differentiable World Models

Two papers published in 1990 introduced what the modern literature now calls a world model. In each, an agent learns a forecaster of its environment from interaction data and uses simulated experience drawn from that forecaster to improve its policy, rather than waiting for real transitions to accumulate. Sutton (1990) arrived at the architecture through dynamic programming and reinforcement learning, framing it as the unification of direct and indirect updates within a single control loop. **?** arrived at it through recurrent neural networks, framing it as a way to make the environment itself differentiable for credit assignment. The two papers do not cite one

another and use different vocabularies, but the architectural commitment is the same. We treat them in parallel rather than in lineage, since each anticipates a distinct branch of the modern literature.

17.2.1 Sutton's integrated architecture

Sutton (1990) proposes Dyna as an architecture that interleaves three processes at every real time step. The agent performs a direct Q-learning update from the observed transition (s, a, r, s'), then updates a learned model $\widehat{P}(s' \mid s, a)$ and $\widehat{r}(s, a)$ from the same transition, then performs K planning updates by sampling previously visited state-action pairs $(\tilde{s}, \tilde{a})$, querying the model for simulated outcomes $(\tilde{r}, \tilde{s}')$, and applying the same Q-update to those synthetic transitions. The ratio K of planning steps to real steps is the lever that controls how heavily the agent exploits its learned model between actions, and is the first explicit statement of the sample-efficiency claim that modern model-based work continues to revisit; the MBPO short-rollout analysis in **?** is in the same family. We defer the algorithm block to §17.3, where we develop Dyna-Q in the notation of this survey.

The architectural openness of Dyna is as important as its sample-efficiency claim. Sutton states explicitly that the learned model can be probabilistic and oftentimes incorrect, with the planner doing useful work as long as the cost of synthetic experience is small relative to the cost of real experience. No structure is imposed on the class of admissible models, on the loss used to fit them, or on the planner that consumes their output. This is the architectural slot that subsequent work fills with tabular counts, Gaussian processes, deep ensembles, recurrent state-space models, and implicit latent dynamics, each plugged into the same outer loop. The conceptual shift from a fixed control algorithm to an open architecture with a model component is what makes Dyna the entry point for the line that runs through §17.7 and §17.8.

17.2.2 Schmidhuber's controller-model architecture

? proposes a paired architecture consisting of a recurrent model network M and a controller network C. The model M receives the controller's outputs together with current observations and is trained, self-supervised, to predict the next inputs to C, including sensory observations and reinforcement signals. The controller C produces actions and is trained jointly with M. Once M is a reasonable forecaster, the environment becomes differentiable from the controller's perspective, since the Jacobian of M stands in for the unknown Jacobian of the true dynamics. Planning is then implemented as gradient descent through the model network, by unrolling C and M together for a fixed horizon and descending on a predicted pain or cost signal, without executing any real actions during the planning phase. This is the first proposal of analytic gradient backpropagation through a learned dynamics model for policy improvement.

In the notation of this survey, let M_θ parameterize the world model with $p_\theta(s_{t+1} \mid s_t, a_t)$ and let C_ϕ parameterize the controller with $\pi_\phi(a_t \mid s_t)$. The model is fit on observed transitions by the negative log-likelihood

$$\mathcal{L}_M(\theta) = -\sum_t \log p_\theta(s_{t+1} \mid s_t, a_t),$$

and the controller is fit by maximizing the expected discounted return under the *learned* model,

$$J(\phi) = \mathbb{E}_{a_t \sim \pi_\phi(\cdot \mid s_t),\, s_{t+1} \sim p_\theta(\cdot \mid s_t, a_t)} \Big[\sum_t \gamma^t r_t \Big],$$

with the planning operator $\phi \leftarrow \phi + \eta \nabla_\phi J(\phi)$ implemented by backpropagating through the H-step unrolled M_θ via reparameterized or score-function gradients. The curiosity unit adds an intrinsic reward $r_t^{\text{int}} = \| s_{t+1} - \mathbb{E}_{M_\theta}[s_{t+1} \mid s_t, a_t] \|^2$ to the extrinsic reward inside J, with

M_θ trained to drive precisely this prediction error toward zero so that regions of state space where M_θ is currently uncertain attract C_ϕ. The idea reappears in the deep era as the intrinsic curiosity of **?**, instantiated with a forward-dynamics network in a learned feature space and used to drive exploration in sparse-reward video games; the full deep realization of the controller-model architecture is the work of **?** and is treated in §17.4.

The contrast with Sutton's Dyna is sharp on two dimensions and shallow on a third. The model class differs, a deterministic tabular Dirac in Dyna versus a differentiable neural network in Schmidhuber. The planner differs, K tabular Q-updates per real step in Dyna versus a gradient step on ϕ through the unrolled M_θ in Schmidhuber. The outer loop is the same: learn M from real data, use M to improve the policy, act in the world.

The symmetry between the two 1990 papers is what makes them an instructive joint origin. Both describe an agent that learns a forecaster of its environment and uses simulated experience from that forecaster to improve its policy, and both do so without the empirical tools that would let the idea work at the scale of pixel inputs or continuous control benchmarks. Both anticipate components that took roughly three decades to mature into reliable practice, intrinsic curiosity in one direction and planning amplification through short model rollouts in the other. In the next subsection we develop Dyna-Q in the notation of this survey and state the empirical claim that the planning-step parameter K trades real samples for synthetic ones, which is the cleanest formal target the 1990 papers leave for the chapters that follow.

17.3 Dyna-Q in Modern Notation

The previous section traced two parallel origins of model-based reinforcement learning in 1990, one from the dynamic-programming tradition and one from the recurrent-neural-network tradition. This section develops the canonical instantiation of Sutton's side of that origin story, the Dyna-Q algorithm, in proper notation. Dyna-Q combines tabular Q-learning with a one-step deterministic model of the environment and applies K planning updates per real interaction step. It is the template that the deep-learning era inherits and modifies, and it is the reference point that the algorithms chapter (§4) points to whenever model-based methods enter the discussion. The section gives the algorithm in formal notation, states the empirical headline from Sutton's original gridworld experiment, and then isolates the two architectural commitments that survive every subsequent reformulation of the idea.

17.3.1 Algorithm

Fix a finite Markov decision process with state space $\mathcal{S}$ and action space $\mathcal{A}$. Let $Q : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ denote the tabular action-value function the agent maintains over the course of interaction. Let $\widehat{P}(s' \mid s, a)$ denote the learned transition model and $\widehat{r}(s, a)$ the learned reward model; together these form the agent's internal forecaster of its environment. The remaining objects are a step size $\alpha \in (0, 1]$, a discount factor $\gamma \in [0, 1)$, an exploration parameter $\varepsilon \in (0, 1)$, and a planning horizon $K \in \mathbb{N}$ counting the number of synthetic updates applied per real step. The agent initializes Q, $\widehat{P}$, and $\widehat{r}$ to arbitrary values and then proceeds as follows.

For each step t:

1. Observe the current state s and select an action $a \in \arg\max_{a'} Q(s, a')$ with ε-greedy exploration, breaking ties uniformly at random.
2. Execute a in the environment and observe the reward r and the next state s'.
3. Direct RL update,

$$Q(s, a) \leftarrow Q(s, a) + \alpha\big[r + \gamma \max_{a''} Q(s', a'') - Q(s, a)\big].$$

4. Model update, $\widehat{r}(s, a) \leftarrow r$ and $\widehat{P}(\cdot \mid s, a) \leftarrow \delta_{s'}$, recording the most recent observed transition as a deterministic prediction.

5. Planning. Repeat K times, on each iteration sampling a previously-visited pair $(\tilde{s}, \tilde{a})$ uniformly from the agent's experience, querying $\widehat{r}(\tilde{s}, \tilde{a})$ and $\widehat{P}(\cdot \mid \tilde{s}, \tilde{a})$ to obtain a synthetic transition $(\tilde{r}, \tilde{s}')$, and applying the same Q-update as in step 3,

$$Q(\tilde{s}, \tilde{a}) \leftarrow Q(\tilde{s}, \tilde{a}) + \alpha\big[\tilde{r} + \gamma \max_{a''} Q(\tilde{s}', a'') - Q(\tilde{s}, \tilde{a})\big].$$

Step 3 is the tabular Q-learning update of Watkins and Dayan (1992a), who prove that under decreasing step sizes satisfying $\sum_t \alpha_t = \infty$ and $\sum_t \alpha_t^2 < \infty$ and infinite visitation of every state-action pair, Q converges with probability one to the optimal action-value $Q^\star$, the fixed point of the Bellman optimality operator $\mathcal{T}^\star$ defined by $(\mathcal{T}^\star Q)(s, a) = \mathbb{E}_{s' \sim P^\star(\cdot \mid s, a)}[r(s, a) + \gamma \max_{a'} Q(s', a')]$. The operator is a γ-contraction in the sup norm, which is the technical ingredient that powers the proof. Step 5 applies the same update with $P^\star$ and r replaced by the learned $\widehat{P}$ and $\widehat{r}$; convergence of the full planning-and-learning loop has no closed-form guarantee in the tabular case beyond the consistency of $\widehat{P}$ and $\widehat{r}$ under the agent's visitation distribution, and the function-approximation setting is studied via Lipschitz-on-model arguments (**?**, discussed in §17.10.2) rather than by a single contraction.

The conceptual content of the indirect-versus-direct distinction collapses into a single observation. Steps 3 and 5 apply identical update rules to identical-looking (s, a, r, s') tuples. The only difference is the provenance of the tuple. In step 3 the tuple is sampled from the true transition kernel of the environment by acting; in step 5 the tuple is sampled from the agent's learned model. Planning, in this architecture, is not a separate algorithm with its own update rule. It is the same update rule applied to model-generated rather than environment-generated experience, and the ratio K is the single lever that controls how heavily the agent leans on its model relative to fresh interaction.

17.3.2 Empirical headline

Sutton (1990) reports a gridworld maze experiment that fixes the empirical claim. The agent navigates a tabular maze with deterministic transitions and sparse reward, learning under three settings of the planning horizon, $K \in \{0, 5, 50\}$. Setting $K = 0$ recovers plain Q-learning, since the planning loop is skipped and only the direct RL update of step 3 fires per interaction. With $K = 50$, the agent reaches near-optimal performance in roughly three episodes; with $K = 0$, the same level of performance requires roughly twenty-five episodes. The ratio of episodes is the original planning-amplification claim. Each real step in the $K = 50$ variant carries the policy-improvement value of about fifty steps in the $K = 0$ variant, at the cost of additional internal computation and a learned model that must be accurate enough for the synthetic updates to be informative. A blocking-maze variant in the same paper shows that a small exploration bonus permits the agent to recover after the environment changes, addressing the obvious failure mode that comes with relying on a learned model.

17.3.3 Two architectural commitments

The first commitment is that the model class is left unspecified. The algorithm box above uses a deterministic one-step Dirac model $\widehat{P}(\cdot \mid s, a) \leftarrow \delta_{s'}$, but this choice is incidental to the architecture. Sutton describes the model as "probabilistic and oftentimes incorrect," which is the conceptual move that allows every subsequent paper to plug a different model class into the same outer loop without changing anything else. The deep era exercises this openness across diverse model classes. **?** use a convolutional variational autoencoder paired with a mixture-density recurrent network, treated in §17.4. The Recurrent State-Space Model that powers the PlaNet and Dreamer lines, treated in §17.5, couples a deterministic recurrent path with a stochastic latent. MuZero, the value-aware end of the spectrum treated in §17.6, dispenses with observation reconstruction entirely and trains the model only against quantities that enter

the planner's value calculation. The MBPO line, treated in §17.7, uses a deep ensemble of probabilistic feed-forward networks. The outer loop in each of these is recognizably Dyna; the model class is the variable that distinguishes them.

The second commitment is that the inner loop applies the same Bellman update on simulated and real data. Step 5 of the algorithm is not a distinct planning operator; it is step 3 with a different source of transitions. This single fact is what permits Dyna to unify direct and indirect reinforcement learning into one architecture rather than two competing approaches. In the dynamic-programming tradition before Sutton, indirect methods estimated a model and then ran value iteration on it as a separate batch computation, while direct methods updated values from interaction without any model. Dyna eliminates the wall between these two camps by routing both kinds of transitions through the same temporal-difference update. The architectural consequence is that the agent does not need to choose between learning from interaction and learning from a model; it can do both, and the ratio K tunes the mixture. Every subsequent model-based method in this chapter inherits this commitment, whether it uses tabular updates, fitted Q-iteration, soft actor-critic, or an MPC controller as its inner loop. Read in economics terms, Dyna is adaptive learning with simulated experience added between observations. The sample-efficiency claim that follows is not a free lunch; simulated experience is useful only where the learned model is accurate enough for the planner's purpose, and most of the failure modes the chapter encounters later are forms of that purpose-relative inaccuracy.

17.3.4 Simulation Study: Planning Amplification in the Blocking Maze

This study is the chapter's first empirical study and is designed to make three claims quantitative. The first is the planning-amplification claim of Sutton (1990): that on the same maze and the same environment-step budget, the value of each real interaction grows roughly in proportion to the number of model-based planning updates applied between interactions. The second is the diagnosis of why plain ε-greedy Q-learning is too slow on a sparse-reward tabular problem of this size and discount, an observation the literature often takes as given without quantification. The third is the model-staleness failure mode that comes with relying on a learned model; the experiment flips the wall mid-experiment to expose whether agents that trust their old model can recover. Figure 40 renders the two phases of the environment side by side.

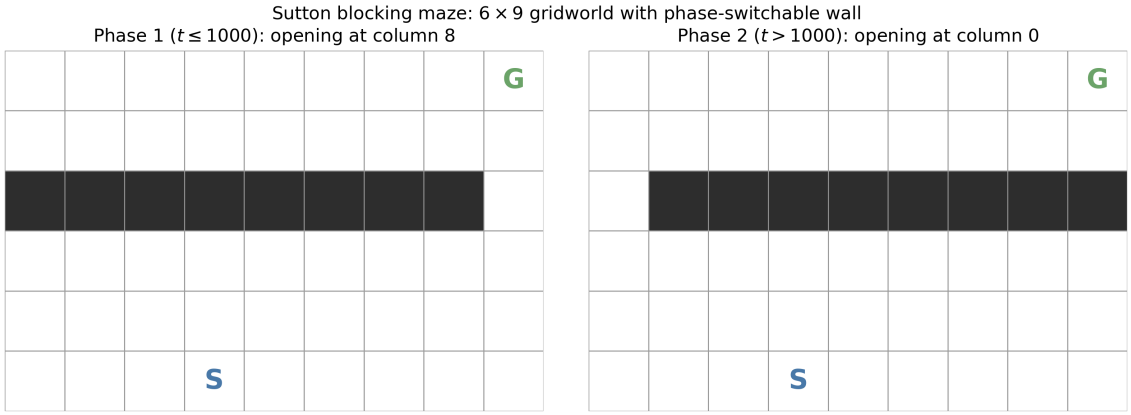

Figure 40: Sutton blocking maze. The agent starts at S and aims for the goal at G; the only opening in the wall along row two shifts from column eight in Phase 1 to column zero in Phase 2 at environment step $t = 1000$. The flip is not signaled to the agent and triggers the model-staleness failure mode that the experiment is designed to expose.

Why plain Q-learning is slow on this maze. The diagnosis is quantitative. With $\gamma = 0.95$, $\alpha = 0.1$, $\varepsilon = 0.1$, Q-values initialized to zero, and a reward signal that is identically zero

everywhere except at the goal, the agent reaches its first goal essentially by random walk; the expected hitting time of a single random walker from S to G on the open part of this 6×9 grid is several hundred steps. Each goal-reach updates exactly one entry of Q by $\alpha \cdot 1 = 0.1$ toward the optimal value γ^{d-1}, where d is the path length from the parent state. Propagating that signal back to the start state requires a chain of subsequent goal-reaches at the right places, and ε-greedy noise at $\varepsilon = 0.1$ diverts the agent off its currently-best path on one in ten steps. The chance of stringing together a long enough sequence of goal-reaches to back the value all the way to S within a few hundred environment steps is correspondingly small. This is the precise form of the slow-per-step-propagation claim that Dyna-Q is designed to address by replaying each successful real transition K times against the model between every pair of real steps.

Setup. The environment is the 6×9 deterministic gridworld of Sutton and Barto (2018), with start at row five column three and goal at row zero column eight. Actions are the four cardinal moves; transitions are deterministic; the reward is one on goal arrival and zero elsewhere; the discount factor is $\gamma = 0.95$. The wall along row two has its only opening at column eight during Phase 1 ($t \leq N_1$ with $N_1 = 1000$) and at column zero during Phase 2 ($N_1 < t \leq N_1 + N_2$ with $N_2 = 2000$). The phase flip happens automatically at $t = N_1$ and is not signaled. The total interaction budget is $N_1 + N_2 = 3000$ environment steps, the per-episode step cap is two hundred, and the experiment runs over thirty independent seeds with $\alpha = 0.1$, $\varepsilon = 0.1$, and zero-initialized Q.

Models. Five agents share the budget. Plain Q-learning with $K = 0$ is the direct-RL baseline, applying only the temporal-difference update of Watkins and Dayan (1992a) per real step. Dyna-Q with $K = 5$ applies five model-based replays per real step from a tabular Dirac model, and Dyna-Q with $K = 50$ applies fifty; both implement the algorithm box of §17.3.1 with the planning-step count set to the named value. Dyna-Q+ with $K = 50$ adds the exploration bonus $\kappa\sqrt{\tau(s,a)}$ to planning targets, with $\tau(s,a)$ the time since the state-action was last visited and $\kappa = 10^{-4}$, instantiating the primitive curiosity mechanism that connects to the prediction-error and ensemble-disagreement variants of §17.10.2. The fifth agent is a faithful realization of the Schmidhuber 1990 controller-model architecture of §17.2.2, with two small multilayer perceptrons: a model network M_θ mapping one-hot state plus one-hot action to next-state logits, fit on observed transitions by cross-entropy, and a controller network C_ϕ mapping one-hot state to action logits, fit by a REINFORCE policy-gradient estimator on imagined trajectories rolled forward through M_θ. Both networks have a single thirty-two-unit hidden layer; planning calls happen every ten real steps with sixteen imagined trajectories of length ten and a moving-average baseline. This is the third 1990-era treatment of the same loop, sitting alongside Sutton's tabular Dyna and the two Dyna-Q variants of $K \in \{5, 50\}$.

Results. Figure 41 and Table 35 report cumulative environment reward across the thirty seeds with mean and standard error bands, with rows in rank order by total reward at $t = 3000$. Dyna-Q at $K = 50$ leads with 52.0 ± 4.2 total goal-reaches, Dyna-Q+ at $K = 50$ follows at 47.0 ± 4.4, Dyna-Q at $K = 5$ at 39.2 ± 4.7, the Schmidhuber controller-model agent at 4.0 ± 0.4, and plain Q-learning closes the field at 3.5 ± 0.5. The top of the table reproduces the planning-amplification claim of Sutton (1990) cleanly under this budget; each real interaction in the $K = 50$ run carries roughly an order of magnitude more policy-improvement weight than the same interaction in $K = 0$, because the synthetic updates of Step 5 propagate the learned reward back through the state graph between every pair of real steps. Dyna-Q+ at $K = 50$ pays a small Phase 1 cost of about five reward units relative to plain Dyna-Q because the exploration bonus eats slightly into exploitation. The cost remains modest because the implementation follows Sutton and Barto (2018) §8.3 and registers all four actions at every visited state with $\tau = 0$ on first encounter, so the bonus drives discovery of untried actions

rather than only perturbing the values of already-tried ones. The Schmidhuber agent tracks plain Q-learning on this maze rather than tabular Dyna-Q because the gradient-based fit of the neural model needs many real samples to specialize from a random initialization, and at three thousand environment steps it has not yet accumulated enough buffer entries near the goal for the REINFORCE estimator to drive the controller toward the discovered path. Phase 2 reveals the recovery story. Dyna-Q at $K = 50$ continues to gain reward at the rate of 9.8 ± 4.0 over the remaining two thousand steps because ε-greedy noise eventually exposes the new corridor and the planning loop quickly retrains the model. Dyna-Q+ matches that rate at 9.6 ± 2.8, with the curiosity bonus accelerating discovery of the newly-open corridor on the opposite side of the wall; the recovery rates are statistically indistinguishable here, and the Phase 1 deficit shrinks but does not close by $t = 3000$. The Schmidhuber agent's Phase 2 gain of 1.5 ± 0.2 is only modestly above plain Q-learning's 0.6 ± 0.2 and tracks the same slow trajectory. The verdict is that the planning-amplification claim of Sutton (1990) reproduces cleanly under this budget, with tabular Dyna-Q at $K = 50$ delivering an order-of-magnitude improvement over $K = 0$, and Dyna-Q+ with faithful untried-action initialization recovering at the same Phase 2 rate as plain Dyna-Q while paying only a small Phase 1 exploration cost. The differentiable Schmidhuber architecture pays the cost of fitting a neural model from scratch on a tabular task it does not need a neural model to represent. The three 1990 commitments, deterministic Dirac model with K planning steps, exploration bonus on a stale-model fix, and a differentiable model with gradient-based planning, sit at three different points on the empirical frontier, with tabular Dyna-Q dominating at this scale and the gradient-based architecture's promise visible only when the state space is large enough that tabular methods cannot follow.

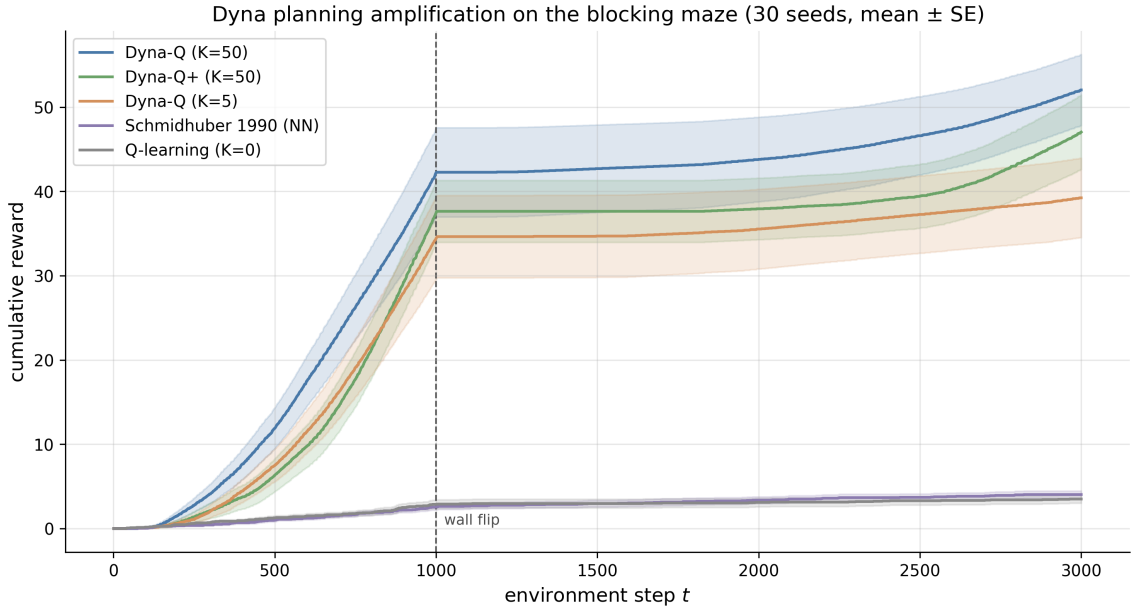

Figure 41: Cumulative reward on the blocking maze for five agents under a shared three-thousand-step environment budget. The dashed vertical line marks the wall flip at $t = 1000$. Lines are means and shaded bands are one standard error across thirty seeds.

17.4 World Models: The Deep Revival

? is the first paper to make latent-space dreaming work at scale on pixel inputs. An agent learns a visual encoder, a recurrent forecaster of the encoder's output, and a controller, with the controller trained against rollouts of the forecaster rather than against real environment frames. The abstract decomposition is a triple of objects with distinct losses, a vision encoder

Table 35: Cumulative reward on the blocking maze. End of Phase 1 is at $t = 1000$; Phase 2 gain is the increase over the remaining two thousand steps; Total is the value at $t = 3000$. Mean ± standard error across thirty seeds.

Agent	End of Phase 1	Phase 2 gain	Total ($t = 3000$)
Dyna-Q (K=50)	42.2 ± 5.3	9.8 ± 4.0	52.0 ± 4.2
Dyna-Q+ (K=50)	37.4 ± 3.7	9.6 ± 2.8	47.0 ± 4.4
Dyna-Q (K=5)	34.4 ± 4.9	4.8 ± 3.0	39.2 ± 4.7
Schmidhuber 1990 (NN)	2.5 ± 0.3	1.5 ± 0.2	4.0 ± 0.4
Q-learning (K=0)	2.9 ± 0.5	0.6 ± 0.2	3.5 ± 0.5

trained against reconstruction, a memory model trained against next-latent likelihood, and a controller trained against imagined returns; this decomposition is the architectural pattern that subsequent deep model-based methods inherit and modify, with the Dreamer line collapsing vision and memory into one jointly trained object and the value-aware line of §17.6 replacing the memory loss outright. On the CarRacing benchmark the resulting controller is the first to cross the conventional solution threshold; on a VizDoom task a controller trained entirely inside the learned forecaster transfers zero-shot to the real environment. The paper is an explicit homage to **?**, the controller-model architecture introduced in §17.2.2, and the line of work it revives is twenty-eight years old. What is new is the toolkit. Convolutional variational autoencoders, mixture-density recurrent networks, and evolution strategies, taken together, make the older architecture empirically tractable at a scale the original could not reach.

Nine recurring terms anchor the rest of the chapter. A *world model* is an estimated model of next states and rewards. A *planning update* is a policy or value update performed using the learned model rather than the environment. A *simulated rollout* is a trajectory generated by the learned model rather than by real interaction. A *latent state* is an internal compressed state used by a neural model, not an observed economic state. A *model loss* is the criterion used to train the world model. A *value-aware loss* is a model loss that cares about preserving values rather than predicting every detail of the next observation. A *short rollout* is a deliberately short simulated trajectory from a real visited state. A *bootstrap* is a value estimate that stands in for rewards beyond the simulated horizon. A *model error* is the gap between the learned model and the true environment, judged by its effect on planning rather than by its statistical fit.

17.4.1 Three components

The vision component V is a convolutional variational autoencoder.[285] The encoder maps an observation $x_t \in \mathbb{R}^{64\times64\times3}$ to a latent vector $z_t \sim \mathcal{N}(\mu(x_t), \sigma(x_t)^2 I)$ with $z_t \in \mathbb{R}^{N_z}$, where $N_z = 32$ for CarRacing and $N_z = 64$ for VizDoom. The decoder is trained jointly with the encoder against the standard ELBO objective,[286] combining a per-pixel reconstruction term with a Kullback-Leibler penalty on the latent. The vision component is trained on observation reconstruction alone, independently of the controller, on a buffer of frames collected by a random policy. The decoder is discarded once training is complete; downstream components see only the encoder's latents. The architectural commitment is that all subsequent processing happens in a low-dimensional vector space rather than in pixel space, which is what makes the rest of the pipeline cheap enough to train and to roll out.

[285] A variational autoencoder compresses high-dimensional observations into a low-dimensional latent vector by training an encoder and a decoder jointly.

[286] Evidence lower bound; the standard variational autoencoder training objective that combines a reconstruction term with a Kullback-Leibler penalty between the encoder's latent and a fixed prior. The Kullback-Leibler term measures the discrepancy between two distributions.

The memory component M is a mixture-density recurrent neural network.[287] Its body is an LSTM with hidden state h_t;[288] its head outputs the parameters of a Gaussian mixture over the next latent, conditional on the current action, the current latent, and the recurrent state, modeling $P(z_{t+1} \mid a_t, z_t, h_t)$ as a mixture of five diagonal Gaussians. A temperature parameter on the mixture controls how aggressively the forecaster samples from its tails, and is exposed as a tuning lever for controller training. The memory component is trained on next-latent likelihood alone, again independently of the controller, using the same buffer of rollouts under a random policy that trained V. The mixture form matters in environments with discrete event structure such as enemy fireballs in VizDoom, where the next latent has multiple plausible values that a unimodal Gaussian forecaster would average over and so render unusable for planning.

The controller C is a single linear layer. It outputs an action $a_t = W_c[z_t, h_t] + b_c$ from the concatenation of the visual latent and the recurrent state. On CarRacing the controller has roughly 870 parameters, compared to about 5 million in V and M together. It is trained by covariance matrix adaptation evolution strategy,[289] evaluating candidate parameter vectors against rollouts drawn from M rather than against real environment frames. Evolution strategies sidestep the need for differentiability through M at the cost of higher sample complexity per parameter, which the linear policy keeps manageable. The controller is the only component whose loss depends on reward, and it is the only component whose training requires interaction, in the form of imagined trajectories through the memory network.

17.4.2 Empirical headline

On CarRacing-v0 the full agent scores 906 ± 21 over 100 random tracks. The benchmark requires an average above 900 to count as solved, and this is the first reported solution. On the VizDoom DoomTakeCover task the controller is trained inside the memory network, with the visual encoder and recurrent forecaster fitted on a buffer of random rollouts and the controller never exposed to real frames during policy search. Transferred zero-shot to the real environment, the controller scores 1092 ± 556 against a solution threshold of 750. The variance is large but the mean clears the threshold, and the policy is constructed without any direct contact with the real environment after the data collection phase.

17.4.3 Architectural reading

The modular training schedule, with V fitted on reconstruction, M fitted on next-latent likelihood, and C fitted on dream rollouts, is the practical workaround for joint end-to-end training, which the paper notes is hard at this scale. Each component has its own loss, its own optimizer, and its own data pipeline, and the three are composed in sequence rather than co-trained. The cost of the workaround is that the representation learned by V is agnostic to what the controller will use it for, and the forecaster learned by M is agnostic to what the controller will reward. The line of work that eventually unifies V and M into a single jointly-trained recurrent state-space model is the PlaNet and Dreamer family, treated in §17.5.

The combination of V and M does the representation work; the controller is tiny by comparison and carries the parameter budget of a linear policy. This anticipates the modern observation that a strong world model lets a small actor produce competitive policies, a point pressed further by Dreamer in §17.5 and by TD-MPC2 in §17.8, where the latent space and the value function carry most of the inductive load and the policy network is relatively light. The training-in-dream-then-transfer protocol on VizDoom is the strongest demonstration to date that a learned

[287] A mixture-density recurrent network is a recurrent network whose output parameterizes a Gaussian mixture over the next latent rather than a single mean; this matters when the next state is genuinely multimodal.

[288] Long short-term memory; a gated recurrent cell that propagates information across time and is the dominant pre-Transformer architecture for sequential data.

[289] A gradient-free black-box optimizer that maintains a Gaussian over candidate parameter vectors and updates the Gaussian by re-fitting to the best-scoring samples each generation.

forecaster can stand in for the environment during policy search, with the agent's only contact with the real environment being the random data collection used to fit V and M in the first place.

17.5 The Recurrent State-Space Model and the Dreamer Line

The previous section described the modular Vision-Memory-Controller architecture of **?**, trained in three separate stages. The Recurrent State-Space Model collapses that sequence into a single jointly trained network and becomes the workhorse architecture of modern world-model reinforcement learning. The line carries through PlaNet, DreamerV1, DreamerV2, and DreamerV3 and is treated here in version-agnostic notation; the differences across versions are summarized at the end of the subsection. The pivot at the end is to the question of what loss the model should be trained against, which the next section picks up.

17.5.1 The Recurrent State-Space Model

Let o_t denote the observation at time t, a_t the action, and r_t the reward. The Recurrent State-Space Model factors the latent state into a deterministic recurrent component h_t and a stochastic latent s_t, coupled by

$$h_t = f_\theta(h_{t-1}, s_{t-1}, a_{t-1}), \qquad s_t \sim p_\theta(s_t \mid h_t).$$

The deterministic path f_θ is a recurrent cell (in practice a Gated Recurrent Unit); the prior $p_\theta(s_t \mid h_t)$ is either a diagonal Gaussian or a product of categoricals, the choice a design parameter rather than a structural commitment. A posterior network $q_\theta(s_t \mid h_t, o_t)$ encodes the current observation into a refined latent for inference at training time. The joint loss for a length-T trajectory has three named terms,

$$\mathcal{L}_\theta = \underbrace{-\sum_t \log p_\theta(o_t \mid h_t, s_t)}_{\text{reconstruction}} \; \underbrace{-\sum_t \log p_\theta(r_t \mid h_t, s_t)}_{\text{reward log-likelihood}} \; + \; \underbrace{\sum_t \mathrm{KL}[q_\theta(s_t \mid h_t, o_t) \,\|\, p_\theta(s_t \mid h_t)]}_{\text{prior-posterior consistency}}.$$

Optionally a latent-overshooting term enforces multi-step consistency in latent space without decoding. The point of the architecture is to separate deterministic memory from stochastic uncertainty so that the model forecasts a distribution over futures while still propagating an information bottleneck through time. The model class is a flexible neural object, but the algorithmic outer loop is recognizably the Dyna template of §17.3, with $\widehat{P}$ and $\widehat{r}$ replaced by jointly trained networks.

17.5.2 Actor-critic in imagination

The policy is trained inside the learned world model rather than from environment interaction directly. An actor $\pi_\phi(a_t \mid h_t, s_t)$ and a critic $V_\psi(h_t, s_t)$ are updated on imagined trajectories rolled forward through the latent dynamics, starting from real states drawn from the agent's replay buffer. The imagined return objective is

$$J(\phi) = \mathbb{E}_{\pi_\phi, p_\theta}\left[\sum_{\tau=t}^{t+H} \gamma^{\tau-t} \hat{r}_\tau + \gamma^{H+1} V_\psi(h_{t+H+1}, s_{t+H+1})\right],$$

where the expectation is taken over actions sampled from the actor and latent states evolved by the world model, H is an imagination horizon, $\hat{r}_\tau$ is the reward predicted by the trained reward head, and the bootstrap term V_ψ supplies a value estimate past the imagination horizon. Because the world-model dynamics are differentiable and the latent samples are reparameterized,

$\nabla_\phi J$ is obtained by backpropagating value gradients through the unrolled latent trajectory. The critic is fit by minimizing a squared error against λ-return targets in the same imagined trajectories.

This is the deep-learning instantiation of the inner planning loop of Dyna-Q, with the model now a flexible neural object and the planner an analytic policy-gradient operator rather than a discrete Q-update. Convergence guarantees are not available in closed form for this setting; the nearest analogue is the Lipschitz contraction bound for model-based value error of **?**, discussed in §17.10.2, which states that value error grows geometrically along the rollout at rate K_F (the Lipschitz constant of the learned dynamics) and remains finite when $\gamma K_F < 1$.[290]

The line trains the world model against likelihood-style losses (reconstruction, reward log-likelihood, prior-posterior consistency), which rewards the model for matching observed distributions irrespective of how those distributions enter the value calculation. Read in economics terms, the recurrent state-space model is a high-dimensional perceived law of motion, fit by likelihood rather than by hand-chosen moments, with a planner that operates entirely on the learned latent. The next section asks whether this is the right objective. The argument, developed by **?** and **?** and instantiated empirically in MuZero, is that the model should be trained against the value functional directly rather than against pixel and reward likelihoods. The Dreamer line achieves its results despite training to the wrong target; the value-aware line asks how much further the field could go if the loss were aligned with the downstream decision.

17.6 What Should the Model Be Trained Against?

Throughout the preceding subsections the model has been trained against a likelihood or reconstruction loss. The Recurrent State-Space Model line uses a pixel reconstruction term, a reward log-likelihood, and a Kullback-Leibler consistency term between prior and posterior; the original Dyna model is a tabular maximum-likelihood estimate of the empirical transition; the controller-model architecture of **?** fits its variational autoencoder and its mixture-density recurrent network to observation and next-latent likelihoods respectively. In each case the loss rewards the model for matching observed distributions, regardless of how those distributions enter the value calculation the agent actually optimizes. The objection raised in the present subsection is that the agent does not care about predicting the next observation perfectly; it cares about getting the value function right. This subsection traces the line that took that observation seriously, from value-aware model learning in **?** through the value-equivalence principle of **??**, the value-targeted regret bound of **?**, the MuZero instantiation of **?**, and the recent calibration analysis of **?**.

17.6.1 Value-aware model learning

? formalize the objection as a loss. Fix a value function class $\mathcal{V}$, a data distribution μ over state-action pairs, and a model class within which the learner selects an estimate $\widehat{P}$ of the true transition kernel $P^\star$. The value-aware model learning loss is

$$\mathcal{L}_{\mathrm{VAML}}(\widehat{P}) = \sup_{V \in \mathcal{V}} \left\| (\widehat{P}V)(\cdot) - (P^\star V)(\cdot) \right\|_\mu ,$$

where $(\widehat{P}V)(s,a) = \int V(s')\widehat{P}(\mathrm{d}s' \mid s,a)$ is the expected next-state value under the learned model and $(P^\star V)(s,a)$ is the same quantity under the true kernel. The loss penalizes the worst-case

[290] The four canonical instantiations of the architecture cover the major design choices. PlaNet (**?**) introduces the RSSM with cross-entropy planning. DreamerV1 (**?**) replaces planning with actor-critic in imagination. DreamerV2 (**?**) switches the Gaussian latent for categoricals with a Kullback-Leibler balancing trick, surpassing top model-free agents at the Atari 200M budget. DreamerV3 (**?**) fixes one hyperparameter configuration across more than one hundred and fifty tasks via symlog reward normalization and free-bits Kullback-Leibler, including the Minecraft diamond from scratch.

discrepancy in expected next-state value across $\mathcal{V}$, measured under μ. A model that places mass on features of the next observation irrelevant to every value function in $\mathcal{V}$ is not penalized; a model that distorts the value expectation is. **?** embed the same idea inside approximate value iteration, replacing the supremum over a value class with the realized value iterate at each step, which yields the practical IterVAML estimator and a fitted error bound that propagates through the Bellman backups rather than through the model alone. **?** read the construction through optimal transport, observing that under a Lipschitz assumption on the value class the VAML loss reduces to a Wasserstein distance between $\widehat{P}$ and $P^\star$, which connects the value-aware loss to the substantial empirical literature on Wasserstein generative modeling.

VAML is the formal statement of the intuition that the model should be wrong in ways that do not matter for the value functional. This is the technical answer to the conceptual openness of Sutton's original Dyna architecture, in which the model was permitted to be probabilistic and oftentimes incorrect (§17.3.3) without specifying what kind of error was tolerable. The likelihood-trained Recurrent State-Space Model line (§17.5) commits to a particular notion of error, namely the reconstruction loss on pixels and the Kullback-Leibler divergence on latents, and violates the value-aware principle whenever the pixel distribution is rich in detail orthogonal to the reward. Whether this matters in practice depends on the task; the empirical claim of the value-aware line is that it does, and that the gap widens as observation dimensionality grows.

17.6.2 Value equivalence and MuZero

? sharpen the VAML idea into a clean equivalence relation. Fix a policy set Π and a value function set $\mathcal{V}$. Two transition kernels $\widehat{P}$ and $P^\star$ are value-equivalent with respect to $(\Pi, \mathcal{V})$ when their Bellman operators agree on every pair $(\pi, V) \in \Pi \times \mathcal{V}$, that is when $(T^\pi_{\widehat{P}} V)(s) = (T^\pi_{P^\star} V)(s)$ for all s and all (π, V) in the chosen sets. The equivalence class shrinks monotonically as Π and $\mathcal{V}$ grow, collapsing to the singleton $\{P^\star\}$ in the limit. The practical implication is that a small model class can be sufficient for a restricted set of policies and value functions, even if it would be insufficient as a generative model of the environment. **?** subsequently introduce proper value equivalence, which restricts the value class to fixed points of the chosen Bellman operators and gives the variant of value equivalence that aligns most directly with the MuZero training objective. **?** push the value-aware idea into the regret-bound literature with value-targeted regression, which trains the model against value targets and matches the dimension-dependent rates of optimism-based reinforcement learning under standard assumptions.

The empirical instantiation is MuZero. **?** train a latent recurrence jointly with a policy network and a value network against three losses applied at the next step of the recurrence, namely a reward prediction loss, a value prediction loss, and a policy prediction loss matched to the output of a Monte Carlo tree search rooted at the same latent. There is no reconstruction loss; the latent dynamics are never asked to decode observations. The transition kernel itself has no explicit target, since the only quantities that enter any loss are the reward, the value, and the policy at the rollout step. Empirically the construction matches AlphaZero on Go, chess, and shogi without access to the rules of those games, and reaches state-of-the-art performance on the Atari benchmark. **?** extend the construction to stochastic transitions by inserting a learned afterstate node, the game-tree analogue of a chance node, between the agent action and the next state. Read in economics terms, value-aware model learning treats the world model as a task-relevant misspecification, kept correct on the value functional even when it is wrong on

the full observation distribution.[291][292]

17.7 Short Rollouts and Ensembles: The Modern Dyna

The line of work culminating in **?** is the continuous-control inheritor of Dyna (§17.3). The architectural recipe is a deep ensemble of probabilistic dynamics models paired with a model-free off-policy actor-critic, soft actor-critic in the canonical implementation, with synthetic experience generated by branching short imagined rollouts from real states drawn out of the replay buffer. The framing slogan, present in the paper and in the surrounding literature, is to not trust the model too far. Breaking the dependence between the task horizon and the model rollout horizon is the move that makes the model-error penalty tractable, and is what separates this line from the trajectory-optimization approach of **?** and the on-policy approach of **?**. The conceptual lineage runs back to Sutton (1990), with the same outer loop of direct learning from real transitions and indirect learning from imagined ones, only with a deep ensemble in place of the tabular model and an off-policy actor-critic in place of tabular Q-learning.

17.7.1 Branched short rollouts

As an abstract operator, the branched rollout maps the pair (real-state distribution, learned dynamics) to a synthetic transition distribution of length k, and decouples the task horizon from the model horizon by anchoring every imagined trajectory at a state the agent has actually visited. The MBPO branching scheme is straightforward. At each gradient step the algorithm samples a real state s from the replay buffer, rolls the learned dynamics ensemble forward for k steps from s under the current policy, and trains the actor-critic on the resulting model-generated transitions alongside real ones. The actor-critic update does not distinguish synthetic transitions from real transitions; the only architectural change relative to model-free soft actor-critic is the distribution from which training tuples are drawn. Real states act as anchors. Because every imagined trajectory is no more than k steps long and originates at a state the agent has actually visited, the model is never asked to extrapolate from its own predictions for more than k steps, even when the task itself runs for hundreds of steps.

The returns bound in **?**, Theorem 4.2 formalizes the trade-off. With ϵ_m the model error measured in total variation, ϵ_π the policy distribution shift relative to the data-collection policy, k the branching length, and $r_{\max}$ the reward bound,

$$\eta[\pi] \geq \eta^{\text{branch}}[\pi] - 2r_{\max}\Big[\frac{\gamma^{k+1}\epsilon_\pi}{(1-\gamma)^2} + \frac{\gamma^k + 2}{1-\gamma}\epsilon_\pi + \frac{k}{1-\gamma}(\epsilon_m + 2\epsilon_\pi)\Big].$$

The first two terms penalize policy shift and shrink in k; the third penalizes model error and grows linearly in k. Theorem 4.3 then shows that an optimal $k^\star > 0$ exists whenever the model error is small enough, justifying short rollouts as the right way to spend a fixed model-error budget. The practical takeaway, stated in the paper's ablations, is that $k = 1$ is competitive across the MuJoCo suite, that k in the range of roughly five to fifteen typically gives the best returns, and that values much beyond about 25 are too inaccurate to help. The empirical sweet

[291] **?** revisit this family and observe that the standard sampled losses, including the canonical MuZero objective, are uncalibrated surrogate losses whose minima do not in general coincide with the minima of the underlying target loss on the value function; value-equivalence is a property of an in-distribution policy and value class, and a model trained purely against value targets has no incentive to remain accurate once the policy drifts. The paper proposes a calibrated variant that restores the correctness property in expectation and leaves the broader question of value-aware accuracy under policy drift open.

[292] The same move appears in the operations-research line on decision-focused learning, in which a predictor is trained with a loss aligned to the downstream decision quality rather than to a statistical scoring rule on the forecast itself (**?**). The inner solver there is a convex program rather than a Bellman backup, but the conceptual move, to align the loss with what the decision-maker actually needs and to treat likelihood as the wrong default whenever the model class is misspecified, is shared.

spot is well inside the regime where the model is treated as a local data augmentation rather than as a long-horizon simulator.

Read in economics terms, the branched-rollout architecture is the computational version of using a locally valid structural model for counterfactual policy search; the model is trusted only over short windows around states the agent has already visited, and the rest of the long-horizon credit is left to the off-policy critic. The dynamics ensemble used in this line, typically five to seven neural networks each predicting a Gaussian over the next-state delta, plays two roles. As an aggregate predictor the ensemble averages over members and trims tails to give a calibrated forecast that a single deterministic network would not produce; as a disagreement signal the spread across members is a cheap proxy for epistemic uncertainty, large where the agent has not visited often and small where coverage is good.[293][294]

17.8 Convergence Point: TD-MPC2

TD-MPC2 (**?**) is the convergence point of three lines that the preceding sections traced separately. Model predictive control with learned dynamics, in the style of PETS and MBPO (§17.7), supplies the planning frame. Recurrent latent world models, in the Dreamer line (§17.5), supply the idea that the model should live in a learned latent and need not decode observations. Value-aware learning, articulated by MuZero and the value-equivalence literature (§17.6), supplies the target the model is trained against. TD-MPC2 keeps the planner of the first line, drops the observation decoder of the second, and adopts the value functional as the training signal of the third. The single empirical commitment that organizes the paper is that one fixed hyperparameter configuration should work across many tasks and many model sizes, with no per-task tuning and no rescaling of components as the network grows.

17.8.1 Components and joint loss

The agent learns five components jointly. An encoder h maps an observation s_t to a normalized latent $z_t = h(s_t)$. A latent dynamics model d predicts the next latent, $\hat{z}_{t+1} = d(z_t, a_t)$. A reward model R predicts the immediate reward from the latent and action, $\hat{r}_t = R(z_t, a_t)$. A terminal action-value function Q supplies a bootstrap beyond the planning horizon, $\hat{q}_t = Q(z_t, a_t)$. A policy prior p supplies an amortized action proposal $\hat{a}_t = p(z_t)$ used both to warm-start the planner and to seed the bootstrap targets. The joint loss is the sum of a latent consistency term that pushes the predicted next latent to agree with the encoded next observation, $\|d(z_t, a_t) - \mathrm{sg}(h(s_{t+1}))\|^2$ under a stop-gradient on the target encoding, a cross-entropy loss on the reward prediction, a cross-entropy loss on the temporal-difference targets for Q, and a behavioral cloning term that regresses p on the actions selected by the planner. Notably absent is an observation reconstruction loss; the model is never asked to decode pixels or proprioceptive features and is trained only against quantities the value calculation actually

[293]The ensemble appears in two prior methods with different commitments. **?** pairs the ensemble with on-policy trust-region policy optimization, estimating value gradients while clipping updates that would push the data distribution into regions where members disagree; the on-policy constraint bounds ϵ_π explicitly but keeps the long-horizon dependence on the model. **?** wraps the same ensemble in model-predictive control and bypasses policy learning entirely by propagating particle trajectories through ensemble members at planning time, which solves the horizon problem by conflating the task and planning horizons and limits scaling to long-horizon tasks where MPC becomes prohibitive. MBPO's branched-rollout architecture pushes the two terms of the returns bound into a regime where standard off-policy machinery can absorb them, and the same recipe carries from low-dimensional locomotion to humanoid without changes to the outer loop.

[294]On the MuJoCo continuous-control benchmark **?** matches the asymptotic return of soft actor-critic with roughly an order of magnitude fewer environment samples; the Ant task reaches the soft actor-critic return at around 300,000 real steps against the latter's roughly 3,000,000, and on Humanoid (a 376-dimensional state with a 17-dimensional action) MBPO converges where PETS fails to learn, consistent with the architectural reading that the task horizon dwarfs PETS's planning horizon. **?** reports a similar order-of-magnitude sample-efficiency gain for ME-TRPO over TRPO/PPO/DDPG on the same suite.

consumes. This is the architectural commitment that distinguishes TD-MPC2 from the Dreamer line (§17.5), in which the world model carries a reconstruction term throughout.

17.8.2 MPPI planning with value bootstrap

Action selection at each environment step uses Model Predictive Path Integral control, a sampling-based optimizer over Gaussian action sequences (**?**). The planner maintains a sequence of Gaussian distributions $(\mathcal{N}(\mu_h, \sigma_h^2))_{h=t}^{t+H-1}$ over the next H actions, draws a population of candidate sequences, rolls each one forward through the learned latent dynamics, scores the imagined returns, and refits (μ_h, σ_h) to a softmax-weighted average of the top-scoring sequences. The scoring rule is the finite-horizon return augmented with a terminal value bootstrap,

$$\mu^\star, \sigma^\star \;=\; \arg\max_{(\mu,\sigma)} \; \mathbb{E}_{(a_t,\ldots,a_{t+H})\sim\mathcal{N}(\mu,\sigma^2)}\Big[\, \gamma^H Q(z_{t+H}, a_{t+H}) \;+\; \sum_{h=t}^{H-1} \gamma^{h-t} R(z_h, a_h) \,\Big],$$

where the trajectory expectation is taken under the learned latent dynamics d rolled from the current encoded state $z_t = h(s_t)$. The terminal Q extends the planner's effective horizon past the finite window H, so the agent does not need to roll long open-loop sequences through the learned model to capture long-horizon credit. This is the architectural reason TD-MPC2 sidesteps the long-rollout compounding error that limits PETS (§17.7), which has no terminal value and must extend H to capture distant reward. After the planner converges, the agent executes only $a_t \sim \mathcal{N}(\mu_t^\star, (\sigma_t^\star)^2)$ and re-plans at the next step. The amortized policy p seeds the next planning call and supplies the action used in the Q-bootstrap.

17.8.3 Empirical headline and scaling

The paper evaluates TD-MPC2 across 104 continuous-control tasks spanning four benchmarks, DeepMind Control, Meta-World, ManiSkill2, and MyoSuite, under a single fixed hyperparameter configuration with no per-task tuning. The task suite includes action dimensionalities up to 39, sparse rewards, multi-object manipulation, musculoskeletal motor control, and locomotion on dog and humanoid embodiments. Across all four benchmarks TD-MPC2 outperforms SAC, the original TD-MPC, and DreamerV3 at comparable parameter counts. The contribution is not that any single task is solved better than ever before, but that one configuration solves all of them, which is the property the field had previously assigned to model-free baselines under heavy per-task tuning.

The scaling experiment trains a single multi-task agent on 80 tasks drawn from DeepMind Control and Meta-World and varies the world model size from 1 million parameters to 317 million. The normalized score moves from 16.0 at 1M parameters to 70.6 at 317M, roughly log-linear in parameter count over the two and a half orders of magnitude examined, with no observed saturation at the upper end. This is the first model-based reinforcement learning paper to report a clean monotone scaling curve over this range. The same hyperparameters are used at every model size; no learning rate, batch size, or capacity-specific schedule is retuned. The empirical claim of TD-MPC2 is that an MPC-shaped model-based agent, built from the components developed in the three earlier lines, scales with parameters in the same way that supervised and self-supervised models do.

17.9 Dual Simulation: Cobweb and Fishery

The chapter compares seven learning paradigms on two single-agent economic decision problems. The first is a cobweb model with adjustment cost, a self-referential environment in which the agent's own action moves the contemporaneous price. The second is a logistic-growth fishery, an exogenous environment in which the agent's action depletes a renewable stock but does not feed

back through expectations. The seven paradigms are an oracle that knows the true parameters, a constant-rule baseline that does not learn, recursive least squares adaptive learning (Marcet and Sargent, 1989), the genetic algorithm of **?** without the election operator, tabular Q-learning (Watkins and Dayan, 1992a), a model-based learner that estimates demand and cost coefficients by least squares and plans through the closed-form linear-quadratic Bellman equation, and the full MBPO of **?** with bootstrap-ensemble dynamics, branched rollouts, and a linear policy trained by REINFORCE. The experiment is deliberately favorable to structured learners; the two environments have low-dimensional state, smooth payoffs, and stable parametric form, and the purpose is to locate the methods on an inductive-bias frontier rather than to rank them universally.

17.9.1 Cobweb with adjustment cost

The cobweb panel is designed to expose three things in turn: where each paradigm sits on the inductive-bias-vs-data trade-off, how parameter recovery rates differ across regimes, and where regret performance and policy quality come apart for methods that bake in correct functional form. The cobweb stability parameter b/c varies across regimes to modulate the curvature of the value function rather than to drive the kind of E-stability divergence Marcet and Sargent (1989) report for the multi-agent expectational cobweb, which does not arise in this single-agent monopoly variant.

Setup. The state is $s_t = (q_{t-1}, p_{t-1}) \in \mathbb{R}^2$ and the action is $q_t \in [0, 4]$. The price clears contemporaneously as $p_t = a - bq_t + \varepsilon_t$ with $\varepsilon_t \sim \mathcal{N}(0, \sigma^2)$, and the reward is $r_t = p_t q_t - (c/2)q_t^2 - (\phi/2)(q_t - q_{t-1})^2$, so the agent faces a linear demand, a quadratic production cost, and a quadratic adjustment cost. The sweep is $b/c \in \{0.5, 1.0, 2.0\}$ with $a = 4$, $c = 1$, $\phi = 0.2$, $\sigma = 0.1$, $\gamma = 0.95$, $T = 500$, and twenty independent seeds. The oracle policy is the affine feedback rule $q_t^\star = K_0 + K_q q_{t-1}$ recovered by quadratic-form fixed-point iteration on the Bellman equation, and cumulative regret is computed per seed against the oracle's realized return on the same noise sequence.

Models. Seven paradigms share the budget. The oracle knows (a, b, c, ϕ) and applies the closed-form optimal feedback rule. The constant-rule baseline plays $q_t = 1.4$ at every step regardless of state, providing the absolute no-learning floor. Recursive least squares of Marcet and Sargent (1989) estimates $(\hat{a}, \hat{b})$ from observed prices, treats (c, ϕ) as known, and re-solves the linear-quadratic planner with point estimates each period. The genetic algorithm of **?** maintains a population of binary-encoded production rules, evolves them under fitness-proportional selection on the running mean of realized observed profit, single-point crossover, bit-flip mutation, and two-elite preservation, and plays the population's incumbent rule; the original Arifovic 1994 election operator scored hypothetical offspring against their parents using the true demand and cost parameters, and is omitted here so the agent uses only realized observed profit. Tabular Q-learning of Watkins and Dayan (1992a) discretizes the state-action space onto a $20 \times 20 \times 25$ grid and applies ε-greedy temporal-difference updates with $\alpha = 0.1$ and ε decaying linearly from 0.3 to 0.01. Two model-based learners share the same parameter-estimation step but differ in how they plan. The model-based LQ learner exploits the linear-Gaussian structure: it solves the closed-form linear-quadratic Bellman equation by Riccati iteration on current point estimates of $(\hat{a}, \hat{b}, \hat{c}, \hat{\phi})$ each step and acts under the resulting affine feedback rule with Gaussian exploration noise. The MBPO learner of **?** maintains a bootstrap ensemble of five linear-Gaussian demand models, parameterizes the policy as $q = K_0 + K_q q_{t-1}$, and at each real step samples ten rollouts of horizon five from buffer-uniform initial states under random ensemble members, accumulating discounted returns from the learned reward model and updating (K_0, K_q) by REINFORCE with a moving-average baseline.

Results. Table 36 reports cumulative regret at $T = 500$ for each paradigm and regime; Figure 42 shows the corresponding regret trajectories. Paradigms are presented in rank order by mean regret across regimes. Recursive least squares attains regret of about five units, indistinguishable across regimes within standard error, because it has the correct functional form for the price map and known cost parameters and need only estimate the two demand coefficients. The model-based LQ learner trails by an order of magnitude with regret rising from about twelve to forty-three units as the regime moves from stable to unstable, reflecting the larger curvature it must resolve in the unstable region and the additional cost coefficients it must learn alongside the demand map. The genetic algorithm of **?** without the election operator is a further order of magnitude behind, with regret of ninety to three hundred units, reflecting the cost of gradient-free population search over a binary chromosome with no parametric knowledge of demand or cost. The constant rule shifts with the regime in a way that reflects the position of the fixed action relative to the regime's optimum and is included as the absolute no-learning floor, not as a learning competitor. The full MBPO learner of **?** sits next, with regret that varies more than two orders of magnitude across regimes (six hundred and fifty in stable, one hundred and twelve in borderline, forty-nine in unstable); the variance reflects REINFORCE's sensitivity to the curvature of the return surface, which is flat in the stable regime where small differences in (K_0, K_q) matter little for return and sharp in the unstable regime where the policy gradient carries usable signal. Tabular Q-learning finishes near a thousand units of regret in every regime, the empirical signature of model-free tabular methods in continuous problems under a tight environment budget. The take-away is that on this cobweb, under the present hyperparameters, branched-rollout REINFORCE underperforms closed-form planning across all three regimes, with the gap widest in the stable regime where the return surface is flattest for the REINFORCE gradient estimator. The two methods share the same parameter-estimation step; the difference is what the planner does with the point estimates.

Regret captures the integrated cost of learning, not the asymptotic quality of the recovered model or policy. Figure 43 plots the parameter trajectories $\hat{a}_t$ and $\hat{b}_t$ for recursive least squares, the model-based LQ learner, and the MBPO ensemble mean against the true values, and Table 37 reports the final $|\hat{\theta} - \theta|$ error per paradigm and regime. All three methods recover the demand coefficients to within four percent of the truth in every regime; the two model-based learners additionally recover the cost coefficients (c, ϕ) to within machine precision[295] because the reward signal pins them down once the residual $r_t - p_t q_t$ is regressed on $(q_t^2, (q_t - q_{t-1})^2)$. Recovery is fastest in the unstable regime because the curvature of the demand map sharpens the information content of each observation; this is the same observation that explains why the regret values themselves are similar across regimes even though the absolute return at the oracle's policy is lower in the unstable regime.

Figure 44 reports the distance from each learner's noise-free action at a regime-specific reference state to the oracle's action at the same state, on a log scale. The ordering is by inductive bias rather than recovery accuracy. The model-based LQ learner attains the smallest residual policy distance in every regime, two to three times smaller than recursive least squares in absolute terms, because by estimating (c, ϕ) it places its planner at the correct curvature of the value function. Recursive least squares is bounded below by the discrepancy between the prior on (c, ϕ) that fixes its planner and the realized environment, which is small here only because the prior was chosen to match. The genetic algorithm tracks a policy distance an order of magnitude larger, reflecting the discretization of the chromosome over the action space rather than incomplete recovery of any continuous parameter. The MBPO learner sits between the genetic algorithm and tabular Q-learning in the stable regime where REINFORCE has not converged its linear policy, and approaches the model-based LQ learner's accuracy in

[295]The exact recovery reflects noiseless reward in this simulation: given observed (p_t, q_t, q_{t-1}), the equation $r_t = p_t q_t - (c/2)q_t^2 - (\phi/2)(q_t - q_{t-1})^2$ is exactly solvable for (c, ϕ) from any two tuples that span the cost feature space. With additive reward noise $\sigma_r > 0$, recovery would degrade at rate $O(\sigma_r/\sqrt{N})$.

the unstable regime where the gradient signal is strong; this regime dependence is the practical cost of the REINFORCE update relative to the analytical planner. Tabular Q-learning sits at order one, a direct consequence of the coarseness of the action grid (twenty-five points over a range of four units) compounded by the constant exploration noise injected into the rollout. The constant rule is flat by construction at a value that depends on how far the fixed action $q_{\text{fixed}} = 1.4$ lies from the regime's optimum, and the value distinguishes how much of the no-learning floor is luck of the regime rather than any property of learning.

The verdict is that recursive least squares wins the regret comparison because its early decisions are nearly optimal under correct functional form and known cost parameters, while the model-based LQ learner wins the asymptotic policy-distance comparison because it ultimately fits more of the problem's structure; the two methods occupy adjacent positions on the inductive-bias frontier, with recursive least squares paying a higher per-step penalty as long as its cost-parameter prior is correct and the model-based LQ learner paying a transient information cost in exchange for a tighter asymptote. The MBPO learner with branched rollouts and REINFORCE is the third position on this frontier: same model-estimation step as the LQ learner, but a slower policy update that pays off only when the closed-form structure is unavailable. On this cobweb the closed-form structure is available and the comparison favors the LQ learner; in environments where it is not, the comparison would invert.

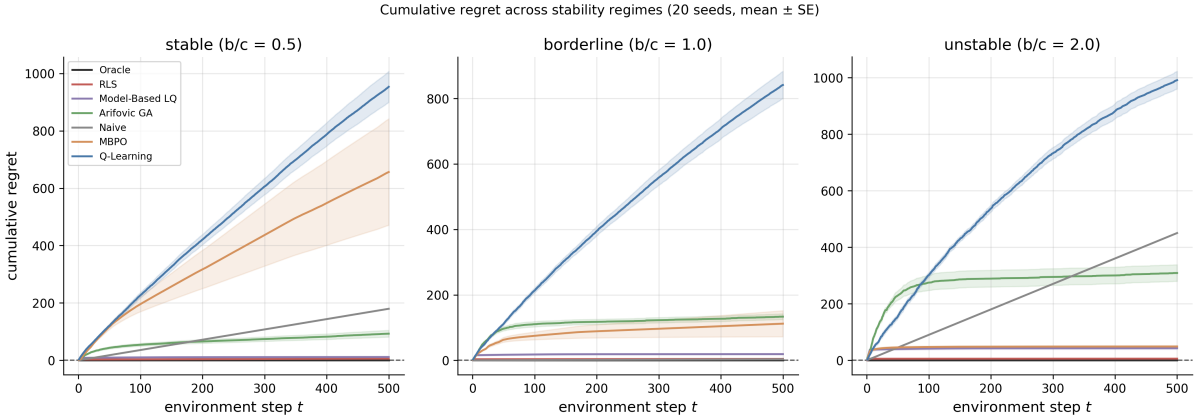

Figure 42: Cumulative regret across stability regimes for seven learning paradigms on the cobweb with adjustment cost. Lines are means and shaded bands are one standard error across twenty seeds. Panels correspond to $b/c \in \{0.5, 1.0, 2.0\}$. Oracle is the zero reference.

Table 36: Cumulative regret at $T = 500$ on the cobweb with adjustment cost, mean ± standard error across twenty seeds. Lower is better; Oracle is the reference.

Paradigm	Stable ($b/c = 0.5$)	Borderline ($b/c = 1.0$)	Unstable ($b/c = 2.0$)
Oracle	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
RLS	5.89 ± 1.04	4.38 ± 0.43	5.87 ± 0.18
Model-Based LQ	11.65 ± 0.93	18.96 ± 1.73	42.90 ± 3.75
Arifovic GA	92.89 ± 11.69	133.43 ± 8.65	308.88 ± 29.16
Naive	179.73 ± 0.34	3.37 ± 0.04	450.34 ± 0.34
MBPO	656.60 ± 185.41	112.06 ± 39.78	48.87 ± 3.47
Q-Learning	953.69 ± 53.79	841.50 ± 41.63	991.15 ± 31.19

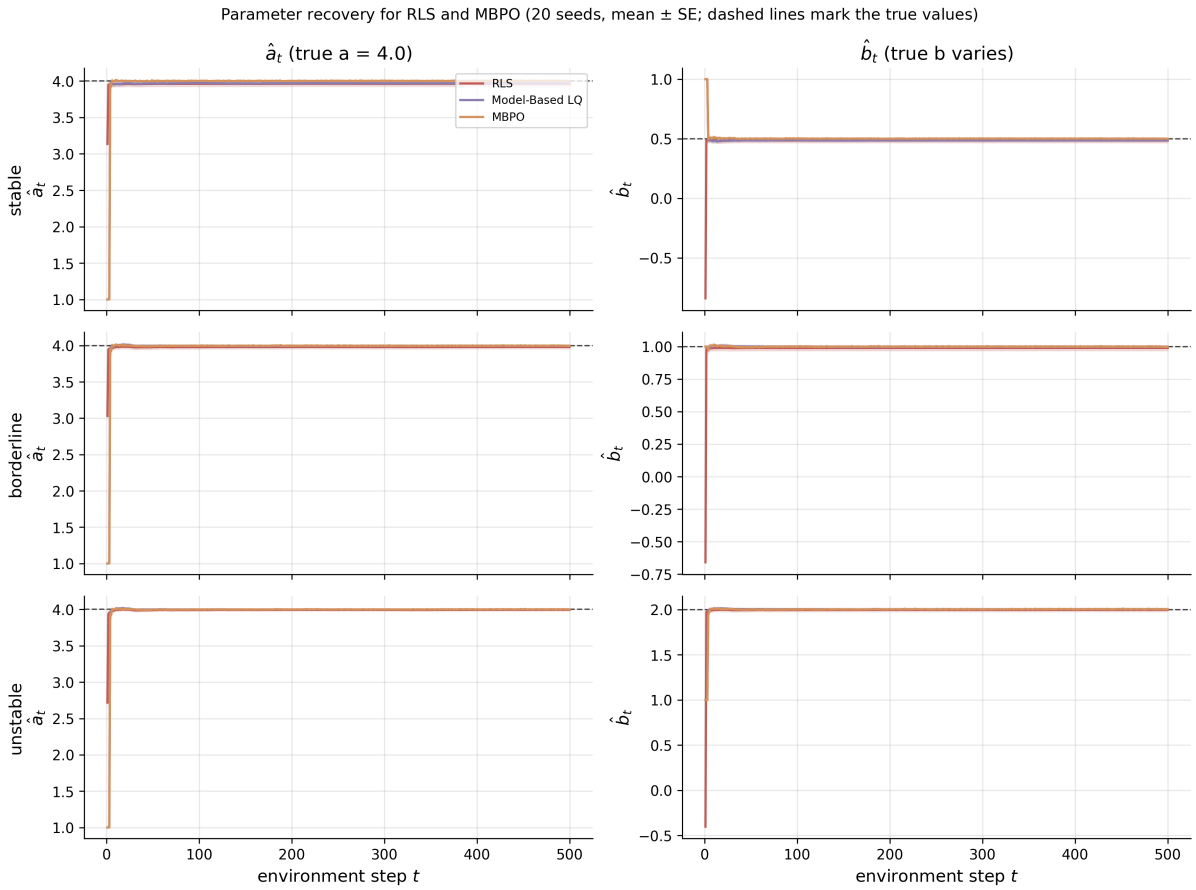

Figure 43: Parameter recovery for recursive least squares, the model-based LQ learner, and the MBPO ensemble mean across stability regimes. Solid lines are means and shaded bands one standard error across twenty seeds; dashed horizontal lines mark the true values of a and b. All three methods converge to the truth within a few dozen environment steps.

17.9.2 Fishery with logistic growth

The fishery panel runs the structured-learner subset of the cobweb's seven paradigms on a logistic-growth fishery, an exogenous environment that isolates the sample-efficiency story from the self-referential pricing of the cobweb. The reward structure is again linear-quadratic, but the transition dynamics are non-linear, so the oracle is a grid-based dynamic-programming solve rather than a closed-form linear-quadratic Riccati.

Setup. The state is the stock biomass $s_t \in [0, 1.5K]$ and the action is the harvest $h_t \in [0, \min(s_t, h_{\max})]$. The dynamics are $s_{t+1} = \max(0, s_t + r s_t(1 - s_t/K) - h_t + \varepsilon_t)$ with $\varepsilon_t \sim \mathcal{N}(0, \sigma^2)$, and the reward is $r_t = p h_t - (c/2) h_t^2$. Parameters are $r = 0.4$, $K = 10$, $p = 2.0$, $c = 0.2$, $\sigma = 0.3$, $\gamma = 0.95$, $T = 500$, with twenty independent seeds and $s_0 = K$ (initial stock at carrying capacity). The deterministic maximum-sustainable-yield harvest is $h_{\mathrm{MSY}} = rK/4 = 1$ at $s^\star = K/2 = 5$.

Models. Six paradigms share the budget, a subset of the cobweb panel's seven (the full MBPO with branched rollouts and REINFORCE is omitted here because the fishery's non-linear dynamics make a closed-form policy parameterization unavailable). The oracle solves a grid-based value-iteration discounted dynamic program on 50 stock × 25 action points and applies the greedy policy. The constant-rule baseline plays $h = 0.5$ at every step, a reasonable steady-state guess that lies between zero and the analytic MSY. Recursive least squares estimates

Table 37: Final parameter recovery error $|\hat{\theta} - \theta|$ at $t = T$, mean ± standard error across twenty seeds. Cells marked — indicate parameters the paradigm does not estimate (recursive least squares assumes c and ϕ known); both model-based learners estimate all four.

Paradigm	Regime	$\lvert\hat{a} - a\rvert$	$\lvert\hat{b} - b\rvert$	$\lvert\hat{c} - c\rvert$	$\lvert\hat{\phi} - \phi\rvert$
RLS	stable	0.037 ± 0.035	0.015 ± 0.018	—	—
RLS	borderline	0.019 ± 0.026	0.011 ± 0.020	—	—
RLS	unstable	0.004 ± 0.016	0.003 ± 0.019	—	—
Model-Based LQ	stable	0.029 ± 0.019	0.013 ± 0.009	0.000 ± 0.000	0.000 ± 0.000
Model-Based LQ	borderline	0.004 ± 0.011	0.002 ± 0.008	0.000 ± 0.000	0.000 ± 0.000
Model-Based LQ	unstable	0.001 ± 0.007	0.003 ± 0.008	0.000 ± 0.000	0.000 ± 0.000
MBPO	stable	0.000 ± 0.005	0.000 ± 0.002	0.000 ± 0.000	0.000 ± 0.000
MBPO	borderline	0.005 ± 0.006	0.003 ± 0.004	0.000 ± 0.000	0.000 ± 0.000
MBPO	unstable	0.000 ± 0.005	0.002 ± 0.006	0.000 ± 0.000	0.000 ± 0.000

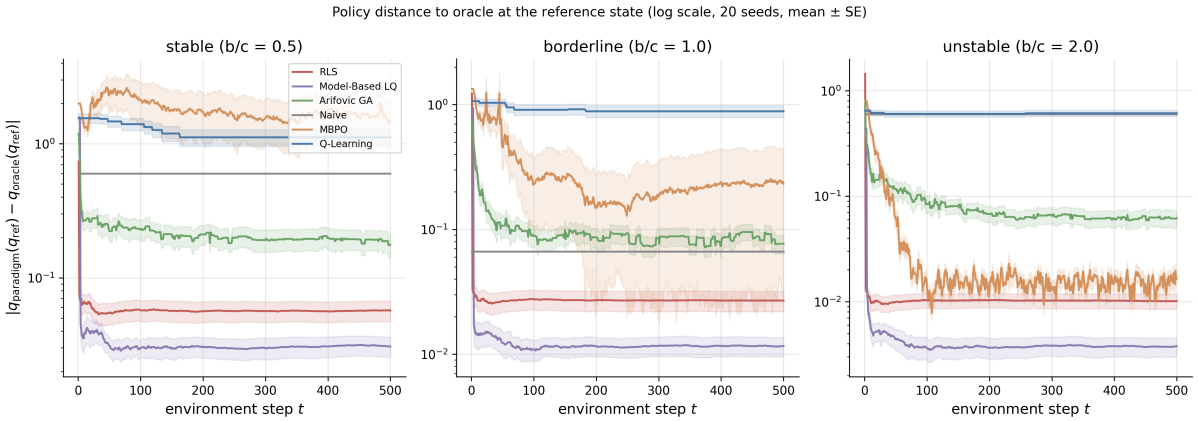

Figure 44: Policy distance to the oracle at a regime-specific reference state, log scale. The reference state is the static optimum quantity for each regime. Lines are means and bands one standard error across twenty seeds. Oracle is identically zero by construction and is omitted.

$(\hat{r}, \hat{K})$ from a linear-in-parameters regression of $\Delta s_t + h_t$ on $(s_t, -s_t^2)$, plans by re-solving the DP with point estimates every twenty-five observations, and applies the resulting greedy policy with known cost parameters (p, c). The model-based LQ learner estimates $(\hat{r}, \hat{K}, \hat{p}, \hat{c})$ jointly by least squares, refits the DP every twenty-five observations, and acts with Gaussian exploration noise that decays over the episode. Tabular Q-learning discretizes (s, h) onto a 30×21 grid with $\alpha = 0.1$ and ε decaying from 0.3 to 0.01. The genetic algorithm of **?** maintains thirty binary-encoded constant harvest rules and evolves them under fitness-proportional selection and the election operator with known cost parameters.

Results. Table 38 reports cumulative regret at $T = 500$ for each paradigm in rank order; Figure 45 shows the regret trajectories. Recursive least squares and the model-based LQ learner sit near the oracle (about thirteen and fifteen regret units respectively), tabular Q-learning recovers a usable policy at roughly two hundred and seventy-five units of regret, the constant rule lands at four hundred and forty-seven, and the genetic algorithm finishes at seven hundred. The fishery's exogenous dynamics give the structured learners a cleaner identification problem than the cobweb because each (s_t, s_{t+1}) pair carries direct information about (r, K) that does not get confounded by the agent's own action through expectations, and the consequence is

that parameter recovery converges within the first hundred environment steps and the residual regret reflects exploration cost rather than mis-identification. The verdict mirrors the cobweb's inductive-bias frontier with one twist: tabular Q-learning beats both the constant rule and the genetic algorithm on this environment because the discretized state-action grid happens to be coarse enough to be tractable yet fine enough to track the optimal feedback rule's monotone structure, while the genetic algorithm's binary-encoded constant harvest is the wrong functional form for an environment where the optimal action varies smoothly with the stock.

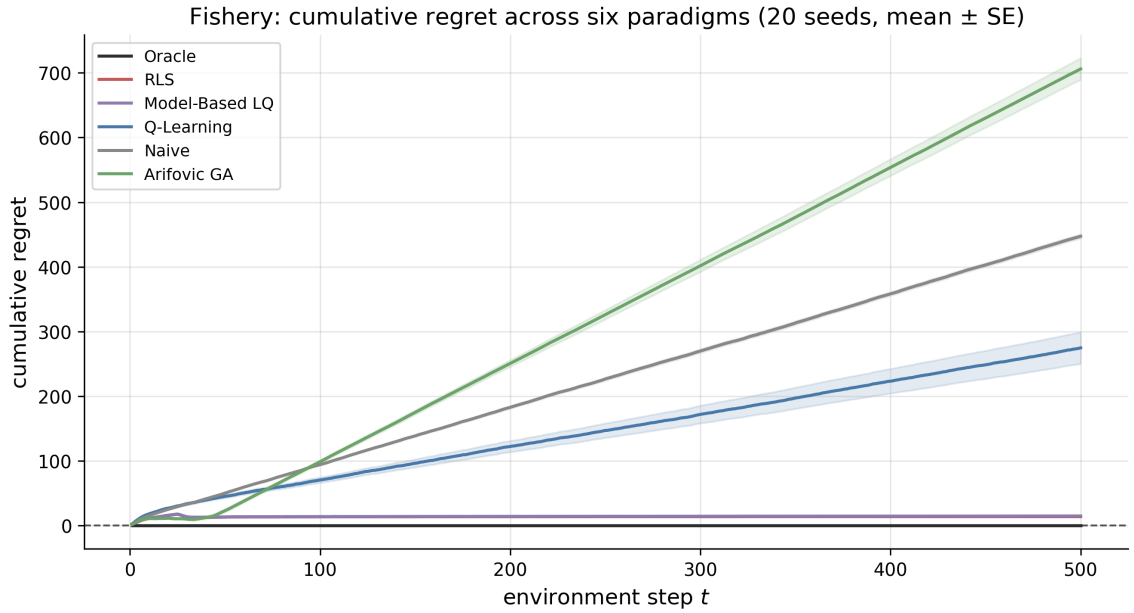

Figure 45: Cumulative regret on the logistic-growth fishery for six paradigms (twenty seeds, mean and one-standard-error bands). Paradigms are presented in rank order by final regret.

Table 38: Final cumulative regret at $T = 500$ on the fishery, mean ± standard error across twenty seeds. Lower is better; oracle is the zero reference. Paradigms are presented in rank order.

Paradigm	Final regret
Oracle	0.00 ± 0.00
RLS	13.67 ± 0.43
Model-Based LQ	14.69 ± 0.73
Q-Learning	274.71 ± 24.36
Naive	447.35 ± 3.24
Arifovic GA	706.13 ± 16.65

17.10 Synthesis

The chapter has traced the model-based reinforcement learning line from two 1990 origins, Sutton's Dyna and Schmidhuber's controller-model architecture, through the deep revival in **?**, the recurrent state-space model that became the workhorse of the Dreamer line, the value-aware question that runs from VAML through value-equivalence to MuZero, the modern Dyna in continuous control associated with the MBPO ensemble line, and the convergence point at TD-MPC2. The remainder of the section notes where model-based methods win, where they fail, where the surrounding boundary makes the model unnecessary, and what remains open.

17.10.1 Where model-based RL wins

Sample efficiency is the headline gain. Model-based methods outperform model-free baselines when environment interaction is scarce or expensive, including pixel-based continuous control in PlaNet and Dreamer, discrete-state planning in MuZero, and continuous control with sparse reward in MBPO and TD-MPC2. The learned model serves as a data amplifier through the Dyna planning step and as a representation amplifier, since the RSSM and TD-MPC2 latents transfer across tasks within their respective suites. When transferable representations or finite-data regimes are the binding constraint, model-based methods win.

17.10.2 Where model-based RL fails

The first failure mode is misspecification and the objective mismatch between training and use. **?** document that maximum-likelihood model fitting and reward optimization are decoupled objectives, so a model trained to high likelihood is not necessarily a model whose induced policy is high-return; their sweeps show likelihood-reward correlations as low as 0.07 on Half-Cheetah and adversarial perturbations that preserve likelihood while halving return. **?** extend this point to MuZero and show that value-equivalent representations are systematically miscalibrated under distribution shift. The value-aware line (§17.6) is the partial answer, reformulating the training target around the value functional rather than the full distribution. A fully principled treatment of off-distribution calibration remains open.

The second failure mode is compounding error and the data-coverage gaps that exploration must fill. **?** give the local-operator Lipschitz bound for model-based RL, in which one-step prediction error compounds geometrically along the rollout at rate K_F, the Lipschitz constant of the learned dynamics, and the induced value-error bound on the planner stays finite when $\gamma K_F < 1$ while diverging as $\gamma K_F \to 1$. When the contraction condition fails, model-based planning diverges. **?** documents that retraining a single model on its own predictions can destabilize the model itself and proposes a self-correcting hallucinated-rollout scheme as one fix. **?** and **?** attack the same problem from the data side, with prediction-error-driven curiosity in the former and ensemble-disagreement-driven planning in the latter; both push the agent's data distribution beyond what greedy exploitation alone produces, with measurable gains in regions of state space the agent would otherwise miss.

17.10.3 When the model is not needed

Not every sequential decision problem rewards the cost of learning a forecaster. When the decision reduces to a supervised prediction followed by a fixed downstream rule, the boundary studied by **?** in deep inventory management, a direct supervised reduction can match or beat a learned-world-model pipeline at much lower engineering cost. The boundary itself is closed-loop feedback. When the agent's action does not feed back into the data-generating process, a supervised reduction suffices; when it does, the learned model becomes load-bearing because forecasts and decisions are then jointly determined. This is the boundary that places model-based reinforcement learning inside the sequential decision-making space and separates it from one-shot forecasting.

17.10.4 Open questions

The dual simulation in §17.9 maps where model-based reinforcement learning sits on the sample-efficiency frontier among the other paradigms in a single-agent cobweb with adjustment cost and a logistic-growth fishery. In the cobweb setting the Marcet-Sargent divergence-in-the-unstable-region story does not arise because the environment is self-referential through prices alone and not through expectations, and the more telling result is the order-of-magnitude separation between recursive least squares with correct functional form, a model-based learner that must

estimate the demand and the cost coefficients, a population-based gradient-free search, and a tabular model-free agent. Whether the divergence story would reappear in an expectational cobweb in which beliefs about future prices drive supply, and in particular how a learned-neural-network world model would behave in that regime, is one of the open questions the chapter does not yet settle. A second open question surfaces from the gap between the closed-form linear-quadratic planner and the branched-rollout REINFORCE planner on the cobweb panel: when the environment admits a closed-form planner, branched rollouts pay a variance cost that closed-form planning does not, and whether this gap survives on environments without analytical planners (precisely where deep MBPO is deployed in practice) is the empirical question the deep continuous-control benchmarks were designed to answer. The analogy to E-stability is suggestive rather than formal; the operators, fixed points, and misspecification objects differ, and a stability theory for model-based reinforcement learning in economic environments remains to be built. **?** is the closest current analogue to a stability condition for the reinforcement-learning side, stating that value error grows geometrically along the rollout at rate K_F (the Lipschitz constant of the learned dynamics) and remains finite when $\gamma K_F < 1$, but the result is a per-rollout contraction bound on value error rather than a fixed-point characterization of the joint learning dynamics in the sense of Marcet and Sargent (1989).

Read against the monograph as a whole, this chapter is where the simulator dependence of reinforcement learning becomes explicit. The agent's policy is only as good as the model it plans against, and the question of when a learned model is accurate enough for an economic policy counterfactual is the open one the monograph carries into its conclusion.

18 Discussion

This concluding section develops concrete research agendas at the intersection of reinforcement learning and the applied sciences as well as lists the bottlenecks and open challenges.

18.1 How Domain Structure Improves Reinforcement Learning

The largest-scale RL successes (Atari, Go, protein folding) share a common ingredient: a cheap, fast, and accurate simulator that generates unlimited training data. Most applied domains lack this ingredient. Every application in Section 7 demanded a custom environment, and the engineering cost of building these environments often dominated the cost of training the RL agent itself. Structural models can fill this gap: they encode variable selection, causal structure, and institutional constraints that determine how agents respond to interventions. The Lucas critique warns that correlational simulators, trained on observational data without structural assumptions, will break under policy changes (Section 14). Building simulators that respect causal identification and correctly model agent responses to rule changes is an important problem.

Domain structure, when available, can dramatically reduce the sample complexity of learning. The knowledge ladder in Section 11 shows that imposing demand structure on a pricing problem can reduce cumulative regret from $\Theta(T)$ to $O(\log T)$. The pattern extends beyond bandits: structural assumptions yield similar gains in dynamic estimation (Section 8) and preference learning (Section 12). These reflect the difference between learning in an unstructured space and learning in one shaped by theory. Formalizing this intuition, identifying which structural assumptions yield which complexity reductions and under what conditions, is an active research frontier.

The RLHF and DPO frameworks studied in Section 12 rely on the Bradley-Terry model, one of the simplest members of the discrete choice family. The discrete choice literature offers a rich toolkit for moving beyond it. Mixed logit models accommodate heterogeneous preferences across annotators. Revealed preference theory provides axiomatic consistency constraints, such as

GARP and stochastic transitivity, that can serve as regularizers on learned reward models. The preference learning simulation in Section 12 illustrates the cost of misspecification (Figure 29). Integrating tools from econometrics for preference elicitation, model selection, and specification testing into the RLHF pipeline is a natural direction.

18.2 How Reinforcement Learning Advances Applied Modeling

Dynamic programming has always offered a prescriptive capability: given a model, compute the optimal policy. In practice, this capability has been limited by the curse of dimensionality to models with small, discrete state spaces. RL relaxes this constraint. TD-based methods make structural models tractable at state-space scales where NFXP is infeasible (Section 8), and real-world deployments confirm that RL can compute policies of practical value (Section 7). These results suggest a prescriptive role for RL: not merely estimating model parameters (the traditional estimation task), but computing the policies those parameters imply.

Social science and policy research work with vast quantities of observational data from settings where controlled experimentation is impossible or unethical. Offline RL, which learns policies from logged data without further interaction, is a natural fit. The simulation in Section 12 illustrates both the promise and the limits: algorithms with distributional shift correction exceed the behavioral baseline, while those without it degrade below it, and performance depends critically on data support (Table 24, Figure 26). Offline RL has clearly delineated conditions under which learning is possible, conditions that connect directly to familiar statistical concepts like overlap and common support. Developing standardized benchmarks and digital twins for offline RL can enable firms and policy-makers to design optimal policies from data generated under old ones.

RL also offers a descriptive model of how boundedly rational agents learn in strategic environments. The simulations in Section 10 demonstrate that independent Q-learning agents converge to Nash equilibrium through trial and error, providing "as-if" microfoundations for equilibrium concepts: the equilibrium arises not from common knowledge of rationality, but from a simple adaptive process.

When RL-trained agents are actually deployed in markets, they become market actors subject to empirical scrutiny. Algorithmic pricing agents, for instance, have been shown to sustain supra-competitive prices through reward-based learning without explicit communication. Competition authorities in the EU, US, and UK are actively investigating whether algorithmic coordination constitutes tacit collusion.[296] As algorithmic agents proliferate in pricing, trading, content recommendation, and resource allocation, the empirical study of algorithmic market behavior is a growing area of research.

World models, surveyed in Section 17, are a third locus where reinforcement learning advances applied modeling. The cobweb and fishery simulations in that chapter show that a small learned dynamics model paired with planning can outperform model-free baselines on economic environments where the state space is modest and the dynamics are partially known to be self-referential or exogenous-stochastic. Read against the Lucas critique raised earlier in this conclusion, world-model methods make the model itself a first-class object the analyst can inspect and constrain, rather than a black-box simulator behind a policy. The cost is calibration. Value-aware losses give task-relevant accuracy where it matters for decisions but offer weaker guarantees against policy drift, a tension the literature on calibrated value-aware models is just beginning to address.

[296] The companion thesis (Rawat, 2026) develops a framework for evaluating algorithmic inefficiency and collusion risk in algorithmically mediated markets, combining simulators with factorial experimental designs.

18.3 Open Challenges

The deadly triad (function approximation, bootstrapping, off-policy learning) remains unresolved. Deep RL algorithms exhibit seed sensitivity, overestimation cascades, and plasticity loss that make reproducibility difficult even in controlled settings (Section 6).

Multi-agent RL faces fundamental problems (Section 10). Computing Nash equilibria is PPAD-complete, and RL does not escape this hardness. In games with multiple equilibria, agents performing Bellman backups may select different equilibria for different states, causing value iteration to cycle or diverge. The literature on equilibrium refinement, focal points, and coordination mechanisms may offer partial resolutions, but a general solution remains open.

The absence of standardized simulators is part of a broader infrastructure gap. Industrial RL deployments require dedicated engineering teams, GPU clusters, and months of hyperparameter tuning (Section 7, Section 6). Reducing these barriers through shared simulation environments and accessible software libraries would accelerate adoption.

18.4 Conclusion

Reinforcement learning extends the reach of dynamic programming to problems that were previously intractable, and it connects to established statistical frameworks through shared mathematical foundations (Section 2.3). The research agendas outlined above (structural simulators, the role of structural assumptions in sample complexity, inference procedures for learned policies) are concrete and tractable. They draw on domain knowledge for structure and institutional context and on RL for computation.

References

Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 22–31, 2017.

Karun Adusumilli and Dita Eckardt. Temporal-difference estimation of dynamic discrete choice models. *arXiv preprint arXiv:2209.15174*, 2022.

Alekh Agarwal, Sham Kakade, and Lin F. Yang. Model-based reinforcement learning with a generative model is minimax optimal. In *Conference on Learning Theory (COLT)*, 2020a.

Alekh Agarwal, Sham M. Kakade, Jason D. Lee, and Gaurav Mahajan. On the theory of policy gradient methods: Optimality, approximation, and distribution shift. *Journal of Machine Learning Research*, 22(98), 2021a.

Rishabh Agarwal, Dale Schuurmans, and Mohammad Norouzi. An optimistic perspective on offline reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2020b.

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, volume 34, 2021b.

Shipra Agrawal and Wei Tang. Dynamic pricing with reference price effects. *arXiv preprint arXiv:2301.02497*, 2024.

S. Rao Aiyagari. Uninsured idiosyncratic risk and aggregate saving. *The Quarterly Journal of Economics*, 109(3):659–684, 1994. doi: 10.2307/2118417.

Anurag Ajay, Yilun Du, Abhishek Gupta, Joshua B. Tenenbaum, Tommi S. Jaakkola, and Pulkit Agrawal. Is conditional generative modeling all you need for decision-making? In *International Conference on Learning Representations*, 2023. arXiv:2211.15657.

Robert Almgren and Neil Chriss. Optimal execution of portfolio transactions. *Journal of Risk*, 3(2):5–40, 2001.

Eitan Altman. *Constrained Markov Decision Processes*. Chapman and Hall/CRC, 1999.

Shun-ichi Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10(2): 251–276, 1998.

Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphael Marinier, Leonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, Sylvain Gelly, and Olivier Bachem. What matters for on-policy deep actor-critic methods? a large-scale study. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

András Antos, Csaba Szepesvári, and Rémi Munos. Learning near-optimal policies with Bellman-residual minimization based fitted policy iteration and a single sample path. *Machine Learning*, 71(1):89–129, 2008. doi: 10.1007/s10994-007-5038-2.

Peter Arcidiacono and Robert A. Miller. Conditional choice probability estimation of dynamic discrete choice models with unobserved heterogeneity. *Econometrica*, 79(6):1823–1867, 2011.

Amanda Askell, Yuntao Bai, Anna Chen, Dawn Drain, Deep Ganguli, Tom Henighan, Andy Jones, Nicholas Joseph, Ben Mann, Nova DasSarma, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Jackson Kernion, Kamal Ndousse, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A general language assistant as a laboratory for alignment. *arXiv preprint arXiv:2112.00861*, 2021.

John Asker, Chaim Fershtman, Jihye Jeon, and Ariel Pakes. A computational framework for analyzing dynamic auctions: The market impact of information sharing. *The RAND Journal of Economics*, 51(3):805–839, 2020.

Tohid Atashbar and Rui Aruhan Shi. Deep reinforcement learning: Emerging trends in macroeconomics and future prospects. Working Paper 2022/259, International Monetary Fund, 2022.

Tohid Atashbar and Rui Aruhan Shi. AI and macroeconomic modeling: Deep reinforcement learning in an RBC model. IMF Working Paper WP/23/40, International Monetary Fund, February 2023.

Susan Athey and Stefan Wager. Policy learning with observational data. *Econometrica*, 89(1): 133–161, 2021.

Adrien Auclert, Bence Bardóczy, Matthew Rognlie, and Ludwig Straub. Using the sequence-space Jacobian to solve and estimate heterogeneous-agent models. *Econometrica*, 89(5):2375–2408, 2021. doi: 10.3982/ECTA17434.

Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2):235–256, 2002a.

Peter Auer, Nicolò Cesa-Bianchi, Yoav Freund, and Robert E. Schapire. The nonstochastic multiarmed bandit problem. *SIAM Journal on Computing*, 32(1):48–77, 2002b.

Lawrence M. Ausubel and Raymond J. Deneckere. Reputation in bargaining and durable goods monopoly. *Econometrica*, 57(3):511–531, 1989.

Lawrence M. Ausubel, Peter Cramton, and Raymond J. Deneckere. Bargaining with incomplete information. In Robert J. Aumann and Sergiu Hart, editors, *Handbook of Game Theory with Economic Applications*, volume 3, pages 1897–1945. Elsevier, 2002.

Mohammad Gheshlaghi Azar, Rémi Munos, and Hilbert J Kappen. Minimax pac bounds on the sample complexity of reinforcement learning with a generative model. *Machine Learning*, 91(1):7–32, 2013.

Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Remi Munos. A general theoretical paradigm to understand learning from human preferences. In *Proceedings of the 27th International Conference on Artificial Intelligence and Statistics*, volume 238 of *Proceedings of Machine Learning Research*, 2024.

Marlon Azinovic-Yang and Jan Žemlička. Deep learning in the sequence space. *arXiv preprint arXiv:2509.13623*, 2026.

Ashwinkumar Badanidiyuru, Robert Kleinberg, and Aleksandrs Slivkins. Bandits with knapsacks. *IEEE 54th Annual Symposium on Foundations of Computer Science*, pages 207–216, 2013.

Leemon Baird. Residual algorithms: Reinforcement learning with function approximation. In *Proceedings of the Twelfth International Conference on Machine Learning*, pages 30–37. Morgan Kaufmann, 1995.

Leemon C. Baird. Advantage updating. Technical Report WL-TR-93-1146, Wright Laboratory, Wright-Patterson AFB, 1993.

James Bannon, Brad Langlois, Raimundo Fernandez, and Danielle Maddix. Causality and batch reinforcement learning: Complementary approaches to planning in unknown domains. In *NeurIPS Workshop on Causal Discovery and Causality-Inspired Machine Learning*, 2020.

Elias Bareinboim and Judea Pearl. Causal inference and the data-fusion problem. *Proceedings of the National Academy of Sciences*, 113(27):7345–7352, 2016.

Elias Bareinboim, Andrew Forney, and Judea Pearl. Bandits with unobserved confounders: A causal approach. In *Advances in Neural Information Processing Systems*, volume 28, pages 1342–1350, 2015.

Francisco Barillas, Lars Peter Hansen, and Thomas J. Sargent. Doubts or variability? *Journal of Economic Theory*, 144(6):2388–2418, 2009.

Andrew G. Barto, Richard S. Sutton, and Charles W. Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-13(5):834–846, 1983.

Nicole Bäuerle and Jonathan Ott. Markov decision processes with average-value-at-risk criteria. *Mathematical Methods of Operations Research*, 74:361–379, 2011.

Marc G. Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 449–458, 2017.

Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

Jess Benhabib, Stephanie Schmitt-Grohé, and Martín Uribe. The perils of taylor rules. *Journal of Economic Theory*, 96(1–2):40–69, 2001. doi: 10.1006/jeth.1999.2585.

Andrew Bennett and Nathan Kallus. Proximal reinforcement learning: Efficient off-policy evaluation in partially observed Markov decision processes. *Operations Research*, 72(3):1071–1086, 2023. doi: 10.1287/opre.2021.0781.

Dimitri P. Bertsekas. *Dynamic Programming and Optimal Control*. Athena Scientific, 1996.

Dimitri P. Bertsekas. Lessons from AlphaZero for optimal, model predictive, and adaptive control. *Athena Scientific Reports*, 2021.

Dimitri P. Bertsekas. *Abstract Dynamic Programming*. Athena Scientific, Belmont, MA, 3rd edition, 2022a.

Dimitri P. Bertsekas. Newton's method for reinforcement learning and model predictive control. *Results in Control and Optimization*, 7:100121, 2022b.

Dimitri P. Bertsekas and John N. Tsitsiklis. *Neuro-Dynamic Programming*. Athena Scientific, Belmont, MA, 1996.

Jalaj Bhandari, Daniel Russo, and Raghav Singal. A finite time analysis of temporal difference learning with linear function approximation. *Operations Research*, 69(3), 2021.

Aurélien Bibaut, Antoine Chambaz, Maria Dimakopoulou, Nathan Kallus, and Mark van der Laan. Post-contextual-bandit inference. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Palok Biswas et al. JUSTICE: Multi-objective multi-agent reinforcement learning for equitable climate policy. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

David Blackwell. Discounted dynamic programming. *Annals of Mathematical Statistics*, 36(1): 226–235, 1965.

David M. Blei, Alp Kucukelbir, and Jon D. McAuliffe. Variational inference: A review for statisticians. *Journal of the American Statistical Association*, 112(518):859–877, 2017.

Han Bleichrodt, Kirsten I. M. Rohde, and Peter P. Wakker. Koopmans' constant discounting for intertemporal choice: A simplification and a generalization. *Journal of Mathematical Psychology*, 52:341–347, 2008.

Tilman Börgers and Rajiv Sarin. Learning through reinforcement and replicator dynamics. *Journal of Economic Theory*, 77(1):1–14, 1997.

Vivek S. Borkar. Stochastic approximation with two time scales. *Systems & Control Letters*, 29(5):291–294, 1997.

Vivek S. Borkar and Sean P. Meyn. The ODE method for convergence of stochastic approximation and reinforcement learning. *SIAM Journal on Control and Optimization*, 38(2):447–469, 2000.

Michael Bowling and Manuela Veloso. Multiagent learning using a variable learning rate. *Artificial Intelligence*, 136(2):215–250, 2002.

Michael Bowling, Neil Burch, Michael Johanson, and Oskari Tammelin. Heads-up limit hold'em poker is solved. *Science*, 347(6218):145–149, 2015.

Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.

David Brandfonbrener, Alberto Bietti, Jacob Buckman, Romain Laroche, and Joan Bruna. When does return-conditioned supervised learning work for offline reinforcement learning? In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Gianluca Brero, Alon Eden, Matthias Gerstgrasser, David C. Parkes, and Duncan Rheingans-Yoo. Reinforcement learning of sequential price mechanisms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 13662–13670, 2021.

William A. Brock and Leonard J. Mirman. Optimal economic growth and uncertainty: The discounted case. *Journal of Economic Theory*, 4(3):479–513, 1972.

Josef Broder and Paat Rusmevichientong. Dynamic pricing under a general parametric choice model. *Operations Research*, 60(4):965–980, 2012.

George W. Brown. Iterative solution of games by fictitious play. In Tjalling C. Koopmans, editor, *Activity Analysis of Production and Allocation*, pages 374–376. John Wiley & Sons, 1951.

Noam Brown and Tuomas Sandholm. Superhuman ai for heads-up no-limit poker: Libratus beats top professionals. *Science*, 359(6374):418–424, 2018.

Noam Brown and Tuomas Sandholm. Superhuman ai for multiplayer poker. *Science*, 365(6456): 885–890, 2019.

Noam Brown, Adam Lerer, Sam Gross, and Tuomas Sandholm. Deep counterfactual regret minimization. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97, pages 793–802. PMLR, 2019.

Axel Brunnbauer, Julian Lemmel, Sophie Neubauer, Zahra Babaiee, and Radu Grosu. Scalable offline reinforcement learning for mean field games. *arXiv preprint arXiv:2410.17898*, 2024.

David A. Bruns-Smith. Model-free and model-based policy evaluation when causality is uncertain. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 1116–1126. PMLR, 2021.

Simone Brusatin, Tommaso Padoan, Domenico Delli Gatti, Andrea Coletta, and Aldo Glielmo. Simulating the economic impact of rationality through reinforcement learning and agent-based modelling. In *Proceedings of the 5th ACM International Conference on AI in Finance (ICAIF)*, 2024. doi: 10.1145/3677052.3698621.

Lars Buesing, Theophane Weber, Yori Zwols, Sebastien Racaniere, Arthur Guez, Jean-Baptiste Lespiau, and Nicolas Heess. Woulda, coulda, shoulda: Counterfactually-guided policy search. In *International Conference on Learning Representations*, 2019.

Jeremy I. Bulow. Durable-goods monopolists. *Journal of Political Economy*, 90(2):314–332, 1982.

Junhui Cai, Ran Chen, Martin J. Wainwright, and Linda Zhao. Doubly high-dimensional contextual bandits: An interpretable model for joint assortment-pricing. *arXiv preprint arXiv:2309.08634*, 2023.

Emilio Calvano, Giacomo Calzolari, Vincenzo Denicolò, and Sergio Pastorello. Artificial intelligence, algorithmic pricing, and collusion. *American Economic Review*, 110(10):3267–3297, 2020.

Haoyang Cao, Samuel N. Cohen, and Lukasz Szpruch. Identifiability in inverse reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Shicong Cen, Chen Cheng, Yuxin Chen, Yuting Wei, and Yuejie Chi. Fast global convergence of natural policy gradient methods with entropy regularization. *Operations Research*, 70(4), 2022.

Souradip Chakraborty, Jiahao Qiu, Hui Yuan, Alec Koppel, Dinesh Manocha, Furong Huang, Amrit Singh Bedi, and Mengdi Wang. MaxMin-RLHF: Alignment with diverse human preferences. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

Ji Chen, Yifan Xu, Peiwen Yu, and Jun Zhang. A reinforcement learning approach for hotel revenue management with evidence from field experiments. *Journal of Operations Management*, 69(7):1176–1201, 2023a. doi: 10.1002/joom.1246.

Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Mingli Chen, Andreas Joseph, Michael Kumhof, Xinlei Pan, and Xuan Zhou. Deep reinforcement learning in a monetary model. *arXiv preprint arXiv:2104.09368*, 2023b. Bank of England Staff Working Paper, January 2023; first arXiv version April 2021.

Yiting Chen, Tracy Xiao Liu, You Shan, and Songfa Zhong. The emergence of economic rationality of GPT. *Proceedings of the National Academy of Sciences*, 120(51):e2316205120, 2023c. doi: 10.1073/pnas.2316205120.

Yuxin Chen, Jieming Mao, and Rui Miao. Dynamic pricing with fairness constraints. *arXiv preprint arXiv:2402.07834*, 2025.

Victor Chernozhukov, Denis Chetverikov, Mert Demirer, Esther Duflo, Christian Hansen, Whitney Newey, and James Robins. Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21(1):C1–C68, 2018. doi: 10.1111/ectj.12097.

Victor Chernozhukov, Whitney Newey, Rahul Singh, and Vasilis Syrgkanis. Automatic debiased machine learning for dynamic treatment effects and general nested functionals. *arXiv preprint arXiv:2203.13887*, 2023.

Khai Xiang Chiong, Alfred Galichon, and Matt Shum. Duality in dynamic discrete-choice models. *Quantitative Economics*, 7(1):83–115, 2016.

Yinlam Chow, Aviv Tamar, Shie Mannor, and Marco Pavone. Risk-sensitive and robust decision-making via CVaR optimization in Markov decision processes. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

Sayak Ray Chowdhury, Anush Kini, and Nagarajan Natarajan. Provably robust DPO: Aligning language models with noisy feedback. *arXiv preprint arXiv:2403.00409*, 2024.

Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Kamil Ciosek, Quan Vuong, Robert Lierowski, and Katja Hofmann. Better exploration with optimistic actor-critic. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Andrew J. Clark and Herbert Scarf. Optimal policies for a multi-echelon inventory problem. *Management Science*, 6(4):475–490, 1960.

Pierre Clavier, Erwan Le Pennec, and Matthieu Geist. Towards minimax optimality of model-based robust reinforcement learning. In *Conference on Uncertainty in Artificial Intelligence (UAI)*, 2024.

Ronald H. Coase. Durability and monopoly. *Journal of Law and Economics*, 15(1):143–149, 1972.

Vincent Conitzer, Rachel Freedman, Jobst Heitzig, Wesley H. Holliday, Bob M. Jacobs, Nathan Lambert, Milan Mosse, Eric Pacuit, Stuart Russell, Hailey Schoelkopf, Emanuel Tewolde, and William S. Zwicker. Position: Social choice should guide AI alignment in dealing with diverse human feedback. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 9346–9360, 2024.

Matias Covarrubias. Dynamic oligopoly and monetary policy: A deep reinforcement learning approach. Job Market Paper, New York University, 2022.

Michael Curry, Alexander Trott, Soham Phade, Yu Bai, and Stephan Zheng. Analyzing micro-founded general equilibrium models with many agents using deep reinforcement learning. In *arXiv preprint arXiv:2201.01163*, 2022.

Cristiano da Costa Cunha, Wei Liu, Tim French, and Ajmal Mian. Unifying causal reinforcement learning: Survey, taxonomy, algorithms and applications. *arXiv preprint arXiv:2512.18135*, 2025.

Will Dabney, Georg Ostrovski, David Silver, and Rémi Munos. Implicit quantile networks for distributional reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 1096–1105, 2018a.

Will Dabney, Mark Rowland, Marc G. Bellemare, and Rémi Munos. Distributional reinforcement learning with quantile regression. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018b.

Constantinos Daskalakis, Paul W. Goldberg, and Christos H. Papadimitriou. The complexity of computing a nash equilibrium. *SIAM Journal on Computing*, 39(1):195–259, 2009.

Peter Dayan. The convergence of TD(λ) for general λ. *Machine Learning*, 8(3–4):341–362, 1992.

Eric V. Denardo. Contraction mappings in the theory underlying dynamic programming. *SIAM Review*, 9(2):165–177, 1967.

Zhihong Deng, Jing Jiang, Guodong Long, and Chengqi Zhang. Causal reinforcement learning: A survey. *Transactions on Machine Learning Research*, 2023.

Esther Derman, Matthieu Geist, and Shie Mannor. Twice regularized MDPs and the equivalence between robustness and regularization. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Dongsheng Ding, Kaiqing Zhang, Tamer Başar, and Mihailo Jovanović. Natural policy gradient primal-dual method for constrained Markov decision processes. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

Shibhansh Dohare, J. Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A. Mahmoud, and Richard S. Sutton. Loss of plasticity in deep continual learning. *Nature*, 632: 768–774, 2024.

Pierluca D'Oro, Max Schwarzer, Evgenii Nikishin, Pierre-Luc Bacon, Marc G. Bellemare, and Aaron Courville. Sample-efficient reinforcement learning by breaking the replay ratio barrier. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7 (1):1–26, 1979.

Theresa Eimer, Marius Lindauer, and Roberta Raileanu. Hyperparameters in reinforcement learning and how to tune them. In *Proceedings of the 40th International Conference on Machine Learning*, Proceedings of Machine Learning Research, pages 9104–9149, 2023.

Scott Emmons, Benjamin Eysenbach, Ilya Kostrikov, and Sergey Levine. RvS: What is essential for offline RL via supervised learning? In *International Conference on Learning Representations (ICLR)*, 2022.

Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. Implementation matters in deep RL: A case study on PPO and TRPO. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.

Damien Ernst, Pierre Geurts, and Louis Wehenkel. Tree-based batch mode reinforcement learning. *Journal of Machine Learning Research*, 6:503–556, 2005.

Lasse Espeholt, Hubert Soyer, Rémi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In *International Conference on Machine Learning*, 2018.

Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. KTO: Model alignment as prospect theoretic optimization. *arXiv preprint arXiv:2402.01306*, 2024.

George W. Evans and Seppo Honkapohja. Policy interaction, expectations and the liquidity trap. *Review of Economic Dynamics*, 8(2):303–323, 2005. doi: 10.1016/j.red.2005.01.002.

Eyal Even-Dar and Yishay Mansour. Learning rates for q-learning. *Journal of Machine Learning Research*, 5:1–25, 2003.

Jianqing Fan, Yongyi Guo, and Mengxin Yu. Semiparametric dynamic pricing. *arXiv preprint*, 2024. arXiv ID pending verification; the ID 2401.01136 previously listed resolved to an unrelated paper.

Nan Fang, Wei Gong, and Guiliang Liu. Offline inverse constrained reinforcement learning for safe-critical decision making in healthcare. *arXiv preprint arXiv:2410.07525*, 2024.

John Fearnley. Exponential lower bounds for policy iteration. In *International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 551–562, 2010.

William Fedus, Prajit Ramachandran, Rishabh Agarwal, Yoshua Bengio, Hugo Larochelle, Mark Rowland, and Marc G. Bellemare. Revisiting fundamentals of experience replay. In *Proceedings of the 37th International Conference on Machine Learning*, Proceedings of Machine Learning Research, 2020.

Mattie Fellows, Matthew J. A. Smith, and Shimon Whiteson. Why target networks stabilise temporal difference methods. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 9886–9909. PMLR, 2023.

Jesús Fernández-Villaverde, Galo Nuño, and Jesse Perla. Taming the curse of dimensionality: Quantitative economics with deep learning. NBER Working Paper 33117, National Bureau of Economic Research, 2024.

Jesús Fernández-Villaverde, Galo Nuño, and Jesse Perla. Taming the curse of dimensionality: Quantitative economics with deep learning. Working Paper 33117, National Bureau of Economic Research, 2024.

Chaim Fershtman and Ariel Pakes. Dynamic games with asymmetric information: A framework for empirical work. *The Quarterly Journal of Economics*, 127(4):1611–1661, 2012.

Wendell H. Fleming and William M. McEneaney. Risk-sensitive control on an infinite time horizon. *SIAM Journal on Control and Optimization*, 33(6):1881–1915, 1995.

Mogens Fosgerau and Jesper R.-V. Sørensen. How McFadden met Rockafellar and learned to do more with less. *Journal of Mathematical Economics*, 100:102629, 2022.

Dylan J. Foster and Vasilis Syrgkanis. Orthogonal statistical learning. *Annals of Statistics*, 51 (3):879–908, 2023.

Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *Proceedings of the 35th International Conference on Machine Learning*, Proceedings of Machine Learning Research, pages 1587–1596, 2018.

Scott Fujimoto, Edoardo Conti, Mohammad Ghavamzadeh, and Joelle Pineau. Benchmarking batch deep reinforcement learning algorithms. *arXiv preprint arXiv:1910.01708*, 2019a. Introduces the discrete BCQ-D variant that thresholds on the behavioral action probability.

Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2052–2062. PMLR, 2019b.

Scott Fujimoto, David Meger, Doina Precup, Ofir Nachum, and Shixiang Shane Gu. Why should I trust you, Bellman? the Bellman error is a poor replacement for value error. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, 2022.

Federico Gabriele, Aldo Glielmo, and Marco Taboga. Heterogeneous RBCs via deep multi-agent reinforcement learning. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2026. doi: 10.65109/VZHC1838.

Ravi Ganti, Matyas Sustik, Quoc Tran, and Brian Seaman. Thompson sampling for dynamic pricing. *arXiv preprint arXiv:1802.03050*, 2018.

Lin Ge, Jitao Wang, Chengchun Shi, Zhenke Wu, and Rui Song. A reinforcement learning framework for dynamic mediation analysis. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 11050–11097. PMLR, 2023.

Luise Ge, Daniel Halpern, Evi Micha, Ariel D. Procaccia, Itai Shapira, Yevgeniy Vorobeychik, and Junlin Wu. Axioms for AI alignment from human feedback. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

Matthieu Geist, Bruno Scherrer, and Olivier Pietquin. A theory of regularized Markov decision processes. In *International Conference on Machine Learning (ICML)*, 2019.

Joren Gijsbrechts, Robert N. Boute, Jan A. Van Mieghem, and Dennis J. Zhang. Can deep reinforcement learning improve inventory management? Performance on dual sourcing, lost sales, and multi-echelon problems. *Manufacturing & Service Operations Management*, 24(3): 1349–1368, 2022.

Itzhak Gilboa and David Schmeidler. Maxmin expected utility with non-unique prior. *Journal of Mathematical Economics*, 18(2):141–153, 1989.

John C. Gittins. Bandit processes and dynamic allocation indices. *Journal of the Royal Statistical Society: Series B (Methodological)*, 41(2):148–164, 1979.

Ali Goli and Amandeep Singh. Frontiers: Can large language models capture human preferences? *Marketing Science*, 43(4):709–722, 2024. doi: 10.1287/mksc.2023.0306.

Christian Gourieroux, Alain Monfort, and Eric Renault. Indirect inference. *Journal of Applied Econometrics*, 8(S1):S85–S118, 1993.

Julien Grand-Clément and Christian Kroer. First-order methods for Wasserstein distributionally robust MDP. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pages 2010–2019, 2021.

Amy Greenwald and Keith Hall. Correlated q-learning. In *Proceedings of the 20th International Conference on Machine Learning*, pages 242–249, 2003.

Faruk Gul, Hugo Sonnenschein, and Robert Wilson. Foundations of dynamic monopoly and the Coase conjecture. *Journal of Economic Theory*, 39(1):155–190, 1986.

Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement learning with deep energy-based policies. In *Proceedings of the 34th International Conference on Machine Learning*, 2017.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *Proceedings of the 35th International Conference on Machine Learning*, pages 1861–1870, 2018.

Vitor Hadad, David A. Hirshberg, Ruohan Zhan, Stefan Wager, and Susan Athey. Confidence intervals for policy evaluation in adaptive experiments. *Proceedings of the National Academy of Sciences*, 118(15):e2014602118, 2021.

Ben Hambly, Renyuan Xu, and Huining Yang. Recent advances in reinforcement learning in finance. *Mathematical Finance*, 33(3):437–503, 2023.

Jiequn Han, Yucheng Yang, and Weinan E. DeepHAM: A global solution method for heterogeneous agent models with aggregate shocks. *arXiv preprint arXiv:2112.14377*, 2022a.

Xiao Han, Weijian Zhang, Jiaxin Wang, Fan Zhang, and Jieping Ye. A better match for drivers and riders: Reinforcement learning at Lyft. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2927–2936, 2022b.

Lars Peter Hansen and Thomas J. Sargent. Robust control and model uncertainty. *American Economic Review*, 91(2):60–66, 2001.

Lars Peter Hansen and Thomas J. Sargent. Risk, ambiguity, and misspecification: Decision theory, robust control, and statistics. *Journal of Applied Econometrics*, 39(6):969–999, 2024.

Moritz Hardt, Nimrod Megiddo, Christos Papadimitriou, and Mary Wootters. Strategic classification. In *Proceedings of the 2016 ACM Conference on Innovations in Theoretical Computer Science (ITCS)*, pages 111–122, 2016.

Jia Lin Hau, Erick Delage, Mohammad Ghavamzadeh, and Marek Petrik. On dynamic programming decompositions of static risk measures in Markov decision processes. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Johannes Heinrich and David Silver. Deep reinforcement learning from self-play in imperfect-information games. *arXiv preprint arXiv:1603.01121*, 2016.

Jobst Heitzig et al. Conditional commitments for stable climate cooperation. In *AAAI Workshop on AI for Climate Change*, 2023.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, 2018.

Natascha Hinterlang and Tobias Taenzer. Optimal monetary policy using reinforcement learning. *Journal of Monetary Economics*, 2024.

Brett Hollenbeck. Horizontal mergers and innovation in concentrated industries. *Quantitative Marketing and Economics*, 2019.

Jiwoo Hong, Noah Lee, and James Thorne. ORPO: Monolithic preference optimization without reference model. *arXiv preprint arXiv:2403.07691*, 2024a.

Mao Hong, Zhengling Qi, and Yanxun Xu. A policy gradient method for confounded POMDPs. In *International Conference on Learning Representations*, 2024b.

V. Joseph Hotz and Robert A. Miller. Conditional choice probabilities and the estimation of dynamic models. *The Review of Economic Studies*, 60(3):497–529, 1993.

Ronald A. Howard. *Dynamic Programming and Markov Processes*. The Technology Press of M.I.T. and John Wiley and Sons, New York, NY, 1960.

Junling Hu and Michael P Wellman. Nash q-learning for general-sum stochastic games. *Journal of Machine Learning Research*, 4(Nov):1039–1069, 2003.

Yingyao Hu and Matthew Shum. Nonparametric identification of dynamic models with unobserved state variables. *Journal of Econometrics*, 171(1):32–44, 2012.

Yingyao Hu and Fangzhu Yang. Estimation of dynamic discrete choice models with unobserved state variables using reinforcement learning. *Working Paper, Johns Hopkins University*, 2025.

Shengyi Huang, Rousslan Fernand Julien Dossa, Antonin Raffin, Anssi Kanervisto, and Weng Wang. The 37 implementation details of proximal policy optimization. *ICLR Blog Track*, 2022.

Julian Ibarz, Jie Tan, Chelsea Finn, Mrinal Kalakrishnan, Peter Pastor, and Sergey Levine. How to train your robot with deep reinforcement learning: Lessons we have learned. *The International Journal of Robotics Research*, 40(4-5):698–721, 2021.

Takanori Ida, Takunori Ishihara, Koichiro Ito, Daido Kido, Toru Kitagawa, Shosei Sakaguchi, and Shusaku Sasaki. Choosing who chooses: Selection-driven targeting in energy rebate programs. Technical Report 32561, National Bureau of Economic Research, 2024.

Mitsuru Igami. Artificial intelligence as structural estimation: Deep blue, bonanza, and alphago. *The Econometrics Journal*, 23(3):S1–S24, 2020.

Andrew Ilyas, Logan Engstrom, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. A closer look at deep policy gradients. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.

Fedor Iskhakov, John Rust, and Bertel Schjerning. Machine learning and structural econometrics: Contrasts and synergies. *The Econometrics Journal*, 23(S1):S81–S124, 2020.

Garud N. Iyengar. Robust dynamic programming. *Mathematics of Operations Research*, 30(2): 257–280, 2005.

Tommi Jaakkola, Michael I. Jordan, and Satinder P. Singh. On the convergence of stochastic iterative dynamic programming algorithms. In *Advances in Neural Information Processing Systems 6*, pages 703–710. Morgan Kaufmann, 1994.

David H. Jacobson. Optimal stochastic linear systems with exponential performance criteria and their relation to deterministic differential games. *IEEE Transactions on Automatic Control*, 18(2):124–131, 1973.

Michael Janner, Qiyang Li, and Sergey Levine. Offline reinforcement learning as one big sequence modeling problem. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Michael Janner, Yilun Du, Joshua B. Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. In *International Conference on Machine Learning*, 2022. arXiv:2205.09991.

Adel Javanmard and Hamid Nazerzadeh. Dynamic pricing in high-dimensions. *Journal of Machine Learning Research*, 20(9):1–49, 2019.

Ying Jin, Zhuoran Yang, and Zhaoran Wang. Is pessimism provably efficient for offline RL? In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 5084–5096. PMLR, 2021.

Sham M. Kakade. A natural policy gradient. *Advances in Neural Information Processing Systems*, 14, 2001.

Sham M. Kakade. *A Natural Policy Gradient*. PhD thesis, University College London, 2002.

Dmitry Kalashnikov, Alex Irpan, Peter Pastor, Julian Ibarz, Alexander Herzog, Eric Jang, Deirdre Quillen, Ethan Holly, Mrinal Kalakrishnan, Vincent Vanhoucke, and Sergey Levine. Scalable deep reinforcement learning for vision-based robotic manipulation. In *Conference on Robot Learning*, 2018.

Nathan Kallus and Masatoshi Uehara. Efficiently breaking the curse of horizon in off-policy evaluation with double reinforcement learning. *Operations Research*, 70(6):3282–3302, 2022. doi: 10.1287/opre.2021.2249.

Nathan Kallus and Angela Zhou. Confounding-robust policy evaluation in infinite-horizon reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

Leon J. Kamin. Predictability, surprise, attention and conditioning. In Byron A. Campbell and Russell M. Church, editors, *Punishment and Aversive Behavior*, pages 279–296. Appleton-Century-Crofts, 1969.

Greg Kaplan, Benjamin Moll, and Giovanni L. Violante. Monetary policy according to HANK. *American Economic Review*, 108(3):697–743, 2018. doi: 10.1257/aer.20160042.

Brandon Kaplowitz. Reinforcement learning and consumption-savings behavior. Working paper, New York University, Stern School of Business, October 2025. arXiv:2510.20748.

Hilbert J. Kappen. Optimal control theory and the linear Bellman equation. *Inference and Learning in Dynamic Models*, pages 363–387, 2011.

Maximilian Kasy and Anja Sautmann. Adaptive treatment assignment in experiments for policy choice. *Econometrica*, 89(1):113–132, 2021.

Pramod Kaushik, Sneha Kummetha, Perusha Moodley, and Raju S. Bapi. A conservative Q-learning approach for handling distribution shift in sepsis treatment strategies. *arXiv preprint arXiv:2203.13884*, 2022.

Michael Kearns, Yishay Mansour, and Andrew Y. Ng. A sparse sampling algorithm for near-optimal planning in large Markov decision processes. *Machine Learning*, 49(2–3):193–208, 2002.

Tata Khundadze and Willi Semmler. European sovereign debt control through reinforcement learning. *Frontiers in Artificial Intelligence*, 8:1569395, 2025. doi: 10.3389/frai.2025.1569395.

Taylor W. Killian, Marzyeh Ghassemi, and Shalmali Joshi. Counterfactually guided off-policy transfer in clinical settings. In *Conference on Health, Inference, and Learning (CHIL)*, 2022.

Kuno Kim, Shivam Garg, Kirankumar Shiragur, and Stefano Ermon. Reward identification in inverse reinforcement learning. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *PMLR*, 2021.

Toru Kitagawa and Aleksey Tetenov. Who should be treated? Empirical welfare maximization methods for treatment choice. *Econometrica*, 86(2):591–616, 2018.

Robert Kleinberg and Tom Leighton. The value of knowing a demand curve: Bounds on regret for online posted-price auctions. In *Proceedings of the 44th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 594–605, 2003.

Thomas Kleine Buening, Jiarui Gan, Debmalya Mandal, and Marta Kwiatkowska. Strategyproof reinforcement learning from human feedback. *arXiv preprint arXiv:2503.09561*, 2025.

David L. Kleinman. On an iterative technique for Riccati equation computations. *IEEE Transactions on Automatic Control*, 13(1):114–115, 1968.

W. Bradley Knox, Stephane Hatgis-Kessell, Serena Booth, Scott Niekum, Peter Stone, and Alessandro Allievi. Models of human preference for learning reward functions. *arXiv preprint arXiv:2206.02231*, 2022.

Vijay R. Konda and John N. Tsitsiklis. Actor-critic algorithms. In *Advances in Neural Information Processing Systems 12*, pages 1008–1014. MIT Press, 2000.

Flemming Kondrup, Thomas Jiralerspong, Elaine Lau, Nathan de Lara, Jacob Shkrob, My Duc Tran, Doina Precup, and Sumana Basu. Towards safe mechanical ventilation treatment using deep offline reinforcement learning. In *AAAI Conference on Artificial Intelligence (Innovative Applications)*, 2023.

Tjalling C. Koopmans. Stationary ordinal utility and impatience. *Econometrica*, 28:287–309, 1960.

Tomasz Korbak, Ethan Perez, and Christopher L. Buckley. RL with KL penalties is better viewed as Bayesian inference. *arXiv preprint arXiv:2205.11275*, 2022.

Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit Q-Learning. In *International Conference on Learning Representations*, 2022.

Per Krusell and Anthony A. Smith. Income and wealth heterogeneity in the macroeconomy. *Journal of Political Economy*, 106(5):867–896, 1998. doi: 10.1086/250034.

Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-Learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

Aviral Kumar, Rishabh Agarwal, Dibya Ghosh, and Sergey Levine. Implicit under-parameterization inhibits data-efficient deep reinforcement learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

Artem Kuriksha. An economy of neural networks: Learning from heterogeneous experiences. Working paper, University of Pennsylvania, 2021. arXiv:2110.11582.

Harold J. Kushner and Dean S. Clark. *Stochastic Approximation Methods for Constrained and Unconstrained Systems*. Springer-Verlag, New York, 1978.

Tze Leung Lai and Herbert Robbins. Asymptotically efficient adaptive allocation rules. *Advances in Applied Mathematics*, 6(1):4–22, 1985.

Sascha Lange, Thomas Gabel, and Martin Riedmiller. Batch reinforcement learning. *Reinforcement Learning: State-of-the-Art*, pages 45–73, 2012.

Finnian Lattimore, Tor Lattimore, and Mark D. Reid. Causal bandits: Learning good interventions via causal inference. In *Advances in Neural Information Processing Systems*, volume 29, pages 1181–1189, 2016.

Tor Lattimore. Regret analysis of the finite-horizon Gittins index strategy for multi-armed bandits. In *Proceedings of the 29th Conference on Learning Theory (COLT)*, pages 1214–1245, 2016.

Sergey Levine. Reinforcement learning and control as probabilistic inference: Tutorial and review. *arXiv preprint arXiv:1805.00909*, 2018.

Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.

Arthur Lewbel. The identification zoo: Meanings of identification in econometrics. *Journal of Economic Literature*, 57(4):835–903, 2019.

Greg Lewis and Vasilis Syrgkanis. Double/debiased machine learning for dynamic treatment effects. In *Advances in Neural Information Processing Systems*, volume 34, pages 22695–22707, 2021.

Gen Li, Yuting Wei, Yuejie Chi, and Yuxin Chen. Softmax policy gradient methods can take exponential time to converge. *Mathematical Programming*, 196:579–632, 2022.

Gen Li, Laixi Shi, Yuxin Chen, Yuting Wei, and Yuejie Chi. Is q-learning minimax optimal? a tight sample complexity analysis. *Operations Research*, 72(1), 2024a.

Gen Li, Yuting Wei, Yuejie Chi, Yuantao Gu, and Yuxin Chen. Breaking the sample size barrier in model-based reinforcement learning with a generative model. *Operations Research*, 72(1), 2024b.

Mingxuan Li, Junzhe Zhang, and Elias Bareinboim. Causally aligned curriculum learning. In *International Conference on Learning Representations*, 2024c.

Minne Li, Zhiwei Qin, Yan Jiao, Yaodong Yang, Zheng Gong, Jun Wang, Changjie Wang, Gauge Wu, and Jieping Ye. Efficient ridesharing order dispatching with mean field multi-agent reinforcement learning. In *Proceedings of the 2019 World Wide Web Conference (WWW)*, pages 983–994, 2019.

Luofeng Liao, Zuyue Fu, Zhuoran Yang, Yixin Wang, Dingli Ma, Mladen Kolar, and Zhaoran Wang. Instrumental variable value iteration for causal offline reinforcement learning. *Journal of Machine Learning Research*, 25(303):1–56, 2024.

Peng Liao, Predrag Klasnja, and Susan A. Murphy. Off-policy estimation of long-term average outcomes with applications to mobile health. *Journal of the American Statistical Association*, 116(533):382–391, 2021. doi: 10.1080/01621459.2020.1807993.

Han-Dong Lim and Donghwan Lee. Regularized q-learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Long-Ji Lin. *Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching*. PhD thesis, Carnegie Mellon University, 1992. Also published in *Machine Learning*, 8(3–4):293–321, 1992.

Michael L Littman. Markov games as a framework for multi-agent reinforcement learning. In *Machine Learning Proceedings 1994*, pages 157–163. Morgan Kaufmann, 1994.

Michael L. Littman. Friend-or-foe q-learning in general-sum games. In *Proceedings of the 18th International Conference on Machine Learning*, pages 322–328, 2001.

Allen Liu, Jingwen Yang, Yining Wang, and Jianghao Sun. Contextual dynamic pricing with strategic buyers. *arXiv preprint arXiv:2307.04055*, 2024a.

Jiaxi Liu, Xiaoqing Wang, Yuming Deng, Xingyu Wu, and Yidong Zhang. Dynamic pricing on E-commerce platform with deep reinforcement learning: A field experiment. *arXiv preprint arXiv:1912.02572*, 2019.

Yanli Liu, Kaiqing Ding, and Javad Lavaei. Policy optimization for constrained MDPs with provably fast convergence. *arXiv preprint arXiv:2111.00552*, 2022.

Zuxin Liu, Zijian Guo, Zhepeng Cen, Huan Zhang, Jie Tan, Bo Li, and Ding Zhao. Datasets and benchmarks for offline safe reinforcement learning. *Journal of Data-centric Machine Learning Research*, 2024b.

Nikolay Lomys and Luca Magnolfi. Estimation of games under no regret: Structural econometrics for ai. Working Paper NET Institute Working Paper No. 24-05, Social Science Research Network (SSRN), 2024. Available at SSRN: https://ssrn.com/abstract=4717195 or http://dx.doi.org/10.2139/ssrn.4717195.

Daniel J. Luckett, Eric B. Laber, Anna R. Kahkoska, David M. Maahs, Elizabeth Mayer-Davis, and Michael R. Kosorok. Estimating dynamic treatment regimes in mobile health using V-learning. *Journal of the American Statistical Association*, 115(530):692–706, 2020.

Zhiyao Luo, Mingcheng Zhu, Fenglin Liu, Jiali Li, Yangchen Pan, Jiandong Zhou, and Tingting Zhu. DTR-Bench: An in silico environment and benchmark platform for reinforcement learning based dynamic treatment regime. *arXiv preprint arXiv:2405.18610*, 2024. Under review.

Clare Lyle, Mark Rowland, and Will Dabney. Understanding and preventing capacity loss in reinforcement learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

Clare Lyle, Mark Rowland, Will Dabney, Marta Kwiatkowska, and Yarin Gal. Understanding plasticity in neural networks. In *Proceedings of the 40th International Conference on Machine Learning*, Proceedings of Machine Learning Research, 2023.

Clare Lyle, Zeyu Zheng, Evgenii Nikishin, Bernardo Avila Pires, Razvan Pascanu, and Will Dabney. Disentangling causes of plasticity loss in neural networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

Fabio Maccheroni, Massimo Marinacci, and Aldo Rustichini. Ambiguity aversion, robustness, and the variational representation of preferences. *Econometrica*, 74(6):1447–1498, 2006.

Lorenzo Magnino, Zida Wu, Kai Shao, Jiacheng Shen, and Mathieu Laurière. Solving continuous mean field games: Deep reinforcement learning for non-stationary dynamics. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Lilia Maliar, Serguei Maliar, and Pablo Winant. Deep learning for solving dynamic economic models. *Journal of Monetary Economics*, 122:76–101, 2021.

Alan S. Manne. Linear programming and sequential decisions. *Management Science*, 6(3): 259–267, 1960.

Charles F. Manski. *Partial Identification of Probability Distributions*. Springer, New York, 2003.

Albert Marcet and Thomas J. Sargent. Convergence of least squares learning mechanisms in self-referential linear stochastic models. *Journal of Economic Theory*, 48(2):337–368, 1989.

Amir massoud Farahmand, Rémi Munos, and Csaba Szepesvári. Error propagation for approximate policy and value iteration. In *Advances in Neural Information Processing Systems 22 (NeurIPS)*, pages 568–576, 2010.

Roberto-Rafael Maura-Rivero, Marc Lanctot, Francesco Visin, and Kate Larson. Jackpot! alignment as a maximal lottery. *arXiv preprint arXiv:2501.19266*, 2025.

Daniel McFadden. Conditional logit analysis of qualitative choice behavior. *Frontiers in Econometrics*, pages 105–142, 1974a.

Daniel McFadden. Conditional logit analysis of qualitative choice behavior. In Paul Zarembka, editor, *Frontiers in Econometrics*, pages 105–142. Academic Press, 1974b. Working paper circulated 1973.

Daniel McFadden. Modeling the choice of residential location. In Anders Karlqvist, Lars Lundqvist, Folke Snickars, and Jörgen W. Weibull, editors, *Spatial Interaction Theory and Planning Models*, pages 75–96. North-Holland, 1978.

Jincheng Mei, Chenjun Xiao, Csaba Szepesvari, and Dale Schuurmans. On the global convergence rates of softmax policy gradient methods. In *International Conference on Machine Learning*, 2020.

Yu Meng, Mengzhou Xia, and Danqi Chen. SimPO: Simple preference optimization with a reference-free reward. *arXiv preprint arXiv:2405.14734*, 2024.

Thomas Mesnard, Theophane Weber, Fabio Viola, Shantanu Thakoor, Alaa Saade, Anna Harutyunyan, Will Dabney, Thomas S. Stepleton, Nicolas Heess, Arthur Guez, Eric Moulines, Marcus Hutter, Lars Buesing, and Remi Munos. Counterfactual credit assignment in model-free reinforcement learning. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 7654–7664. PMLR, 2021.

Qirui Mi, Siyu Xia, Songyuan Zhu, Haifeng Zhang, Bo Min, and Mengyue Yang. TaxAI: A dynamic economic simulator and benchmark for multi-agent reinforcement learning. In *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2024. arXiv:2309.16307.

Qirui Mi, Zhiyu Zhao, Chengdong Ma, Siyu Xia, Yan Song, Mengyue Yang, Jun Wang, and Haifeng Zhang. Learning macroeconomic policies through dynamic stackelberg mean-field games. In *Proceedings of the European Conference on Artificial Intelligence (ECAI), Frontiers in Artificial Intelligence and Applications*, 2025. arXiv:2403.12093.

Nolan Miller, Paul Resnick, and Richard Zeckhauser. Eliciting informative feedback: The peer-prediction method. *Management Science*, 51(9):1359–1373, 2005.

Abhilash Mishra. AI alignment and social choice: Fundamental limitations and policy implications. *arXiv preprint arXiv:2310.16048*, 2023.

Kanishka Misra, Eric M. Schwartz, and Jacob Abernethy. Dynamic online pricing with incomplete information using multiarmed bandit experiments. *Marketing Science*, 38(2):226–252, 2019.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, pages 1928–1937. PMLR, 2016.

Thomas M. Moerland, Joost Broekens, Aske Plaat, and Catholijn M. Jonker. Model-based reinforcement learning: A survey. *Foundations and Trends in Machine Learning*, 2023.

Peyman Mohajerin Esfahani and Daniel Kuhn. Data-driven distributionally robust optimization using the Wasserstein metric: Performance guarantees and tractable reformulations. *Mathematical Programming*, 171:115–166, 2018.

Tianshi Mu, Pranjal Rawat, John Rust, Chengjun Zhang, and Qixuan Zhong. Who is more bayesian: Humans or ChatGPT? *arXiv preprint arXiv:2504.10636*, 2025.

Jonas Mueller, Vasilis Syrgkanis, and Matt Taddy. Low-rank bandit methods for high-dimensional dynamic pricing. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Rémi Munos and Csaba Szepesvári. Finite-time bounds for fitted value iteration. *Journal of Machine Learning Research*, 9:815–857, 2008.

Rémi Munos and Csaba Szepesvári. Finite-time bounds for fitted value iteration. *Journal of Machine Learning Research*, 9(27):815–857, 2008.

Rémi Munos, Tom Stepleton, Anna Harutyunyan, and Marc G. Bellemare. Safe and efficient off-policy reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

Remi Munos, Michal Valko, Daniele Calandriello, Mohammad Gheshlaghi Azar, Mark Rowland, Daniel Guo, Yunhao Tang, Matthieu Geist, Thomas Mesnard, Come Fiegel, Andrea Michi, Marco Selvi, Sertan Girgin, Nikola Momchev, Olivier Bachem, Daniel J. Mankowitz, Doina Precup, and Bilal Piot. Nash learning from human feedback. *arXiv preprint arXiv:2312.00886*, 2023.

S. A. Murphy. Optimal dynamic treatment regimes. *Journal of the Royal Statistical Society, Series B*, 65(2):331–355, 2003.

Hongseok Namkoong, Ramtin Keramati, Steve Yadlowsky, and Emma Brunskill. Off-policy policy evaluation for sequential decisions under unobserved confounding. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

Yuriy Nevmyvaka, Yi Feng, and Michael Kearns. Reinforcement learning for optimized trade execution. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 673–680, 2006.

Xinkun Nie, Emma Brunskill, and Stefan Wager. Learning when-to-treat policies. *Journal of the American Statistical Association*, 116(533):392–409, 2021.

Evgenii Nikishin, Max Schwarzer, Pierluca D'Oro, Pierre-Luc Bacon, and Aaron Courville. The primacy bias in deep reinforcement learning. In *Proceedings of the 39th International Conference on Machine Learning*, Proceedings of Machine Learning Research, 2022.

Arnab Nilim and Laurent El Ghaoui. Robust control of Markov decision processes with uncertain transition matrices. *Operations Research*, 53(5):780–798, 2005.

William D. Nordhaus. An optimal transition path for controlling greenhouse gases. *Science*, 258(5086):1315–1319, 1992. doi: 10.1126/science.258.5086.1315.

William D. Nordhaus. Climate clubs: Overcoming free-riding in international climate policy. *American Economic Review*, 105(4):1339–1370, 2015. doi: 10.1257/aer.15000001.

Michael Oberst and David Sontag. Counterfactual off-policy evaluation with Gumbel-Max structural causal models. In *Proceedings of the 36th International Conference on Machine Learning*, pages 4923–4932, 2019.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022.

Alizée Pace, Hugo Yèche, David Ha, Gunnar Rätsch, and Guy Tennenholtz. Delphic offline reinforcement learning under nonidentifiable hidden confounding. In *International Conference on Learning Representations*, 2024.

Ariel Pakes and Paul McGuire. Computing Markov-perfect Nash equilibria: Numerical implications of a dynamic differentiated product model. *RAND Journal of Economics*, 25(4): 555–589, 1994.

Kishan Panaganti and Dileep Kalathil. Sample complexity of robust reinforcement learning with a generative model. In *Proceedings of the 25th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2022.

Fabio Pardo, Arash Tavakoli, Vitaly Levdik, and Petar Kormushev. Time limits in reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning*, Proceedings of Machine Learning Research, 2018.

Chanwoo Park, Mingyang Liu, Dingwen Kong, Kaiqing Zhang, and Asuman E. Ozdaglar. RLHF from heterogeneous feedback via personalization and preference aggregation. *arXiv preprint arXiv:2405.00254*, 2024.

Keiran Paster, Sheila McIlraith, and Jimmy Ba. You can't count on luck: Why decision transformers and RvS fail in stochastic environments. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Santiago Paternain, Luiz Chamon, Miguel Calvo-Fullana, and Alejandro Ribeiro. Constrained reinforcement learning has zero duality gap. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Andrew Patterson, Samuel Neumann, Martha White, and Adam White. Empirical design in reinforcement learning. In *arXiv preprint arXiv:2304.01315*, 2024.

Ivan P. Pavlov. *Conditioned Reflexes: An Investigation of the Physiological Activity of the Cerebral Cortex*. Oxford University Press, 1927.

Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, Cambridge, 2nd edition, 2009.

Judea Pearl and Elias Bareinboim. External validity: From do-calculus to transportability across populations. *Statistical Science*, 29(4):579–595, 2014.

Nianli Peng and Yilin Wang. Efficient and scalable deep reinforcement learning for mean field control games. *arXiv preprint arXiv:2501.00052*, 2025. Harvard course 6.S890 paper.

Ian R. Petersen, Matthew R. James, and Paul Dupuis. Minimax optimal control of stochastic uncertain systems with relative entropy constraints. *IEEE Transactions on Automatic Control*, 45(3):398–412, 2000.

Lerrel Pinto, James Davidson, Rahul Sukthankar, and Abhinav Gupta. Robust adversarial reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *PMLR*, pages 2817–2826, 2017.

Silviu Pitis. Rethinking the discount factor in reinforcement learning: A decision theoretic approach. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, 2019.

Moshe A. Pollatschek and Benjamin Avi-Itzhak. Algorithms for stochastic games with geometrical interpretation. *Management Science*, 15(7):399–415, 1969.

L. A. Prashanth, Cheng Jie, Michael Fu, Steven Marcus, and Csaba Szepesvári. Cumulative prospect theory meets reinforcement learning: Prediction and control. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 1406–1415, 2016.

Doina Precup, Richard S. Sutton, and Satinder P. Singh. Eligibility traces for off-policy policy evaluation. In *Proceedings of the 17th International Conference on Machine Learning (ICML)*, pages 759–766, 2000.

Drazen Prelec. A bayesian truth serum for subjective data. *Science*, 306(5695):462–466, 2004.

Martin L. Puterman and Shelby L. Brumelle. On the convergence of policy iteration in stationary dynamic programming. *Mathematics of Operations Research*, 4(1):60–69, 1979.

Marek Pycia and Peter Troyan. A theory of simplicity in games and mechanism design. *Econometrica*, 91(4):1495–1526, 2023.

Liqun Qi and Jie Sun. A nonsmooth version of Newton's method. *Mathematical Programming*, 58(1–3):353–367, 1993.

Zhiwei Qin, Xiaocheng Tang, Yan Jiao, Fan Zhang, Zhe Xu, Hongtu Zhu, and Jieping Ye. Ride-hailing order dispatching at DiDi via reinforcement learning. *INFORMS Journal on Applied Analytics*, 51(3):272–286, 2021.

Guannan Qu and Adam Wierman. Finite-time analysis of asynchronous stochastic approximation and q-learning. *Journal of Machine Learning Research*, 21(1):1–28, 2020.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Paria Rashidinejad, Banghua Zhu, Cong Ma, Jiantao Jiao, and Stuart Russell. Bridging offline reinforcement learning and imitation learning: A tale of pessimism. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Sai Srivatsa Ravindranath, Zhe Feng, Di Wang, Manzil Zaheer, Aranyak Mehta, and David C. Parkes. Deep reinforcement learning for sequential combinatorial auctions. Submitted to ICLR 2025, 2024. Available at https://openreview.net/forum?id=SVd9Ffcdp8.

Pranjal Rawat. Designing auctions when algorithms learn to bid. *Working Paper*, 2026.

Michael Reiter. Solving heterogeneous-agent models by projection and perturbation. *Journal of Economic Dynamics and Control*, 33(3):649–665, 2009. doi: 10.1016/j.jedc.2008.08.010.

Robert A. Rescorla and Allan R. Wagner. A theory of Pavlovian conditioning: Variations in the effectiveness of reinforcement and nonreinforcement. In Abraham H. Black and William F. Prokasy, editors, *Classical Conditioning II: Current Research and Theory*, pages 64–99. Appleton-Century-Crofts, 1972.

Herbert Robbins. Some aspects of the sequential design of experiments. *Bulletin of the American Mathematical Society*, 58(5):527–535, 1952.

James M. Robins. A new approach to causal inference in mortality studies with a sustained exposure period — application to control of the healthy worker survivor effect. *Mathematical Modelling*, 7(9-12):1393–1512, 1986.

James M. Robins. Optimal structural nested models for optimal sequential decisions. In D. Y. Lin and P. J. Heagerty, editors, *Proceedings of the Second Seattle Symposium in Biostatistics*, volume 179 of *Lecture Notes in Statistics*, pages 189–326. Springer, New York, 2004.

James M. Robins, Andrea Rotnitzky, and Lue Ping Zhao. Estimation of regression coefficients when some regressors are not always observed. *Journal of the American Statistical Association*, 89(427):846–866, 1994.

James M. Robins, Miguel Á. Hernán, and Babette Brumback. Marginal structural models and causal inference in epidemiology. *Epidemiology*, 11(5):550–560, 2000.

Luca F. Roggeveen, Ali el Hassouni, Harm-Jan de Grooth, Armand R. J. Girbes, Mark Hoogendoorn, and Paul W. G. Elbers. Reinforcement learning for intensive care medicine: Actionable clinical insights from novel approaches to reward shaping and off-policy model evaluation. *Intensive Care Medicine Experimental*, 12(1):32, 2024. doi: 10.1186/s40635-024-00614-x.

Paul R. Rosenbaum. *Observational Studies*. Springer, New York, 2 edition, 2002. doi: 10.1007/978-1-4757-3692-2.

Stéphane Ross and J. Andrew Bagnell. Agnostic system identification for model-based reinforcement learning. In *Proceedings of the 29th International Conference on Machine Learning*, 2012.

Michael Rothschild. A two-armed bandit theory of market pricing. *Journal of Economic Theory*, 9(2):185–202, 1974.

Donald B. Rubin. Bayesian inference for causal effects: The role of randomization. *The Annals of Statistics*, 6(1):34–58, 1978.

G. A. Rummery and M. Niranjan. On-line q-learning using connectionist systems. Technical report, Cambridge University, 1994.

John Rust. Optimal replacement of gmc bus engines: An empirical model of harold zurcher. *Econometrica*, 55(5):999–1033, 1987.

John Rust. Structural estimation of Markov decision processes. In Robert F. Engle and Daniel McFadden, editors, *Handbook of Econometrics*, volume 4, chapter 51, pages 3081–3143. Elsevier, 1994.

John Rust. Numerical dynamic programming in economics. In Hans M. Amman, David A. Kendrick, and John Rust, editors, *Handbook of Computational Economics*, volume 1, pages 619–729. Elsevier, 1996.

John Rust. Dynamic programming. In *The New Palgrave Dictionary of Economics*. Palgrave Macmillan, London, 2nd edition, 2008.

John Rust and Pranjal Rawat. Structural econometrics and inverse reinforcement learning: Inferring preferences and beliefs from human behavior. Working Paper, Georgetown University, 2026.

Andrzej Ruszczyński. Risk-averse dynamic programming for Markov decision processes. *Mathematical Programming*, 125(2):235–261, 2010. doi: 10.1007/s10107-010-0393-3.

Emmanuel Saez. Using elasticities to derive optimal income tax rates. *The Review of Economic Studies*, 68(1):205–229, 2001. doi: 10.1111/1467-937X.00166.

Shosei Sakaguchi. Policy learning for optimal dynamic treatment regimes with observational data. *arXiv preprint arXiv:2404.00221*, 2024. Revise-and-resubmit at *Journal of Econometrics*.

Arthur L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229, 1959.

Manuel S. Santos and John Rust. Convergence properties of policy iteration. *SIAM Journal on Control and Optimization*, 42(6):2094–2115, 2004.

Thomas J. Sargent. *Bounded Rationality in Macroeconomics*. Oxford University Press, 1993.

Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.

John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. *Proceedings of the 32nd International Conference on Machine Learning*, pages 1889–1897, 2015.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Phillip J. Schulte, Anastasios A. Tsiatis, Eric B. Laber, and Marie Davidian. Q- and A-learning methods for estimating optimal dynamic treatment regimes. *Statistical Science*, 29(4):640–661, 2014.

Avner Seror. The moral mind(s) of large language models. *arXiv preprint arXiv:2412.04476*, 2024.

Lior Shani, Yonathan Efroni, and Shie Mannor. Adaptive trust region policy optimization: Global convergence and faster rates for regularized MDPs. In *AAAI Conference on Artificial Intelligence*, 2020.

Claude E. Shannon. Programming a computer for playing chess. *Philosophical Magazine*, 41 (314):256–275, 1950.

Lloyd S. Shapley. Stochastic games. *Proceedings of the National Academy of Sciences*, 39(10): 1095–1100, 1953.

Lloyd S. Shapley. Some topics in two-person games. In M. Dresher, L. S. Shapley, and A. W. Tucker, editors, *Advances in Game Theory*, pages 1–28. Princeton University Press, 1964.

Chengchun Shi, Masatoshi Uehara, Jiawei Huang, and Nan Jiang. A minimax learning approach to off-policy evaluation in confounded partially observable markov decision processes. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 20057–20094. PMLR, 2022.

Chengchun Shi, Shikai Luo, Yuan Le, Hongtu Zhu, and Rui Song. Statistically efficient advantage learning for offline reinforcement learning in infinite horizons. *Journal of the Royal Statistical Society, Series B*, 2024a. Accepted; arXiv:2202.13163.

Chengchun Shi, Jin Zhu, Ye Shen, Shikai Luo, Hongtu Zhu, and Rui Song. Off-policy confidence interval estimation with confounded markov decision process. *Journal of the American Statistical Association*, 119(545):273–284, 2024b. doi: 10.1080/01621459.2022.2110878.

Rui Aruhan Shi. Learning from zero: How to make consumption-saving decisions in a stochastic environment with an AI algorithm. CESifo Working Paper 9255, CESifo, 2022. arXiv:2105.10099.

Yoav Shoham, Rob Powers, and Trond Grenager. If multi-agent learning is the answer, what is the question? *Artificial Intelligence*, 171(7):365–377, 2007.

Aaron Sidford, Mengdi Wang, Xian Wu, Lin F. Yang, and Yinyu Ye. Near-optimal time and sample complexities for solving Markov decision processes with a generative model. In *Advances in Neural Information Processing Systems*, 2018.

David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy

Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359, 2017.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018a.

Tom Silver, Kelsey Allen, Josh Tenenbaum, and Leslie Kaelbling. Residual policy learning. *arXiv preprint arXiv:1812.06298*, 2018b.

Satinder Singh, Tommi Jaakkola, Michael L Littman, and Csaba Szepesvári. Convergence results for single-step on-policy reinforcement-learning algorithms. *Machine learning*, 38: 287–308, 2000.

Satinder P. Singh and Richard C. Yee. An upper bound on the loss from approximate optimal-value functions. *Machine Learning*, 16(3):227–233, 1994.

Anand Siththaranjan, Cassidy Laidlaw, and Dylan Hadfield-Menell. Distributional preference learning: Understanding and accounting for hidden context in RLHF. In *International Conference on Learning Representations*, 2024.

Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Joar Max Viktor Skalse, Matthew Farrugia-Roberts, Stuart Russell, Alessandro Abate, and Adam Gleave. Invariance in policy optimisation and partial identifiability in reward learning. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 32033–32058, 2023.

Ghada Sokar, Rishabh Agarwal, Pablo Samuel Castro, and Utku Evci. The dormant neuron phenomenon in deep reinforcement learning. In *Proceedings of the 40th International Conference on Machine Learning*, Proceedings of Machine Learning Research, 2023.

Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning to summarize with human feedback. In *Advances in Neural Information Processing Systems*, volume 33, pages 3008–3021, 2020.

Nancy L. Stokey. Rational expectations and durable goods pricing. *Bell Journal of Economics*, 12(1):112–128, 1981.

Adam Stooke, Joshua Achiam, and Pieter Abbeel. Responsive safety in reinforcement learning by PID lagrangian methods. In *International Conference on Machine Learning (ICML)*, 2020.

Tomasz Strzalecki. Axiomatic foundations of multiplier preferences. *Econometrica*, 79(1):47–73, 2011.

Haoran Sun, Yurong Chen, Siwei Wang, Wei Chen, and Xiaotie Deng. Mechanism design for LLM fine-tuning with multiple reward models. *arXiv preprint arXiv:2405.16276*, 2024.

Richard S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3(1):9–44, 1988. doi: 10.1023/A:1022633531479.

Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Proceedings of the Seventh International Conference on Machine Learning*, pages 216–224. Morgan Kaufmann, 1990.

Richard S. Sutton and Andrew G. Barto. Time-derivative models of Pavlovian reinforcement. In Michael Gabriel and John Moore, editors, *Learning and Computational Neuroscience: Foundations of Adaptive Networks*, pages 497–537. MIT Press, 1990.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, 2nd edition, 2018.

Richard S. Sutton, David A. McAllester, Satinder P. Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems*, 12, 1999.

Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 2000.

Richard S. Sutton, Hamid Reza Maei, Doina Precup, Shalabh Bhatnagar, David Silver, Csaba Szepesvári, and Eric Wiewiora. Fast gradient-descent methods for temporal-difference learning with linear function approximation. In *Proceedings of the 26th International Conference on Machine Learning*, pages 993–1000. ACM, 2009.

Csaba Szepesvári. *Algorithms for Reinforcement Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool Publishers, 2010.

Andrea Tacchetti, Raphael Koster, Jan Balaguer, Liu Leqi, Mîruna Pislar, Matthew M. Botvinick, Karl Tuyls, David C. Parkes, and Christopher Summerfield. Deep mechanism design: Learning social and economic policies for human benefit. *Proceedings of the National Academy of Sciences*, 2025.

Aviv Tamar, Yinlam Chow, Mohammad Ghavamzadeh, and Shie Mannor. Policy gradient for coherent risk measures. *Advances in Neural Information Processing Systems*, 28, 2015.

Oskari Tammelin. Solving large imperfect information games using cfr+. *arXiv preprint arXiv:1407.5042*, 2014.

Zhiqiang Tan. A distributional approach for causal inference using propensity scores. *Journal of the American Statistical Association*, 101(476):1619–1637, 2006. doi: 10.1198/016214506000000023.

Shengpu Tang and Jenna Wiens. Counterfactually-augmented importance sampling for semi-offline policy evaluation. *Transactions on Machine Learning Research*, 2023. arXiv:2310.17146.

Xiaocheng Tang, Zhiwei Qin, Fan Zhang, Zhaodong Wang, Zhe Xu, Yintai Ma, Hongtu Zhu, and Jieping Ye. A deep value-network based approach for multi-driver order dispatching. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1780–1790, 2019.

Gerald Tesauro. Td-gammon, a self-teaching backgammon program, achieves master-level play. *Neural Computation*, 6(2):215–219, 1994.

Chen Tessler, Daniel J. Mankowitz, and Shie Mannor. Reward constrained policy optimization. In *International Conference on Learning Representations (ICLR)*, 2019.

William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3–4):285–294, 1933.

Edward L. Thorndike. *Animal Intelligence: An Experimental Study of the Associative Processes in Animals*. Macmillan, 1898. Psychological Review Monograph Supplements, No. 8.

Sebastian Thrun and Anton Schwartz. Issues in using function approximation for reinforcement learning. In *Proceedings of the Fourth Connectionist Models Summer School*, 1993.

Haoxing Tian, Ioannis Ch. Paschalidis, and Alex Olshevsky. Convergence of actor-critic methods with multi-layer neural networks. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society, Series B*, 58(1):267–288, 1996.

Emanuel Todorov. Linearly-solvable Markov decision problems. In *Advances in Neural Information Processing Systems*, volume 19, 2006.

Nenad Tomasev, Ulrich Paquet, Demis Hassabis, and Vladimir Kramnik. Assessing game balance with AlphaZero: Exploring alternative rule sets in chess. *arXiv preprint arXiv:2009.04374*, 2020.

Mark Towers, Ariel Kwiatkowski, Jordan K. Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulao, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tze, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. In *arXiv preprint arXiv:2407.17032*, 2024.

John N. Tsitsiklis. Asynchronous stochastic approximation and q-learning. *Machine Learning*, 16(3):185–202, 1994. doi: 10.1007/bf00993306.

John N. Tsitsiklis. On the convergence of optimistic policy iteration. *Journal of Machine Learning Research*, 3:59–72, 2002.

John N. Tsitsiklis and Benjamin Van Roy. Analysis of temporal-difference learning with function approximation. In *Advances in Neural Information Processing Systems 9 (NIPS)*, 1997.

Daniele Tullii, Adel Javanmard, Matteo Pirotta, and Pierre Lezaud. Contextual dynamic pricing with strategic buyers under unknown valuations. *arXiv preprint*, 2024. arXiv ID pending verification; the ID 2307.04895 previously listed resolved to an unrelated paper.

Amos Tversky and Daniel Kahneman. Advances in prospect theory: Cumulative representation of uncertainty. *Journal of Risk and Uncertainty*, 5(4):297–323, 1992.

Masatoshi Uehara, Haruka Kiyohara, Andrew Bennett, Victor Chernozhukov, Nan Jiang, Nathan Kallus, Chengchun Shi, and Wen Sun. Future-dependent value-based off-policy evaluation in POMDPs. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Lars van der Laan, Aurelien Bibaut, and Nathan Kallus. Efficient inference for inverse reinforcement learning and dynamic discrete choice models. *arXiv preprint arXiv:2512.24407*, 2025.

Hado van Hasselt. Double q-learning. In *Advances in Neural Information Processing Systems*, volume 23, 2010.

Hado van Hasselt, Arthur Guez, Matteo Hessel, Volodymyr Mnih, and David Silver. Learning values across many orders of magnitude. In *Advances in Neural Information Processing Systems*, volume 29, 2016a.

Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-Learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, 2016b.

Hado van Hasselt, Yotam Doron, Florian Strub, Matteo Hessel, Nicolas Sonnerat, and Joseph Modayil. Deep reinforcement learning and the deadly triad. In *arXiv preprint arXiv:1812.02648*, 2018.

Harm van Seijen, A. Rupam Mahmood, Patrick M. Pilarski, Marlos C. Machado, and Richard S. Sutton. True online temporal-difference learning. *Journal of Machine Learning Research*, 17 (145):1–40, 2016.

Martin J. Wainwright. Stochastic approximation with cone-contractive operators: Sharp ℓ_∞-bounds for Q-learning. *Annals of Statistics*, 47(6):3168–3197, 2019.

Jiayi Wang, Zhengling Qi, and Chengchun Shi. Blessing from human-AI interaction: Super policy learning in confounded environments. *Journal of the American Statistical Association*, 2025a. doi: 10.1080/01621459.2025.2574706. In press.

Tony Wang, Kyle Feinstein, and Sheryl Chen. Reinforcement learning for monetary policy under macroeconomic uncertainty: Analyzing tabular and function approximation methods. *arXiv preprint arXiv:2512.17929*, 2025b. Stanford CS course paper; preliminary.

Yue Wang, Tomasz Żak, and Csaba Szepesvári. Near-optimal sample complexity for iterated CVaR reinforcement learning with a generative model. *arXiv preprint arXiv:2503.08934*, 2025c.

Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992a.

Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3-4):279–292, 1992b. doi: 10.1007/bf00992698.

Ian N. Weaver, Vineet Kumar, and Lalit Jain. Nonparametric pricing bandits leveraging informational externalities to learn the demand curve. *Marketing Science*, 44(6):1299–1320, 2025. doi: 10.1287/mksc.2022.0247.

Peter Whittle. Risk-sensitive linear/quadratic/gaussian control. *Advances in Applied Probability*, 13(4):764–777, 1981.

Clarisse Wibault, Johannes Forkel, Sebastian Towers, Tiphaine Wibault, Juan Duque, George Whittle, Andreas Schaab, Yucheng Yang, Chiyuan Wang, Michael Osborne, Benjamin Moll, and Jakob Foerster. Recurrent structural policy gradient for partially observable mean field games. *arXiv preprint arXiv:2602.20141*, 2026.

Wolfram Wiesemann, Daniel Kuhn, and Berç Rustem. Robust Markov decision processes. *Mathematics of Operations Research*, 38(1):153–183, 2013.

Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

Ronald J. Williams and Jing Peng. Function optimization using connectionist reinforcement learning algorithms. *Connection Science*, 3(3):241–268, 1991.

Di Wu, Xiujun Chen, Xun Yang, Hao Wang, Qing Tan, Xiaoxun Zhang, Jian Xu, and Kun Gai. Budget constrained bidding by model-free reinforcement learning in display advertising. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1443–1451, 2018.

Yue Wu, Weitong Zhang, Pan Xu, and Quanquan Gu. A finite-time analysis of two time-scale actor-critic methods. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Yue Wu, Zhiqing Sun, Huizhuo Yuan, Kaixuan Ji, Yiming Yang, and Quanquan Gu. Self-play preference optimization for language model alignment. *arXiv preprint arXiv:2405.00675*, 2024.

Lin Xiao. On the convergence rates of policy gradient methods. *Journal of Machine Learning Research*, 23, 2022.

Jianyu Xu and Yu-Xiang Wang. Logarithmic regret in feature-based dynamic pricing. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Menahem E. Yaari. The dual theory of choice under risk. *Econometrica*, 55(1):95–115, 1987.

Shan Yang. Mean-field reinforcement learning without synchrony. *arXiv preprint arXiv:2602.18026*, 2026. National University of Singapore.

Yucheng Yang, Chiyuan Wang, Andreas Schaab, and Benjamin Moll. Structural reinforcement learning for heterogeneous agent macroeconomics. *Working paper, University of Zurich and London School of Economics*, 2025. Preliminary, December 2025.

Yinyu Ye. The simplex and policy-iteration methods are strongly polynomial for the Markov decision problem with a fixed discount rate. *Mathematics of Operations Research*, 36(4): 593–603, 2011.

Changchang Yin, Ruoqi Liu, Jeffrey Caterino, and Ping Zhang. Deconfounding actor-critic network with policy adaptation for dynamic treatment regimes. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022.

Mingzhang Yin, Claudia Shi, Yixin Wang, and David M. Blei. Conformal sensitivity analysis for individual treatment effects. *Journal of the American Statistical Association*, 119(545): 122–135, 2024. doi: 10.1080/01621459.2022.2102503.

Donghao Ying, Yuhao Ding, and Javad Lavaei. A dual approach to constrained Markov decision processes with entropy regularization. *arXiv preprint arXiv:2110.08923*, 2022.

Zhuodong Yu, Ling Dai, Shaohang Xu, Siyang Gao, and Chin Pang Ho. Fast Bellman updates for Wasserstein distributionally robust MDPs. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024.

Andrea Zanette. Exponential lower bounds for batch reinforcement learning: Batch RL can be exponentially harder than online RL. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*. PMLR, 2021.

Junzhe Zhang and Elias Bareinboim. Near-optimal reinforcement learning in dynamic treatment regimes. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Junzhe Zhang and Elias Bareinboim. Designing optimal dynamic treatment regimes: A causal reinforcement learning approach. In *Proceedings of the 37th International Conference on Machine Learning*, pages 11012–11022, 2020.

Shangtong Zhang and Richard S. Sutton. A deeper look at experience replay. In *arXiv preprint arXiv:1712.01275*, 2017.

Shangtong Zhang, Hengshuai Yao, and Shimon Whiteson. Breaking the deadly triad with a target network. In *International Conference on Machine Learning (ICML)*, 2021.

Tianyu Zhang, Andrew Williams, Soham Phade, Sunil Srinivasa, Yang Zhang, Prateek Gupta, Yoshua Bengio, and Stephan Zheng. RICE-N: A climate-economy modeling framework for negotiation. In *AAAI 2022 Workshop on AI for Climate Change*, 2022.

Tianyu Zhang, Andrew Williams, Soham Phade, Sunil Srinivasa, Yang Zhang, Prateek Gupta, Yoshua Bengio, and Stephan Zheng. RICE-N: A multi-region integrated assessment model for negotiation among strategic regions. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

Weipeng Zhang. Distributed randomized multiagent policy iteration in reinforcement learning. *Results in Control and Optimization*, 12:100255, 2023.

Stephan Zheng, Alexander Trott, Sunil Srinivasa, David C. Parkes, and Richard Socher. The AI economist: Optimal economic policy design via two-level deep reinforcement learning. *Science Advances*, 8(18):eabk2607, 2022. doi: 10.1126/sciadv.abk2607.

Zhengyuan Zhou, Susan Athey, and Stefan Wager. Offline multi-action policy learning: Generalization and optimization. *Operations Research*, 71(1):148–183, 2023.

Brian D. Ziebart. *Modeling Purposeful Adaptive Behavior with the Principle of Maximum Causal Entropy*. PhD thesis, Carnegie Mellon University, 2010.

Brian D. Ziebart, Andrew Maas, J. Andrew Bagnell, and Anind K. Dey. Maximum entropy inverse reinforcement learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2008.

Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.

Martin Zinkevich, Michael Johanson, Michael Bowling, and Carmelo Piccione. Regret minimization in games with incomplete information. In *Advances in Neural Information Processing Systems*, volume 20, 2008.

A Glossary of Acronyms and Terms

Acronyms

A2C/A3C	Advantage Actor-Critic / Asynchronous Advantage Actor-Critic. Policy gradient algorithms that use an advantage function baseline to reduce variance (Section 4).
BCQ	Batch-Constrained Q-learning. An offline RL algorithm that restricts the learned policy to actions supported by the behavioral data (Section 12.2).
BwK	Bandits with Knapsacks. A bandit framework with resource constraints, where an agent must balance exploration and exploitation subject to a budget (Section 11).
CCP	Conditional Choice Probabilities. Choice probabilities conditional on state, used in structural estimation as an alternative to full-solution methods (Section 8).

CFR	Counterfactual Regret Minimization. An iterative algorithm for computing Nash equilibria in extensive-form games by minimizing regret at each information set (Section 10).
CQL	Conservative Q-Learning. An offline RL algorithm that adds a penalty for overestimating Q-values on out-of-distribution actions (Section 12.2).
DDC	Dynamic Discrete Choice. A class of structural econometric models in which agents make sequential discrete decisions under uncertainty (Section 8).
DDPG	Deep Deterministic Policy Gradient. An actor-critic algorithm for continuous action spaces that combines a deterministic policy gradient with a learned Q-function (Section 4).
DP	Dynamic Programming. A collection of algorithms that compute optimal policies by solving the Bellman equation, given a known model of the environment (Section 4).
DPO	Direct Preference Optimization. A method that optimizes a language model directly on preference data without training a separate reward model (Section 13).
DQN	Deep Q-Network. Q-learning with a neural network function approximator, target network, and experience replay (Section 4).
ETC	Explore-Then-Commit. A bandit algorithm that explores uniformly for a fixed number of rounds, then commits to the empirically best arm (Section 11).
FQI/FVI	Fitted Q-Iteration / Fitted Value Iteration. Approximate dynamic programming algorithms that use supervised learning to fit value functions from batch data (Section 5.2.5).
GLIE	Greedy in the Limit with Infinite Exploration. A condition on exploration schedules ensuring convergence to the optimal policy (Section 4).
GMM	Generalized Method of Moments. An econometric estimation method that matches sample moments to population moment conditions (Section 8).
IPW	Inverse Probability Weighting. A method for correcting distributional mismatch by reweighting observations by the inverse of their selection probability (Section 14).
IQL	Implicit Q-Learning. An offline RL algorithm that avoids querying out-of-distribution actions by using expectile regression on the value function (Section 12.2).
IRL	Inverse Reinforcement Learning. The problem of inferring a reward function from observed behavior, assuming the agent acts approximately optimally (Section 4).
IV	Instrumental Variables. An econometric technique for identifying causal effects in the presence of endogeneity by exploiting exogenous variation (Section 14).
KL	Kullback-Leibler divergence. A measure of how one probability distribution diverges from a reference distribution, used in trust-region and regularization methods (Section 4).
LLM	Large Language Model. A neural network trained on large text corpora, the primary object of alignment via RLHF and DPO (Section 13).
LQR	Linear-Quadratic Regulator. An optimal control method that computes the policy for linear dynamics with a quadratic cost function via the Riccati equation (Section 7).

MARL	Multi-Agent Reinforcement Learning. RL in settings with multiple interacting agents, where each agent's optimal policy depends on the others' behavior (Section 10).
MC	Monte Carlo. Methods that estimate value functions from complete episode returns rather than bootstrapped estimates (Section 4).
MDP	Markov Decision Process. A formal model of sequential decision-making defined by states, actions, transition probabilities, rewards, and a discount factor (Section 4).
MLE	Maximum Likelihood Estimation. A statistical method that estimates parameters by maximizing the likelihood of the observed data (Section 8).
MPC	Model Predictive Control. A control method that solves a finite-horizon optimization problem at each timestep using an explicit model of the dynamics, then applies only the first action (Section 7).
NE	Nash Equilibrium. A strategy profile in which no player can improve their payoff by unilaterally deviating (Section 10).
NFXP	Nested Fixed Point. Rust's algorithm for structural estimation of DDC models that nests value function iteration inside a maximum likelihood loop (Section 8).
NPG	Natural Policy Gradient. A policy gradient method that preconditions the gradient by the inverse Fisher information matrix (Section 4).
OPE	Off-Policy Evaluation. Estimating the expected return of a target policy using data generated by a different behavioral policy (Section 14).
PEVI	Pessimistic Value Iteration. An offline RL algorithm that subtracts an uncertainty penalty from value estimates to avoid overestimation on unseen state-action pairs (Section 14).
PI	Policy Iteration. A dynamic programming algorithm that alternates between policy evaluation and policy improvement until convergence (Section 4).
PID	Proportional-Integral-Derivative controller. A feedback controller that computes a control signal from the weighted sum of the current error, its integral, and its derivative (Section 7).
POMDP	Partially Observable MDP. An MDP in which the agent cannot directly observe the state and must maintain beliefs from noisy observations (Section 14).
PPO	Proximal Policy Optimization. A policy gradient algorithm that constrains updates to a trust region via a clipped surrogate objective (Section 4).
RL	Reinforcement Learning. A framework for sequential decision-making in which an agent learns a policy by interacting with an environment and observing rewards (Section 4).
RLHF	Reinforcement Learning from Human Feedback. A framework for aligning model behavior with human preferences by training a reward model from pairwise comparisons and optimizing a policy against it (Section 13).
SAC	Soft Actor-Critic. An off-policy actor-critic algorithm that maximizes a combined objective of expected return and policy entropy (Section 4).
SARSA	State-Action-Reward-State-Action. An on-policy TD control algorithm that updates Q-values using the action actually taken in the next state (Section 4).

SFT	Supervised Fine-Tuning. The initial stage of LLM alignment in which the model is trained on curated demonstrations before preference optimization (Section 13).
SPE	Subgame Perfect Equilibrium. A refinement of Nash equilibrium requiring that strategies constitute a Nash equilibrium in every subgame (Section 10).
TD	Temporal Difference. A class of prediction and control algorithms that update value estimates using bootstrapped targets rather than complete returns (Section 5.2).
TRPO	Trust Region Policy Optimization. A policy gradient algorithm that constrains each update to lie within a KL-divergence trust region of the previous policy (Section 4).
TS	Thompson Sampling. A Bayesian bandit algorithm that selects actions by sampling from the posterior distribution over reward parameters (Section 11).
UCB	Upper Confidence Bound. A bandit algorithm that selects the arm with the highest sum of empirical mean and an exploration bonus (Section 11).
VI	Value Iteration. A dynamic programming algorithm that iteratively applies the Bellman optimality operator until the value function converges (Section 4).